Lectures on Quantum Computing and Completely Positive Maps

A formalisation in Isabelle/HOL of Vern Paulsen's Houston 2011 lecture notes

Dustin Bryant

formal statements and proofs: the theories listed in the table of contents

October 23, 2026

Abstract

This document is generated by Isabelle from the project theories. Every definition and theorem shown here has been checked by Isabelle/HOL (proofs are folded). The commentary restates Paulsen's lecture notes lecture by lecture (Day 1 = 22 August 2011, Day 34 = 18 November 2011): definitions and results in the notes' own language, each followed by the formal statement. The informal proofs of the notes are omitted; the formal proofs replace them. Where the formalisation deviates from the notes, the commentary says so. The companion file `NOTES_AUDIT.md` lists, for every lecture, what is formalised and what is not. Two companion entries build on it: the functional calculus of Hermitian matrices with product formulas, and quantum phase estimation with the HHL algorithm.

Contents

1 Reading guide **3**

2 Conventions **5**

I Lectures 1–10: basic mathematics and the postulates of quantum mechanics **5**

2.1 Day 2: the projection does not depend on the orthonormal basis 5
2.2 Day 6: coordinates in a tensor product 6
2.3 Day 8: non-orthogonal states cannot be distinguished 8
2.4 Day 9: measuring one half of a Bell pair 9
2.5 Day 11: SWAP from three CNOTs 10
2.6 Day 5: a basis of $V \oplus W$ (D5-5) 10

3 Day 5: Cholesky's algorithm on the notes' two examples (D5-3) **11**

3.1 One step of Cholesky's algorithm 11
3.2 Small matrices entry by entry 11
3.3 Example 1: a matrix that is not positive 11
3.4 Example 2: a positive matrix, in two steps 12

4 Day 6: the Schmidt rank (D6-4) **13**

5 Days 10, 11 and 15: superdense coding, the CCNOT table, the partial trace over A **15**

5.1 Superdense coding (D10-1) 15
5.2 The CCNOT truth table (D11-2) 16
5.3 The partial trace over A (D15-6) 16

6 Days 4–29: the converse Hermitian tests, composite ensembles, Choi's four conditions, codes **17**
6.1 Day 4: the converse directions for Hermitian matrices (D4-1) . . . 17
6.2 Choi's theorem with (2) isolated (D19-1) . . . 18
6.3 Composite ensembles (D14-5) . . . 18
6.4 Hamming distance and subspace codes (D29-4) . . . 19

II Lectures 11–22 and 29–34: quantum computing, density matrices, completely positive maps, error correction **20**

7 Day 11 (21 September 2011): gates revisited, circuit diagrams, no-cloning **20**
7.1 Cloning and no-cloning . . . 21

8 Day 12 (23 September 2011): quantum parallelism **22**

9 Day 13 (26 September 2011): ensembles and density matrices **22**
9.1 Ensembles . . . 22
9.2 Von Neumann's density matrix . . . 23
9.3 Probabilities via the trace . . . 24

10 Day 14 (28 September 2011): density matrices, pure states, composite ensembles **25**
10.1 Pure states . . . 26
10.2 Composite ensembles . . . 27
10.3 Partial traces: motivation . . . 27

11 Day 15 (30 September 2011): traces, tensor products of operators, partial traces **27**
11.1 The isomorphism $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$ . . . 27
11.2 Partial traces . . . 30
11.3 Partial traces and measurements . . . 32

12 Day 16 (3 October 2011): partial traces in block form; the postulates for density matrices **32**

13 Day 17 (5 October 2011): measurement maps, quantum noise, completely positive maps **33**
13.1 Measurement maps . . . 33
13.2 Kraus maps, positive maps, complete positivity . . . 34

14 Day 18 (7 October 2011): the theory of completely positive maps **38**

15 Day 19 (10 October 2011): Choi's theorem **38**
15.1 From a positive Choi matrix to a Kraus representation . . . 40

16 Day 20 (12 October 2011): positive functionals, adjoints, Choi rank **43**
16.1 The adjoint of a Kraus map . . . 43
16.2 Complete positivity and the UCP/CPTP duality . . . 44
16.3 Positive functionals are completely positive . . . 45
16.4 Choi rank . . . 45

17 Day 21 (14 October 2011): the Kraus rank bound; unitary freedom of representations **46**

18 Day 22 (17 October 2011): minimal Kraus representations, convex sets, UCP and CPTP **47**
18.1 Convex sets and extreme points . . . 47
18.2 The convex sets UCP and CPTP . . . 48

19 Day 29 (7 November 2011): classical error detection and correction **49**
19.1 Binary strings, Hamming weight and distance . . . 49
19.2 Code distance . . . 50
19.3 The parity-check and repetition codes . . . 50

20 Day 30 (9 November 2011): error probabilities; the three-qubit bit-flip code **51**
20.1 The Pauli matrices . . . 51
20.2 Pauli magic (Day 32) . . . 52
20.3 The bit-flip channel (Day 31) . . . 53

21 Day 31 (11 November 2011): the bit-flip code as a channel; fidelity **53**
21.1 Fidelity . . . 54

22 Day 32 (14 November 2011): the phase-flip code, the Shor code, Pauli magic **54**
22.1 The phase-flip code . . . 54
22.2 Projections . . . 56
22.3 More on the repetition code . . . 57
22.4 Algebra of projections; towards Days 33 and 34 . . . 58

23 Extensions beyond the notes **59**
23.1 Rank-one projections . . . 59
23.2 The bit-flip channel at $p = 0$ and $p = 1$ . . . 59
23.3 The phase-flip channel . . . 59
23.4 Unitary-conjugation channels . . . 60
23.5 The bit-phase-flip channel and the identity channel . . . 61
23.6 Composition of channels . . . 62
23.7 Conjugation channels are completely positive . . . 62
23.8 Unitary-conjugation channels are UCP and CPTP . . . 63
23.9 The Pauli channels are UCP and CPTP . . . 64
23.10 Pauli matrices as Kraus operators . . . 64
23.11 Kraus maps of Kraus-operator lists are CPTP . . . 65
23.12 Concatenation of Kraus lists . . . 65
23.13 Fixed points of Pauli channels . . . 66
23.14 The maximally mixed state . . . 67
23.15 Fixed points of unitary-conjugation channels . . . 67
23.16 Channels preserve density matrices . . . 68
23.17 Pauli product identities . . . 69

24 Small matrix and vector lemmas shared by the gap-closure theories **69**

25 Day 12: $H^{\otimes N}$ on $|0\cdots0\rangle$ and the counting probability (D12-1, D12-4) **71**

26 Day 16: $H_A \otimes H_B \cong H_A^{(p)}$ and $L(H_A \otimes H_B) \cong M_p(L(H_A))$ (D16-2) **73**

27 Days 17–18: densities to densities iff positive and trace-preserving (D17-4, D18-2) **75**

28 Day 18: the transpose is positive but not 2-positive (D18-4) **75**

29 Day 20: Choi's rank theorem $\mathrm{cr}(\Phi) = \operatorname{rank} P_\Phi$ (D20-4) **77**
29.1 $\operatorname{rank}(\sum_{l<L} w_l w_l^*) \leq L$ . . . 77
29.2 Independent vectors inside the column space bound the rank from below . . . 78
29.3 The theorem . . . 78

30 Day 21: unitary freedom of rank-one decompositions and Kraus representations (D21-3, D21-5, D21-6) **78**
30.1 The Gram matrix of an independent list is invertible; dual vectors . . . 78
30.2 D21-3: independent vectors give independent rank-one matrices . . . 79
30.3 Isometric mixing of columns preserves VV^* . . . 79
30.4 D21-5: two rank-one decompositions are related by an isometry . . . 80
30.5 D21-6 (Choi 2): unitary freedom of Kraus representations . . . 80

31 Day 21: ranges of positive matrices (D21-4) **81**
31.1 Day 4 revisited: positive definiteness and spanning (D4-8) . . . 81

32 Day 21: the conjecture on averages of unital maps (D21-2) **81**
32.1 Positive matrices as sums of d rank-one terms . . . 83
32.2 Measurement maps . . . 83
32.3 The case $d = 1$: Horn's lemma . . . 84
32.4 Maps with diagonal values . . . 84
32.5 The notes' form (i) is false as written . . . 84

33 D21-2 for $n = 2$ **85**
33.1 Completing an orthonormal family to an orthonormal basis . . . 85
33.2 Singular value decomposition . . . 85
33.3 A contraction is the average of two unitaries . . . 85
33.4 Bessel's inequality for rows that are orthonormal or zero . . . 86
33.5 Factoring a wide matrix . . . 86
33.6 The case $n = 2$ . . . 86

34 Day 22: linear independence bounds the number of Kraus operators (D22-6) **87**

35 Day 22: a minimal Kraus representation is linearly independent (D22-1) **88**
35.1 Mixing a Kraus list by an isometry does not change the map . . . 88
35.2 A unitary with a prescribed first row . . . 89
35.3 D22-1 . . . 89

36 Day 22: the adjoint map, an affine bijection from UCP onto CPTP (D22-5) **89**

37 Day 22: $\mathrm{UCP}(M_n, M_d)$ is compact (D22-4) **91**

38 Day 6 and Day 15: the universal properties in the Kronecker model **93**
38.1 The notes' "Hamming [7,1,3]" code does not correct one error . . . 96
38.2 The Hamming [7,4,3] code corrects one error . . . 97
38.3 Probabilities of undetected and uncorrected errors (Day 30) . . . 97

39 Day 30: the three-qubit bit-flip code and its recovery map (D30-3, D30-4) **98**
39.1 Errors followed by recovery . . . 102
39.2 Minimum fidelities (D31-4) . . . 103

40 Day 32: the Pauli expansion identity for unitaries (D32-7) **103**

41 Days 32–33: composition of Kraus maps and the general recovery proposition (D32-4, D33-2) 104
41.1 Algebra of *foldr*-sums of matrices . . . 104
41.2 Composition of Kraus maps . . . 105
41.3 The general recovery proposition (D32-4) and its linear-combination form (D33-2) 106
41.4 Matrix sums acting on vectors . . . 106
41.5 Codes with orthogonal error images . . . 109
41.6 The Y-flip code . . . 110
41.7 Tensor products of matrices . . . 110
41.8 Single-qubit operators on nine qubits, and on a block of three . . . 111
41.9 The block vectors $|m\rangle \pm |7-m\rangle$ . . . 112
41.10 Code words and the images of the errors . . . 113
41.11 The 22 correctable errors . . . 114
41.12 The code space and its projection . . . 115
41.13 Proposition (Day 33): orthogonal images, and $Z_1 v = Z_2 v = Z_3 v$ . . . 116
41.14 Shor's theorem . . . 117

42 Day 34: the Knill–Laflamme conditions (D34-1, D34-2, D34-3) 118
42.1 Vectors in the range of a projection; rank-one decompositions inside V . . . 118
42.2 Linearity facts for the composite map $\mathcal{R} \circ \mathbb{E}$ . . . 118
42.3 Knill–Laflamme, Theorem 1, necessity: $PE_i^* E_j P = \alpha_{ij} P$ . . . 119
42.4 Kraus lists consisting of multiples of one operator . . . 120
42.5 Theorem 2 . . . 120

III Lectures 23–26: extreme points, operator systems and Arveson's theorem 122

43 Day 23 (19 October 2011): Choi's Theorem 3, extreme points of UCP and CPTP 122
43.1 Extreme UCP maps . . . 123
43.2 UCP-Kraus representation . . . 123
43.3 Unital condition from the Kraus list . . . 124
43.4 UCP-Kraus existence . . . 124
43.5 Sanity facts about *kraus-products* . . . 125
43.6 Quadratic Kraus map (perturbation primitive) . . . 125
43.7 Lemma A: PSD coefficient matrix gives a CP perturbation . . . 128
43.8 Small-ε PSD perturbation of the identity . . . 129
43.9 Symmetric / anti-hermitian split of the coefficient matrix . . . 130
43.10 Bridge: ¬ *mat-lin-indep-real* gives a real *A-orig* witness . . . 131
43.11 Auxiliary lemmas for the assembly . . . 133
43.12 Minimality and the distinctness lemma (Lemma B) . . . 133
43.13 CPTP-Kraus representation and extreme CPTP maps . . . 134
43.14 Vectorisation of Kraus operators . . . 135
43.15 Choi matrix of a Kraus map . . . 135
43.16 Choi matrix is linear in the map . . . 136
43.17 The Choi matrix of a *kraus-quad-map* . . . 136
43.18 CP Radon–Nikodym — the dominated-PSD kernel . . . 137
43.18.1 Diagonal pseudo-inverse list and its identity . . . 138
43.19 Linearity of *kraus-quad-map* in its matrix argument . . . 139
43.20 Choi matrix determines a linear CP map on carrier . . . 140
43.21 CP Radon–Nikodym . . . 140

43.22Choi's Theorem 3 — reverse direction . . . 141
43.23Trace pairing duality and TP-converse . . . 144
43.24Choi's Theorem 3 — forward direction . . . 144
43.25Choi's Theorem 3 (statement) . . . 145
43.26Sanity / easy facts about extreme UCP . . . 146

44 Day 23: Choi's Theorem 3 for unital maps (D23-1) 147
44.1 Linear dependence tests . . . 148
44.2 Example 1: unitary conjugation is extreme . . . 149
44.3 Example 2: an average of two unitary conjugations . . . 149
44.4 Example 4: the diagonal map . . . 150
44.5 Example 3: $X \mapsto \mathrm{Tr}(X)\, I_d$ . . . 151
44.6 $M_p(OS)$ is an operator system . . . 152
44.7 Every element is a combination of four positives . . . 153
44.8 Finite sums of matrices . . . 153
44.9 $E_{ij} \otimes M$ and the block decomposition . . . 154
44.10The correspondence . . . 155
44.11Theorem (Arveson): completely positive iff d-positive iff S_Φ positive . . . 156
44.12Vectors . . . 158
44.13The operator norm . . . 158
44.14Unital functionals . . . 159
44.15Auxiliary facts . . . 160
44.16Matrices as points of the Euclidean space *lmat* . . . 161
44.17Extension of positive functionals . . . 162
44.18Arveson's extension theorem . . . 162

45 Day 26: factoring a CP map through a unital or a trace-preserving one (D26-1) 162

IV Functional-analytic infrastructure (Lecture 25: Hahn–Banach) 164

V Lectures 26–28: separability and entanglement witnesses 164

46 Days 26–28 (28 October – 2 November 2011): separability and entanglement witnesses 164
46.1 The separable cone (Day 27) . . . 165
46.2 Separable density matrices (Day 26) . . . 165
46.3 The separable cone is closed (Day 28) . . . 165
46.4 Separation (Days 27–28) . . . 165
46.5 Entanglement witnesses (Day 28) . . . 165
46.6 Tensor products with a matrix in the first factor . . . 166
46.7 The quadratic form of Φ_f . . . 166
46.8 Proposition (Day 27): $f(\mathrm{Sep}) \geq 0$ iff Φ_f is positive . . . 166
46.9 Proposition (Day 27): positive maps keep separable matrices positive . . . 167
46.10Theorem (Day 28): the positive-map criterion . . . 167

47 Day 27: entangled matrices detected by the transpose (D27-2) 168

48 Day 28: the Horodecki–Peres criterion for two qubits (D28-5) 169

theory *QC-Notes-Overview*
imports
Quantum-Computing-by-VP-3-10.Basic-Math
Quantum-Computing-by-VP.Day02-11-Supplement
Quantum-Computing-by-VP-3-10.Quantum-Computing-3-10
Quantum-Computing-by-VP.Day04-29-Supplement
Quantum-Computing-by-VP.Day05-Cholesky-Examples
Quantum-Computing-by-VP.Day06-15-Universal
Quantum-Computing-by-VP.Day06-Schmidt-Rank
Quantum-Computing-by-VP.Day10-15-Supplement
Quantum-Computing-by-VP.Quantum-Computing11
Quantum-Computing-by-VP.Day12-Counting-Probability
Quantum-Computing-by-VP.Day16-Block-Form
Quantum-Computing-by-VP.Day17-Environment-Model
Quantum-Computing-by-VP.Day18-Positive-Maps
Quantum-Computing-by-VP.Day19-Choi-TP-Unital
Quantum-Computing-by-VP.Day20-Choi-Rank
Quantum-Computing-by-VP.Day21-Average-Conjecture
Quantum-Computing-by-VP.Day21-Average-Two
Quantum-Computing-by-VP.Day21-Ranges
Quantum-Computing-by-VP.Day21-Unitary-Freedom
Quantum-Computing-by-VP.Day22-Adjoint-Maps
Quantum-Computing-by-VP.Day22-Kraus-Independence
Quantum-Computing-by-VP.Day22-Minimal-Kraus
Quantum-Computing-by-VP.Day22-UCP-Compact
Quantum-Computing-by-VP.Day23-Choi3-UCP
Quantum-Computing-by-VP.Day23-ChoiExtremeUCP
Quantum-Computing-by-VP.Day23-Extremality-Examples
Quantum-Computing-by-VP.Day24-Operator-Systems
Quantum-Computing-by-VP.Day24-Separability
Quantum-Computing-by-VP.Day25-Arveson-Correspondence
Quantum-Computing-by-VP.Day25-Unital-Functional-Norm
Quantum-Computing-by-VP-3-10.Matrix-Typed
Quantum-Computing-by-VP.Day26-Arveson-Extension
Quantum-Computing-by-VP.Day26-CP-Factorization
Quantum-Computing-by-VP-3-10-Lmat.Separable-Cone
Quantum-Computing-by-VP.Day27-Entangled-Examples
Quantum-Computing-by-VP.Day27-Positive-Map-Criterion
Quantum-Computing-by-VP.Day28-Horodecki-Peres
Quantum-Computing-by-VP.Day29-Classical-Codes
Quantum-Computing-by-VP.Day30-BitFlip-Code
Quantum-Computing-by-VP.Day31-Code-Fidelity
Quantum-Computing-by-VP.Day32-Pauli-Expansion
Quantum-Computing-by-VP.Day32-Recovery-Proposition
Quantum-Computing-by-VP.Day32-Y-Code
Quantum-Computing-by-VP.Day33-Shor-Code
Quantum-Computing-by-VP.Day34-KL-Sufficiency
Quantum-Computing-by-VP.Day34-Knill-Laflamme
Quantum-Computing-by-VP-3-10.Essential-Functional-Analysis
Quantum-Computing-by-VP-3-10-Lmat.Lmat-Euclidean
Quantum-Computing-by-VP.QC-Prelims
begin

1 Reading guide

Paulsen's course had five parts: (I) basic mathematics, (II) an introduction to quantum computing, (III) the theory of completely positive maps, (IV) entanglement and entanglement witnesses, (V) quantum error correction. The formal development keeps the mathematics of the notes but is organised by theory, since later lectures reuse earlier infrastructure. The table gives the correspondence.

Lectures	Topic
1–2	inner products, matrix units, trace, projections, Parseval
3–10	unitaries, Schur, Hermitian and positive matrices, tensor products, postulates, POVMs, gates
11–22	no-cloning, oracles, density matrices, partial traces, CP maps, Choi's theorem, adjoints, convexit
23	extreme points of CPTP/UCP maps (Choi's Theorem 3), CP Radon–Nikodym
25	Hahn–Banach on matrix spaces
–	Hilbert–Schmidt analysis, Carathéodory, typed matrices as a Euclidean space
26–28	separable cone, separating hyperplanes, entanglement witnesses
29–32	classical codes, bit/phase-flip channels, fidelity, Pauli algebra
12	$H^{\otimes N}\|0\cdots0\rangle$, the counting probability
17	the environment model of noise, $\mathrm{tr}_E(U(\rho\otimes\|\varphi\rangle\langle\varphi\|)U^*)=\sum W_i\rho W_i^*$
17–18	densities to densities iff positive and TP; the transpose
20	$\mathrm{cr}(\Phi)=\operatorname{rank}P_\Phi$
21	rank-one decompositions and Kraus representations up to an isometry (Choi 2)
22	$\mathrm{cr}(\Phi)\le n$ for extreme maps; minimal Kraus lists are independent
24	operator systems, $M_p(S)$, completely positive maps on S
24–25	Arveson: CP $\iff$ d-positive $\iff$ $S_\Phi\ge0$; the correspondence $\Phi\leftrightarrow S_\Phi$
26	Arveson's extension theorem; Kraus form on operator systems
27–28	positive maps and separability: $f(\mathrm{Sep})\ge0\iff\Phi_f$ positive, the positive-map criterion
28	the two-qubit Horodecki–Peres criterion
30	the three-qubit bit-flip code and its recovery theorem
32–33	Pauli expansion of unitaries; the general recovery proposition
32–33	the nine-qubit Shor code and Shor's theorem
34	Knill–Laflamme: states vs. PXP, Theorem 1 (both directions), closure
2–11	basis-independent projections, a basis of $V\oplus W$, tensor coordinates, non-orthogonal states, a Be
29–30	the notes' Hamming code, the $[7,4,3]$ code, undetected-error probabilities
31	the bit-flip code against independent flips: $R\circ E$, minimum fidelities
32	the code for Y flips; codes with orthogonal error images
19	trace preservation and unitality read off the Choi matrix
23	extreme and non-extreme CPTP maps: U^*XU, averages, $\mathrm{Tr}(X)I/d$, diagonals
25	a unital functional is positive iff it has norm one
5	Sylvester's criterion, imported from the AFP (separate session and PDF, see Day 5)
5	Cholesky's algorithm on the notes' two 3×3 examples; the rank-one decomposition
22	the adjoint Φ^* of a linear map; $\Gamma(\Phi)=\Phi^*$ is an affine bijection UCP $\to$ CPTP
22	$\mathrm{UCP}(M_n,M_d)$ is compact (in Choi coordinates)
6, 15	universal properties: bilinear maps factor uniquely through $v\otimes w$; $\Gamma(R\otimes T)=R\otimes T$ determine
21	the conjecture D21-2 stated; Fourier mixing, measurement maps, Horn's lemma, diagonal-valued
21	D21-2 for $n=2$: Bessel, singular values, a contraction as the average of two unitaries
23	Choi's Theorem 3 for unital maps, via the adjoint
16	$H_A\otimes H_B\cong H_A^{(p)}$ isometrically; $X=\sum X_{ij}\otimes E_{ij}$; block action, partial traces
6, 10	the Schmidt rank: rank 1 iff a product state, entangled iff rank >1
10–15	superdense coding table, the CCNOT truth table, the partial trace over A
4–29	converse Hermitian tests, composite ensembles, Choi's four conditions, subspace codes
4, 21	$0\le ww^*\le P\Rightarrow w\in R(P)$; $R(\sum w_lw_l^*)=\mathrm{span}\{w_l\}$; $\sum v_iv_i^*>0$ iff the v_i span
26	CP maps as $V^*\psi(X)V$ with ψ UCP and as $\psi(VXV^*)$ with ψ CPTP
27	the transpose detects $\left(\begin{smallmatrix}E_{ii}&E_{ij}\\E_{ji}&E_{jj}\end{smallmatrix}\right)$ and (E_{ij}) as entangled

The theories of the second and third blocks (shared lemmas in `QC_Prelims`) close the gaps recorded by the first and second audits; the remaining unformalised items are listed in `NOTES_AUDIT.md`. The companion AFP entries *Hermitian Functional Calculus* and *HHL Algorithm* go beyond the

notes: the functional calculus and product formulas, quantum phase estimation and the HHL algorithm.

2 Conventions

- Vectors in $\mathbb{C}^n$ are *complex Matrix.vec* with *dim-vec* $v = n$; matrices in $M_{m,n}$ are *complex Matrix.mat* with $A \in$ *carrier-mat* m n (the Jordan-Normal-Form library). Dimensions are hypotheses, not types, except in the typed wrapper *lmat* used for the topological arguments of Part IV.
- The physicists' adjoint $X^\dagger = X^*$ is $A^\dagger$; $\langle y|x\rangle$ is $\langle y|x\rangle$ (conjugate-linear in the first slot, as in the notes); $|v\rangle\langle w|$ is $v \oslash w$; the Hilbert–Schmidt inner product $\langle Y|X\rangle = \mathrm{Tr}(Y^*X)$ is $\langle\langle$ Y | X $\rangle\rangle$; the matrix unit $E_{ij} \in M_{m,n}$ is $E[\![i,j$; $m,n]\!]$.
- Indices start at 0, as the physicists in the notes prefer: the canonical basis of $\mathbb{C}^n$ is $|0\rangle, \ldots, |n-1\rangle$, formally *unit-vec n i*.
- $\sum_i |v_i\rangle\langle v_i|$ for a list of vectors is *outer-sum n vs* $= VV^\dagger$ with V the matrix whose columns are the v_i.
- Tensor products are Kronecker products: $v \otimes w$ and $A \bigotimes B$, with the first factor as the major index.
- Kraus sums $\sum_i W_i X W_i^\dagger$ over a list are *kraus-map-dim d Ws X*.

Verification status. All theories build with *quick-and-dirty* = *false*; there are no *sorry*s and no axioms beyond those of Isabelle/HOL and the imported libraries.

end

Part I

Lectures 1–10: basic mathematics and the postulates of quantum mechanics

theory *Day02-11-Supplement*
imports *Quantum-Computing11*
begin

Supplementary results for Days 2–11 recorded as gaps by the first audit.

2.1 Day 2: the projection does not depend on the orthonormal basis

definition *col-span* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec set* **where**
col-span n vs = {*mat-of-cols n vs* $*_v$ *c* | *c*. *dim-vec c* = *length vs*}

lemma *outer-sum-fixes-col-span*:
assumes *orth*: *orthonormal-list n vs* **and** *w*: *w* ∈ *col-span n vs*
shows *outer-sum n vs* $*_v$ *w* = *w*
⟨*proof*⟩

lemma *mem-col-span-self*:
assumes ∀ *v* ∈ *set vs*. *dim-vec v* = *n* **and** *j*: *j* < *length vs*

shows *vs* ! *j* ∈ *col-span n vs*
⟨*proof*⟩

lemma *outer-sum-absorbs*:
assumes *ov*: *orthonormal-list n vs* **and** *ow*: *orthonormal-list n ws*
and *sub*: *set ws* ⊆ *col-span n vs*
shows *outer-sum n vs* ∗ *outer-sum n ws* = *outer-sum n ws*
⟨*proof*⟩

D2-8: two orthonormal bases of the same subspace give the same projection.

theorem *outer-sum-basis-independent*:
assumes *ov*: *orthonormal-list n vs* **and** *ow*: *orthonormal-list n ws*
and *span*: *col-span n vs* = *col-span n ws*
shows *outer-sum n vs* = *outer-sum n ws*
⟨*proof*⟩

2.2 Day 6: coordinates in a tensor product

$\sum_j x_j \otimes y_j$ in $\mathbb{C}^N$, written as in *schmidt-rank*.

definition *tensor-combination* :: *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec* **where**
tensor-combination N xs ys = *foldr* (λ(*x*,*y*) *acc*. *tensor-vec x y* + *acc*) (*zip xs ys*) (0$_v$ *N*)

lemma *tensor-combination-dim* [*simp*]: *dim-vec* (*tensor-combination N xs ys*) = *N*
⟨*proof*⟩

lemma *tensor-combination-index*:
assumes ∀ *x* ∈ *set xs*. *dim-vec x* = *k* ∀ *y* ∈ *set ys*. *dim-vec y* = *m length xs* = *length ys p* < *k* ∗ *m*
shows *tensor-combination* (*k* ∗ *m*) *xs ys* $ *p* = (∑*j*<*length xs*. *xs* ! *j* $ (*p div m*) ∗ *ys* ! *j* $ (*p mod m*))
⟨*proof*⟩

A basis $f_1, \ldots, f_m$ of $\mathbb{C}^m$ is a list whose column matrix F is invertible; the coordinates use $G = F^{-1}$.

A vector of $\mathbb{C}^k \otimes \mathbb{C}^m$ as a $k \times m$ matrix: entry (i, l) is the coefficient of $e_i \otimes e_l$.

definition *vec-to-mat* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
vec-to-mat k m u = *Matrix.mat k m* (λ(*i*, *l*). *u* $ (*i* ∗ *m* + *l*))

lemma *vec-to-mat-carrier* [*simp*]: *vec-to-mat k m u* ∈ *carrier-mat k m*
⟨*proof*⟩

lemma *vec-to-mat-dims* [*simp*]: *dim-row* (*vec-to-mat k m u*) = *k dim-col* (*vec-to-mat k m u*) = *m*
⟨*proof*⟩

lemma *col-vec-to-mat*: *l* < *m* ⟹ *Matrix.col* (*vec-to-mat k m u*) *l* = *Matrix.vec k* (λ*i*. *u* $ (*i* ∗ *m* + *l*))
⟨*proof*⟩

definition *right-coord* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
right-coord k m G u j = *vec-to-mat k m u* ∗$_v$ *Matrix.row G j*

definition *left-coord* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
left-coord k m Hm u i = (*vec-to-mat k m u*)^T ∗$_v$ *Matrix.row Hm i*

lemma *right-coord-dim* [*simp*]: *dim-vec* (*right-coord k m G u j*) = *k*

⟨*proof*⟩

lemma *left-coord-dim* [*simp*]: *dim-vec* (*left-coord k m Hm u i*) = *m*
⟨*proof*⟩

lemma *right-coord-index*:
G ∈ *carrier-mat m m* ⟹ *i* < *k* ⟹ *right-coord k m G u j* $ *i* = (∑ *l*<*m*. *u* $ (*i* ∗ *m* + *l*) ∗ *G* $$ (*j*,*l*))
⟨*proof*⟩

lemma *left-coord-index*:
Hm ∈ *carrier-mat k k* ⟹ *l* < *m* ⟹ *left-coord k m Hm u i* $ *l* = (∑ *r*<*k*. *Hm* $$ (*i*,*r*) ∗ *u* $ (*r* ∗ *m* + *l*))
⟨*proof*⟩

lemma *basis-inverse*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
obtains *G* **where** *G* ∈ *carrier-mat m m G* ∗ *mat-of-cols m fs* = 1_m *m mat-of-cols m fs* ∗ *G* = 1_m *m*
⟨*proof*⟩

lemma *inv-entry-left*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *GF*: *G* ∗ *mat-of-cols m fs* = 1_m *m* **and** *j*: *j* < *m* **and** *j'*: *j'* < *m*
shows (∑ *l*<*m*. *fs* ! *j'* $ *l* ∗ *G* $$ (*j*,*l*)) = (*if j* = *j' then 1 else 0*)
⟨*proof*⟩

lemma *inv-entry-right*:
fixes *fs* :: *complex Matrix.vec list*
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *FG*: *mat-of-cols m fs* ∗ *G* = 1_m *m* **and** *r*: *r* < *m* **and** *l*: *l* < *m*
shows (∑ *j*<*m*. *fs* ! *j* $ *r* ∗ *G* $$ (*j*,*l*)) = (*if r* = *l then 1 else 0*)
⟨*proof*⟩

lemma *right-coord-unique*:
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *GF*: *G* ∗ *mat-of-cols m fs* = 1_m *m*
and *xs*: *length xs* = *m* ∀ *x* ∈ *set xs*. *dim-vec x* = *k* **and** *j*: *j* < *m*
shows *right-coord k m G* (*tensor-combination* (*k* ∗ *m*) *xs fs*) *j* = *xs* ! *j*
⟨*proof*⟩

lemma *right-coord-exists*:
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *G*: *G* ∈ *carrier-mat m m*
and *FG*: *mat-of-cols m fs* ∗ *G* = 1_m *m* **and** *u*: *dim-vec u* = *k* ∗ *m*
shows *tensor-combination* (*k* ∗ *m*) (*map* (*right-coord k m G u*) [*0*..<*m*]) *fs* = *u*
⟨*proof*⟩

D6-3 (i): for a basis $f_1, \ldots, f_m$ of $\mathbb{C}^m$, every $u \in \mathbb{C}^k \otimes \mathbb{C}^m$ is $\sum_j x_j \otimes f_j$ for unique $x_1, \ldots, x_m \in \mathbb{C}^k$.

theorem *tensor-right-coordinates*:
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
and *u*: *dim-vec u* = *k* ∗ *m*
shows ∃!*xs*. *length xs* = *m* ∧ (∀ *x* ∈ *set xs*. *dim-vec x* = *k*) ∧ *u* = *tensor-combination* (*k* ∗ *m*) *xs fs*
⟨*proof*⟩

lemma *left-coord-unique*:
assumes *ed*: ∀ *e* ∈ *set es*. *dim-vec e* = *k length es* = *k* **and** *Hm*: *Hm* ∈ *carrier-mat k k*
and *HE*: *Hm* ∗ *mat-of-cols k es* = 1_m *k*

and *ys*: *length ys* = *k* ∀ *y* ∈ *set ys*. *dim-vec y* = *m* **and** *i*: *i* < *k*
shows *left-coord k m Hm* (*tensor-combination* (*k* ∗ *m*) *es ys*) *i* = *ys* ! *i*
⟨*proof*⟩

lemma *left-coord-exists*:
assumes *ed*: ∀ *e* ∈ *set es*. *dim-vec e* = *k length es* = *k* **and** *Hm*: *Hm* ∈ *carrier-mat k k*
and *EH*: *mat-of-cols k es* ∗ *Hm* = 1_m *k* **and** *u*: *dim-vec u* = *k* ∗ *m*
shows *tensor-combination* (*k* ∗ *m*) *es* (*map* (*left-coord k m Hm u*) [*0*..<*k*]) = *u*
⟨*proof*⟩

D6-3 (ii): likewise $u = \sum_i e_i \otimes y_i$ for unique $y_1, \ldots, y_k \in \mathbb{C}^m$.

theorem *tensor-left-coordinates*:
assumes *ed*: ∀ *e* ∈ *set es*. *dim-vec e* = *k length es* = *k* **and** *inv*: *invertible-mat* (*mat-of-cols k es*)
and *u*: *dim-vec u* = *k* ∗ *m*
shows ∃!*ys*. *length ys* = *k* ∧ (∀ *y* ∈ *set ys*. *dim-vec y* = *m*) ∧ *u* = *tensor-combination* (*k* ∗ *m*) *es ys*
⟨*proof*⟩

Corollary: $\mathbb{C}^k \otimes \mathbb{C}^m \cong \mathbb{C}^k \oplus \cdots \oplus \mathbb{C}^k$ (m times) by the linear bijection $(x_1, \ldots, x_m) \mapsto \sum_j x_j \otimes f_j$, and $\cong \mathbb{C}^m \oplus \cdots \oplus \mathbb{C}^m$ (k times) likewise; the identification depends on the chosen basis.

lemma *tensor-combination-add-left*:
assumes *l*: *length xs* = *length fs length xs′* = *length fs*
and *d*: ∀ *x* ∈ *set xs*. *dim-vec x* = *k* ∀ *x* ∈ *set xs′*. *dim-vec x* = *k* **and** *fd*: ∀ *y* ∈ *set fs*. *dim-vec y* = *m*
shows *tensor-combination* (*k* ∗ *m*) (*map2* (+) *xs xs′*) *fs* = *tensor-combination* (*k* ∗ *m*) *xs fs* + *tensor-combination* (*k* ∗ *m*) *xs′ fs*
⟨*proof*⟩

lemma *tensor-combination-smult-left*:
assumes *l*: *length xs* = *length fs* **and** *d*: ∀ *x* ∈ *set xs*. *dim-vec x* = *k* **and** *fd*: ∀ *y* ∈ *set fs*. *dim-vec y* = *m*
shows *tensor-combination* (*k* ∗ *m*) (*map* (λ*x*. *a* ·$_v$ *x*) *xs*) *fs* = *a* ·$_v$ *tensor-combination* (*k* ∗ *m*) *xs fs*
⟨*proof*⟩

corollary *tensor-iso-direct-sum*:
assumes *fd*: ∀ *f* ∈ *set fs*. *dim-vec f* = *m length fs* = *m* **and** *inv*: *invertible-mat* (*mat-of-cols m fs*)
shows *bij-betw* (λ*xs*. *tensor-combination* (*k* ∗ *m*) *xs fs*)
{*xs*. *length xs* = *m* ∧ (∀ *x* ∈ *set xs*. *dim-vec x* = *k*)} (*carrier-vec* (*k* ∗ *m*))
⟨*proof*⟩

2.3 Day 8: non-orthogonal states cannot be distinguished

lemma *meas-sum-norm*:
assumes *ms*: *measurement-system n Ms* **and** *v*: *dim-vec v* = *n*
shows (∑ *l*<*length Ms*. ‖*Ms* ! *l* ∗$_v$ *v*‖²) = ‖*v*‖²
⟨*proof*⟩

lemma *inner-minus-smult*:
assumes *d1*: *dim-vec u* = *n* **and** *d2*: *dim-vec w* = *n*
shows ⟨*w* − *a* ·$_v$ *u* | *w* − *a* ·$_v$ *u*⟩ = ⟨*w*|*w*⟩ − *a* ∗ ⟨*w*|*u*⟩ − *cnj a* ∗ ⟨*u*|*w*⟩ + *a* ∗ *cnj a* ∗ ⟨*u*|*u*⟩
⟨*proof*⟩

lemma *norm-sq-minus-component*:
assumes *d1*: *dim-vec u* = *n* **and** *d2*: *dim-vec w* = *n* **and** *u*: ‖*u*‖ = *1*
shows ‖*w* − ⟨*u*|*w*⟩ ·$_v$ *u*‖² = ‖*w*‖² − (*cmod* ⟨*u*|*w*⟩)²
⟨*proof*⟩

D8-2: if outcome i certainly occurs on ψ_1, it occurs on ψ_2 with probability at least $|\langle\psi_1|\psi_2\rangle|^2$.

theorem *nonorthogonal-outcome-prob*:

assumes *ms*: *measurement-system n Ms* **and** *q1*: *quantum-state n ψ1* **and** *q2*: *quantum-state n ψ2*
and *i*: $i < length\ Ms$ **and** *p1*: *meas-prob* (*Ms* ! *i*) *ψ1* = *1*
shows (*cmod* $\langle \psi 1|\psi 2\rangle)^2 \leq$ *meas-prob* (*Ms* ! *i*) *ψ2*
⟨*proof*⟩

corollary *nonorthogonal-not-distinguishable*:
assumes *ms*: *measurement-system n Ms* **and** *q1*: *quantum-state n ψ1* **and** *q2*: *quantum-state n ψ2*
and *no*: $\langle \psi 1|\psi 2\rangle \neq 0$ **and** *i*: $i < length\ Ms$ **and** *p1*: *meas-prob* (*Ms* ! *i*) *ψ1* = *1*
shows *meas-prob* (*Ms* ! *i*) *ψ2* > *0*
⟨*proof*⟩

2.4 Day 9: measuring one half of a Bell pair

definition *e0* :: *complex Matrix.vec* **where** *e0* = *unit-vec 2 0*
definition *e1* :: *complex Matrix.vec* **where** *e1* = *unit-vec 2 1*
definition *proj0* :: *complex Matrix.mat* **where** *proj0* = *e0* ⊘ *e0*
definition *proj1* :: *complex Matrix.mat* **where** *proj1* = *e1* ⊘ *e1*

lemma *e-dims* [*simp*]: *dim-vec e0* = *2 dim-vec e1* = *2*
⟨*proof*⟩

lemma *e-inner* [*simp*]: $\langle e0|e0\rangle$ = *1* $\langle e0|e1\rangle$ = *0* $\langle e1|e0\rangle$ = *0* $\langle e1|e1\rangle$ = *1*
⟨*proof*⟩

lemma *proj-carrier* [*simp*]: *proj0* ∈ *carrier-mat 2 2 proj1* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *proj-apply* [*simp*]:
proj0 $*_v$ *e0* = *e0 proj0* $*_v$ *e1* = 0_v *2 proj1* $*_v$ *e0* = 0_v *2 proj1* $*_v$ *e1* = *e1*
⟨*proof*⟩

lemma *bell-state-e*: *bell-state* = (*1* / *csqrt 2*) $\cdot_v$ (*tensor-vec e0 e0* + *tensor-vec e1 e1*)
⟨*proof*⟩

lemma *tensor-zero-left*: *dim-vec w* = *2* ⟹ *tensor-vec* (0_v *2*) *w* = 0_v *4*
⟨*proof*⟩

lemma *tensor-zero-right*: *dim-vec v* = *2* ⟹ *tensor-vec v* (0_v *2*) = 0_v *4*
⟨*proof*⟩

lemma *A-meas-bell*:
(*proj0* ⨂ 1_m *2*) $*_v$ *bell-state* = (*1* / *csqrt 2*) $\cdot_v$ *tensor-vec e0 e0*
(*proj1* ⨂ 1_m *2*) $*_v$ *bell-state* = (*1* / *csqrt 2*) $\cdot_v$ *tensor-vec e1 e1*
⟨*proof*⟩

lemma *tensor-e-norm*: ‖*tensor-vec e0 e0*‖ = *1* ‖*tensor-vec e1 e1*‖ = *1*
⟨*proof*⟩

D9-6: A measures $|0\rangle\langle 0|, |1\rangle\langle 1|$ on the shared state γ: each outcome has probability $\frac{1}{2}$, and γ collapses to $e_0 \otimes e_0$ resp. $e_1 \otimes e_1$, after which B's measurement $I \otimes M_m$ gives A's outcome with probability 1.

theorem *bell-measurement*:
meas-prob (*proj0* ⨂ 1_m *2*) *bell-state* = *1/2*
meas-prob (*proj1* ⨂ 1_m *2*) *bell-state* = *1/2*
post-meas-state (*proj0* ⨂ 1_m *2*) *bell-state* = *tensor-vec e0 e0*
post-meas-state (*proj1* ⨂ 1_m *2*) *bell-state* = *tensor-vec e1 e1*
meas-prob (1_m *2* ⨂ *proj0*) (*tensor-vec e0 e0*) = *1*
meas-prob (1_m *2* ⨂ *proj1*) (*tensor-vec e1 e1*) = *1*

⟨*proof*⟩

lemma *proj-explicit*:
proj0 = *Matrix.mat 2 2* (λ(*i*,*j*). *if i* = *0* ∧ *j* = *0 then 1 else 0*)
proj1 = *Matrix.mat 2 2* (λ(*i*,*j*). *if i* = *1* ∧ *j* = *1 then 1 else 0*)
⟨*proof*⟩

lemma *proj-measurement-system*: *measurement-system 2* [*proj0*, *proj1*]
⟨*proof*⟩

2.5 Day 11: SWAP from three CNOTs

lemma *less4-cases*: (*i*::*nat*) < *4* ⟷ *i* = *0* ∨ *i* = *1* ∨ *i* = *2* ∨ *i* = *3*
⟨*proof*⟩

CNOT′ is CNOT controlled by the second qubit.

definition *CNOT′* :: *complex Matrix.mat* **where**
CNOT′ = *Matrix.mat 4 4* (λ(*i*,*j*). *if* (*i*,*j*) ∈ {(*0*,*0*),(*1*,*3*),(*2*,*2*),(*3*,*1*)} *then 1 else 0*)

D11-3: SWAP = CNOT · CNOT′ · CNOT (*SWAP* is the AFP's).

theorem *SWAP-three-CNOTs*: *SWAP* = *CNOT* ∗ *CNOT′* ∗ *CNOT*
⟨*proof*⟩

lemma *SWAP-basis*:
assumes *a* < *2 b* < *2*
shows *SWAP* ∗$_v$ *tensor-vec* (*unit-vec 2 a*) (*unit-vec 2 b*) = *tensor-vec* (*unit-vec 2 b*) (*unit-vec 2 a*)
⟨*proof*⟩

lemma *CNOT′-basis*:
assumes *a* < *2 b* < *2*
shows *CNOT′* ∗$_v$ *tensor-vec* (*unit-vec 2 a*) (*unit-vec 2 b*) = *tensor-vec* (*unit-vec 2* ((*a* + *b*) *mod 2*))
(*unit-vec 2 b*)
⟨*proof*⟩

2.6 Day 5: a basis of $V \oplus W$ (D5-5)

A basis of $\mathbb{C}^n$ is a list of n vectors whose column matrix is invertible, as in *tensor-right-coordinates*. The notes: if $v_1, \ldots, v_n$ and $w_1, \ldots, w_k$ are bases of V and W then $(v_i, 0)$, $(0, w_j)$ is a basis of $V \oplus W$, so $\dim(V \oplus W) = \dim V + \dim W$. The column matrix is block diagonal.

definition *direct-sum-basis* :: *nat* ⇒ *nat* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* ⇒ *complex Matrix.vec list* **where**
direct-sum-basis n k vs ws = *map* (λ*v*. *v* ⊕$_v$ *0*$_v$ *k*) *vs* @ *map* (λ*w*. *0*$_v$ *n* ⊕$_v$ *w*) *ws*

lemma *direct-sum-basis-block*:
assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n length vs* = *n* **and** *ws*: ∀ *w* ∈ *set ws*. *dim-vec w* = *k length ws* = *k*
shows *mat-of-cols* (*n* + *k*) (*direct-sum-basis n k vs ws*)
= *four-block-mat* (*mat-of-cols n vs*) (*0*$_m$ *n k*) (*0*$_m$ *k n*) (*mat-of-cols k ws*)
⟨*proof*⟩

theorem *direct-sum-basis-is-basis*:
assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n length vs* = *n* **and** *iv*: *invertible-mat* (*mat-of-cols n vs*)
and *ws*: ∀ *w* ∈ *set ws*. *dim-vec w* = *k length ws* = *k* **and** *iw*: *invertible-mat* (*mat-of-cols k ws*)
shows ∀ *u* ∈ *set* (*direct-sum-basis n k vs ws*). *dim-vec u* = *n* + *k*
and *length* (*direct-sum-basis n k vs ws*) = *n* + *k*
and *invertible-mat* (*mat-of-cols* (*n* + *k*) (*direct-sum-basis n k vs ws*))
⟨*proof*⟩

end
theory *Day05-Cholesky-Examples*
imports *QC-Prelims*
begin

3 Day 5: Cholesky's algorithm on the notes' two examples (D5-3)

3.1 One step of Cholesky's algorithm

For $P = (p_{ij})$ with $p_{11} \neq 0$ the notes subtract $R = (\frac{p_{i1}p_{1j}}{p_{11}})_{ij}$; $P - R$ vanishes in the first row and column, and what remains is the $(n-1) \times (n-1)$ matrix *chol-next P*.

definition *chol-R* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
chol-R P = *Matrix.mat* (*dim-row P*) (*dim-row P*) (λ(*i*, *j*). *P* $$ (*i*, *0*) ∗ *P* $$ (*0*, *j*) / *P* $$ (*0*, *0*))

definition *chol-next* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
chol-next P = *Matrix.mat* (*dim-row P* − *1*) (*dim-row P* − *1*) (λ(*i*, *j*). (*P* − *chol-R P*) $$ (*i* + *1*, *j* + *1*))

lemma *chol-R-carrier* [*simp*]: *chol-R P* ∈ *carrier-mat* (*dim-row P*) (*dim-row P*)
⟨*proof*⟩

lemma *hermitian-entry*:
assumes *hermitian P P* ∈ *carrier-mat n n i* < *n j* < *n*
shows *P* $$ (*i*, *j*) = *cnj* (*P* $$ (*j*, *i*))
⟨*proof*⟩

The test: for $P = P^*$ with $p_{11} > 0$, $P \geq 0$ iff the next matrix is ≥ 0 (from *cholesky-reduction*).

lemma *chol-test*:
assumes *P*: *P* ∈ *carrier-mat n n* **and** *h*: *hermitian P* **and** *n*: *0* < *n* **and** *p*: *P* $$ (*0*, *0*) > *0*
shows *psd P* ⟷ *psd* (*chol-next P*)
⟨*proof*⟩

3.2 Small matrices entry by entry

lemma *mat3-eqI*:
assumes *A* ∈ *carrier-mat 3 3 B* ∈ *carrier-mat 3 3*
and ⋀*i j*. *i* ∈ {*0*, *1*, *2*} ⟹ *j* ∈ {*0*, *1*, *2*} ⟹ *A* $$ (*i*, *j*) = *B* $$ (*i*, *j*)
shows *A* = *B*
⟨*proof*⟩

lemma *mat2-eqI*:
assumes *A* ∈ *carrier-mat 2 2 B* ∈ *carrier-mat 2 2*
and ⋀*i j*. *i* ∈ {*0*, *1*} ⟹ *j* ∈ {*0*, *1*} ⟹ *A* $$ (*i*, *j*) = *B* $$ (*i*, *j*)
shows *A* = *B*
⟨*proof*⟩

3.3 Example 1: a matrix that is not positive

The notes: for $P = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 7 \end{pmatrix}$, $R = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}$ and $P - R = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & -1 & -2 \end{pmatrix}$, so $P \not\geq 0$. Each matrix is checked below; the last step is the test vector $(1, 1)$ for the remaining 2×2 block.

definition *notes-P1* :: *complex Matrix.mat* **where**

notes-P1 = mat-of-rows-list 3 [[1, 2, 3], [2, 4, 5], [3, 5, 7]]

lemma *notes-P1-carrier*: *notes-P1 ∈ carrier-mat 3 3*
⟨*proof*⟩

lemma *notes-P1-hermitian*: *hermitian notes-P1*
⟨*proof*⟩

lemma *notes-P1-R*: *chol-R notes-P1 = mat-of-rows-list 3 [[1, 2, 3], [2, 4, 6], [3, 6, 9]]*
⟨*proof*⟩

lemma *notes-P1-diff*: *notes-P1 − chol-R notes-P1 = mat-of-rows-list 3 [[0, 0, 0], [0, 0, −1], [0, −1, −2]]*
⟨*proof*⟩

lemma *notes-P1-next*: *chol-next notes-P1 = mat-of-rows-list 2 [[0, −1], [−1, −2]]*
⟨*proof*⟩

lemma *not-psd-witness*:
assumes *A*: *A ∈ carrier-mat n n* **and** *x*: *dim-vec x = n* **and** *neg*: *Re (Quantum.inner-prod x (A ∗v x)) < 0*
shows *¬ psd A*
⟨*proof*⟩

$\begin{pmatrix} 0 & -1 \\ -1 & -2 \end{pmatrix}$ has the value -4 at $(1,1)$, so it is not positive.

lemma *notes-P1-next-not-psd*: *¬ psd (mat-of-rows-list 2 [[0, −1], [−1, −2]])*
⟨*proof*⟩

theorem *notes-example-1*: *hermitian notes-P1 ∧ ¬ psd notes-P1*
⟨*proof*⟩

3.4 Example 2: a positive matrix, in two steps

The notes: for $P = \begin{pmatrix} 4 & 2 & 3 \\ 2 & 5 & 6 \\ 3 & 6 & 8 \end{pmatrix}$, $R = \frac{1}{4}\begin{pmatrix} 16 & 8 & 12 \\ 8 & 4 & 6 \\ 12 & 6 & 9 \end{pmatrix}$ and $P - R = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 4 & \frac{9}{2} \\ 0 & \frac{9}{2} & \frac{23}{4} \end{pmatrix}$. Repeating on $Q = \begin{pmatrix} 4 & \frac{9}{2} \\ \frac{9}{2} & \frac{23}{4} \end{pmatrix}$: $Q - \frac{1}{4}\begin{pmatrix} 4 \\ \frac{9}{2} \end{pmatrix}\begin{pmatrix} 4 & \frac{9}{2} \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & \frac{11}{16} \end{pmatrix}$, so $P \geq 0$.

definition *notes-P2* :: *complex Matrix.mat* **where**
notes-P2 = mat-of-rows-list 3 [[4, 2, 3], [2, 5, 6], [3, 6, 8]]

lemma *notes-P2-carrier*: *notes-P2 ∈ carrier-mat 3 3*
⟨*proof*⟩

lemma *notes-P2-hermitian*: *hermitian notes-P2*
⟨*proof*⟩

lemma *notes-P2-R*: *chol-R notes-P2 = (1/4) ·m mat-of-rows-list 3 [[16, 8, 12], [8, 4, 6], [12, 6, 9]]*
⟨*proof*⟩

lemma *notes-P2-diff*: *notes-P2 − chol-R notes-P2 = mat-of-rows-list 3 [[0, 0, 0], [0, 4, 9/2], [0, 9/2, 23/4]]*
⟨*proof*⟩

definition *notes-Q2* :: *complex Matrix.mat* **where**

notes-Q2 = mat-of-rows-list 2 [[4, 9/2], [9/2, 23/4]]

lemma *notes-P2-next*: *chol-next notes-P2 = notes-Q2*
⟨*proof*⟩

lemma *notes-Q2-carrier*: *notes-Q2 ∈ carrier-mat 2 2*
⟨*proof*⟩

lemma *notes-Q2-hermitian*: *hermitian notes-Q2*
⟨*proof*⟩

lemma *notes-Q2-R*: *chol-R notes-Q2 = (1/4) ·$_m$ mat-of-rows-list 2 [[16, 18], [18, 81/4]]*
⟨*proof*⟩

lemma *notes-Q2-diff*: *notes-Q2 − chol-R notes-Q2 = mat-of-rows-list 2 [[0, 0], [0, 11/16]]*
⟨*proof*⟩

The last matrix is $(\frac{11}{16}) = \frac{11}{16}\,|e\rangle\langle e|$, which is positive.

lemma *notes-Q2-next*: *chol-next notes-Q2 = complex-of-real (11/16) ·$_m$ (Matrix.vec 1 (λ-. 1) ⊘ Matrix.vec 1 (λ-. 1))*
⟨*proof*⟩

lemma *notes-Q2-next-psd*: *psd (chol-next notes-Q2)*
⟨*proof*⟩

theorem *notes-example-2*: *hermitian notes-P2 ∧ psd notes-P2*
⟨*proof*⟩

The notes' remark: the algorithm writes P as a sum of rank-one matrices, $P = \frac{1}{4}|v_1\rangle\langle v_1| + \frac{1}{4}|v_2\rangle\langle v_2| + \frac{11}{16}|e_3\rangle\langle e_3|$ with $v_1 = (4, 2, 3)$, $v_2 = (0, 4, \frac{9}{2})$, $e_3 = (0, 0, 1)$ (the first columns of P, of $P - R$ and of the last step).

lemma *outer-prod-of-list*:
vec-of-list l ⊘ vec-of-list l = Matrix.mat (length l) (length l) (λ(i, j). l ! i ∗ cnj (l ! j))
⟨*proof*⟩

lemma *notes-P2-rank-one-sum*:
notes-P2 = complex-of-real (1/4) ·$_m$ (vec-of-list [4, 2, 3] ⊘ vec-of-list [4, 2, 3])
+ complex-of-real (1/4) ·$_m$ (vec-of-list [0, 4, 9/2] ⊘ vec-of-list [0, 4, 9/2])
+ complex-of-real (11/16) ·$_m$ (vec-of-list [0, 0, 1] ⊘ vec-of-list [0, 0, 1])
⟨*proof*⟩

end
theory *Day06-Schmidt-Rank*
imports *Day02-11-Supplement*
begin

4 Day 6: the Schmidt rank (D6-4)

The notes define the Schmidt rank $\mathrm{rank}_S(u)$ of $u \in X \otimes Y$ as the least number of elementary tensors in an expression for u, and use on Day 10 that a two-photon state ψ is a product $\psi_1 \otimes \psi_2$ iff $\mathrm{rank}_S(\psi) = 1$; otherwise it is entangled. Here: every $u \in \mathbb{C}^n \otimes \mathbb{C}^m$ has an expression of length n, so the least length exists; $\mathrm{rank}_S(u) = 0$ iff $u = 0$; for $u \neq 0$, $\mathrm{rank}_S(u) = 1$ iff u is a product and u is entangled iff $\mathrm{rank}_S(u) > 1$; the Bell state has Schmidt rank 2.

A representation of $u \in \mathbb{C}^n \otimes \mathbb{C}^m$ by k elementary tensors.

definition *schmidt-repr* :: *nat ⇒ nat ⇒ complex Matrix.vec ⇒ nat ⇒ bool* **where**

schmidt-repr n m u k $\longleftrightarrow$ ($\exists$ *vs ws. length vs* $=$ *k* $\wedge$ *length ws* $=$ *k* $\wedge$
($\forall$ *i* $<$ *k. dim-vec* (*vs* ! *i*) $=$ *n*) $\wedge$ ($\forall$ *i* $<$ *k. dim-vec* (*ws* ! *i*) $=$ *m*) $\wedge$
u $=$ *tensor-combination* (*n* $*$ *m*) *vs ws*)

lemma *schmidt-rank-eq*:
assumes *u*: *dim-vec u* $=$ *n* $*$ *m* **and** *m*: *0* $<$ *m*
shows *schmidt-rank u m* $=$ (*LEAST k. schmidt-repr n m u k*)
⟨*proof*⟩

Every u is a sum of n elementary tensors, $u = \sum_i e_i \otimes u_i$ with $u_i = (u_{im+j})_j$, so the least length exists.

lemma *schmidt-repr-exists*:
assumes *u*: *dim-vec u* $=$ *n* $*$ *m*
shows *schmidt-repr n m u n*
⟨*proof*⟩

lemma *schmidt-rank-repr*:
assumes *u*: *dim-vec u* $=$ *n* $*$ *m* **and** *m*: *0* $<$ *m*
shows *schmidt-repr n m u* (*schmidt-rank u m*) **and** $\bigwedge$*k. schmidt-repr n m u k* $\Longrightarrow$ *schmidt-rank u m* $\leq$ *k*
⟨*proof*⟩

lemma *schmidt-repr-0*: *schmidt-repr n m u 0* $\longleftrightarrow$ *u* $=$ 0_v (*n* $*$ *m*)
⟨*proof*⟩

lemma *schmidt-repr-1*:
schmidt-repr n m u 1 $\longleftrightarrow$ ($\exists$ *v w. dim-vec v* $=$ *n* $\wedge$ *dim-vec w* $=$ *m* $\wedge$ *u* $=$ *tensor-vec v w*)
⟨*proof*⟩

theorem *schmidt-rank-zero-iff*:
assumes *u*: *dim-vec u* $=$ *n* $*$ *m* **and** *m*: *0* $<$ *m*
shows *schmidt-rank u m* $=$ *0* $\longleftrightarrow$ *u* $=$ 0_v (*n* $*$ *m*)
⟨*proof*⟩

The notes: $\psi = \psi_1 \otimes \psi_2$ iff $\text{rank}_S(\psi) = 1$ (for $\psi \neq 0$).

theorem *schmidt-rank-one-iff-product*:
assumes *u*: *dim-vec u* $=$ *n* $*$ *m* **and** *m*: *0* $<$ *m* **and** *u0*: *u* $\neq$ 0_v (*n* $*$ *m*)
shows *schmidt-rank u m* $=$ *1* $\longleftrightarrow$ ($\exists$ *v w. dim-vec v* $=$ *n* $\wedge$ *dim-vec w* $=$ *m* $\wedge$ *u* $=$ *tensor-vec v w*)
⟨*proof*⟩

corollary *entangled-iff-schmidt-rank*:
assumes *u*: *dim-vec u* $=$ *n* $*$ *m* **and** *m*: *0* $<$ *m* **and** *u0*: *u* $\neq$ 0_v (*n* $*$ *m*)
shows *entangled n m u* $\longleftrightarrow$ *1* $<$ *schmidt-rank u m*
⟨*proof*⟩

corollary *schmidt-rank-bell-state*: *schmidt-rank bell-state 2* $=$ *2*
⟨*proof*⟩

end
theory *Day10-15-Supplement*
imports *QC-Prelims*
begin

5 Days 10, 11 and 15: superdense coding, the CCNOT table, the partial trace over A

5.1 Superdense coding (D10-1)

Lab A and Lab B share $\gamma = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, Lab A holding the first qubit. Lab A multiplies its qubit by I, $\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$, $\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$ or $\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix}$ and sends it to Lab B, which then holds $\frac{1}{\sqrt{2}}(|00\rangle \pm |11\rangle)$ or $\frac{1}{\sqrt{2}}(|01\rangle \pm |10\rangle)$: four orthonormal states, so one qubit carries two bits. The four matrices are I, Z, X and ZX (*sd-op-paulis*).

lemma *less4*: $i < (4::nat) \longleftrightarrow i = 0 \vee i = 1 \vee i = 2 \vee i = 3$ ⟨*proof*⟩

lemma *tensor-I2-apply*:
fixes A :: *complex Matrix.mat* **and** v :: *complex Matrix.vec*
assumes *A*: $A \in$ *carrier-mat 2 2* **and** *v*: *dim-vec v = 4* **and** *i*: $i < 4$
shows (*Tensor.tensor-mat A* (1_m 2) $*_v$ *v*) $ *i* =
($\sum j<4$. *A* $$ (*i div 2*, *j div 2*) * (*if i mod 2 = j mod 2 then 1 else 0*) * *v* $ *j*)
⟨*proof*⟩

definition *superdense-gamma* :: *complex Matrix.vec* **where**
superdense-gamma = bell-basis ! *0*

Lab A applies one of the notes' four matrices to its qubit.

definition *sd-op* :: *nat* ⇒ *complex Matrix.mat* **where**
sd-op k = (*if k = 0 then mat-of-rows-list 2* [[*1*, *0*], [*0*, *1*]]
else if k = 1 then mat-of-rows-list 2 [[*1*, *0*], [*0*, *−1*]]
else if k = 2 then mat-of-rows-list 2 [[*0*, *1*], [*1*, *0*]]
else mat-of-rows-list 2 [[*0*, *1*], [*−1*, *0*]])

lemma *sd-op-carrier*: *sd-op k* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *sd-op-paulis*:
sd-op 0 = 1$_m$ *2 sd-op 1 = Pauli-Z sd-op 2 = Pauli-X sd-op 3 = Pauli-Z * Pauli-X*
⟨*proof*⟩

The notes' table: the four results are the four Bell states.

lemma *superdense-coding-0*: *Tensor.tensor-mat* (*sd-op 0*) (1_m 2) $*_v$ *superdense-gamma = bell-basis* ! *0*
⟨*proof*⟩

lemma *superdense-coding-1*: *Tensor.tensor-mat* (*sd-op 1*) (1_m 2) $*_v$ *superdense-gamma = bell-basis* ! *1*
⟨*proof*⟩

lemma *superdense-coding-2*: *Tensor.tensor-mat* (*sd-op 2*) (1_m 2) $*_v$ *superdense-gamma = bell-basis* ! *2*
⟨*proof*⟩

lemma *superdense-coding-3*: *Tensor.tensor-mat* (*sd-op 3*) (1_m 2) $*_v$ *superdense-gamma = bell-basis* ! *3*
⟨*proof*⟩

theorem *superdense-coding*:
assumes *k*: $k < 4$
shows *Tensor.tensor-mat* (*sd-op k*) (1_m 2) $*_v$ *superdense-gamma = bell-basis* ! *k*

⟨*proof*⟩

corollary *superdense-coding-orthonormal*:
orthonormal-list 4 (*map* (λ*k*. *Tensor.tensor-mat* (*sd-op k*) (*1*$_m$ *2*) ∗$_v$ *superdense-gamma*) [*0*..<*4*])
⟨*proof*⟩

5.2 The CCNOT truth table (D11-2)

lemma *CCNOT-unit-vec*:
assumes *k*: *k* < *8*
shows *CCNOT* ∗$_v$ *unit-vec 8 k* = *unit-vec 8* (*if k* = *6 then 7 else if k* = *7 then 6 else k*)
⟨*proof*⟩

On basis states $|a, b, c\rangle = e_a \otimes e_b \otimes e_c$: when $a = 0$ nothing happens; when $a = 1$, CNOT acts on b, c.

theorem *CCNOT-basis*:
assumes *a*: *a* < *2* **and** *b*: *b* < *2* **and** *c*: *c* < *2*
shows *CCNOT* ∗$_v$ *tensor-vec* (*unit-vec 2 a*) (*tensor-vec* (*unit-vec 2 b*) (*unit-vec 2 c*)) =
tensor-vec (*unit-vec 2 a*) (*tensor-vec* (*unit-vec 2 b*) (*unit-vec 2* (*if a* = *1* ∧ *b* = *1 then 1* − *c else c*)))
⟨*proof*⟩

corollary *CCNOT-truth-table*:
fixes *e* :: *nat* ⇒ *complex Matrix.vec*
defines *e* ≡ λ*a*. *unit-vec 2 a*
shows *CCNOT* ∗$_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 0*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 0*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 1*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 0*) (*e 1*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 0*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 0*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 1*)) = *tensor-vec* (*e 0*) (*tensor-vec* (*e 1*) (*e 1*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 0*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 0*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 1*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 0*) (*e 1*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 0*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 1*))
and *CCNOT* ∗$_v$ *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 1*)) = *tensor-vec* (*e 1*) (*tensor-vec* (*e 1*) (*e 0*))
⟨*proof*⟩

5.3 The partial trace over A (D15-6)

Reordering the basis: for a bijection σ of $\{0, \ldots, N-1\}$ the matrix $(x_{\sigma(r)\sigma(s)})$ has the trace of X and is Hermitian, resp. positive, with X.

definition *perm-conj* :: *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
perm-conj N σ *X* = *Matrix.mat N N* (λ(*r*, *s*). *X* $$ (σ *r*, σ *s*))

lemma *perm-conj-carrier* [*simp*]: *perm-conj N* σ *X* ∈ *carrier-mat N N*
⟨*proof*⟩

lemma *perm-conj-dims* [*simp*]: *dim-row* (*perm-conj N* σ *X*) = *N dim-col* (*perm-conj N* σ *X*) = *N*
⟨*proof*⟩

lemma *perm-conj-trace*:
assumes σ: *bij-betw* σ {..<*N*} {..<*N*} **and** *X*: *X* ∈ *carrier-mat N N*
shows *Tr* (*perm-conj N* σ *X*) = *Tr X*
⟨*proof*⟩

lemma *perm-conj-hermitian*:
assumes σ: ⋀*r*. *r* < *N* ⟹ σ *r* < *N* **and** *X*: *X* ∈ *carrier-mat N N* **and** *h*: *hermitian X*
shows *hermitian* (*perm-conj N* σ *X*)

⟨*proof*⟩

lemma *perm-conj-psd*:
assumes σ: *bij-betw* σ {..<*N*} {..<*N*} **and** *X*: $X \in$ *carrier-mat N N* **and** *ps*: *psd X*
shows *psd* (*perm-conj N* σ *X*)
⟨*proof*⟩

Exchanging the factors: with $\sigma(im+k) = kp+i$ $(i < p,\ k < m)$, tracing out A is tracing out B after the exchange.

definition *swap-index* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* **where**
swap-index m p r = (*r mod m*) ∗ *p* + *r div m*

lemma *swap-index-less*:
assumes *r* < *m* ∗ *p* **shows** *swap-index m p r* < *m* ∗ *p*
⟨*proof*⟩

lemma *swap-index-bij*: *bij-betw* (*swap-index m p*) {..<*m* ∗ *p*} {..<*m* ∗ *p*}
⟨*proof*⟩

lemma *partial-trace-A-as-B*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)
shows *partial-trace-A m p X* = *partial-trace-B p m* (*perm-conj* (*m* ∗ *p*) (*swap-index m p*) *X*)
⟨*proof*⟩

theorem *partial-trace-A-trace*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*) **and** *m*: *0* < *m*
shows *Tr* (*partial-trace-A m p X*) = *Tr X*
⟨*proof*⟩

theorem *partial-trace-A-hermitian*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*) **and** *h*: *hermitian X*
shows *hermitian* (*partial-trace-A m p X*)
⟨*proof*⟩

theorem *partial-trace-A-is-density*:
assumes *dm*: *density-matrix* (*m* ∗ *p*) *X* **and** *m*: *0* < *m*
shows *density-matrix p* (*partial-trace-A m p X*)
⟨*proof*⟩

end
theory *Day04-29-Supplement*
imports *Day27-Entangled-Examples Day29-Classical-Codes*
begin

6 Days 4–29: the converse Hermitian tests, composite ensembles, Choi's four conditions, codes

Completions of variant rows of the audit: on Day 4 the directions (b)⇒(a) and (c)⇒(a) of the characterisation of Hermitian matrices (D4-1); on Day 14 the density of a composite ensemble (D14-5); on Day 19 Choi's theorem with the n-positivity condition (2) stated in its own right (D19-1); on Day 29 the triangle inequality, the distance of a subspace code and unique decoding (D29-4).

6.1 Day 4: the converse directions for Hermitian matrices (D4-1)

lemma *unitary-real-diag-imp-hermitian*:

assumes *U*: $U \in$ *carrier-mat n n* **and** *u*: *unitary U* **and** *H*: $H \in$ *carrier-mat n n*
and *D*: *length D* $= n$ **and** *spec*: $U^\dagger * H * U =$ *diag-mat-of-real D*
shows *hermitian H*
⟨*proof*⟩

lemma *eigenbasis-real-imp-hermitian*:
assumes *H*: $H \in$ *carrier-mat n n* **and** *orth*: *orthonormal-list n vs* **and** *len*: *length vs* $= n$
and *eig*: $\bigwedge i.\ i < n \Longrightarrow H *_v vs\ !\ i =$ *complex-of-real* $(lam\ !\ i) \cdot_v vs\ !\ i$ **and** *lam*: *length lam* $= n$
shows *hermitian H*
⟨*proof*⟩

6.2 Choi's theorem with (2) isolated (D19-1)

lemma *choi-seed-psd*: *psd* (*choi-seed n*)
⟨*proof*⟩

lemma *choi-psd-if-n-positive*:
assumes *n*: $0 < n$ **and** *d*: $0 < d$ **and** *pp*: *p-positive n d n* Φ
shows *psd* (*choi-matrix n d* Φ)
⟨*proof*⟩

theorem *choi-four-conditions*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *n*: $0 < n$ **and** *d*: $0 < d$
shows (*completely-positive n d* Φ $\longleftrightarrow$ *p-positive n d n* Φ)
$\wedge$ (*p-positive n d n* Φ $\longleftrightarrow$ *psd* (*choi-matrix n d* Φ))
$\wedge$ (*psd* (*choi-matrix n d* Φ) $\longleftrightarrow$
$(\exists Ws.\ (\forall W \in set\ Ws.\ W \in$ *carrier-mat d n*$) \wedge (\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A =$ *kraus-map-dim d Ws A*$)))$
⟨*proof*⟩

6.3 Composite ensembles (D14-5)

$|\psi \otimes \varphi\rangle\langle\psi \otimes \varphi| = |\psi\rangle\langle\psi| \otimes |\varphi\rangle\langle\varphi|$.

lemma *outer-sum-tensor*:
assumes ψ: *dim-vec* $\psi = n$ **and** φ: *dim-vec* $\varphi = m$
shows *outer-sum* $(n * m)$ [*tensor-vec* ψ φ] = *outer-sum n* [ψ] $\bigotimes$ *outer-sum m* [φ]
⟨*proof*⟩

For ensembles $\{p_i, \psi_i\}$ on $\mathcal{H}_A$ and $\{q_j, \varphi_j\}$ on $\mathcal{H}_B$, the composite ensemble is $\{p_i q_j, \psi_i \otimes \varphi_j\}$.

definition *product-ensemble* :: (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* **where**
product-ensemble EA EB = *map* (λ((*p*, ψ), (*q*, φ)). (*p* ∗ *q*, *tensor-vec* ψ φ)) (*List.product EA EB*)

lemma *sum-list-product*:
fixes *f* :: *'a* ⇒ *real* **and** *g* :: *'b* ⇒ *real*
shows $(\sum (x,\ y) \leftarrow$ *List.product xs ys. f x* ∗ *g y*$) =$ $(\sum x \leftarrow xs.\ f\ x) * (\sum y \leftarrow ys.\ g\ y)$
⟨*proof*⟩

lemma *product-ensemble-ensemble*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *ensemble* $(n * m)$ (*product-ensemble EA EB*)
⟨*proof*⟩

The notes' composite density $\sum_{i,j} p_i q_j \, |\psi_i\rangle\langle\psi_i| \otimes |\varphi_j\rangle\langle\varphi_j|$ is the density of the composite ensemble, hence a density matrix.

definition *composite-density* :: *nat* ⇒ *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *complex Matrix.mat* **where**
composite-density n m EA EB = *foldr* (λ((*p*, *ψ*), (*q*, *φ*)) *acc*.
complex-of-real (*p* ∗ *q*) ·$_m$ (*outer-sum n* [*ψ*] ⨂ *outer-sum m* [*φ*]) + *acc*) (*List.product EA EB*) (*0*$_m$ (*n* ∗ *m*) (*n* ∗ *m*))

lemma *composite-density-eq*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *composite-density n m EA EB* = *ensemble-density* (*n* ∗ *m*) (*product-ensemble EA EB*)
⟨*proof*⟩

theorem *composite-density-is-dm*:
assumes *A*: *ensemble n EA* **and** *B*: *ensemble m EB*
shows *density-matrix* (*n* ∗ *m*) (*composite-density n m EA EB*)
⟨*proof*⟩

6.4 Hamming distance and subspace codes (D29-4)

lemma *hamming-dist-xor*: *hamming-dist v w* = *hamming-weight* (*xor-word v w*)
⟨*proof*⟩

The triangle inequality for words of bits of equal length.

lemma *hamming-dist-triangle*:
assumes *length u* = *length v length v* = *length w*
and ∀ *x* ∈ *set u*. *x* < *2* ∀ *x* ∈ *set v*. *x* < *2* ∀ *x* ∈ *set w*. *x* < *2*
shows *hamming-dist u w* ≤ *hamming-dist u v* + *hamming-dist v w*
⟨*proof*⟩

For a subspace code (closed under addition mod 2) the distance is the least weight of a nonzero code word.

definition *zero-word* :: *nat* ⇒ *nat list* **where**
zero-word m = *replicate m 0*

lemma *xor-word-self*: *xor-word v v* = *zero-word* (*length v*)
⟨*proof*⟩

lemma *xor-word-zero*: ∀ *x* ∈ *set v*. *x* < *2* ⟹ *xor-word v* (*zero-word* (*length v*)) = *v*
⟨*proof*⟩

lemma *xor-word-eq-zero-iff*:
assumes *length v* = *length w* ∀ *x* ∈ *set v*. *x* < *2* ∀ *x* ∈ *set w*. *x* < *2*
shows *xor-word v w* = *zero-word* (*length v*) ⟷ *v* = *w*
⟨*proof*⟩

lemma *hamming-weight-zero-word* [*simp*]: *hamming-weight* (*zero-word m*) = *0*
⟨*proof*⟩

lemma *hamming-weight-eq-zero-iff*: ∀ *x* ∈ *set v*. *x* < *2* ⟹ *hamming-weight v* = *0* ⟷ *v* = *zero-word* (*length v*)
⟨*proof*⟩

definition *subspace-code* :: *nat* ⇒ *nat list set* ⇒ *bool* **where**
subspace-code m S ⟷ (∀ *v* ∈ *S*. *length v* = *m* ∧ (∀ *x* ∈ *set v*. *x* < *2*))
∧ (∀ *v* ∈ *S*. ∀ *w* ∈ *S*. *xor-word v w* ∈ *S*)

lemma *xor-word-bits*: $\forall\, x \in$ *set* (*xor-word v w*). $x < 2$
⟨*proof*⟩

theorem *subspace-code-distance*:
assumes *S*: *subspace-code m S* **and** *fin*: *finite S* **and** *two*: $\exists\, v \in S.\ \exists\, w \in S.\ v \neq w$
shows *code-distance S* = *Min* {*hamming-weight v* | *v*. $v \in S \wedge v \neq$ *zero-word m*}
⟨*proof*⟩

A code of distance at least $2t + 1$ corrects t errors: a received word is within distance t of at most one code word.

theorem *corrects-errors-unique-decoding*:
assumes *ce*: *corrects-errors S t* **and** *fin*: *finite S*
and *S*: $\forall\, v \in S.$ *length* $v = m \wedge (\forall\, x \in$ *set v*. $x < 2)$
and *r*: *length* $r = m\ \forall\, x \in$ *set r*. $x < 2$
and *c*: $c \in S$ *hamming-dist r c* $\leq t$ **and** *c'*: $c' \in S$ *hamming-dist r c'* $\leq t$
shows $c = c'$
⟨*proof*⟩

end

Part II

Lectures 11–22 and 29–34: quantum computing, density matrices, completely positive maps, error correction

theory *Quantum-Computing11*
imports
Quantum-Computing-by-VP-3-10.Basic-Math
Quantum-Computing-by-VP-3-10.Quantum-Computing-3-10
begin

7 Day 11 (21 September 2011): gates revisited, circuit diagrams, no-cloning

The lecture opens with two corrections: $H^2 = I$ (not X), with $He_0 = (e_0 + e_1)/\sqrt{2}$, $Xe_0 = e_1$, $Xe_1 = e_0$, $He_1 = (e_0 - e_1)/\sqrt{2}$, so that X and H are reflections; and the truth tables of CNOT ($|a, b\rangle \mapsto |a, b + a\rangle$, arithmetic in $\mathbb{Z}_2$) and of CCNOT ($|a, b, c\rangle \mapsto |a, b, c + ab\rangle$). Circuit diagrams are introduced; the CNOT with control on the second wire has matrix $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$, and three alternating CNOTs compose to the swap of the two wires (*SWAP-three-CNOTs* in theory Day02_11_Supplement).

Definition (notes). The Toffoli gate CCNOT is the 8×8 permutation matrix that fixes the first six basis vectors and exchanges $|110\rangle$ and $|111\rangle$.

definition *CCNOT* :: *complex Matrix.mat* **where**

CCNOT ≡ *Matrix.mat 8 8*
(λ(*i*,*j*). *if* (*i* = *6* ∧ *j* = *7*) ∨ (*i* = *7* ∧ *j* = *6*)
then 1
else if i = *j* ∧ *i* < *6 then 1 else 0*)

lemma *CCNOT-dagger* [*simp*]:
CCNOT† = *CCNOT*
⟨*proof*⟩

lemma *CCNOT-dim-row* [*simp*]:
dim-row CCNOT = *8*
⟨*proof*⟩

lemma *CCNOT-dim-col* [*simp*]:
dim-col CCNOT = *8*
⟨*proof*⟩

lemma *CCNOT-inv* [*simp*]:
CCNOT ∗ *CCNOT* = 1_m *8*
⟨*proof*⟩

lemma *CCNOT-unitary*: *unitary CCNOT*
⟨*proof*⟩

7.1 Cloning and no-cloning

CNOT clones the computational basis: $U(e_a \otimes e_0) = e_a \otimes e_a$, and more generally $|i_1 \cdots i_n\rangle \otimes |0 \cdots 0\rangle \mapsto |i_1 \cdots i_n\rangle \otimes |i_1 \cdots i_n\rangle$ for some unitary. But for $\psi = \alpha e_0 + \beta e_1$ with $\alpha\beta \neq 0, U(\psi \otimes e_0) \neq \psi \otimes \psi$.

Theorem (notes) (No-cloning)**.** There is no state ϕ and unitary U on $\mathcal{H} \otimes \mathcal{H}$ with $U(\psi \otimes \phi) = \psi \otimes \psi$ for all $\psi \in \mathcal{H}$.

The notes argue with $-\psi$; the formal proof (for $\dim \mathcal{H} = n \geq 2$ and unit vectors ψ) uses two states with inner product neither 0 nor 1.

definition *is-cloner* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *bool* **where**
is-cloner n U φ ≡
U ∈ *carrier-mat* (*n*^*2*) (*n*^*2*) ∧ *unitary U* ∧ *dim-vec* φ = *n* ∧
(∀ ψ. *quantum-state n* ψ ⟶
U ∗$_v$ *tensor-vec* ψ φ = *tensor-vec* ψ ψ)

Two states whose inner product is neither 0 nor 1 exist once $n \geq 2$.

lemma *two-states-nontrivial-inner-prod*:
assumes *n*: *2* ≤ *n*
obtains ψ_1 ψ_2 :: *complex Matrix.vec*
where *quantum-state n* ψ_1 **and** *quantum-state n* ψ_2
and ⟨ψ_1 | ψ_2⟩ ≠ (*0* :: *real*) **and** ⟨ψ_1 | ψ_2⟩ ≠ (*1* :: *real*)
⟨*proof*⟩

theorem *no-cloning*:
assumes *n2*: *2* ≤ *n*
shows ¬ (∃ *U* φ. *is-cloner n U* φ)
⟨*proof*⟩

8 Day 12 (23 September 2011): quantum parallelism

Since $He_0 = (e_0 + e_1)/\sqrt{2}$,

$$H^{\otimes N}(e_0 \otimes \cdots \otimes e_0) = \left(\tfrac{1}{\sqrt{2}}\right)^N \sum_{J \in \mathbb{Z}_2^N} e_J = \left(\tfrac{1}{\sqrt{2}}\right)^N \sum_J |J\rangle.$$

Only the case $N = 1$ is stated here; the general identity is *hadamard-uniform-N* in *Day12-Counting-Probability* (with *tensor-pow-mat* for $H^{\otimes N}$).

Definition (notes) (Oracle)**.** For $f : \mathbb{Z}_2^N \to \mathbb{Z}_2$, the vectors $e_J \otimes e_{i+f(J)}$ form an orthonormal basis of $(\mathbb{C}^2)^{\otimes N} \otimes \mathbb{C}^2$, so there is a unitary U_f with $U_f(e_J \otimes e_i) = e_J \otimes e_{i+f(J)}$, the *oracle* for f.

Proposition (notes). For a unit vector γ, the measurement $P_0 = |\gamma\rangle\langle\gamma|$, $P_1 = I - P_0$ has $p_0(\psi) = |\langle\gamma|\psi\rangle|^2$ and $p_1(\psi) = 1 - |\langle\gamma|\psi\rangle|^2$.

Application: with $x = (He_0)^{\otimes N}$, $U_f(x \otimes e_0) = 2^{-N/2} \sum_J e_J \otimes e_{f(J)}$, and measuring against $\gamma = x \otimes e_0$ gives $\langle\gamma|U_f(x \otimes e_0)\rangle = M/2^N$ with $M = \#\{J : f(J) = 0\}$, hence $p_0 = (M/2^N)^2$: repeating the experiment estimates M (a Bernoulli trial). The existence of the oracle is formalised below; the proposition is *rank-one-meas-prob* (Day 8); the counting identity is *counting-probability* in *Day12-Counting-Probability*.

definition *oracle-unitary* :: (*nat* ⇒ *nat*) ⇒ *nat* ⇒ *bool* **where**
  oracle-unitary f N ≡
    ∃ *U* :: *complex Matrix.mat*.
      U ∈ *carrier-mat* (*2^(N+1)*) (*2^(N+1)*) ∧
      unitary U ∧
      (∀ *J* < *2^N*. ∀ *i* < *2*.
        U *$_v$ (*tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 i*)) =
        tensor-vec (*unit-vec* (*2^N*) *J*) (*unit-vec 2* ((*i* + *f J*) *mod 2*)))

theorem *oracle-exists*:
  fixes *f* :: *nat* ⇒ *nat*
  assumes *f-range*: ∀ *J* < *2^*(*N* :: *nat*). *f J* < *2*
  shows *oracle-unitary f N*
⟨*proof*⟩

$M = \#\{J < 2^N : f(J) = 0\}$, the quantity estimated by the counting experiment.

definition *count-zeros* :: (*nat* ⇒ *nat*) ⇒ *nat* ⇒ *nat* **where**
  count-zeros f N ≡ *card* {*J*. *J* < *2^N* ∧ *f J* = *0*}

9 Day 13 (26 September 2011): ensembles and density matrices

9.1 Ensembles

Motivation: measuring $\psi = a|0\rangle + b|1\rangle$ with $M_0 = |0\rangle\langle 0|$, $M_1 = |1\rangle\langle 1|$ leaves the state $\frac{a}{|a|}|0\rangle$ with probability $|a|^2$ and $\frac{b}{|b|}|1\rangle$ with probability $|b|^2$; a later measurement $\{\tilde{M}_\alpha\}$ then has outcome probabilities $p_\alpha = |a|^2\|\tilde{M}_\alpha \frac{a}{|a|}|0\rangle\|^2 + |b|^2\|\tilde{M}_\alpha \frac{b}{|b|}|1\rangle\|^2$.

Definition (notes). An *ensemble* $\{p_i, \psi_i\}_{i=1}^L$ is a set of states with probabilities $p_i \geq 0$, $\sum_i p_i = 1$. For a measurement system $\{M_\alpha\}$ the probability of outcome α is $p_\alpha(\{p_i, \psi_i\}) = \sum_i p_i \|M_\alpha \psi_i\|^2$.

A second source of ensembles: a unit vector $\psi = \sum_l \phi_l \otimes f_l \in \mathcal{H}_A \otimes \mathcal{H}_E$ (environment $\mathcal{H}_E$ with orthonormal basis f_l) behaves, for measurements $M_\alpha \otimes I$, exactly like the ensemble $\{\|\phi_l\|^2, \phi_l/\|\phi_l\|\}$.

definition *ensemble* :: *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *bool* **where**
ensemble n E ≡
(∀ (*p*, *ψ*) ∈ *set E*. *p* ≥ *0* ∧ *quantum-state n ψ*) ∧
(∑ (*p*, -) ← *E*. *p*) = *1*

9.2 Von Neumann's density matrix

Measurements cannot distinguish ψ from $e^{i\theta}\psi$, and $|e^{i\theta}\psi\rangle\langle e^{i\theta}\psi| = |\psi\rangle\langle\psi| = (\alpha_i\overline{\alpha_j})$: the object attached to a state is the projection, not the vector. For any γ, $\|\gamma\|^2 = \operatorname{Tr}|\gamma\rangle\langle\gamma|$, and

$$p_\alpha(\psi) = \|M_\alpha\psi\|^2 = \operatorname{Tr}\big((M_\alpha^* M_\alpha)(\psi\psi^*)\big), \qquad p_\alpha(\{p_l,\psi_l\}) = \operatorname{Tr}\Big((M_\alpha^* M_\alpha)\sum_l p_l\psi_l\psi_l^*\Big).$$

Definition (notes). $P = \sum_l p_l|\psi_l\rangle\langle\psi_l|$ is the *density matrix* of the ensemble.

Formally *pure-density* ψ is $|\psi\rangle\langle\psi|$, *ensemble-density n E* is P, and *density-matrix n ϱ* is the Day 14 characterisation ($\rho \geq 0$, $\operatorname{Tr}\rho = 1$).

definition *density-matrix* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
density-matrix n ϱ ≡ *ϱ* ∈ *carrier-mat n n* ∧ *psd ϱ* ∧ *Tr ϱ* = *1*

definition *pure-density* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
pure-density ψ ≡
let n = *dim-vec ψ in*
outer-sum n [*ψ*]

lemma *Tr-outer-sum-single*:
assumes *dim*: *dim-vec ψ* = *n*
shows *Tr* (*outer-sum n* [*ψ*]) = ⟨*ψ* | *ψ*⟩
⟨*proof*⟩

lemma *pure-density-is-density-matrix*:
assumes *quantum-state n ψ*
shows *density-matrix n* (*pure-density ψ*)
⟨*proof*⟩

definition *ensemble-density* :: *nat* ⇒ (*real* × *complex Matrix.vec*) *list* ⇒ *complex Matrix.mat* **where**
ensemble-density n E ≡
foldr (λ(*p*,*ψ*) *acc*. *complex-of-real p* ·$_m$ *outer-sum n* [*ψ*] + *acc*)
E (*0*$_m$ *n n*)

lemma *psd-zero-mat* [*simp*]:
psd (*0*$_m$ *n n*)
⟨*proof*⟩

lemma *psd-add-psd*:
assumes *A*: *psd A* **and** *B*: *psd B*
and *Ac*: *A* ∈ *carrier-mat n n* **and** *Bc*: *B* ∈ *carrier-mat n n*
shows *psd* (*A* + *B*)
⟨*proof*⟩

lemma *ensemble-density-Nil* [*simp*]:
ensemble-density n [] = *0*$_m$ *n n*
⟨*proof*⟩

lemma *ensemble-density-Cons*:
ensemble-density n ((p, ψ) # E) =
complex-of-real p ·$_m$ outer-sum n [ψ] + ensemble-density n E
⟨*proof*⟩

lemma *ensemble-density-carrier*:
assumes ∀(*p*, *ψ*) ∈ *set E*. *quantum-state n ψ*
shows *ensemble-density n E* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *ensemble-density-psd-aux*:
assumes ∀(*p*, *ψ*) ∈ *set E*. *p* ≥ *0* ∧ *quantum-state n ψ*
shows *psd* (*ensemble-density n E*)
⟨*proof*⟩

lemma *Tr-zero-mat′* [*simp*]:
Tr (*0$_m$ n n*) = *0*
⟨*proof*⟩

lemma *ensemble-density-Tr*:
assumes ∀(*p*, *ψ*) ∈ *set E*. *quantum-state n ψ*
shows *Tr* (*ensemble-density n E*) = *complex-of-real* (∑(*p*, -) ← *E*. *p*)
⟨*proof*⟩

theorem *ensemble-density-is-dm*:
assumes *E*: *ensemble n E*
shows *density-matrix n* (*ensemble-density n E*)
⟨*proof*⟩

9.3 Probabilities via the trace

$p_\alpha(\psi) = \|M_\alpha\psi\|^2 = \mathrm{Tr}(M_\alpha^* M_\alpha |\psi\rangle\langle\psi|)$. For an ensemble with density matrix ρ, $p_\alpha = \mathrm{Tr}(M_\alpha^* M_\alpha \rho) = \langle M_\alpha^* M_\alpha | \rho \rangle$ (*ensemble-prob-trace-eq* below).

lemma *pure-state-prob-trace*:
fixes *M* :: *complex Matrix.mat* **and** *ψ* :: *complex Matrix.vec*
assumes *st*: *quantum-state n ψ*
and *Msq*: *M* ∈ *carrier-mat n n*
shows *meas-prob M ψ = Re* (*Tr* ((*M*†) ∗ *M* ∗ *outer-sum n* [*ψ*]))
⟨*proof*⟩

For an ensemble: $p_\alpha(\{p_i, \psi_i\}) = \sum_i p_i \|M_\alpha \psi_i\|^2 = \mathrm{Tr}(M_\alpha^* M_\alpha \rho)$.

lemma *ensemble-prob-trace-eq*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat n n*
and *E*: ∀(*p*, *ψ*) ∈ *set E*. *quantum-state n ψ*
shows (∑(*p*, *ψ*) ← *E*. *p* ∗ *meas-prob M ψ*)
= *Re* (*Tr* (((*M*†) ∗ *M*) ∗ *ensemble-density n E*))
⟨*proof*⟩

lemma *Tr-psd-Re-nonneg*:
assumes *P*: *psd P* **and** *Pc*: *P* ∈ *carrier-mat n n*
shows *0* ≤ *Re* (*Tr P*)
⟨*proof*⟩

lemma *ensemble-prob-trace*:

fixes *Ms* :: *complex Matrix.mat list*
assumes *ms*: *measurement-system n Ms*
and *dm*: *density-matrix n ϱ*
and *Min*: $M \in$ *set Ms*
shows *Re* (*Tr* (($M^\dagger$) ∗ *M* ∗ *ϱ*)) ≥ *0*
⟨*proof*⟩

10 Day 14 (28 September 2011): density matrices, pure states, composite ensembles

Proposition (notes). $\rho \in L(\mathcal{H}_A)$ is the density matrix of some ensemble if and only if $\rho \geq 0$ and $\operatorname{Tr}\rho = 1$.

Definition (notes). ρ is a *density matrix* when $\rho \geq 0$ and $\operatorname{Tr}\rho = 1$. The ensemble $\{\lambda_i, \psi_i\}$ of eigenvalues and an orthonormal eigenbasis is the *spectral ensemble*.

Many ensembles have the same density matrix: writing $\rho = XX^* = \sum_i c_i c_i^*$ with columns c_i gives the ensemble $\{\|c_i\|^2, c_i/\|c_i\|\}$.

theorem *density-matrix-characterisation*:
fixes *ϱ* :: *complex Matrix.mat*
assumes *sq*: *ϱ* ∈ *carrier-mat n n*
shows *density-matrix n ϱ* ⟷ *psd ϱ* ∧ *Tr ϱ* = *1*
⟨*proof*⟩

lemma *ensemble-density-carrier-dim*:
shows *ensemble-density n E* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *outer-sum-singleton-index*:
assumes *r*: *r* < *n* **and** *c*: *c* < *n*
shows *outer-sum n* [*ψ*] $$ (*r*, *c*) = *ψ* $ *r* ∗ *cnj* (*ψ* $ *c*)
⟨*proof*⟩

Day 13: $|e^{i\theta}\psi\rangle\langle e^{i\theta}\psi| = |\psi\rangle\langle\psi|$; the density matrix forgets the phase.

lemma *pure-density-smult-unit*:
fixes *ψ* :: *complex Matrix.vec* **and** *c* :: *complex*
assumes *c*: *cmod c* = *1*
shows *pure-density* (*c* ·$_v$ *ψ*) = *pure-density ψ*
⟨*proof*⟩

lemma *ensemble-density-index*:
assumes *r*: *r* < *n* **and** *c*: *c* < *n*
shows *ensemble-density n E* $$ (*r*, *c*) =
(∑ (*p*, *ψ*) ← *E*. *complex-of-real p* ∗ *ψ* $ *r* ∗ *cnj* (*ψ* $ *c*))
⟨*proof*⟩

The spectral ensemble: every density matrix comes from an ensemble.

theorem *spectral-ensemble-exists*:
fixes *ϱ* :: *complex Matrix.mat*
assumes *dm*: *density-matrix n ϱ*
shows ∃ *E*. *ensemble n E* ∧ *ensemble-density n E* = *ϱ*
⟨*proof*⟩

corollary *density-matrix-iff-ensemble*:
fixes *ϱ* :: *complex Matrix.mat*

shows *density-matrix n ϱ* ⟷
(∃ *E*. *ensemble n E* ∧ *ensemble-density n E* = *ϱ*)
⟨*proof*⟩

10.1 Pure states

If $\rho = |\psi\rangle\langle\psi| = \sum_i p_i|\psi_i\rangle\langle\psi_i|$ then for $\gamma \perp \psi$, $0 = \langle\gamma|\rho\gamma\rangle = \sum_i p_i|\langle\psi_i|\gamma\rangle|^2$, so every ψ_i with $p_i > 0$ is parallel to ψ: $\psi_i = \beta_i\psi$ with $|\beta_i| = 1$ and $|\psi_i\rangle\langle\psi_i| = |\psi\rangle\langle\psi|$. These density matrices are the *pure states*.

definition *pure-state-density* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
pure-state-density n ϱ ≡
density-matrix n ϱ ∧
(∃ *ψ*. *quantum-state n ψ* ∧ *ϱ* = *pure-density ψ*)

lemma *pure-state-density-is-density*:
assumes *pure-state-density n ϱ*
shows *density-matrix n ϱ*
⟨*proof*⟩

lemma *pure-density-is-pure-state-density*:
assumes *quantum-state n ψ*
shows *pure-state-density n* (*pure-density ψ*)
⟨*proof*⟩

A companion fact not in the notes: a density matrix is pure iff it is idempotent.

theorem *pure-density-iff-idempotent*:
fixes *ϱ* :: *complex Matrix.mat*
assumes *dm*: *density-matrix n ϱ*
shows *pure-state-density n ϱ* ⟷ *ϱ* ∗ *ϱ* = *ϱ*
⟨*proof*⟩

lemma *outer-sum-singleton-mult-vec*:
assumes *ψ-dim*: *dim-vec ψ* = *n* **and** *γ-dim*: *dim-vec γ* = *n*
shows *outer-sum n* [*ψ*] ∗*v γ* = ⟨*ψ*|*γ*⟩ ·*v ψ*
⟨*proof*⟩

lemma *ensemble-density-quad-form*:
assumes *E-dim*: ∀ (*p*, *ψ*) ∈ *set E*. *dim-vec ψ* = *n*
and *γ-dim*: *dim-vec γ* = *n*
shows ⟨*γ* | *ensemble-density n E* ∗*v γ*⟩ =
(∑ (*p*, *ψ*) ← *E*. *complex-of-real p* ∗ ⟨*ψ*|*γ*⟩ ∗ *cnj* ⟨*ψ*|*γ*⟩)
⟨*proof*⟩

theorem *pure-ensemble-parallel*:
fixes *ψ* :: *complex Matrix.vec*
assumes *st*: *quantum-state n ψ*
and *ens*: *ensemble n E*
and *dens*: *ensemble-density n E* = *pure-density ψ*
and *mem*: (*p*, *ψi*) ∈ *set E*
and *ppos*: *0* < *p*
shows ∃ *β*. *ψi* = *β* ·*v ψ*
⟨*proof*⟩

10.2 Composite ensembles

For ensembles $\{p_i, \psi_i\}$ in lab A and $\{q_l, \phi_l\}$ in lab B the composite is the ensemble $\{p_i q_l, \psi_i \otimes \phi_l\}$ with density matrix $\rho = \sum_{i,l} p_i q_l |\psi_i\rangle\langle\psi_i| \otimes |\phi_l\rangle\langle\phi_l|$, because $|\psi \otimes \phi\rangle\langle\psi \otimes \phi| = |\psi\rangle\langle\psi| \otimes |\phi\rangle\langle\phi|$ (checked on elementary tensors $h \otimes k$). The identity is formalised below; that $\rho_1 \otimes \rho_2$ is again a density matrix is *density-matrix-tensor* (Day 15).

lemma *tensor-pure-density*:
fixes ψ φ :: *complex Matrix.vec*
assumes ψ-*dim*: *dim-vec* $\psi = m$
and φ-*dim*: *dim-vec* $\varphi = p$
and *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
shows *pure-density* (*tensor-vec* ψ φ) =
(*pure-density* ψ) $\bigotimes$ (*pure-density* φ)
⟨*proof*⟩

10.3 Partial traces: motivation

For a density matrix $\rho \in L(\mathcal{H}_A \otimes \mathcal{H}_B)$ and measurements $\{M_\alpha\}$ in lab A, which act as $M_\alpha \otimes I$, $p_\alpha(\rho) = \mathrm{Tr}((M_\alpha^* M_\alpha \otimes I)\rho)$. The notes announce a density matrix $\rho_A \in L(\mathcal{H}_A)$ with $\mathrm{Tr}_{\mathcal{H}_A \otimes \mathcal{H}_B}((M^*M \otimes I)\rho) = \mathrm{Tr}_{\mathcal{H}_A}(M^* M \rho_A)$ for all M; it is constructed on Day 15.

11 Day 15 (30 September 2011): traces, tensor products of operators, partial traces

Proposition (notes). For any orthonormal basis $\{u_1, \ldots, u_n\}$ of $\mathbb{C}^n$, $\mathrm{tr}(A) = \sum_i \langle u_i | A u_i \rangle$.

theorem *trace-via-onb*:
fixes A :: *complex Matrix.mat*
assumes *sq*: $A \in$ *carrier-mat n n*
and *orth*: *orthonormal-list n vs*
and *full*: *length vs* $= n$
shows *Tr* $A = (\sum i{<}n.\ \langle vs\ !\ i \mid A *_v vs\ !\ i\rangle)$
⟨*proof*⟩

Proposition (notes). $\mathrm{tr}(R \otimes T) = \mathrm{tr}(R)\,\mathrm{tr}(T)$.

theorem *trace-tensor*:
fixes R T :: *complex Matrix.mat*
assumes *Rsq*: $R \in$ *carrier-mat m m*
and *Tsq*: $T \in$ *carrier-mat p p*
and *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
shows *Tr* $(R \bigotimes T) =$ *Tr* $R *$ *Tr* T
⟨*proof*⟩

11.1 The isomorphism $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$

Proposition (notes). In finite dimensions the map $\Gamma : L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \to L(\mathcal{H}_A \otimes \mathcal{H}_B)$, $R \otimes T \mapsto R \otimes T$, is a vector-space isomorphism (both sides have dimension $m^2 p^2$ and Γ is injective).

Formally, $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is represented in coordinates as M_{m^2,p^2} and Γ as the reindexing $M_{m^2,p^2} \to M_{mp}$; it is an explicit linear isomorphism with explicit inverse.

Postulate $4'$ (Day 16) and the composite ensembles of Day 14: tensor products of positive matrices are positive, and tensor products of density matrices are density matrices.

lemma *psd-tensor*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat m m* **and** *B*: *B* ∈ *carrier-mat p p*
and *pA*: *psd A* **and** *pB*: *psd B*
shows *psd* (*A* ⊗ *B*)
⟨*proof*⟩

lemma *density-matrix-tensor*:
fixes *A B* :: *complex Matrix.mat*
assumes *dA*: *density-matrix m A* **and** *dB*: *density-matrix p B*
and *m*: *0* < *m* **and** *p*: *0* < *p*
shows *density-matrix* (*m* ∗ *p*) (*A* ⊗ *B*)
⟨*proof*⟩

proposition *L-tensor-dim*:
fixes *m p* :: *nat*
shows *m*^2 ∗ *p*^2 = (*m* ∗ *p*)^2
⟨*proof*⟩

definition *operator-tensor-iso-map* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
operator-tensor-iso-map m p c ≡
Matrix.mat (*m*∗*p*) (*m*∗*p*)
(λ(*r*,*s*). *c* $$ ((*r div p*) ∗ *m* + (*s div p*), (*r mod p*) ∗ *p* + (*s mod p*)))

lemma *operator-tensor-iso-map-carrier* [*simp*]:
operator-tensor-iso-map m p c ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-map-entry*:
assumes *r* < *m*∗*p* **and** *s* < *m*∗*p*
shows *operator-tensor-iso-map m p c* $$ (*r*,*s*) =
c $$ ((*r div p*) ∗ *m* + (*s div p*), (*r mod p*) ∗ *p* + (*s mod p*))
⟨*proof*⟩

lemma *operator-tensor-iso-map-row-bound*:
fixes *m p r s* :: *nat*
assumes *p-pos*: *0* < *p* **and** *r* < *m*∗*p* **and** *s* < *m*∗*p*
shows (*r div p*) ∗ *m* + (*s div p*) < *m*∗*m*
⟨*proof*⟩

lemma *operator-tensor-iso-map-col-bound*:
fixes *p r s* :: *nat*
assumes *p-pos*: *0* < *p*
shows (*r mod p*) ∗ *p* + (*s mod p*) < *p*∗*p*
⟨*proof*⟩

definition *operator-tensor-iso-inv* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
operator-tensor-iso-inv m p x ≡
Matrix.mat (*m*∗*m*) (*p*∗*p*)
(λ(*a*,*b*). *x* $$ ((*a div m*) ∗ *p* + (*b div p*), (*a mod m*) ∗ *p* + (*b mod p*)))

definition *operator-tensor-coordinate-space* ::
nat ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
operator-tensor-coordinate-space m p ≡ *mat-linear-structure* (*m*∗*m*) (*p*∗*p*)

definition *bipartite-operator-space* ::
nat ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
bipartite-operator-space m p ≡ *mat-linear-structure* (*m*∗*p*) (*m*∗*p*)

lemma *operator-tensor-iso-inv-carrier* [*simp*]:
operator-tensor-iso-inv m p x ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-inv-entry*:
assumes *a* < *m*∗*m* **and** *b* < *p*∗*p*
shows *operator-tensor-iso-inv m p x* $$ (*a*,*b*) =
x $$ ((*a div m*) ∗ *p* + (*b div p*), (*a mod m*) ∗ *p* + (*b mod p*))
⟨*proof*⟩

lemma *operator-tensor-iso-inv-row-bound*:
fixes *m p a b* :: *nat*
assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p* **and** *a* < *m*∗*m* **and** *b* < *p*∗*p*
shows (*a div m*) ∗ *p* + (*b div p*) < *m*∗*p*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-col-bound*:
fixes *m p a b* :: *nat*
assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows (*a mod m*) ∗ *p* + (*b mod p*) < *m*∗*p*
⟨*proof*⟩

lemma *operator-tensor-iso-map-add*:
assumes *p-pos*: *0* < *p*
and *x*: *x* ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
and *y*: *y* ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
shows *operator-tensor-iso-map m p* (*x* + *y*) =
operator-tensor-iso-map m p x + *operator-tensor-iso-map m p y*
⟨*proof*⟩

lemma *operator-tensor-iso-map-zero*:
assumes *p-pos*: *0* < *p*
shows *operator-tensor-iso-map m p* (*0*$_m$ (*m*∗*m*) (*p*∗*p*)) = *0*$_m$ (*m*∗*p*) (*m*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-map-smult*:
assumes *p-pos*: *0* < *p*
and *x*: *x* ∈ *carrier-mat* (*m*∗*m*) (*p*∗*p*)
shows *operator-tensor-iso-map m p* (*c* ·$_m$ *x*) =
c ·$_m$ *operator-tensor-iso-map m p x*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-add*:
assumes *m-pos*: *0* < *m*
and *p-pos*: *0* < *p*
and *x*: *x* ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
and *y*: *y* ∈ *carrier-mat* (*m*∗*p*) (*m*∗*p*)
shows *operator-tensor-iso-inv m p* (*x* + *y*) =
operator-tensor-iso-inv m p x + *operator-tensor-iso-inv m p y*
⟨*proof*⟩

lemma *operator-tensor-iso-inv-zero*:
assumes *m-pos*: *0* < *m* **and** *p-pos*: *0* < *p*
shows *operator-tensor-iso-inv m p* (*0*$_m$ (*m*∗*p*) (*m*∗*p*)) = *0*$_m$ (*m*∗*m*) (*p*∗*p*)
⟨*proof*⟩

lemma *operator-tensor-iso-inv-smult*:
assumes *m-pos*: $0 < m$
and *p-pos*: $0 < p$
and *x*: $x \in$ *carrier-mat* $(m*p)$ $(m*p)$
shows *operator-tensor-iso-inv m p* $(c \cdot_m x) =$
$c \cdot_m$ *operator-tensor-iso-inv m p x*
⟨*proof*⟩

proposition *operator-tensor-space-isomorphism*:
assumes *m-pos*: $0 < m$
and *p-pos*: $0 < p$
shows *is-isomorphism-between*
(*operator-tensor-coordinate-space m p*)
(*bipartite-operator-space m p*)
(*operator-tensor-iso-map m p*)
⟨*proof*⟩

corollary *operator-tensor-space-isomorphic*:
assumes $0 < m$ **and** $0 < p$
shows *is-isomorphic-to*
(*operator-tensor-coordinate-space m p*)
(*bipartite-operator-space m p*)
⟨*proof*⟩

11.2 Partial traces

Definition (notes). Identify $L(\mathcal{H}_A \otimes \mathcal{H}_B) = L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ and define $\mathrm{tr}_B(\sum_l R_l \otimes T_l) = \sum_l \mathrm{tr}(T_l) R_l \in L(\mathcal{H}_A)$ and $\mathrm{tr}_A(\sum_l R_l \otimes T_l) = \sum_l \mathrm{tr}(R_l) T_l \in L(\mathcal{H}_B)$; also written $X^A = \mathrm{tr}_B(X)$, $X^B = \mathrm{tr}_A(X)$. Well defined by the universal property, since $(R, T) \mapsto \mathrm{tr}(T)R$ is bilinear.

The formal definitions use the block formulas of Day 16: an $mp \times mp$ matrix is indexed by pairs (i, k), $i < m$, $k < p$, with flat index $ip + k$ ($\mathcal{H}_A$ is the major factor, matching $(A \otimes B)_{(r,c)} = A_{(r \div p,\, c \div p)} B_{(r \bmod p,\, c \bmod p)}$), and $(\mathrm{tr}_B\, X)_{ij} = \sum_{k<p} X_{ip+k,\, jp+k}$, $(\mathrm{tr}_A\, X)_{ij} = \sum_{k<m} X_{kp+i,\, kp+j}$. The defining property on elementary tensors is proved below.

record *'lab qspace* =
qdim :: *nat*
qlabel :: *'lab*

record *'lab bipartite-space* =
left-space :: *'lab qspace*
right-space :: *'lab qspace*

definition *partial-trace-B* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
partial-trace-B m p x ≡
Matrix.mat m m (λ(*i*, *j*). ∑ *k*<*p*. *x* $$ (*i* ∗ *p* + *k*, *j* ∗ *p* + *k*))

definition *partial-trace-A* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
partial-trace-A m p x ≡
Matrix.mat p p (λ(*i*, *j*). ∑ *k*<*m*. *x* $$ (*k* ∗ *p* + *i*, *k* ∗ *p* + *j*))

definition *partial-trace-on* ::
'lab qspace ⇒ *'lab bipartite-space* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
partial-trace-on s ab x ≡
(*if s* = *right-space ab then*
partial-trace-B (*qdim* (*left-space ab*)) (*qdim* (*right-space ab*)) *x*
else if s = *left-space ab then*
partial-trace-A (*qdim* (*left-space ab*)) (*qdim* (*right-space ab*)) *x*

else 0_m *0 0*)

notation *partial-trace-on* (*tr*$_{-}^{-}$ - [*1000*, *1000*, *1000*] *1000*)

lemma *partial-trace-on-right*:
fixes S :: *'lab qspace* **and** AB :: *'lab bipartite-space*
assumes S = *right-space* AB
shows $tr_S^{AB}\ x$ =
partial-trace-B (*qdim* (*left-space* AB)) (*qdim* (*right-space* AB)) x
⟨*proof*⟩

lemma *partial-trace-on-left*:
fixes S :: *'lab qspace* **and** AB :: *'lab bipartite-space*
assumes S = *left-space* AB
and *left-space* $AB \neq$ *right-space* AB
shows $tr_S^{AB}\ x$ =
partial-trace-A (*qdim* (*left-space* AB)) (*qdim* (*right-space* AB)) x
⟨*proof*⟩

lemma *dim-row-partial-trace-B* [*simp*]: *dim-row* (*partial-trace-B* m p x) = m
⟨*proof*⟩

lemma *dim-col-partial-trace-B* [*simp*]: *dim-col* (*partial-trace-B* m p x) = m
⟨*proof*⟩

lemma *partial-trace-B-carrier* [*simp*]:
partial-trace-B m p $x \in$ *carrier-mat* m m
⟨*proof*⟩

lemma *dim-row-partial-trace-A* [*simp*]: *dim-row* (*partial-trace-A* m p x) = p
⟨*proof*⟩

lemma *dim-col-partial-trace-A* [*simp*]: *dim-col* (*partial-trace-A* m p x) = p
⟨*proof*⟩

lemma *partial-trace-A-carrier* [*simp*]:
partial-trace-A m p $x \in$ *carrier-mat* p p
⟨*proof*⟩

lemma *partial-trace-B-tensor*:
fixes R T :: *complex Matrix.mat*
assumes *Rsq*: $R \in$ *carrier-mat* m m
and *Tsq*: $T \in$ *carrier-mat* p p
and *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
shows *partial-trace-B* m p ($R \otimes T$) = *Tr* T $\cdot_m$ R
⟨*proof*⟩

lemma *partial-trace-A-tensor*:
fixes R T :: *complex Matrix.mat*
assumes *Rsq*: $R \in$ *carrier-mat* m m
and *Tsq*: $T \in$ *carrier-mat* p p
and *m-pos*: $0 < m$ **and** *p-pos*: $0 < p$
shows *partial-trace-A* m p ($R \otimes T$) = *Tr* R $\cdot_m$ T
⟨*proof*⟩

11.3 Partial traces and measurements

Writing $\rho = \sum_l R_l \otimes T_l$,

$$p_\alpha(\rho) = \mathrm{tr}((M_\alpha^* M_\alpha \otimes I)\rho) = \sum_l \mathrm{tr}(M_\alpha^* M_\alpha R_l)\,\mathrm{tr}(T_l) = \mathrm{tr}\big(M_\alpha^* M_\alpha\ \mathrm{tr}_B(\rho)\big) = p_\alpha(\rho^A)\,:$$

lab A only sees ρ^A, and symmetrically lab B only sees ρ^B. Check: $\mathrm{tr}(\rho^A) = \mathrm{tr}\,\rho = 1$, and $\rho \geq 0$ implies $\rho^A \geq 0$.

12 Day 16 (3 October 2011): partial traces in block form; the postulates for density matrices

With matrix units E_{ij} of $L(\mathcal{H}_B)$, every $X \in L(\mathcal{H}_A) \otimes L(\mathcal{H}_B)$ is uniquely $X = \sum_{i,j} X_{ij} \otimes E_{ij}$ with $X_{ij} \in L(\mathcal{H}_A)$, and $\mathrm{tr}_B(X) = \sum_i X_{ii}$, $\mathrm{tr}_A(X) = (\mathrm{tr}\, X_{ij})_{ij}$; symmetrically for $X = \sum E_{kl} \otimes Y_{kl}$. Choosing a basis $f_1, \ldots, f_p$ of $\mathcal{H}_B$ identifies $\mathcal{H}_A \otimes \mathcal{H}_B \cong \mathcal{H}_A^{(p)} = \mathcal{H}_A \oplus \cdots \oplus \mathcal{H}_A$ and $L(\mathcal{H}_A \otimes \mathcal{H}_B) \cong M_p(L(\mathcal{H}_A))$, block matrices (X_{ij}) acting by $(X_{ij})(h_j)_j = (\sum_j X_{ij} h_j)_i$; in block notation $\mathrm{tr}_B((X_{ij})) = X_{11} + \cdots + X_{pp}$ and $\mathrm{tr}_A((X_{ij})) = (\mathrm{tr}\, X_{ij})$.

Reformulated postulates. $1'$: the state of a system is a density matrix $\rho \in L(\mathcal{H}_A)$. $2'$: closed evolution is $\rho \mapsto U\rho U^*$. $3'$: a measurement system $\{M_\alpha\}$ with $\sum M_\alpha^* M_\alpha = I$ yields outcome α with probability $\mathrm{tr}(M_\alpha^* M_\alpha \rho)$. $4'$: systems with densities ρ_1, ρ_2 compose to $\rho_1 \otimes \rho_2$ on $\mathcal{H}_A \otimes \mathcal{H}_B$.

lemma *partial-trace-B-trace*:
  fixes *X* :: *complex Matrix.mat*
  assumes *Xsq*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)
    and *p-pos*: *0* < *p*
  shows *Tr* (*partial-trace-B m p X*) = *Tr X*
⟨*proof*⟩

lemma *partial-trace-B-hermitian*:
  fixes *X* :: *complex Matrix.mat*
  assumes *Xsq*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*)
    and *herm*: *hermitian X*
  shows *hermitian* (*partial-trace-B m p X*)
⟨*proof*⟩

lemma *partial-trace-B-is-density*:
  fixes *X* :: *complex Matrix.mat*
  assumes *dm*: *density-matrix* (*m* ∗ *p*) *X*
    and *p-pos*: *0* < *p*
  shows *density-matrix m* (*partial-trace-B m p X*)
⟨*proof*⟩

Key identity: measurements in lab A only see $\rho^A = \mathrm{tr}_B\,\rho$.

theorem *partial-trace-measurement*:
  fixes *meas* :: *complex Matrix.mat* **and** ϱ :: *complex Matrix.mat*
  assumes *M-sq*: *meas* ∈ *carrier-mat m m*
    and *dm*: *density-matrix* (*m* ∗ *p*) ϱ
    and *p-pos*: *0* < *p*
  shows *Tr* (((*meas*† ∗ *meas*) ⨂ *1*$_m$ *p*) ∗ ϱ) =
      Tr (*meas*† ∗ *meas* ∗ *partial-trace-B m p* ϱ)
⟨*proof*⟩

13 Day 17 (5 October 2011): measurement maps, quantum noise, completely positive maps

13.1 Measurement maps

Measuring ψ with $\{M_\alpha\}$ and keeping the outcome as an ensemble gives the density matrix $\sum_\alpha p_\alpha(\psi)|M_\alpha\psi/\|M_\alpha\psi\|\rangle\langle M_\alpha\psi/\|M_\alpha\psi\|| = \sum_\alpha M_\alpha(\psi\psi^*)M_\alpha^*$; for an ensemble with density ρ_1 one obtains $M(\rho_1) = \sum_\alpha M_\alpha\rho_1 M_\alpha^*$: a linear map on $L(\mathcal{H}_A)$ sending density matrices to density matrices.

Classical noise flips a bit with some probability; probability vectors evolve by stochastic matrices (non-negative entries, columns summing to one; doubly stochastic if rows do too). Quantum noise: with an environment $\mathcal{H}_E$ in state $|\phi\rangle$, the system $\rho\otimes|\phi\rangle\langle\phi|$ evolves by a unitary $U = (U_{ij})$ on $\mathcal{H}_A \otimes \mathcal{H}_E$ and lab A sees

$$\mathrm{tr}_E(U(\rho\otimes|\phi\rangle\langle\phi|)U^*) = \sum_i W_i\rho W_i^*, \qquad W_i = \sum_k U_{ik}\alpha_k, \qquad \sum_i W_i^*W_i = I.$$

Measurement by an intruder and environmental noise are maps of the same form $\rho \mapsto \sum_\alpha X_\alpha\rho X_\alpha^*$ with $\sum X_\alpha^*X_\alpha = I$. The environment computation is *environment-model* in *Day17-Environment-Model*.

Third way, axiomatic: a quantum event is a linear $\Phi : L(\mathcal{H}_A) \to L(\mathcal{H}_A)$.

Proposition (notes). Φ maps density matrices to density matrices iff Φ is positive ($\rho \geq 0 \Rightarrow \Phi(\rho) \geq 0$) and trace preserving ($\mathrm{tr}\,\Phi(X) = \mathrm{tr}\,X$); then $\Phi \otimes I_{L(\mathcal{H}_B)}$ also maps density matrices to density matrices for every $\mathcal{H}_B$.

Definition (notes). Φ is *completely positive* if $\Phi\otimes I_{L(\mathcal{H}_B)}$ is positive for every finite-dimensional $\mathcal{H}_B$.

Theorem (notes) (Choi–Kraus)**.** $\Phi : L(\mathcal{H}_A) \to L(\mathcal{H}_A)$ is completely positive and trace preserving iff $\Phi(X) = \sum_i E_i X E_i^*$ for matrices E_i with $\sum_i E_i^*E_i = I$.

The proposition is *density-preserving-iff-positive-tp* in *Day18-Positive-Maps*; complete positivity is defined below and the Choi–Kraus theorem is *choi-kraus-cptp* on Day 19.

definition *meas-map* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
meas-map Ms ϱ ≡
foldr (λ*M acc*. *M* ∗ *ϱ* ∗ (*M*†) + *acc*) *Ms* (0*m* (*dim-row ϱ*) (*dim-col ϱ*))

lemma *meas-map-Nil* [*simp*]:
meas-map [] *ϱ* = 0*m* (*dim-row ϱ*) (*dim-col ϱ*)
⟨*proof*⟩

lemma *meas-map-Cons* [*simp*]:
meas-map (*M* # *Ms*) *ϱ* = *M* ∗ *ϱ* ∗ (*M*†) + *meas-map Ms ϱ*
⟨*proof*⟩

lemma *meas-map-carrier*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
shows *meas-map Ms ϱ* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *meas-map-psd*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and *ϱ-c*: *ϱ* ∈ *carrier-mat n n*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*meas-map Ms ϱ*)

⟨*proof*⟩

lemma *foldr-MdagM-carrier*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
shows *foldr* (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (0$_m$ *n n*) ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *meas-map-trace-aux*:
assumes *Ms-c*: ∀ *M* ∈ *set Ms*. *M* ∈ *carrier-mat n n*
and ϱ*-c*: ϱ ∈ *carrier-mat n n*
shows *Tr* (*meas-map Ms* ϱ) =
Tr (*foldr* (λ*M acc*. (*M*†) ∗ *M* + *acc*) *Ms* (0$_m$ *n n*) ∗ ϱ)
⟨*proof*⟩

lemma *meas-map-density*:
assumes *ms*: *measurement-system n Ms*
and *dm*: *density-matrix n* ϱ
shows *density-matrix n* (*meas-map Ms* ϱ)
⟨*proof*⟩

13.2 Kraus maps, positive maps, complete positivity

A *Kraus map* $\rho \mapsto \sum_i W_i \rho W_i^\dagger$ is *kraus-map-dim d Ws*; the list *Ws* is a list of *Kraus operators* (*kraus-operators n d Ws*) when $\sum_i W_i^\dagger W_i = I_n$. Positive, trace-preserving and unital maps are defined on the carrier M_n. The amplification $\Phi^{(p)} = \Phi \otimes \mathrm{id}_p$ acts on block matrices by $(X_{ab}) \mapsto (\Phi(X_{ab}))$ (*amplification n d p* Φ), and Φ is p-positive when $\Phi^{(p)}$ is positive, completely positive when it is p-positive for all p. The main result of the subsection: Kraus maps are completely positive.

definition *kraus-map-dim* :: *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-map-dim d Ws ϱ ≡
foldr (λ*W acc*. *W* ∗ ϱ ∗ (*W*†) + *acc*) *Ws* (0$_m$ *d d*)

definition *kraus-map* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-map Ws ϱ ≡ *kraus-map-dim* (*dim-row* ϱ) *Ws* ϱ

definition *kraus-operators* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *bool* **where**
kraus-operators n d Ws ≡
(∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*) ∧
foldr (λ*W acc*. (*W*†) ∗ *W* + *acc*) *Ws* (0$_m$ *n n*) = 1$_m$ *n*

lemma *kraus-map-dim-Nil* [*simp*]:
kraus-map-dim d [] ϱ = 0$_m$ *d d*
⟨*proof*⟩

lemma *kraus-map-dim-Cons* [*simp*]:
kraus-map-dim d (*W* # *Ws*) ϱ =
W ∗ ϱ ∗ (*W*†) + *kraus-map-dim d Ws* ϱ
⟨*proof*⟩

lemma *kraus-map-dim-carrier*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and ϱ*-c*: ϱ ∈ *carrier-mat n n*
shows *kraus-map-dim d Ws* ϱ ∈ *carrier-mat d d*
⟨*proof*⟩

lemma *kraus-map-dim-psd*:

assumes *Ws-c*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and *ϱ-c*: $\varrho \in carrier\text{-}mat\ n\ n$
and *ϱ-psd*: $psd\ \varrho$
shows $psd\ (kraus\text{-}map\text{-}dim\ d\ Ws\ \varrho)$
⟨*proof*⟩

definition *positive-map* :: $nat \Rightarrow nat \Rightarrow (complex\ Matrix.mat \Rightarrow complex\ Matrix.mat) \Rightarrow bool$ **where**
$positive\text{-}map\ n\ d\ \Phi \equiv$
$(\forall\, A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ A \in carrier\text{-}mat\ d\ d) \wedge$
$(\forall\, P.\ P \in carrier\text{-}mat\ n\ n \longrightarrow psd\ P \longrightarrow psd\ (\Phi\ P))$

lemma *kraus-map-dim-positive-map*:
assumes *Ws*: $kraus\text{-}operators\ n\ d\ Ws$
shows $positive\text{-}map\ n\ d\ (kraus\text{-}map\text{-}dim\ d\ Ws)$
⟨*proof*⟩

definition *trace-preserving* :: $nat \Rightarrow (complex\ Matrix.mat \Rightarrow complex\ Matrix.mat) \Rightarrow bool$ **where**
$trace\text{-}preserving\ n\ \Phi \equiv$
$\forall\, X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow Tr\ (\Phi\ X) = Tr\ X$

definition *unital-map* :: $nat \Rightarrow nat \Rightarrow (complex\ Matrix.mat \Rightarrow complex\ Matrix.mat) \Rightarrow bool$ **where**
$unital\text{-}map\ n\ d\ \Phi \equiv \Phi\ (1_m\ n) = 1_m\ d$

definition *block-of-mat* :: $nat \Rightarrow complex\ Matrix.mat \Rightarrow nat \Rightarrow nat \Rightarrow complex\ Matrix.mat$ **where**
$block\text{-}of\text{-}mat\ n\ x\ a\ b \equiv$
$Matrix.mat\ n\ n\ (\lambda(r,c).\ x\ \$\$\ (a{*}n + r,\ b{*}n + c))$

lemma *block-of-mat-carrier* [*simp*]:
$block\text{-}of\text{-}mat\ n\ X\ a\ b \in carrier\text{-}mat\ n\ n$
⟨*proof*⟩

lemma *block-of-mat-index*:
assumes $r < n$ **and** $c < n$
shows $block\text{-}of\text{-}mat\ n\ X\ a\ b\ \$\$\ (r,c) = X\ \$\$\ (a{*}n + r,\ b{*}n + c)$
⟨*proof*⟩

lemma *block-of-mat-single*:
assumes *X*: $X \in carrier\text{-}mat\ n\ n$
shows $block\text{-}of\text{-}mat\ n\ X\ 0\ 0 = X$
⟨*proof*⟩

definition *amplification* :: $nat \Rightarrow nat \Rightarrow nat \Rightarrow$
$(complex\ Matrix.mat \Rightarrow complex\ Matrix.mat) \Rightarrow complex\ Matrix.mat \Rightarrow complex\ Matrix.mat$ **where**
$amplification\ n\ d\ p\ \Phi\ x \equiv$
$Matrix.mat\ (p{*}d)\ (p{*}d)$
$(\lambda(i,j).\ \Phi\ (block\text{-}of\text{-}mat\ n\ x\ (i\ div\ d)\ (j\ div\ d))\ \$\$$
$(i\ mod\ d,\ j\ mod\ d))$

lemma *amplification-carrier* [*simp*]:
$amplification\ n\ d\ p\ \Phi\ X \in carrier\text{-}mat\ (p{*}d)\ (p{*}d)$
⟨*proof*⟩

lemma *amplification-index*:
assumes $i < p{*}d$ **and** $j < p{*}d$
shows $amplification\ n\ d\ p\ \Phi\ X\ \$\$\ (i,j) =$
$\Phi\ (block\text{-}of\text{-}mat\ n\ X\ (i\ div\ d)\ (j\ div\ d))\ \$\$$
$(i\ mod\ d,\ j\ mod\ d)$
⟨*proof*⟩

lemma *amplification-one*:
assumes *X*: $X \in carrier\text{-}mat\ n\ n$
and *range*: $\Phi\ X \in carrier\text{-}mat\ d\ d$
shows $amplification\ n\ d\ 1\ \Phi\ X = \Phi\ X$
⟨*proof*⟩

lemma *tensor-id-kraus-carrier*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$
shows $(1_m\ p \bigotimes W) \in carrier\text{-}mat\ (p*d)\ (p*n)$
⟨*proof*⟩

lemma *tensor-id-kraus-list-carrier*:
assumes *Ws*: $\forall W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
shows $\forall L \in set\ (map\ (\lambda W.\ 1_m\ p \bigotimes W)\ Ws).$
$L \in carrier\text{-}mat\ (p*d)\ (p*n)$
⟨*proof*⟩

lemma *tensor-id-kraus-index*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$
and *r*: $r < p*d$
and *c*: $c < p*n$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $(1_m\ p \bigotimes W)\ \$\$\ (r,c) =$
$(if\ r\ div\ d = c\ div\ n\ then\ W\ \$\$\ (r\ mod\ d,\ c\ mod\ n)\ else\ 0)$
⟨*proof*⟩

lemma *sum-block-delta*:
fixes *f* :: $nat \Rightarrow complex$
assumes *a*: $a < p$
and *npos*: $0 < n$
shows $(\sum t < p*n.\ if\ a = t\ div\ n\ then\ f\ (t\ mod\ n)\ else\ 0) =$
$(\sum r < n.\ f\ r)$
⟨*proof*⟩

lemma *sum-block-delta-index*:
fixes *h* :: $nat \Rightarrow complex$
assumes *a*: $a < p$
and *npos*: $0 < n$
shows $(\sum t < p*n.\ if\ a = t\ div\ n\ then\ h\ t\ else\ 0) =$
$(\sum r < n.\ h\ (a*n + r))$
⟨*proof*⟩

lemma *tensor-id-kraus-left-mult-index*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$
and *x*: $x \in carrier\text{-}mat\ (p*n)\ q$
and *i*: $i < p*d$
and *s*: $s < q$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $((1_m\ p \bigotimes W) * x)\ \$\$\ (i,s) =$
$(\sum t < n.\ W\ \$\$\ (i\ mod\ d,\ t) * x\ \$\$\ ((i\ div\ d) * n + t,\ s))$
⟨*proof*⟩

lemma *tensor-id-kraus-right-mult-index*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$
and *y*: $y \in carrier\text{-}mat\ q\ (p*n)$

and i: $i < q$
and j: $j < p*d$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $(y * (1_m\ p \bigotimes (W^\dagger)))$ \$\$ $(i,j) =$
$(\sum s < n.\ y$ \$\$ $(i, (j\ div\ d) * n + s) * (W^\dagger)$ \$\$ $(s, j\ mod\ d))$
⟨*proof*⟩

lemma *kraus-term-index*:
assumes W: $W \in carrier\text{-}mat\ d\ n$
and B: $B \in carrier\text{-}mat\ n\ n$
and i: $i < d$
and j: $j < d$
shows $(W * B * (W^\dagger))$ \$\$ $(i,j) =$
$(\sum s < n. (\sum t < n.\ W$ \$\$ $(i,t) * B$ \$\$ $(t,s)) * (W^\dagger)$ \$\$ $(s,j))$
⟨*proof*⟩

lemma *kraus-amplification-single*:
assumes W: $W \in carrier\text{-}mat\ d\ n$
and x: $x \in carrier\text{-}mat\ (p*n)\ (p*n)$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $amplification\ n\ d\ p\ (\lambda\varrho.\ W * \varrho * (W^\dagger))\ x =$
$((1_m\ p \bigotimes W) * x) * (1_m\ p \bigotimes (W^\dagger))$
⟨*proof*⟩

lemma *amplification-zero-map*:
assumes *dpos*: $0 < d$
shows $amplification\ n\ d\ p\ (\lambda\varrho.\ 0_m\ d\ d)\ x = 0_m\ (p*d)\ (p*d)$
⟨*proof*⟩

lemma *amplification-add*:
assumes *dpos*: $0 < d$
and F: $\bigwedge a\ b.\ F\ (block\text{-}of\text{-}mat\ n\ x\ a\ b) \in carrier\text{-}mat\ d\ d$
and G: $\bigwedge a\ b.\ G\ (block\text{-}of\text{-}mat\ n\ x\ a\ b) \in carrier\text{-}mat\ d\ d$
shows $amplification\ n\ d\ p\ (\lambda\varrho.\ F\ \varrho + G\ \varrho)\ x =$
$amplification\ n\ d\ p\ F\ x + amplification\ n\ d\ p\ G\ x$
⟨*proof*⟩

lemma *amplification-kraus-map-dim*:
assumes *Ws*: $\forall W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and x: $x \in carrier\text{-}mat\ (p*n)\ (p*n)$
and *dpos*: $0 < d$
and *npos*: $0 < n$
shows $amplification\ n\ d\ p\ (kraus\text{-}map\text{-}dim\ d\ Ws)\ x =$
$kraus\text{-}map\text{-}dim\ (p*d)\ (map\ (\lambda W.\ 1_m\ p \bigotimes W)\ Ws)\ x$
⟨*proof*⟩

definition *p-positive* :: $nat \Rightarrow nat \Rightarrow nat \Rightarrow (complex\ Matrix.mat \Rightarrow complex\ Matrix.mat) \Rightarrow bool$
where
$p\text{-}positive\ n\ d\ p\ \Phi \equiv$
$\forall X :: complex\ Matrix.mat.\ X \in carrier\text{-}mat\ (p*n)\ (p*n) \longrightarrow psd\ X \longrightarrow$
$psd\ (amplification\ n\ d\ p\ \Phi\ X)$

definition *completely-positive* :: $nat \Rightarrow nat \Rightarrow (complex\ Matrix.mat \Rightarrow complex\ Matrix.mat) \Rightarrow bool$
where
$completely\text{-}positive\ n\ d\ \Phi \equiv \forall p > 0.\ p\text{-}positive\ n\ d\ p\ \Phi$

theorem *kraus-is-cp*:
assumes *Ws*: *kraus-operators n d Ws*
and *npos*: *0 < n*
and *dpos*: *0 < d*
shows *completely-positive n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

14 Day 18 (7 October 2011): the theory of completely positive maps

Proposition (notes). Every $X \in M_n$ is $X = (P_1 - P_2) + i(P_3 - P_4)$ with $P_i \geq 0$: $\mathrm{span}(M_n^+) = M_n$.

(Write $X = H + iK$ with H, K Hermitian and split each spectral decomposition into its non-negative and negative eigenvalues.)

Definition (notes). A linear $\Phi : M_n \to M_d$ is *positive* if $P \geq 0$ implies $\Phi(P) \geq 0$; it is *p-positive* if $\Phi^{(p)} : M_p(M_n) \to M_p(M_d)$, $(X_{ij}) \mapsto (\Phi(X_{ij}))$, is positive, and *completely positive* if it is p-positive for every p.

Proposition (notes). A linear $\Phi : M_n \to M_d$ maps density matrices to density matrices iff it is positive and trace preserving.

Example (notes): the transpose $\Phi(A) = A^t$ on M_2 is positive but not 2-positive, since $\Phi^{(2)}$ sends the positive matrix $(E_{ij})_{i,j}$ to a matrix of determinant -1. The decomposition into positives is formalised below; the proposition is *density-preserving-iff-positive-tp* and the example *transpose-not-2-positive*, both in *Day18-Positive-Maps*.

lemma *diag-mat-of-real-hermitian*:
((*diag-mat-of-real D*)†) = *diag-mat-of-real D*
⟨*proof*⟩

lemma *hermitian-decomp-psd*:
fixes *H* :: *complex Matrix.mat*
assumes *sq*: *H* ∈ *carrier-mat n n*
and *herm*: *hermitian H*
shows ∃ *P1 P2* :: *complex Matrix.mat*.
P1 ∈ *carrier-mat n n* ∧ *P2* ∈ *carrier-mat n n* ∧
psd P1 ∧ *psd P2* ∧
H = *P1* − *P2*
⟨*proof*⟩

lemma *matrix-decomp-psd*:
fixes *X* :: *complex Matrix.mat*
assumes *sq*: *X* ∈ *carrier-mat n n*
shows ∃ *P1 P2 P3 P4* :: *complex Matrix.mat*.
(∀ *i* ∈ {*0..3*}. [*P1*,*P2*,*P3*,*P4*] ! *i* ∈ *carrier-mat n n* ∧ *psd* ([*P1*,*P2*,*P3*,*P4*] ! *i*)) ∧
X = (*P1* − *P2*) + i ·$_m$ (*P3* − *P4*)
⟨*proof*⟩

15 Day 19 (10 October 2011): Choi's theorem

Theorem (notes) (Choi, 1975)**.** For linear $\Phi : M_n \to M_d$ the following are equivalent: (1) Φ is completely positive; (2) Φ is n-positive; (3) $P_\Phi = (\Phi(E_{ij}))_{i,j} \geq 0$ in $M_n(M_d)$; (4) there are $d \times n$ matrices $B_1, \ldots, B_K$ with $\Phi(X) = \sum_k B_k X B_k^*$.

Notes: (1) $\sum_k B_k X B_k^*$ is a *Choi–Kraus representation*; (2) P_Φ is the *Choi–Jamiołkowski matrix*; (3) $CP(M_n, M_d) \leftrightarrow M_n(M_d)^+$ via $\Phi \leftrightarrow P_\Phi$; (4) Φ is trace preserving iff $\mathrm{tr}(\Phi(E_{ij})) = \delta_{ij}$; (5) Φ is unital iff $\sum_i \Phi(E_{ii}) = I_d$.

Formally P_Φ is *choi-matrix n d* Φ (an $nd \times nd$ matrix built from the blocks $\Phi(E_{ij})$); *choi-theorem* is (1)⇔(3), *kraus-rep-of-psd-choi* is (3)⇒(4), and *kraus-is-cp* is (4)⇒(1). The step (2)⇒(3) is the observation that $P_\Phi = \Phi^{(n)}((E_{ij}))$ with $(E_{ij}) \geq 0$ (*choi-matrix-as-amplification*). Linearity of Φ is assumed on the carrier M_n (additivity and homogeneity hypotheses). The trace-preserving case is *choi-kraus-cptp*; notes (4) and (5) in Choi-matrix form are *choi-trace-preserving-iff* and *choi-unital-iff* in theory Day19_Choi_TP_Unital.

definition *choi-matrix* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* **where**
  choi-matrix n d Φ ≡
    Matrix.mat (*n*∗*d*) (*n*∗*d*)
      (λ(*r*,*c*). Φ (*mat-unit* (*r div d*) (*c div d*) *n n*) $$ (*r mod d*, *c mod d*))

notation *choi-matrix* (P_Φ)

definition *choi-seed* :: *nat* ⇒ *complex Matrix.mat* **where**
  choi-seed n ≡
    Matrix.mat (*n*∗*n*) (*n*∗*n*)
      (λ(*r*,*c*). *mat-unit* (*r div n*) (*c div n*) *n n* $$ (*r mod n*, *c mod n*))

lemma *choi-seed-carrier* [*simp*]:
  choi-seed n ∈ *carrier-mat* (*n*∗*n*) (*n*∗*n*)
  ⟨*proof*⟩

lemma *choi-seed-block*:
  assumes *npos*: *n* > *0*
    and *a*: *a* < *n*
    and *b*: *b* < *n*
  shows *block-of-mat n* (*choi-seed n*) *a b* = *mat-unit a b n n*
⟨*proof*⟩

lemma *choi-matrix-as-amplification*:
  assumes *npos*: *n* > *0*
    and *dpos*: *d* > *0*
  shows *choi-matrix n d* Φ = *amplification n d n* Φ (*choi-seed n*)
⟨*proof*⟩

lemma *choi-psd-if-cp*:
  assumes *npos*: *n* > *0*
    and *dpos*: *d* > *0*
    and *cp*: *completely-positive n d* Φ
  shows *psd* (*choi-matrix n d* Φ)
⟨*proof*⟩

lemma *kraus-cp-no-tp*:
  assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
    and *npos*: *0* < *n*
    and *dpos*: *0* < *d*
  shows *completely-positive n d* (*kraus-map-dim d Ws*)
⟨*proof*⟩

15.1 From a positive Choi matrix to a Kraus representation

The notes' proof of (3)⇒(4): write $P_\Phi = \sum_k v_k v_k^*$ with $v_k \in \mathbb{C}^{nd}$, cut each v_k into n blocks $w_1^k, \ldots, w_n^k \in \mathbb{C}^d$, and let $B_k = [w_1^k | \cdots | w_n^k]$; then $B_k E_{ij} B_k^* = w_i^k (w_j^k)^*$ and $\Phi(X) = \sum_{ij} x_{ij} \Phi(E_{ij}) = \sum_k B_k X B_k^*$ by linearity.

Complete positivity from a Kraus list without the trace-preservation constraint: the argument is the same as ⟦*kraus-operators ?n ?d ?Ws*; *0 < ?n*; *0 < ?d*⟧ ⟹ *completely-positive ?n ?d* (*kraus-map-dim ?d ?Ws*), which only uses the carrier hypothesis.

Linearity of the Kraus-sum map in its matrix argument.

lemma *kraus-map-dim-add*:
  assumes *A-c*: *A* ∈ *carrier-mat n n* **and** *B-c*: *B* ∈ *carrier-mat n n*
    and *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
  shows *kraus-map-dim d Ws* (*A* + *B*) =
      kraus-map-dim d Ws A + *kraus-map-dim d Ws B*
  ⟨*proof*⟩

lemma *kraus-map-dim-smult*:
  assumes *A-c*: *A* ∈ *carrier-mat n n*
    and *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
  shows *kraus-map-dim d Ws* (*c* ·$_m$ *A*) = *c* ·$_m$ *kraus-map-dim d Ws A*
  ⟨*proof*⟩

lemma *kraus-map-dim-zero* [*simp*]:
  assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
  shows *kraus-map-dim d Ws* (*0*$_m$ *n n*) = *0*$_m$ *d d*
  ⟨*proof*⟩

Enumeration of [*0*..<*n*] × [*0*..<*n*] as a list of pairs.

definition *mat-unit-pairs* :: *nat* ⇒ (*nat* × *nat*) *list* **where**
  mat-unit-pairs n ≡
    concat (*map* (λ*a*. *map* (*Pair a*) [*0*..<*n*]) [*0*..<*n*])

lemma *mat-unit-pairs-set*:
  set (*mat-unit-pairs n*) = {*p*. *fst p* < *n* ∧ *snd p* < *n*}
  ⟨*proof*⟩

lemma *mat-unit-pairs-distinct*: *distinct* (*mat-unit-pairs n*)
  ⟨*proof*⟩

Generic pair-indexed linear combination of $f\ (E_{ab})$ contributions, weighted by the entries of A.

primrec *pair-comb* ::
  (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒
   nat ⇒ *nat* ⇒ (*nat* × *nat*) *list* ⇒ *complex Matrix.mat* **where**
  pair-comb f A n d [] = *0*$_m$ *d d* |
  pair-comb f A n d (*p* # *ps*) =
    A $$ *p* ·$_m$ *f* (*mat-unit* (*fst p*) (*snd p*) *n n*) + *pair-comb f A n d ps*

lemma *pair-comb-carrier*:
  assumes *f-car*: ⋀*a b*. *a* < *n* ⟹ *b* < *n* ⟹ *f* (*mat-unit a b n n*) ∈ *carrier-mat d d*
    and *ps-valid*: ∀ *p* ∈ *set ps*. *fst p* < *n* ∧ *snd p* < *n*
  shows *pair-comb f A n d ps* ∈ *carrier-mat d d*
  ⟨*proof*⟩

lemma *pair-comb-entry*:
  assumes *f-car*: ⋀*a b*. *a* < *n* ⟹ *b* < *n* ⟹ *f* (*mat-unit a b n n*) ∈ *carrier-mat d d*

and *ps-valid*: $\forall\, p \in set\ ps.\ fst\ p < n \wedge snd\ p < n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows *pair-comb f A n d ps* $\$\$\ (r,c) =$
$(\sum p \leftarrow ps.\ A\ \$\$\ p * f\ (mat\text{-}unit\ (fst\ p)\ (snd\ p)\ n\ n)\ \$\$\ (r,c))$
⟨*proof*⟩

For the identity-like choice $f = id$, the pair combination reconstructs A when *ps* enumerates all basis positions.

lemma *pair-comb-id-reconstructs*:
assumes *A-car*: $A \in carrier\text{-}mat\ n\ n$
shows *pair-comb* $(\lambda X.\ X)\ A\ n\ n\ (mat\text{-}unit\text{-}pairs\ n) = A$
⟨*proof*⟩

For a linear map $f : M_n \to M_d$, applying f to A equals the pair combination of f evaluated on the basis.

lemma *linear-eq-pair-comb*:
fixes *f* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *f-add*: $\bigwedge X\ Y.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow Y \in carrier\text{-}mat\ n\ n \Longrightarrow$
$f\ (X + Y) = f\ X + f\ Y$
and *f-smult*: $\bigwedge c\ X.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow f\ (c \cdot_m X) = c \cdot_m f\ X$
and *f-car*: $\bigwedge X.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow f\ X \in carrier\text{-}mat\ d\ d$
and *A-car*: $A \in carrier\text{-}mat\ n\ n$
and *npos*: $n > 0$ **and** *dpos*: $d > 0$
shows $f\ A = pair\text{-}comb\ f\ A\ n\ d\ (mat\text{-}unit\text{-}pairs\ n)$
⟨*proof*⟩

Conjugation $W\ E_{ab}\ W^\dagger$ has entry $W_{ia} \cdot^\dagger W_{jb}$.

lemma *conjugation-mat-unit-entry*:
assumes *Wc*: $W \in carrier\text{-}mat\ d\ n$
and *a-lt*: $a < n$ **and** *b-lt*: $b < n$
and *i-lt*: $i < d$ **and** *j-lt*: $j < d$
shows $(W * mat\text{-}unit\ a\ b\ n\ n * (W^\dagger))\ \$\$\ (i,\ j) =$
$W\ \$\$\ (i,\ a) * cnj\ (W\ \$\$\ (j,\ b))$
⟨*proof*⟩

Entry-wise formula for *kraus-map-dim* as a sum over the list.

lemma *kraus-map-dim-entry-sum-list*:
assumes *Ws-c*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and *rho-c*: $\varrho \in carrier\text{-}mat\ n\ n$
and *i-lt*: $i < d$ **and** *j-lt*: $j < d$
shows *kraus-map-dim d Ws* ϱ $\$\$\ (i,\ j) =$
$sum\text{-}list\ (map\ (\lambda W.\ (W * \varrho * (W^\dagger))\ \$\$\ (i,\ j))\ Ws)$
⟨*proof*⟩

Kraus representation extracted from a psd Choi matrix: reshape each column of a psd factorization of P_Φ into a Kraus operator, and the resulting list agrees with Φ on all of M_n.

lemma *kraus-rep-of-psd-choi*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow B \in carrier\text{-}mat\ n\ n \Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall\, A.\ A \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ A \in carrier\text{-}mat\ d\ d$
and *npos*: $n > 0$
and *dpos*: $d > 0$
and *psdP*: $psd\ (choi\text{-}matrix\ n\ d\ \Phi)$
shows $\exists\, Ws$ :: *complex Matrix.mat list*.

*length Ws = n * d ∧*
(∀ W ∈ set Ws. W ∈ carrier-mat d n) ∧
(∀ A. A ∈ carrier-mat n n ⟶ Φ A = kraus-map-dim d Ws A)
⟨*proof*⟩

theorem *choi-theorem*:
fixes Φ :: *complex Matrix.mat ⇒ complex Matrix.mat*
assumes *linear*: ⋀*A B. A ∈ carrier-mat n n ⟹ B ∈ carrier-mat n n ⟹*
Φ (A + B) = Φ A + Φ B
and *smult*: ⋀*c A. A ∈ carrier-mat n n ⟹ Φ (c ·$_m$ A) = c ·$_m$ Φ A*
and *range*: *∀ A. A ∈ carrier-mat n n ⟶ Φ A ∈ carrier-mat d d*
and *npos*: *n > 0*
and *dpos*: *d > 0*
shows *completely-positive n d Φ ⟷*
psd (choi-matrix n d Φ)
⟨*proof*⟩

Accumulated $\sum_i\ {W_i}^\dagger\ W_i$ over a Kraus list.

definition *sum-wdag-w :: nat ⇒ complex Matrix.mat list ⇒ complex Matrix.mat* **where**
sum-wdag-w n Ws ≡
*foldr (λW acc. (W†) * W + acc) Ws (0$_m$ n n)*

lemma *sum-wdag-w-Nil* [*simp*]: *sum-wdag-w n [] = 0$_m$ n n*
⟨*proof*⟩

lemma *sum-wdag-w-Cons*:
*sum-wdag-w n (W # Ws) = (W†) * W + sum-wdag-w n Ws*
⟨*proof*⟩

lemma *sum-wdag-w-carrier*:
assumes *∀ W ∈ set Ws. W ∈ carrier-mat d n*
shows *sum-wdag-w n Ws ∈ carrier-mat n n*
⟨*proof*⟩

lemma *kraus-operators-altdef*:
kraus-operators n d Ws ⟷
(∀ W ∈ set Ws. W ∈ carrier-mat d n) ∧ sum-wdag-w n Ws = 1$_m$ n
⟨*proof*⟩

lemma *Tr-kraus-map-dim-as-wdagw*:
fixes *ϱ :: complex Matrix.mat*
assumes *Ws-c*: *∀ W ∈ set Ws. W ∈ carrier-mat d n*
and *X-c*: *ϱ ∈ carrier-mat n n*
shows *Tr (kraus-map-dim d Ws ϱ) = Tr (sum-wdag-w n Ws * ϱ)*
⟨*proof*⟩

The $\sum_i\ {W_i}^\dagger\ W_i$ matrix entry equals a sum-list of $cnj(W_{ika})\cdot W_{ikb}$ contributions (needed to recover 1_n from trace preservation).

lemma *sum-wdag-w-entry*:
assumes *Ws-c*: *∀ W ∈ set Ws. W ∈ carrier-mat d n*
and *a-lt*: *a < n* **and** *b-lt*: *b < n*
shows *sum-wdag-w n Ws $$ (a, b) =*
*sum-list (map (λW. ((W†) * W) $$ (a, b)) Ws)*
⟨*proof*⟩

Pairing with E_{ab} extracts the (b,a) entry under the trace.

lemma *Tr-mult-mat-unit*:

assumes *M-c*: $M \in$ *carrier-mat n n*
and *a-lt*: $a < n$ **and** *b-lt*: $b < n$
shows *Tr* $(M *$ *mat-unit a b n n*$) = M$ \$\$ (b, a)
⟨*proof*⟩

theorem *choi-kraus-cptp*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n* $\Longrightarrow$
Φ $(A + B) =$ Φ $A +$ Φ B
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow$ Φ $(c \cdot_m A) = c \cdot_m$ Φ A
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow$ Φ $A \in$ *carrier-mat d d*
and *npos*: $n > 0$
and *dpos*: $d > 0$
shows (*completely-positive n d* Φ $\wedge$ *trace-preserving n* Φ) $\longleftrightarrow$
($\exists$ *Ws* :: *complex Matrix.mat list.*
kraus-operators n d Ws $\wedge$
$(\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow$ Φ $X =$ *kraus-map-dim d Ws X*))
⟨*proof*⟩

16 Day 20 (12 October 2011): positive functionals, adjoints, Choi rank

Proposition (notes). For linear $\phi : M_n \to \mathbb{C}$: ϕ positive $\Leftrightarrow$ ϕ completely positive $\Leftrightarrow$ $(\phi(E_{ij})) \in M_n^+$.

Proposition (notes). Let $\Phi : M_n \to M_d$ with Hilbert–Schmidt adjoint $\Phi^* : M_d \to M_n$ ($\langle Y|X\rangle = \mathrm{tr}(Y^*X)$). (1) If $\Phi(X) = \sum_i A_i X A_i^*$ then $\Phi^*(Y) = \sum_i A_i^* Y A_i$. (2) Φ is completely positive iff Φ^* is. (3) Φ is unital and completely positive iff Φ^* is completely positive and trace preserving.

Definition (notes). The *Choi rank* of a completely positive Φ is $\mathrm{cr}(\Phi) = \min\{L : \Phi(X) = \sum_{l=1}^{L} B_l X B_l^*\}$.

Theorem (notes) (Choi)**.** $\mathrm{cr}(\Phi) = \mathrm{rank}(P_\Phi)$.

The functional proposition, the adjoint proposition (for Kraus maps) and the definition of the Choi rank are formalised below; the rank theorem is *choi-rank-eq-rank* in *Day20-Choi-Rank* (its lemma $\mathrm{rank}((E_{ij})) = 1$ is not stated separately), and the bound $\mathrm{cr}(\Phi) \le nd$ is below.

16.1 The adjoint of a Kraus map

For $\Phi(X) = \sum_i A_i X A_i^\dagger$ with $A_i \in M_{d,n}$, the adjoint is $\Phi^*(Y) = \sum_i A_i^\dagger Y A_i$ (*kraus-adjoint-dim n As*; the output dimension n is kept explicit for the empty list).

definition *kraus-adjoint-dim* ::
nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
kraus-adjoint-dim n As y ≡
foldr $(\lambda A\ acc.\ (A^\dagger) * y * A + acc)$ *As* $(0_m\ n\ n)$

lemma *kraus-adjoint-dim-Nil* [*simp*]:
fixes *Y* :: *complex Matrix.mat*
shows *kraus-adjoint-dim n* [] *Y* $= 0_m\ n\ n$
⟨*proof*⟩

lemma *kraus-adjoint-dim-Cons* [*simp*]:
fixes *Y* :: *complex Matrix.mat*

shows *kraus-adjoint-dim n (A # As) Y =*
*(A†) * Y * A + kraus-adjoint-dim n As Y*
⟨*proof*⟩

lemma *kraus-adjoint-dim-carrier*:
fixes *Y* :: *complex Matrix.mat*
assumes *As-c*: ∀ *A* ∈ *set As. A* ∈ *carrier-mat d n*
and *Y-c*: *Y* ∈ *carrier-mat d d*
shows *kraus-adjoint-dim n As Y* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *kraus-adjoint-dim-as-kraus-map-dim*:
fixes *Y* :: *complex Matrix.mat*
assumes *As-c*: ∀ *A* ∈ *set As. A* ∈ *carrier-mat d n*
and *Y-c*: *Y* ∈ *carrier-mat d d*
shows *kraus-adjoint-dim n As Y =*
kraus-map-dim n (map (λA. (A†)) As) Y
⟨*proof*⟩

The adjoint identity $\langle Y|\Phi(X)\rangle = \langle \Phi^*(Y)|X\rangle$, by induction on the Kraus list and cyclicity of the trace.

lemma *hs-inner-prod-right-zero*:
⟨⟨ *y* | 0_m *m n* ⟩⟩ = *0*
⟨*proof*⟩

lemma *hs-inner-prod-left-zero*:
fixes *X* :: *complex Matrix.mat*
assumes *X-c*: *X* ∈ *carrier-mat m n*
shows ⟨⟨ 0_m *m n* | *X* ⟩⟩ = *0*
⟨*proof*⟩

lemma *hs-inner-prod-kraus-term*:
fixes *A X Y* :: *complex Matrix.mat*
assumes *A-c*: *A* ∈ *carrier-mat d n*
and *X-c*: *X* ∈ *carrier-mat n n*
and *Y-c*: *Y* ∈ *carrier-mat d d*
shows ⟨⟨ *Y* | *A* * *X* * (*A*†) ⟩⟩ = ⟨⟨ (*A*†) * *Y* * *A* | *X* ⟩⟩
⟨*proof*⟩

theorem *kraus-adjoint-correct*:
fixes *As* :: *complex Matrix.mat list*
and *X* :: *complex Matrix.mat*
and *Y* :: *complex Matrix.mat*
assumes *As-c*: ∀ *A* ∈ *set As. A* ∈ *carrier-mat d n*
and *X-c*: *X* ∈ *carrier-mat n n*
and *Y-c*: *Y* ∈ *carrier-mat d d*
shows ⟨⟨ *Y* | *kraus-map-dim d As X* ⟩⟩
= ⟨⟨ *kraus-adjoint-dim n As Y* | *X* ⟩⟩
⟨*proof*⟩

16.2 Complete positivity and the UCP/CPTP duality

Φ^* is the Kraus map of the adjoint operators, hence completely positive, and Φ is unital iff Φ^* is trace preserving: $\sum_i A_i A_i^* = I$ in both cases, using that $\mathrm{tr}(BY) = \mathrm{tr}(Y)$ for all Y forces $B = I$.

theorem *adjoint-cp-from-kraus*:

assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive d n* (*kraus-adjoint-dim n As*)
⟨*proof*⟩

lemma *kraus-map-dim-one-eq-sum-wdag-w-dagger*:
fixes *As* :: *complex Matrix.mat list*
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
shows *kraus-map-dim d As* (1_m *n*)
= *sum-wdag-w d* (*map* (λA. ($A^\dagger$)) *As*)
⟨*proof*⟩

theorem *cp-unital-iff-adjoint-tp*:
assumes *As-c*: $\forall A \in$ *set As*. $A \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *unital-map n d* (*kraus-map-dim d As*)
$\longleftrightarrow$ *trace-preserving d* (*kraus-adjoint-dim n As*)
⟨*proof*⟩

16.3 Positive functionals are completely positive

The case $d = 1$ of Choi's theorem: for $P = (P_{ij}) \in M_r(M_n)^+$ and $v = (\alpha_i) \in \mathbb{C}^r$, $\langle v|(\phi(P_{ij}))v\rangle = \phi(\sum_{ij} \overline{\alpha_i}\alpha_j P_{ij}) = \phi(Y^*PY) \geq 0$ with Y the block column $(\alpha_i I)_i$.

lemma *phi-entry-double-sum*:
fixes φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *f-add*: $\bigwedge X\ Y.\ X \in$ *carrier-mat n n* $\Longrightarrow$ $Y \in$ *carrier-mat n n*
$\Longrightarrow \varphi\ (X + Y) = \varphi\ X + \varphi\ Y$
and *f-smult*: $\bigwedge c\ X.\ X \in$ *carrier-mat n n* $\Longrightarrow \varphi\ (c \cdot_m X) = c \cdot_m \varphi\ X$
and *f-car*: $\bigwedge X.\ X \in$ *carrier-mat n n* $\Longrightarrow \varphi\ X \in$ *carrier-mat d d*
and *M-c*: $M \in$ *carrier-mat n n*
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\varphi\ M$ \$\$ $(r, c) =$
$(\sum i{<}n.\ \sum j{<}n.\ M$ \$\$ $(i, j) * \varphi$ (*mat-unit i j n n*) \$\$ $(r, c))$
⟨*proof*⟩

theorem *positive-scalar-is-cp*:
fixes φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n*
$\Longrightarrow \varphi\ (A + B) = \varphi\ A + \varphi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \varphi\ (c \cdot_m A) = c \cdot_m \varphi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \varphi\ A \in$ *carrier-mat 1 1*
and *pos*: *positive-map n 1* φ
and *npos*: $0 < n$
shows *completely-positive n 1* φ
⟨*proof*⟩

16.4 Choi rank

choi-rank n d Φ is the least length of a Kraus list representing Φ on the carrier. The theorem $\mathrm{cr}(\Phi) = \mathrm{rank}(P_\Phi)$ is *choi-rank-eq-rank* in *Day20-Choi-Rank*; the bound $\mathrm{cr}(\Phi) \leq nd$ follows from the Kraus-rank bound of Day 21.

definition *choi-rank* ::

nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *nat* **where**
choi-rank n d Φ ≡
Inf { *length As* | *As* :: *complex Matrix.mat list*.
(∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n*)
∧ (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d As X*) }

17 Day 21 (14 October 2011): the Kraus rank bound; unitary freedom of representations

Corollary (notes). A completely positive $\Phi : M_n \to M_d$ has a Kraus representation $\Phi(X) = \sum_{l=1}^{K} A_l X A_l^*$ with $K \le nd$.

The notes then record a conjecture (every CP map is an average of n UCP maps of Choi rank $\le d$) and develop the uniqueness theory:

Lemma (notes). If $T : \mathbb{C}^r \to \mathbb{C}^m$ is one-to-one then T^* is onto.

Proposition (notes). If $v_1, \dots, v_r \in \mathbb{C}^k$ are linearly independent then so are $\{v_i v_j^*\}_{i,j} \subseteq M_k$.

Lemma (notes). If $0 \le ww^* \le P$ then $w \in \mathrm{range}(P)$; hence for $P = \sum_l w_l w_l^* \ge 0$, $\mathrm{range}(P) = \mathrm{span}\{w_l\}$.

Theorem (notes). If $P = \sum_{j=1}^{r} v_j v_j^* = \sum_{l=1}^{m} w_l w_l^*$ with $\mathrm{rank}\, P = r$, there is $U = (\alpha_{ij})_{m \times r}$ with $w_i = \sum_j \alpha_{ij} v_j$ and $U^*U = I_r$.

Theorem (notes) (Choi 2)**.** If Φ is completely positive with $\mathrm{cr}(\Phi) = r$ and $\Phi(X) = \sum_{j=1}^{r} V_j X V_j^* = \sum_{l=1}^{m} W_l X W_l^*$, then $W_i = \sum_j \alpha_{ij} V_j$ for a unique $U = (\alpha_{ij})$ with $U^*U = I_r$.

Only the Kraus rank bound is formalised here. The kernel form of the lemma ($\ker A \subseteq \ker B$ when $0 \le B \le A$) is proved in theory Day23 and used there to prove Choi's Theorem 3 without the unitary-freedom theorem; the remaining statements are audit items D21-3, D21-5, D21-6.

corollary *kraus-rank-bound*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *linear*: ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n*
⟹ Φ (*A* + *B*) = Φ *A* + Φ *B*
and *smult*: ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*
and *range*: ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: *0* < *n*
and *dpos*: *0* < *d*
shows ∃ *As* :: *complex Matrix.mat list*.
length As ≤ *n* ∗ *d* ∧
(∀ *A* ∈ *set As*. *A* ∈ *carrier-mat d n*) ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d As X*)
⟨*proof*⟩

Dimension bookkeeping for $T^\dagger$ (the onto statement is not formalised).

lemma *adjoint-image-in-dom*:
fixes *T* :: *complex Matrix.mat*
assumes *sq*: *T* ∈ *carrier-mat m r*
shows ∀ *w* :: *complex Matrix.vec*. *dim-vec w* = *m*
⟶ *dim-vec* (*T*† ∗$_v$ *w*) = *r*
⟨*proof*⟩

$\mathrm{cr}(\Phi) \le nd$: the defining set of the Choi rank is non-empty.

corollary *choi-rank-le*:

fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n*
$\Longrightarrow \Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *choi-rank n d* $\Phi \leq n * d$
⟨*proof*⟩

18 Day 22 (17 October 2011): minimal Kraus representations, convex sets, UCP and CPTP

Proposition (notes). If Φ is completely positive, $\mathrm{cr}(\Phi) = r$ and $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$, then $V_1, \ldots, V_r$ are linearly independent.

(Not formalised: on Day 23 linear independence of the Kraus operators is a hypothesis.)

Definition (notes). $C \subseteq V$ is *convex* if $tv_1 + (1-t)v_2 \in C$ for $v_1, v_2 \in C$, $0 \leq t \leq 1$; $v \in C$ is *extreme* if $v = \frac{1}{2}(v_1 + v_2)$ with $v_1, v_2 \in C$ forces $v = v_1 = v_2$. The *convex hull* of E is the set of convex combinations of points of E; a map between convex sets is *affine* if it preserves convex combinations.

Theorem (notes) (Krein–Milman)**.** A compact convex subset of a finite-dimensional real vector space is the convex hull of its extreme points.

Examples: $UCP(M_n, M_d)$, the unital completely positive maps, is convex ($P_{t\Phi_1+(1-t)\Phi_2} = tP_{\Phi_1} + (1-t)P_{\Phi_2} \geq 0$) and compact (closed; bounded because $0 \leq \Phi(E_{ii}) \leq I$ gives $|r_{ij}| \leq 1$ for the entries of P_Φ); $CPTP(M_d, M_n)$ is convex and compact, being the image of $UCP(M_n, M_d)$ under the affine bijection $\Gamma(\Phi) = \Phi^*$.

Theorem (notes) (Choi 3)**.** If $\Phi : M_n \to M_d$ is unital and completely positive, $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$ with $r = \mathrm{cr}(\Phi)$, then Φ is an extreme point of $UCP(M_n, M_d)$ iff $\{V_i^* V_j : 1 \leq i, j \leq r\}$ is linearly independent in M_d.

Corollary (notes). If Φ is extreme then $\mathrm{cr}(\Phi) \leq d$. If $\Phi : M_d \to M_n$ is CPTP with $\Phi(X) = \sum_{i=1}^{r} V_i X V_i^*$, $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $CPTP(M_d, M_n)$ iff $\{V_i^* V_j\}$ is linearly independent.

Convexity of UCP and CPTP is proved below; compactness and the affine bijection are not formalised; the corollary $\mathrm{cr}(\Phi) \leq d$ is *extreme-CPTP-choi-rank-le* in *Day22-Kraus-Independence*, and Choi's Theorem 3 is proved in theory Day23 in its CPTP form. Krein–Milman is available in HOL-Analysis (*Krein-Milman-Minkowski*).

18.1 Convex sets and extreme points

definition *convex-set-mat* :: *complex Matrix.mat set* $\Rightarrow$ *bool* **where**
convex-set-mat C $\equiv$
$\forall A \in C.\ \forall B \in C.\ \forall t$ :: *real*. $0 \leq t \wedge t \leq 1 \longrightarrow$
complex-of-real t $\cdot_m$ *A*
$+$ (*1* $-$ *complex-of-real t*) $\cdot_m$ *B* $\in$ *C*

definition *extreme-point-mat* ::
complex Matrix.mat set $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *bool* **where**
extreme-point-mat C v $\equiv$

$v \in C \wedge$
$\neg\,(\exists\, v1 \in C.\ \exists\, v2 \in C.\ v1 \neq v \wedge v \neq v2 \wedge$
$v = (1/2 :: complex) \cdot_m v1 + (1/2 :: complex) \cdot_m v2)$

definition *affine-map-mat* ::
(*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
affine-map-mat tmap ≡
$\forall\, A\ B.\ \forall\, t :: real.\ 0 \leq t \wedge t \leq 1 \longrightarrow$
tmap (*complex-of-real t* $\cdot_m$ *A*
$+ (1 - complex\text{-}of\text{-}real\ t) \cdot_m B) =$
complex-of-real t $\cdot_m$ *tmap A*
$+ (1 - complex\text{-}of\text{-}real\ t) \cdot_m tmap\ B$

18.2 The convex sets UCP and CPTP

definition *UCP* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* **where**
UCP n d ≡ {Φ. *completely-positive n d* Φ ∧ *unital-map n d* Φ}

definition *CPTP* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) *set* **where**
CPTP n d ≡ {Φ. *completely-positive n d* Φ ∧ *trace-preserving n* Φ}

lemma *amplification-smult*:
assumes *dpos*: $0 < d$
and *F-block*: $\bigwedge a\ b.\ F\ (block\text{-}of\text{-}mat\ n\ x\ a\ b) \in carrier\text{-}mat\ d\ d$
shows *amplification n d p* ($\lambda\varrho.\ c \cdot_m F\ \varrho$) *x*
$= c \cdot_m amplification\ n\ d\ p\ F\ x$
⟨*proof*⟩

UCP is convex. The range hypotheses are explicit because *completely-positive* does not constrain the values of Φ_i off the carrier.

lemma *UCP-convex*:
fixes *t* :: *real*
assumes *t-nn*: $0 \leq t$
and *t-le*: $t \leq 1$
and Φ*1-in*: Φ*1* ∈ *UCP n d*
and Φ*2-in*: Φ*2* ∈ *UCP n d*
and *r1*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*1 X* ∈ *carrier-mat d d*
and *r2*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*2 X* ∈ *carrier-mat d d*
and *dpos*: $0 < d$
shows ($\lambda X.$ *complex-of-real t* $\cdot_m$ Φ*1 X*
$+ (1 - complex\text{-}of\text{-}real\ t) \cdot_m$ Φ*2 X*) ∈ *UCP n d*
⟨*proof*⟩

CPTP is convex.

lemma *CPTP-convex*:
fixes *t* :: *real*
assumes *t-nn*: $0 \leq t$
and *t-le*: $t \leq 1$
and Φ*1-in*: Φ*1* ∈ *CPTP n d*
and Φ*2-in*: Φ*2* ∈ *CPTP n d*
and *r1*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*1 X* ∈ *carrier-mat d d*
and *r2*: ∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ*2 X* ∈ *carrier-mat d d*
and *dpos*: $0 < d$
shows ($\lambda X.$ *complex-of-real t* $\cdot_m$ Φ*1 X*

$+ (1 - \textit{complex-of-real } t) \cdot_m \Phi 2\ X) \in \textit{CPTP } n\ d$
⟨*proof*⟩

19 Day 29 (7 November 2011): classical error detection and correction

Days 23–28 are formalised in the theories Day23_ChoiExtremeUCP, Separable_Cone and Day24_Separability.

To transmit $v \in \mathbb{Z}_2^n$ through a channel that may flip bits (+1 in $\mathbb{Z}_2$), one encodes $v \mapsto \phi(v) \in \mathbb{Z}_2^{n+k}$, transmits, and decodes. Examples: repetition $v \mapsto (v, v)$ detects one error but corrects none and doubles the cost; the parity check $(a_1, \ldots, a_n) \mapsto (a_1, \ldots, a_n; a_1 + \cdots + a_n)$ detects one error cheaply; the Hamming $[7, 1, 3]$ code detects and corrects one error (theory Day29_Classical_Codes: the code as written detects one error but cannot correct it; the $[7, 4, 3]$ code corrects one).

Definition (notes). The *Hamming weight* $\|w\|$ of $w \in \mathbb{Z}_2^m$ is its number of non-zero entries; the weight of a code $S \subseteq \mathbb{Z}_2^m$ is $\min\{\|v - w\| : v \neq w \in S\}$, the least number of errors turning one code word into another; for a subspace code it is $\min\{\|v\| : 0 \neq v \in S\}$.

Decoding amounts to choosing a left inverse of ϕ (different left inverses correct differently); the best binary codes are cyclic codes, ideals in $\mathbb{Z}_2[x]/\langle x^m - 1\rangle$ (not formalised).

19.1 Binary strings, Hamming weight and distance

definition *hamming-weight* :: *nat list* ⇒ *nat* **where**
hamming-weight v ≡ *length* (*filter* (λ*b*. *b* ≠ *0*) *v*)

definition *hamming-dist* :: *nat list* ⇒ *nat list* ⇒ *nat* **where**
hamming-dist v w ≡ *hamming-weight* (*map2* (λ*a b*. (*a* + *b*) *mod 2*) *v w*)

lemma *hamming-weight-Nil* [*simp*]: *hamming-weight* [] = *0*
⟨*proof*⟩

lemma *hamming-weight-Cons*:
hamming-weight (*a* # *v*)
= (*if a* ≠ *0 then Suc* (*hamming-weight v*) *else hamming-weight v*)
⟨*proof*⟩

lemma *hamming-dist-Nil* [*simp*]:
hamming-dist [] [] = *0*
⟨*proof*⟩

lemma *hamming-dist-Cons*:
hamming-dist (*a* # *v*) (*b* # *w*)
= (*if* (*a* + *b*) *mod 2* ≠ *0*
then Suc (*hamming-dist v w*)
else hamming-dist v w)
⟨*proof*⟩

lemma *hamming-dist-symm*:
assumes *length v* = *length w*
shows *hamming-dist v w* = *hamming-dist w v*
⟨*proof*⟩

lemma *hamming-dist-zero*:

assumes *lenw*: *length v = length w*
and *bits-v*: $\forall x \in set\ v.\ x < 2$
and *bits-w*: $\forall x \in set\ w.\ x < 2$
shows *hamming-dist v w = 0* $\longleftrightarrow$ *v = w*
⟨*proof*⟩

lemma *hamming-dist-self* [*simp*]:
hamming-dist v v = 0
⟨*proof*⟩

19.2 Code distance

A code of distance at least $2t+1$ can correct t errors (recorded as the definition *corrects-errors*; the decoding theorem itself is not formalised).

definition *code-distance* :: *nat list set* ⇒ *nat* **where**
code-distance code ≡
if ($\exists v \in code.\ \exists w \in code.\ v \neq w$)
then Min {*hamming-dist v w* | *v w*. *v* ∈ *code* ∧ *w* ∈ *code* ∧ *v* ≠ *w*}
else 0

definition *corrects-errors* :: *nat list set* ⇒ *nat* ⇒ *bool* **where**
corrects-errors code t ≡ *code-distance code* ≥ *2*t + 1*

19.3 The parity-check and repetition codes

Parity check: append the sum of the bits; any single error changes the code word. Repetition doubles Hamming distances.

definition *parity-bit* :: *nat list* ⇒ *nat* **where**
parity-bit v ≡ *foldr* (λ*a b*. (*a* + *b*) *mod 2*) *v 0*

definition *parity-check-encode* :: *nat list* ⇒ *nat list* **where**
parity-check-encode v ≡ *v* @ [*parity-bit v*]

lemma *length-parity-check-encode* [*simp*]:
length (*parity-check-encode v*) = *length v* + *1*
⟨*proof*⟩

lemma *hamming-weight-pos-iff*:
0 < *hamming-weight v* $\longleftrightarrow$ ($\exists x \in set\ v.\ x \neq 0$)
⟨*proof*⟩

lemma *hamming-dist-pos-iff*:
assumes *lenw*: *length v = length w*
and *bits-v*: $\forall x \in set\ v.\ x < 2$
and *bits-w*: $\forall x \in set\ w.\ x < 2$
shows *0* < *hamming-dist v w* $\longleftrightarrow$ *v* ≠ *w*
⟨*proof*⟩

lemma *parity-bit-lt-2*: *parity-bit v* < *2*
⟨*proof*⟩

lemma *parity-check-encode-binary*:
assumes $\forall x \in set\ v.\ x < 2$
shows $\forall x \in set$ (*parity-check-encode v*). $x < 2$
⟨*proof*⟩

lemma *parity-check-detects-one-error*:
assumes *lenE*: *length e = length v + 1*
and *bits-v*: $\forall x \in$ *set v*. *x < 2*
and *bits-e*: $\forall x \in$ *set e*. *x < 2*
and *weight*: *hamming-weight e = 1*
shows *parity-check-encode v*
$\neq$ *map2* ($\lambda a\ b$. (*a + b*) *mod 2*) (*parity-check-encode v*) *e*
⟨*proof*⟩

definition *repetition-encode* :: *nat list* ⇒ *nat list* **where**
repetition-encode v ≡ *v* @ *v*

lemma *length-repetition-encode* [*simp*]:
length (*repetition-encode v*) = *2* ∗ *length v*
⟨*proof*⟩

lemma *hamming-dist-repetition-doubles*:
assumes *length v = length w*
shows *hamming-dist* (*repetition-encode v*) (*repetition-encode w*)
= *2* ∗ *hamming-dist v w*
⟨*proof*⟩

20 Day 30 (9 November 2011): error probabilities; the three-qubit bit-flip code

If a bit flips with probability p (independently), the repetition code on three bits leaves an error undetected with probability $3p^2(1-p)^4 + 3p^4(1-p)^2 + p^6$, the parity code with probability $\binom{4}{2}p^2(1-p)^2 + p^4$ (the former is smaller for $p < \frac{1}{2}$), and the majority-rule code $0 \mapsto 000$, $1 \mapsto 111$ decodes wrongly with probability $3p^2(1-p)+p^3$ (theory Day29_Classical_Codes; the comparison holds for all $0 < p < 1$).

Quantum error correction: one cannot clone, but basis vectors can be copied and measurements can decode. With $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ the allowed errors are qubit flips $X \otimes I \otimes I$, $I \otimes X \otimes I$, ... The *three-qubit bit-flip code* encodes $a|0\rangle + b|1\rangle \mapsto a|000\rangle + b|111\rangle$ (two CNOTs). Decoding: (1) syndrome measurement with $P_0 = |000\rangle\langle 000| + |111\rangle\langle 111|$, $P_1 = |100\rangle\langle 100| + |011\rangle\langle 011|$, $P_2 = |010\rangle\langle 010| + |101\rangle\langle 101|$, $P_3 = |001\rangle\langle 001| + |110\rangle\langle 110|$, $\sum P_i^2 = I$, which leaves a state with at most one flip unchanged and reveals where the flip occurred; (2) recovery $R(Y) = P_0 Y P_0 + (X \otimes I \otimes I) P_1 Y P_1 (X \otimes I \otimes I) + \cdots$, a completely positive trace-preserving map with $R(|\psi_i\rangle\langle\psi_i|) = |\psi\rangle\langle\psi|$ after zero or one error. The code's recovery theorem is *bit-flip-code-recovery* in *Day30-BitFlip-Code* (syndrome projections, $\sum P_i = I$, R CPTP); the Pauli matrices and the bit-flip channel are below.

20.1 The Pauli matrices

The notes use two sign conventions for Y: $\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ on Day 10 (the library's Y) and $\begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}$ on Day 32. *Pauli-Y* follows Day 32; both are Hermitian unitary involutions, and the Pauli identities below carry the corresponding signs. *Pauli-X* and *Pauli-Z* are the library's *Quantum.X* and *Quantum.Z*.

definition *Pauli-X* :: *complex Matrix.mat* **where**
Pauli-X ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if* (*i = 0* ∧ *j = 1*) ∨ (*i = 1* ∧ *j = 0*) *then 1 else 0*)

definition *Pauli-Y* :: *complex Matrix.mat* **where**
Pauli-Y ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if i = 0 ∧ j = 1 then* i
else if i = 1 ∧ j = 0 then −i *else 0*)

definition *Pauli-Z* :: *complex Matrix.mat* **where**
Pauli-Z ≡ *Matrix.mat 2 2*
(λ(*i*,*j*). *if i = 0 ∧ j = 0 then 1*
else if i = 1 ∧ j = 1 then −1 else 0)

lemma *Pauli-X-carrier* [*simp*]: *Pauli-X* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-Y-carrier* [*simp*]: *Pauli-Y* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-Z-carrier* [*simp*]: *Pauli-Z* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *Pauli-X-dagger*: *Pauli-X*† = *Pauli-X*
⟨*proof*⟩

lemma *Pauli-X-squared*: *Pauli-X* ∗ *Pauli-X* = *1*$_m$ *2*
⟨*proof*⟩

lemma *Pauli-X-unitary*: *unitary Pauli-X*
⟨*proof*⟩

lemma *Pauli-Y-dagger*: *Pauli-Y*† = *Pauli-Y*
⟨*proof*⟩

lemma *Pauli-Y-squared*: *Pauli-Y* ∗ *Pauli-Y* = *1*$_m$ *2*
⟨*proof*⟩

lemma *Pauli-Y-unitary*: *unitary Pauli-Y*
⟨*proof*⟩

lemma *Pauli-Z-dagger*: *Pauli-Z*† = *Pauli-Z*
⟨*proof*⟩

lemma *Pauli-Z-squared*: *Pauli-Z* ∗ *Pauli-Z* = *1*$_m$ *2*
⟨*proof*⟩

lemma *Pauli-Z-unitary*: *unitary Pauli-Z*
⟨*proof*⟩

20.2 Pauli magic (Day 32)

Proposition (notes). I, X, Y, Z are orthogonal in M_2 and all have 2-norm $\sqrt{2}$.

proposition *pauli-orthogonal*:
shows ⟨⟨ *1*$_m$ *2* | *Pauli-X* ⟩⟩ = *0*
and ⟨⟨ *1*$_m$ *2* | *Pauli-Y* ⟩⟩ = *0*
and ⟨⟨ *1*$_m$ *2* | *Pauli-Z* ⟩⟩ = *0*
and ⟨⟨ *Pauli-X* | *Pauli-Y* ⟩⟩ = *0*
and ⟨⟨ *Pauli-X* | *Pauli-Z* ⟩⟩ = *0*
and ⟨⟨ *Pauli-Y* | *Pauli-Z* ⟩⟩ = *0*

⟨*proof*⟩

proposition *pauli-norm*:
  shows ⟨⟨ 1_m 2 | (1_m 2 :: *complex Matrix.mat*) ⟩⟩ = 2
    and ⟨⟨ *Pauli-X* | *Pauli-X* ⟩⟩ = 2
    and ⟨⟨ *Pauli-Y* | *Pauli-Y* ⟩⟩ = 2
    and ⟨⟨ *Pauli-Z* | *Pauli-Z* ⟩⟩ = 2
⟨*proof*⟩

20.3 The bit-flip channel (Day 31)

Doing nothing to ψ while a flip occurs with probability p produces the ensemble $\{1-p, \psi\}, \{p, X\psi\}$, i.e. the error map $E(Y) = (1-p)Y + pXYX^*$, which is completely positive and trace preserving.

definition *bit-flip-channel* ::
  real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
  bit-flip-channel p ϱ ≡
    complex-of-real (1 − *p*) ·$_m$ *ϱ*
    + *complex-of-real p* ·$_m$ (*Pauli-X* ∗ *ϱ* ∗ (*Pauli-X*†))

lemma *bit-flip-channel-carrier*:
  assumes *ϱ* ∈ *carrier-mat 2 2*
  shows *bit-flip-channel p ϱ* ∈ *carrier-mat 2 2*
⟨*proof*⟩

lemma *bit-flip-channel-trace-preserving*:
  fixes *p* :: *real*
  assumes *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
  shows *Tr* (*bit-flip-channel p ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *bit-flip-channel-psd*:
  fixes *p* :: *real*
  assumes *p-nn*: 0 ≤ *p*
    and *p-le-1*: *p* ≤ 1
    and *ϱ-c*: *ϱ* ∈ *carrier-mat 2 2*
    and *ϱ-psd*: *psd ϱ*
  shows *psd* (*bit-flip-channel p ϱ*)
⟨*proof*⟩

21 Day 31 (11 November 2011): the bit-flip code as a channel; fidelity

Encoding $\tilde{\psi} = a|000\rangle + b|111\rangle$ and flipping each qubit independently with probability p gives the error map $E(Y) = (1-p)^3 Y + p(1-p)^2 \sum_j (X_j) Y (X_j)^* + \cdots + p^3 (X \otimes X \otimes X) Y (X \otimes X \otimes X)^*$, and with the recovery R of Day 30,

$$R \circ E(Y) = \big((1-p)^3 + 3p(1-p)^2\big) Y + \big(3p^2(1-p) + p^3\big)(X \otimes X \otimes X) Y (X \otimes X \otimes X).$$

The code helps iff $(1-p)^3 + 3p(1-p)^2 \geq 1-p$ iff $p \leq \frac{1}{2}$ (theory Day31_Code_Fidelity, where the formula for $R \circ E$ holds for Y supported on the code space). Below: the encoding isometry.

definition *encode-mat* :: *complex Matrix.mat* **where**
  encode-mat ≡ *Matrix.mat 8 2* (λ(*i*,*j*).
    if i = 0 ∧ *j* = 0 *then* 1

else if $i = 7 \wedge j = 1$ *then* 1
else 0)

lemma *encode-mat-carrier* [*simp*]: *encode-mat* ∈ *carrier-mat* 8 2
⟨*proof*⟩

21.1 Fidelity

A state ψ perturbed to $\psi' = a\psi + b\psi^\perp$ is close to ψ when $|a|$ is close to 1, so the *fidelity* of states is $F = |\langle\psi'|\psi\rangle|$; for the density matrices $\rho = |\psi\rangle\langle\psi|$, $\rho' = |\psi'\rangle\langle\psi'|$ one has $\langle\rho|\rho'\rangle = \mathrm{Tr}(\rho\rho') = |\langle\psi'|\psi\rangle|^2$, so $F = \sqrt{\mathrm{Tr}(\rho\rho')}$. Error correction is compared by $F = \sqrt{\langle\psi|R \circ E(|\psi\rangle\langle\psi|)\psi\rangle}$, minimised or averaged over ψ: doing nothing gives $\min F = \sqrt{1-p}$, the bit-flip code gives $\min F = \sqrt{(1-p)^3 + 3p(1-p)^2}$ (theory Day31_Code_Fidelity). Formally *fidelity* ψ ϱ is $\sqrt{\Re\langle\psi|\rho\psi\rangle}$.

definition *fidelity* ::
complex Matrix.vec ⇒ *complex Matrix.mat* ⇒ *real* **where**
fidelity ψ ϱ ≡ *sqrt* (*Re* (⟨ψ | ϱ *$_v$ ψ⟩))

lemma *fidelity-def-eq*:
fidelity ψ ϱ = *sqrt* (*Re* (⟨ψ | ϱ *$_v$ ψ⟩)))
⟨*proof*⟩

lemma *fidelity-pure-states*:
fixes ψ ψ' :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
and ψ'-*qs*: *quantum-state n* ψ'
shows *fidelity* ψ ((*mat-of-cols n* [ψ']) ∗ ((*mat-of-cols n* [ψ'])†))
= *cmod* (⟨ψ | ψ'⟩)
⟨*proof*⟩

lemma *fidelity-self*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows *fidelity* ψ ((*mat-of-cols n* [ψ]) ∗ ((*mat-of-cols n* [ψ])†)) = 1
⟨*proof*⟩

lemma *fidelity-orthogonal*:
fixes ψ ψ' :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
and ψ'-*qs*: *quantum-state n* ψ'
and *ortho*: ⟨ψ | ψ'⟩ = 0
shows *fidelity* ψ ((*mat-of-cols n* [ψ']) ∗ ((*mat-of-cols n* [ψ'])†)) = 0
⟨*proof*⟩

22 Day 32 (14 November 2011): the phase-flip code, the Shor code, Pauli magic

22.1 The phase-flip code

With $Z|0\rangle = |0\rangle$, $Z|1\rangle = -|1\rangle$ and $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$ one has $|+\rangle \perp |-\rangle$, $Z|+\rangle = |-\rangle$, $Z|-\rangle = |+\rangle$: Z acts on $\{|+\rangle, |-\rangle\}$ as the bit flip acts on $\{|0\rangle, |1\rangle\}$. Encoding $|0\rangle \mapsto |++\rangle$, $|1\rangle \mapsto |--\rangle$ gives the three-qubit *phase-flip code*, which detects and corrects phase flips

exactly as the bit-flip code corrects bit flips; with $\beta = Y|+\rangle$ (orthogonal to $|+\rangle$, $Y\beta = |+\rangle$) the same works for Y.

The *Shor code* encodes $|0\rangle \mapsto |0_L\rangle = 2^{-3/2}(|000\rangle + |111\rangle)^{\otimes 3}$ and $|1\rangle \mapsto |1_L\rangle = 2^{-3/2}(|000\rangle - |111\rangle)^{\otimes 3}$ in $\mathbb{C}^{2^9}$.

Theorem (notes) (Shor)**.** There is a completely positive trace-preserving $R : M_{2^9} \to M_{2^9}$ such that for every 2×2 unitary U, every position j, $\tilde{U} = I \otimes \cdots \otimes U \otimes \cdots \otimes I$, and every $\psi = a|0_L\rangle + b|1_L\rangle$, $R(\tilde{U}|\psi\rangle\langle\psi|\tilde{U}^*) = |\psi\rangle\langle\psi|$.

Proposition (notes). If $V \subseteq \mathbb{C}^n$ is a subspace and $U_1, \ldots, U_t$ are unitaries with $U_iV \perp U_jV$ for $i \neq j$, then there is a CPTP R with $R(U_i|\psi\rangle\langle\psi|U_i^*) = |\psi\rangle\langle\psi|$ for all $\psi \in V$ and all i (take $R(X) = \sum_i U_i^* P_i X P_i U_i$ with P_i the projection onto U_iV, $P_0 = I - \sum P_i$, $U_0 = I$).

The $3 \cdot 9 + 1$ *1-Pauli* operators $I \otimes \cdots \otimes U \otimes \cdots \otimes I$ (U a Pauli matrix) map $V = \mathrm{span}\{|0_L\rangle, |1_L\rangle\}$ to mutually orthogonal subspaces (up to the coincidences $Z_1 v = Z_2 v = Z_3 v$ etc.). The proposition is *recovery-single-error* in *Day32-Recovery-Proposition*, together with its linear-combination form *recovery-linear-combination* (Day 33); the Pauli expansion identity is *pauli-expansion-unitary* in *Day32-Pauli-Expansion*. The Shor code and Shor's theorem are *Day33-Shor-Code* (*shor-code-theorem*). As first stated, the orthogonality claim is false ($Z_1 \neq Z_2$ but $Z_1|0_L\rangle = Z_2|0_L\rangle$, *shor-notes-Z1-Z2*); Day 33 corrects it.

definition *ket-plus* :: *complex Matrix.vec* **where**
ket-plus ≡ (*1 / sqrt 2* :: *complex*) ·$_v$ (*unit-vec 2 0 + unit-vec 2 1*)

definition *ket-minus* :: *complex Matrix.vec* **where**
ket-minus ≡ (*1 / sqrt 2* :: *complex*) ·$_v$ (*unit-vec 2 0 − unit-vec 2 1*)

lemma *ket-plus-carrier* [*simp*]: *ket-plus* ∈ *carrier-vec 2*
⟨*proof*⟩

lemma *ket-minus-carrier* [*simp*]: *ket-minus* ∈ *carrier-vec 2*
⟨*proof*⟩

lemma *ket-plus-dim* [*simp*]: *dim-vec ket-plus = 2*
⟨*proof*⟩

lemma *ket-minus-dim* [*simp*]: *dim-vec ket-minus = 2*
⟨*proof*⟩

lemma *ket-plus-index-0*: *ket-plus* $ *0 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-plus-index-1*: *ket-plus* $ *1 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-minus-index-0*: *ket-minus* $ *0 = 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-minus-index-1*: *ket-minus* $ *1 = − 1 / sqrt 2*
⟨*proof*⟩

lemma *ket-plus-unit*: ‖*ket-plus*‖ = *1*
⟨*proof*⟩

lemma *ket-minus-unit*: ‖*ket-minus*‖ = *1*
⟨*proof*⟩

lemma *ket-plus-minus-orthog*: $\langle \textit{ket-plus} \mid \textit{ket-minus} \rangle = 0$
⟨*proof*⟩

lemma *Pauli-Z-ket-plus*: $\textit{Pauli-Z} *_v \textit{ket-plus} = \textit{ket-minus}$
⟨*proof*⟩

lemma *Pauli-Z-ket-minus*: $\textit{Pauli-Z} *_v \textit{ket-minus} = \textit{ket-plus}$
⟨*proof*⟩

lemma *Pauli-X-ket-plus*: $\textit{Pauli-X} *_v \textit{ket-plus} = \textit{ket-plus}$
⟨*proof*⟩

lemma *Pauli-X-ket-minus*: $\textit{Pauli-X} *_v \textit{ket-minus} = - \textit{ket-minus}$
⟨*proof*⟩

The code words $|+++\rangle$ and $|---\rangle$ of the three-qubit phase-flip code (an earlier version of this file called them *phase-code-zero*/*phase-code-one*; the nine-qubit Shor code words are not defined).

definition *phase-code-zero* :: *complex Matrix.vec* **where**
phase-code-zero ≡
let v = *unit-vec 2 0* + *unit-vec 2 1* *in*
(*1* / (*2* ∗ *sqrt 2*) :: *complex*)
$\cdot_v$ *tensor-vec* (*tensor-vec v v*) *v*

definition *phase-code-one* :: *complex Matrix.vec* **where**
phase-code-one ≡
let v = *unit-vec 2 0* − *unit-vec 2 1* *in*
(*1* / (*2* ∗ *sqrt 2*) :: *complex*)
$\cdot_v$ *tensor-vec* (*tensor-vec v v*) *v*

lemma *phase-code-zero-dim* [*simp*]: *dim-vec phase-code-zero* = *8*
⟨*proof*⟩

lemma *phase-code-one-dim* [*simp*]: *dim-vec phase-code-one* = *8*
⟨*proof*⟩

lemma *phase-code-zero-carrier* [*simp*]: *phase-code-zero* ∈ *carrier-vec 8*
⟨*proof*⟩

lemma *phase-code-one-carrier* [*simp*]: *phase-code-one* ∈ *carrier-vec 8*
⟨*proof*⟩

22.2 Projections

A *projection* on $\mathbb{C}^n$ is a Hermitian idempotent, the notion of Day 2; projections are the building blocks of syndrome measurements and recovery maps (Days 30–34). Day 2's theorem that such a P is the orthogonal projection onto its range follows the basic facts.

definition *projection* :: *nat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
projection n P ≡
$P \in$ *carrier-mat n n* ∧ *hermitian P* ∧ $P * P = P$

lemma *projection-carrier*:
projection n P ⟹ $P \in$ *carrier-mat n n*
⟨*proof*⟩

lemma *projection-hermitian*:
projection n P $\Longrightarrow$ *hermitian P*
⟨*proof*⟩

lemma *projection-idempotent*:
projection n P $\Longrightarrow$ $P * P = P$
⟨*proof*⟩

lemma *projection-zero*: *projection n* (0_m *n n*)
⟨*proof*⟩

lemma *projection-one*: *projection n* (1_m *n*)
⟨*proof*⟩

lemma *projection-psd*:
assumes *projection n P*
shows *psd P*
⟨*proof*⟩

The theorem of Day 2: a Hermitian idempotent P is the orthogonal projection onto its range, i.e. $Pv = v$ for v in the range and $Pw = 0$ for w orthogonal to the range.

lemma *projection-fixes-range*:
fixes *P* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
assumes *P*: *projection n P* **and** *x*: *dim-vec x* = *n*
shows $P *_v (P *_v x) = P *_v x$
⟨*proof*⟩

lemma *projection-annihilates-orthogonal*:
fixes *P* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
assumes *P*: *projection n P* **and** *w*: *dim-vec w* = *n*
and *orth*: $\bigwedge x$ :: *complex Matrix.vec*. *dim-vec x* = *n* $\Longrightarrow$ $\langle P *_v x \mid w\rangle = 0$
shows $P *_v w = 0_v n$
⟨*proof*⟩

lemma *projection-complement-idempotent*:
assumes *projection n P*
shows (1_m *n* − *P*) ∗ (1_m *n* − *P*) = 1_m *n* − *P*
⟨*proof*⟩

22.3 More on the repetition code

lemma *hamming-weight-append*:
hamming-weight (*xs* @ *ys*) = *hamming-weight xs* + *hamming-weight ys*
⟨*proof*⟩

lemma *hamming-weight-repetition-encode*:
hamming-weight (*repetition-encode v*) = 2 ∗ *hamming-weight v*
⟨*proof*⟩

lemma *repetition-encode-binary*:
assumes ∀ *x* ∈ *set v*. *x* < 2
shows ∀ *x* ∈ *set* (*repetition-encode v*). *x* < 2
⟨*proof*⟩

22.4 Algebra of projections; towards Days 33 and 34

Day 33 constructs the Shor recovery map from the projections onto the 22 orthogonal subspaces $V, X_1V, \ldots, X_9V, Y_1V, \ldots, Y_9V, Z_1V, Z_4V, Z_7V$ and proves, using $U = a_0I + a_1X + a_2Y + a_3Z$ with $\sum |a_k|^2 = 1$ for unitary U (Pauli magic), that it corrects every single-qubit unitary error. Day 34 gives the general theory:

Proposition (notes). For $V \subseteq \mathbb{C}^n$ with projection P and CPTP maps E (errors) and R (recovery), $R \circ E(|\psi\rangle\langle\psi|) = |\psi\rangle\langle\psi|$ for all $\psi \in V$ iff $R \circ E(PXP) = PXP$ for all X.

Theorem (notes) (1)**.** Let $E(X) = \sum_{i=1}^{r} E_i X E_i^*$ be CPTP. There is a CPTP R with $R \circ E(PXP) = PXP$ for all X iff $PE_i^*E_jP = \alpha_{ij}P$ for some scalars α_{ij}.

Theorem (notes) (2)**.** If R corrects E as in Theorem 1 and $G_i \in \mathrm{span}\{E_j\}$ with $\sum G_i^*G_i = I$, then R also corrects $G(X) = \sum_i G_i X G_i^*$.

Of the Knill–Laflamme results, the equivalence of Theorem 1's two readings (*recovery-states-iff-PXP*), the necessity of $PE_i^*E_jP = \alpha_{ij}P$ (*knill-laflamme-necessity*) and Theorem 2 (*knill-laflamme-closure*) are proved in *Day34-Knill-Laflamme*; the converse, the construction of R from the conditions, and the equivalence itself (*knill-laflamme*) are in *Day34-KL-Sufficiency*. The elementary algebra of orthogonal projections used by the proofs follows.

lemma *projection-complement*:
  assumes *projection n P*
  shows *projection n* (1_m *n* − *P*)
⟨*proof*⟩

Two projections are orthogonal if their products in both orders vanish.

definition *orthogonal-projections* ::
  nat ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *bool* **where**
  orthogonal-projections n P Q ≡
    projection n P ∧ *projection n Q* ∧
    P ∗ *Q* = 0_m *n n* ∧ *Q* ∗ *P* = 0_m *n n*

lemma *orthogonal-projections-carrier-left*:
  orthogonal-projections n P Q ⟹ *P* ∈ *carrier-mat n n*
  ⟨*proof*⟩

lemma *orthogonal-projections-carrier-right*:
  orthogonal-projections n P Q ⟹ *Q* ∈ *carrier-mat n n*
  ⟨*proof*⟩

lemma *orthogonal-projections-symm*:
  orthogonal-projections n P Q ⟷ *orthogonal-projections n Q P*
  ⟨*proof*⟩

lemma *orthogonal-projections-sum*:
  assumes *opq*: *orthogonal-projections n P Q*
  shows *projection n* (*P* + *Q*)
⟨*proof*⟩

lemma *orthogonal-projections-decomp-id*:
  assumes *orthogonal-projections n P Q*
  shows *P* + *Q* + (1_m *n* − *P* − *Q*) = 1_m *n*
⟨*proof*⟩

23 Extensions beyond the notes

Nothing from here on is in Paulsen's notes. These are standard quantum-information facts proved with the machinery above: rank-one projections, the bit-flip, phase-flip, bit-phase-flip and unitary-conjugation channels and their membership in UCP and CPTP, composition of channels, Kraus lists, the maximally mixed state, and the Pauli product identities.

23.1 Rank-one projections

lemma *outer-sum-singleton-hermitian*:
fixes ψ :: *complex Matrix.vec*
shows $(\textit{outer-sum}\ n\ [\psi])^\dagger = \textit{outer-sum}\ n\ [\psi]$
⟨*proof*⟩

lemma *outer-sum-singleton-idempotent*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows $\textit{outer-sum}\ n\ [\psi] * \textit{outer-sum}\ n\ [\psi] = \textit{outer-sum}\ n\ [\psi]$
⟨*proof*⟩

lemma *rank1-projection*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows *projection n* (*outer-sum n* $[\psi]$)
⟨*proof*⟩

lemma *rank1-projection-acts-identity*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows $\textit{outer-sum}\ n\ [\psi] *_v \psi = \psi$
⟨*proof*⟩

lemma *rank1-projection-trace-one*:
fixes ψ :: *complex Matrix.vec*
assumes ψ-*qs*: *quantum-state n* ψ
shows *Tr* (*outer-sum n* $[\psi]$) = *1*
⟨*proof*⟩

23.2 The bit-flip channel at $p = 0$ and $p = 1$

lemma *bit-flip-channel-zero*:
assumes $\varrho \in$ *carrier-mat 2 2*
shows *bit-flip-channel 0* $\varrho = \varrho$
⟨*proof*⟩

lemma *bit-flip-channel-one*:
assumes $\varrho \in$ *carrier-mat 2 2*
shows *bit-flip-channel 1* $\varrho = \textit{Pauli-X} * \varrho * (\textit{Pauli-X}^\dagger)$
⟨*proof*⟩

23.3 The phase-flip channel

definition *phase-flip-channel* ::
real ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
phase-flip-channel p $\varrho \equiv$

complex-of-real $(1 - p) \cdot_m \varrho$
$+$ *complex-of-real* $p \cdot_m$ $(Pauli\text{-}Z * \varrho * (Pauli\text{-}Z^\dagger))$

lemma *phase-flip-channel-carrier*:
assumes $\varrho \in$ *carrier-mat 2 2*
shows *phase-flip-channel* p $\varrho \in$ *carrier-mat 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-trace-preserving*:
fixes p :: *real*
assumes *ϱ-c*: $\varrho \in$ *carrier-mat 2 2*
shows *Tr* (*phase-flip-channel* p ϱ) $=$ *Tr* ϱ
⟨*proof*⟩

lemma *phase-flip-channel-psd*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$
and *p-le-1*: $p \leq 1$
and *ϱ-c*: $\varrho \in$ *carrier-mat 2 2*
and *ϱ-psd*: *psd* ϱ
shows *psd* (*phase-flip-channel* p ϱ)
⟨*proof*⟩

23.4 Unitary-conjugation channels

For a unitary $U \in M_n$ and $0 \leq p \leq 1$, $\rho \mapsto (1-p)\rho + pU\rho U^\dagger$ is trace preserving and positive; the bit-flip and phase-flip channels are the instances $U = X$, $U = Z$.

definition *unitary-conj-channel* ::
complex Matrix.mat ⇒ *real* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
unitary-conj-channel U p $\varrho \equiv$
complex-of-real $(1 - p) \cdot_m \varrho$
$+$ *complex-of-real* $p \cdot_m$ $(U * \varrho * (U^\dagger))$

lemma *unitary-conj-channel-carrier*:
assumes *U-c*: $U \in$ *carrier-mat n n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *unitary-conj-channel* U p $\varrho \in$ *carrier-mat n n*
⟨*proof*⟩

lemma *unitary-conj-channel-trace-preserving*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary* U
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *Tr* (*unitary-conj-channel* U p ϱ) $=$ *Tr* ϱ
⟨*proof*⟩

lemma *unitary-conj-channel-psd*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$
and *p-le-1*: $p \leq 1$
and *U-c*: $U \in$ *carrier-mat n n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
and *ϱ-psd*: *psd* ϱ
shows *psd* (*unitary-conj-channel* U p ϱ)
⟨*proof*⟩

lemma *bit-flip-eq-unitary-conj*:
bit-flip-channel p ϱ = unitary-conj-channel Pauli-X p ϱ
⟨*proof*⟩

lemma *phase-flip-eq-unitary-conj*:
phase-flip-channel p ϱ = unitary-conj-channel Pauli-Z p ϱ
⟨*proof*⟩

23.5 The bit-phase-flip channel and the identity channel

definition *bit-phase-flip-channel* ::
real ⇒ complex Matrix.mat ⇒ complex Matrix.mat **where**
bit-phase-flip-channel p ϱ ≡ unitary-conj-channel Pauli-Y p ϱ

lemma *bit-phase-flip-channel-carrier*:
assumes *ϱ ∈ carrier-mat 2 2*
shows *bit-phase-flip-channel p ϱ ∈ carrier-mat 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-trace-preserving*:
fixes *p* :: *real*
assumes *ϱ-c*: *ϱ ∈ carrier-mat 2 2*
shows *Tr* (*bit-phase-flip-channel p ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *bit-phase-flip-channel-psd*:
fixes *p* :: *real*
assumes *p-nn*: *0 ≤ p*
and *p-le-1*: *p ≤ 1*
and *ϱ-c*: *ϱ ∈ carrier-mat 2 2*
and *ϱ-psd*: *psd ϱ*
shows *psd* (*bit-phase-flip-channel p ϱ*)
⟨*proof*⟩

definition *identity-channel* ::
complex Matrix.mat ⇒ complex Matrix.mat **where**
identity-channel ϱ ≡ ϱ

lemma *identity-channel-carrier*:
ϱ ∈ carrier-mat n n ⟹ identity-channel ϱ ∈ carrier-mat n n
⟨*proof*⟩

lemma *identity-channel-trace-preserving*:
ϱ ∈ carrier-mat n n ⟹ Tr (*identity-channel ϱ*) = *Tr ϱ*
⟨*proof*⟩

lemma *identity-channel-psd*:
ϱ ∈ carrier-mat n n ⟹ psd ϱ ⟹ psd (*identity-channel ϱ*)
⟨*proof*⟩

lemma *identity-channel-eq-unitary-conj-zero*:
assumes *ϱ-c*: *ϱ ∈ carrier-mat n n*
shows *identity-channel ϱ = unitary-conj-channel* (*1*$_m$ *n*) *0 ϱ*
⟨*proof*⟩

23.6 Composition of channels

lemma *compose-trace-preserving*:
assumes *F-tp*: *trace-preserving n F*
and *G-tp*: *trace-preserving n G*
and *F-range*: $\forall \varrho.\ \varrho \in$ *carrier-mat n n* $\longrightarrow F\ \varrho \in$ *carrier-mat n n*
shows *trace-preserving n* $(\lambda\varrho.\ G\ (F\ \varrho))$
⟨*proof*⟩

lemma *identity-channel-compose-left*:
$(\lambda\varrho.$ *identity-channel* $(F\ \varrho)) = F$
⟨*proof*⟩

lemma *identity-channel-compose-right*:
$(\lambda\varrho.\ F$ (*identity-channel* $\varrho)) = F$
⟨*proof*⟩

lemma *compose-unital-map*:
assumes *F-un*: *unital-map n n F*
and *G-un*: *unital-map n n G*
shows *unital-map n n* $(\lambda\varrho.\ F\ (G\ \varrho))$
⟨*proof*⟩

lemma *identity-channel-unital*: *unital-map n n identity-channel*
⟨*proof*⟩

lemma *identity-channel-trace-preserving-full*:
trace-preserving n identity-channel
⟨*proof*⟩

lemma *identity-channel-completely-positive*:
assumes $0 < n$
shows *completely-positive n n identity-channel*
⟨*proof*⟩

23.7 Conjugation channels are completely positive

definition *conjugation-channel* ::
complex Matrix.mat ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
conjugation-channel U $\varrho \equiv U * \varrho * (U^\dagger)$

lemma *conjugation-channel-eq-kraus*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and *ϱ-c*: $\varrho \in$ *carrier-mat n n*
shows *conjugation-channel U* $\varrho =$ *kraus-map-dim d* $[U]\ \varrho$
⟨*proof*⟩

lemma *conjugation-channel-cp*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive n d* (*kraus-map-dim d* $[U]$)
⟨*proof*⟩

lemma *identity-channel-UCP*:
assumes $0 < n$
shows *identity-channel* $\in$ *UCP n n*
⟨*proof*⟩

lemma *identity-channel-CPTP*:
assumes $0 < n$
shows *identity-channel* $\in$ *CPTP n n*
⟨*proof*⟩

lemma *map-conj-cp*:
assumes *U-c*: $U \in$ *carrier-mat d n*
and *U-range*: $\forall \varrho.\ \varrho \in$ *carrier-mat n n*
$\longrightarrow U * \varrho * (U^\dagger) \in$ *carrier-mat d d*
and *npos*: $0 < n$
and *dpos*: $0 < d$
shows *completely-positive n d* $(\lambda\varrho.\ U * \varrho * (U^\dagger))$
⟨*proof*⟩

lemma *map-conj-unital*:
fixes U :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
shows $U * (1_m\ n) * (U^\dagger) = 1_m\ n$
⟨*proof*⟩

23.8 Unitary-conjugation channels are UCP and CPTP

lemma *map-conj-in-UCP*:
fixes U :: *complex Matrix.mat*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *npos*: $0 < n$
shows $(\lambda\varrho.\ U * \varrho * (U^\dagger)) \in$ *UCP n n*
⟨*proof*⟩

lemma *unitary-conj-channel-in-UCP*:
fixes U :: *complex Matrix.mat* **and** p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *npos*: $0 < n$
shows *unitary-conj-channel U p* $\in$ *UCP n n*
⟨*proof*⟩

lemma *unitary-conj-channel-in-CPTP*:
fixes U :: *complex Matrix.mat* **and** p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$

and *p-le*: $p \leq 1$
and *npos*: $0 < n$
shows *unitary-conj-channel U p* $\in$ *CPTP n n*
⟨*proof*⟩

23.9 The Pauli channels are UCP and CPTP

lemma *bit-flip-channel-eq*:
bit-flip-channel p = *unitary-conj-channel Pauli-X p*
⟨*proof*⟩

lemma *phase-flip-channel-eq*:
phase-flip-channel p = *unitary-conj-channel Pauli-Z p*
⟨*proof*⟩

lemma *bit-phase-flip-channel-eq*:
bit-phase-flip-channel p = *unitary-conj-channel Pauli-Y p*
⟨*proof*⟩

lemma *bit-flip-channel-UCP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-flip-channel p* $\in$ *UCP 2 2*
⟨*proof*⟩

lemma *bit-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-flip-channel p* $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-UCP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *phase-flip-channel p* $\in$ *UCP 2 2*
⟨*proof*⟩

lemma *phase-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *phase-flip-channel p* $\in$ *CPTP 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-UCP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-phase-flip-channel p* $\in$ *UCP 2 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-CPTP*:
fixes *p* :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
shows *bit-phase-flip-channel p* $\in$ *CPTP 2 2*
⟨*proof*⟩

23.10 Pauli matrices as Kraus operators

lemma *Pauli-X-kraus-operators*:

kraus-operators 2 2 [Pauli-X]
⟨*proof*⟩

lemma *Pauli-Y-kraus-operators*:
kraus-operators 2 2 [Pauli-Y]
⟨*proof*⟩

lemma *Pauli-Z-kraus-operators*:
kraus-operators 2 2 [Pauli-Z]
⟨*proof*⟩

23.11 Kraus maps of Kraus-operator lists are CPTP

lemma *kraus-is-trace-preserving*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws*: *kraus-operators n d Ws*
shows *trace-preserving n* (*kraus-map-dim d Ws*)
⟨*proof*⟩

lemma *kraus-is-cptp*:
fixes *Ws* :: *complex Matrix.mat list*
assumes *Ws*: *kraus-operators n d Ws*
and *npos*: *0 < n*
and *dpos*: *0 < d*
shows *kraus-map-dim d Ws ∈ CPTP n d*
⟨*proof*⟩

lemma *Pauli-X-kraus-map-CPTP*:
kraus-map-dim 2 [Pauli-X] ∈ CPTP 2 2
⟨*proof*⟩

lemma *Pauli-Y-kraus-map-CPTP*:
kraus-map-dim 2 [Pauli-Y] ∈ CPTP 2 2
⟨*proof*⟩

lemma *Pauli-Z-kraus-map-CPTP*:
kraus-map-dim 2 [Pauli-Z] ∈ CPTP 2 2
⟨*proof*⟩

lemma *identity-kraus-operators*:
kraus-operators n n [1$_m$ *n]*
⟨*proof*⟩

lemma *identity-channel-eq-kraus-map-dim*:
fixes *n* :: *nat*
assumes *ϱ-c*: *ϱ ∈ carrier-mat n n*
shows *identity-channel ϱ = kraus-map-dim n [1*$_m$ *n] ϱ*
⟨*proof*⟩

23.12 Concatenation of Kraus lists

lemma *sum-wdag-w-append*:
fixes *Ws Vs* :: *complex Matrix.mat list*
assumes *Ws-c*: *∀ W ∈ set Ws. W ∈ carrier-mat d n*

and *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
shows *sum-wdag-w n* (*Ws* @ *Vs*)
= *sum-wdag-w n Ws* + *sum-wdag-w n Vs*
⟨*proof*⟩

lemma *kraus-map-dim-append*:
fixes *Ws Vs* :: *complex Matrix.mat list*
assumes *Ws-c*: $\forall\, W \in set\ Ws.\ W \in carrier\text{-}mat\ d\ n$
and *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *ϱ-c*: $\varrho \in carrier\text{-}mat\ n\ n$
shows *kraus-map-dim d* (*Ws* @ *Vs*) *ϱ*
= *kraus-map-dim d Ws ϱ* + *kraus-map-dim d Vs ϱ*
⟨*proof*⟩

23.13 Fixed points of Pauli channels

lemma *Pauli-X-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: $M \in carrier\text{-}mat\ 2\ 2$
and *commute*: *Pauli-X* ∗ *M* = *M* ∗ *Pauli-X*
shows *Pauli-X* ∗ *M* ∗ (*Pauli-X*†) = *M*
⟨*proof*⟩

lemma *bit-flip-channel-fixed-point-commute*:
fixes *p* :: *real*
assumes *M-c*: $M \in carrier\text{-}mat\ 2\ 2$
and *commute*: *Pauli-X* ∗ *M* = *M* ∗ *Pauli-X*
shows *bit-flip-channel p M* = *M*
⟨*proof*⟩

lemma *Pauli-Z-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: $M \in carrier\text{-}mat\ 2\ 2$
and *commute*: *Pauli-Z* ∗ *M* = *M* ∗ *Pauli-Z*
shows *Pauli-Z* ∗ *M* ∗ (*Pauli-Z*†) = *M*
⟨*proof*⟩

lemma *phase-flip-channel-fixed-point-commute*:
fixes *p* :: *real*
assumes *M-c*: $M \in carrier\text{-}mat\ 2\ 2$
and *commute*: *Pauli-Z* ∗ *M* = *M* ∗ *Pauli-Z*
shows *phase-flip-channel p M* = *M*
⟨*proof*⟩

lemma *unitary-conj-channel-identity-op*:
fixes *p* :: *real*
assumes *ϱ-c*: $\varrho \in carrier\text{-}mat\ n\ n$
shows *unitary-conj-channel* (1_m *n*) *p ϱ* = *ϱ*
⟨*proof*⟩

lemma *unitary-conj-channel-identity-op-eq*:
fixes *p* :: *real*

shows $(\lambda\varrho.\ \mathit{if}\ \varrho \in \mathit{carrier\text{-}mat}\ n\ n$
$\quad \mathit{then}\ \mathit{unitary\text{-}conj\text{-}channel}\ (1_m\ n)\ p\ \varrho\ \mathit{else}\ \mathit{identity\text{-}channel}\ \varrho)$
$= \mathit{identity\text{-}channel}$
⟨*proof*⟩

23.14 The maximally mixed state

definition *maximally-mixed-state* :: *nat* ⇒ *complex Matrix.mat* **where**
maximally-mixed-state n ≡ *complex-of-real* (*1* / *real n*) ·$_m$ (*1*$_m$ *n*)

lemma *maximally-mixed-state-carrier* [*simp*]:
maximally-mixed-state n ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *Tr-one-mat*:
Tr (*1*$_m$ *n* :: *complex Matrix.mat*) = *of-nat n*
⟨*proof*⟩

lemma *maximally-mixed-state-trace*:
assumes *npos*: *0* < *n*
shows *Tr* (*maximally-mixed-state n*) = *1*
⟨*proof*⟩

lemma *maximally-mixed-state-psd*:
psd (*maximally-mixed-state n*)
⟨*proof*⟩

lemma *maximally-mixed-state-density-matrix*:
assumes *npos*: *0* < *n*
shows *density-matrix n* (*maximally-mixed-state n*)
⟨*proof*⟩

23.15 Fixed points of unitary-conjugation channels

lemma *unitary-conj-commute*:
fixes *M* :: *complex Matrix.mat*
assumes *U-c*: *U* ∈ *carrier-mat n n*
and *U-unit*: *unitary U*
and *M-c*: *M* ∈ *carrier-mat n n*
and *commute*: *U* ∗ *M* = *M* ∗ *U*
shows *U* ∗ *M* ∗ (*U*†) = *M*
⟨*proof*⟩

lemma *unitary-conj-channel-fixed-point-commute*:
fixes *p* :: *real*
assumes *U-c*: *U* ∈ *carrier-mat n n*
and *U-unit*: *unitary U*
and *M-c*: *M* ∈ *carrier-mat n n*
and *commute*: *U* ∗ *M* = *M* ∗ *U*
shows *unitary-conj-channel U p M* = *M*
⟨*proof*⟩

lemma *maximally-mixed-state-commutes*:
fixes *U* :: *complex Matrix.mat*
assumes *U-c*: *U* ∈ *carrier-mat n n*
shows *U* ∗ *maximally-mixed-state n* = *maximally-mixed-state n* ∗ *U*
⟨*proof*⟩

lemma *unitary-conj-channel-fixes-max-mixed*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
shows *unitary-conj-channel U p* (*maximally-mixed-state n*)
$=$ *maximally-mixed-state n*
⟨*proof*⟩

lemma *bit-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *bit-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

lemma *phase-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *phase-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

lemma *bit-phase-flip-channel-fixes-max-mixed*:
fixes p :: *real*
shows *bit-phase-flip-channel p* (*maximally-mixed-state 2*)
$=$ *maximally-mixed-state 2*
⟨*proof*⟩

23.16 Channels preserve density matrices

lemma *unitary-conj-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *U-c*: $U \in$ *carrier-mat n n*
and *U-unit*: *unitary U*
and *p-nn*: $0 \leq p$
and *p-le*: $p \leq 1$
and *dm*: *density-matrix n* ϱ
shows *density-matrix n* (*unitary-conj-channel U p* ϱ)
⟨*proof*⟩

lemma *bit-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*bit-flip-channel p* ϱ)
⟨*proof*⟩

lemma *phase-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$
and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*phase-flip-channel p* ϱ)
⟨*proof*⟩

lemma *bit-phase-flip-channel-preserves-density-matrix*:
fixes p :: *real*
assumes *p-nn*: $0 \leq p$ **and** *p-le*: $p \leq 1$

and *dm*: *density-matrix 2* ϱ
shows *density-matrix 2* (*bit-phase-flip-channel p* ϱ)
⟨*proof*⟩

23.17 Pauli product identities

With the Day 32 convention for Y: $XY = -iZ$, $YZ = -iX$, $ZX = -iY$.

lemma *Pauli-X-times-Pauli-Y*: *Pauli-X* $*$ *Pauli-Y* $= -\mathrm{i} \cdot_m$ *Pauli-Z*
⟨*proof*⟩

lemma *Pauli-Y-times-Pauli-Z*: *Pauli-Y* $*$ *Pauli-Z* $= -\mathrm{i} \cdot_m$ *Pauli-X*
⟨*proof*⟩

lemma *Pauli-Z-times-Pauli-X*: *Pauli-Z* $*$ *Pauli-X* $= -\mathrm{i} \cdot_m$ *Pauli-Y*
⟨*proof*⟩

end
theory *QC-Prelims*
imports *Day23-ChoiExtremeUCP*
begin

24 Small matrix and vector lemmas shared by the gap-closure theories

Index case splits, trace facts, entrywise vector sums, rank-one identities and rectangular *foldr* sums. Nothing here is specific to a lecture; the theories *Day12-Counting-Probability*, *Day18-Positive-Maps*, *Day22-Kraus-Independence*, *Day28-Horodecki-Peres*, *Day30-BitFlip-Code* and *Day32-Pauli-Expansion* build on it.

lemma *cnj-mult-self-cmod*: *cnj z* $*$ *z* $=$ *complex-of-real* $((cmod\ z)^2)$
⟨*proof*⟩

lemma *Re-cnj-mult-self*: *Re* $(cnj\ z * z) = (cmod\ z)^2$
⟨*proof*⟩

lemma *Tr-hermitian-Im-zero*:
assumes *Pc*: *P* $\in$ *carrier-mat n n* **and** *herm*: *hermitian P*
shows *Im* $(Tr\ P) = 0$
⟨*proof*⟩

lemma *Tr-hermitian-real*:
assumes *Pc*: *P* $\in$ *carrier-mat n n* **and** *herm*: *hermitian P*
shows *Tr P* $=$ *complex-of-real* $(Re\ (Tr\ P))$
⟨*proof*⟩

A positive semidefinite matrix of trace zero is zero (the notes: "given a non-zero $P \geq 0$, $\mathrm{Tr}(P) \neq 0$").

lemma *psd-Tr-zero-imp-zero*:
assumes *Pc*: *P* $\in$ *carrier-mat n n* **and** *pos*: *psd P* **and** *tr0*: *Tr P* $= 0$
shows *P* $= 0_m$ *n n*
⟨*proof*⟩

lemma *smult-smult-mat*:
fixes *A* :: *complex Matrix.mat*
shows *a* $\cdot_m$ $(b \cdot_m A) = (a * b) \cdot_m A$

⟨*proof*⟩

lemma *zero-smult-mat-carrier*:
fixes A :: *complex Matrix.mat*
assumes $A \in$ *carrier-mat m k*
shows $0 \cdot_m A = 0_m\ m\ k$
⟨*proof*⟩

Finite sums of vectors, entrywise.

definition *vec-sum* :: *nat* ⇒ $'c$ *set* ⇒ ($'c$ ⇒ *complex Matrix.vec*) ⇒ *complex Matrix.vec* **where**
vec-sum n Js w = *Matrix.vec n* (λ*i*. *sum* (λ*J*. *w J* $ *i*) *Js*)

lemma *vec-sum-dim* [*simp*]: *dim-vec* (*vec-sum n Js w*) = *n*
⟨*proof*⟩

lemma *vec-sum-index* [*simp*]: *i* < *n* ⟹ *vec-sum n Js w* $ *i* = *sum* (λ*J*. *w J* $ *i*) *Js*
⟨*proof*⟩

lemma *vec-sum-cong*:
assumes ⋀*J*. *J* ∈ *Js* ⟹ *w J* = *w*′ *J*
shows *vec-sum n Js w* = *vec-sum n Js w*′
⟨*proof*⟩

lemma *mult-mat-vec-vec-sum*:
assumes *Ac*: *A* ∈ *carrier-mat m n* **and** *fin*: *finite Js*
and *wd*: ⋀*J*. *J* ∈ *Js* ⟹ *dim-vec* (*w J*) = *n*
shows *A* ∗$_v$ *vec-sum n Js w* = *vec-sum m Js* (λ*J*. *A* ∗$_v$ *w J*)
⟨*proof*⟩

lemma *inner-prod-vec-sum-right*:
assumes *ud*: *dim-vec u* = *n*
and *wd*: ⋀*J*. *J* ∈ *Js* ⟹ *dim-vec* (*w J*) = *n*
shows ⟨*u* | *vec-sum n Js w*⟩ = *sum* (λ*J*. ⟨*u* | *w J*⟩) *Js*
⟨*proof*⟩

lemma *mult-mat-vec-smult-vec*:
fixes *A* :: *complex Matrix.mat* **and** *v* :: *complex Matrix.vec*
assumes *Ac*: *A* ∈ *carrier-mat m n* **and** *vd*: *dim-vec v* = *n*
shows *A* ∗$_v$ (*c* ·$_v$ *v*) = *c* ·$_v$ (*A* ∗$_v$ *v*)
⟨*proof*⟩

lemma *inner-prod-unit-vec-right*:
assumes *ud*: *dim-vec u* = *n* **and** *j*: *j* < *n*
shows ⟨*u* | *unit-vec n j*⟩ = *cnj* (*u* $ *j*)
⟨*proof*⟩

lemma *outer-prod-self-psd*:
assumes *dim-vec v* = *n*
shows *psd* (*v* ⊘ *v*)
⟨*proof*⟩

Conjugating a rank-one matrix: $W|\varphi\rangle\langle\varphi|W^* = |W\varphi\rangle\langle W\varphi|$.

lemma *conj-outer-prod*:
fixes *W* :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes *Wc*: *W* ∈ *carrier-mat m n* **and** φ*d*: *dim-vec* φ = *n*
shows *W* ∗ (φ ⊘ φ) ∗ (*W*†) = (*W* ∗$_v$ φ) ⊘ (*W* ∗$_v$ φ)
⟨*proof*⟩

lemma *outer-prod-zero-vec*: $(0_v\ n :: complex\ Matrix.vec) \oslash (0_v\ n) = 0_m\ n\ n$
⟨*proof*⟩

lemma *idx8*: $k < (8::nat) \Longrightarrow k = 0 \vee k = 1 \vee k = 2 \vee k = 3 \vee k = 4 \vee k = 5 \vee k = 6 \vee k = 7$
⟨*proof*⟩

lemma *dagger-mult-diag-inner*:
fixes $Y :: complex\ Matrix.mat$
assumes *Yc*: $Y \in carrier\text{-}mat\ p\ q$ **and** *j*: $j < q$
shows $((Y^\dagger) * Y)\ \$\$\ (j,j) = \langle Matrix.col\ Y\ j \mid Matrix.col\ Y\ j\rangle$
⟨*proof*⟩

lemma *foldr-add-mat-carrier-rect*:
assumes $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ p\ q$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ p\ q) \in carrier\text{-}mat\ p\ q$
⟨*proof*⟩

lemma *foldr-add-mat-index-rect*:
assumes $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ p\ q$ **and** $i < p$ **and** $j < q$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ p\ q)\ \$\$\ (i,j) = (\sum x \leftarrow xs.\ f\ x\ \$\$\ (i,j))$
⟨*proof*⟩

end
theory *Day12-Counting-Probability*
imports *QC-Prelims*
begin

25 Day 12: $H^{\otimes N}$ on $|0\cdots0\rangle$ and the counting probability (D12-1, D12-4)

N-fold tensor powers (last factor least significant, matching $(\otimes)$).

primrec *tensor-pow-mat* :: $complex\ Matrix.mat \Rightarrow nat \Rightarrow complex\ Matrix.mat$ **where**
$tensor\text{-}pow\text{-}mat\ A\ 0 = 1_m\ 1$
| $tensor\text{-}pow\text{-}mat\ A\ (Suc\ k) = Tensor.tensor\text{-}mat\ (tensor\text{-}pow\text{-}mat\ A\ k)\ A$

primrec *tensor-pow-vec* :: $complex\ Matrix.vec \Rightarrow nat \Rightarrow complex\ Matrix.vec$ **where**
$tensor\text{-}pow\text{-}vec\ v\ 0 = Matrix.vec\ 1\ (\lambda\text{-}.\ 1)$
| $tensor\text{-}pow\text{-}vec\ v\ (Suc\ k) = tensor\text{-}vec\ (tensor\text{-}pow\text{-}vec\ v\ k)\ v$

lemma *tensor-pow-mat-carrier*:
assumes $A \in carrier\text{-}mat\ 2\ 2$
shows $tensor\text{-}pow\text{-}mat\ A\ k \in carrier\text{-}mat\ (2\hat{}k)\ (2\hat{}k)$
⟨*proof*⟩

lemma *tensor-pow-vec-dim*:
assumes $dim\text{-}vec\ v = 2$
shows $dim\text{-}vec\ (tensor\text{-}pow\text{-}vec\ v\ k) = 2\hat{}k$
⟨*proof*⟩

lemma *tensor-pow-mat-vec*:
assumes *Ac*: $A \in carrier\text{-}mat\ 2\ 2$ **and** *vd*: $dim\text{-}vec\ v = 2$
shows $tensor\text{-}pow\text{-}mat\ A\ k *_v tensor\text{-}pow\text{-}vec\ v\ k = tensor\text{-}pow\text{-}vec\ (A *_v v)\ k$
⟨*proof*⟩

lemma *tensor-pow-vec-const*:
$tensor\text{-}pow\text{-}vec\ (Matrix.vec\ 2\ (\lambda\text{-}.\ c))\ k = Matrix.vec\ (2\hat{}k)\ (\lambda\text{-}.\ c\hat{}k)$

⟨*proof*⟩

definition *uniform-vec* :: *nat* ⇒ *complex Matrix.vec* **where**
uniform-vec N = *Matrix.vec* (*2^N*) (λ-. *complex-of-real* (*1* / *sqrt 2*) ^ *N*)

lemma *uniform-vec-dim* [*simp*]: *dim-vec* (*uniform-vec N*) = *2^N*
⟨*proof*⟩

lemma *hadamard-e0-const*: *Hadamard* $*_v$ *unit-vec 2 0* = *Matrix.vec 2* (λ-. *complex-of-real* (*1* / *sqrt 2*))
⟨*proof*⟩

theorem *hadamard-uniform-N*:
tensor-pow-mat Hadamard N $*_v$ *tensor-pow-vec* (*unit-vec 2 0*) *N* = *uniform-vec N*
⟨*proof*⟩

lemma *inner-prod-unit-vec-2*:
assumes *k* < *2*
shows ⟨*unit-vec 2 0* | *unit-vec 2 k*⟩ = (*if k* = *0 then 1 else 0*)
⟨*proof*⟩

lemma *inner-prod-uniform-unit*:
assumes *J* < *2^N*
shows ⟨*uniform-vec N* | *unit-vec* (*2^N*) *J*⟩ = *complex-of-real* (*1* / *sqrt 2*) ^ *N*
⟨*proof*⟩

lemma *half-sqrt2-sq*: (*1* / *sqrt 2* :: *real*) * (*1* / *sqrt 2*) = *1* / *2*
⟨*proof*⟩

lemma *uniform-coeff-sq*:
complex-of-real (*1* / *sqrt 2*) ^ *N* * *complex-of-real* (*1* / *sqrt 2*) ^ *N* = *complex-of-real* ((*1*/*2*) ^ *N*)
⟨*proof*⟩

lemma *uniform-vec-self*: ⟨*uniform-vec N* | *uniform-vec N*⟩ = *1*
⟨*proof*⟩

lemma *uniform-tensor-e0-as-sum*:
tensor-vec (*uniform-vec N*) (*unit-vec 2 0*) =
vec-sum (*2^(N+1)*) {*J*. *J* < *2^N*}
(λ*J*. *complex-of-real* (*1* / *sqrt 2*) ^ *N* $·_v$ *tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 0*))
⟨*proof*⟩

lemma *uniform-e0-state*: *quantum-state* (*2^(N+1)*) (*tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*))
⟨*proof*⟩

The notes' computation: with $x = H^{\otimes N}|0\cdots0\rangle$ and $\gamma = x \otimes e_0$, $p_0(U_f(x \otimes e_0)) = (M/2^N)^2$ where $M = \#\{J : f(J) = 0\}$.

theorem *counting-probability*:
fixes *f* :: *nat* ⇒ *nat* **and** *U* :: *complex Matrix.mat*
assumes *f-range*: ∀ *J* < *2^N*. *f J* < *2*
and *Uc*: *U* ∈ *carrier-mat* (*2^(N+1)*) (*2^(N+1)*)
and *orac*: ∀ *J* < *2^N*. ∀ *i* < *2*.
U $*_v$ (*tensor-vec* (*unit-vec* (*2^N*) *J*) (*unit-vec 2 i*)) =
tensor-vec (*unit-vec* (*2^N*) *J*) (*unit-vec 2* ((*i* + *f J*) *mod 2*))
shows *meas-prob* (*tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*) ⊘ *tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*))
(*U* $*_v$ *tensor-vec* (*uniform-vec N*) (*unit-vec 2 0*))
= (*real* (*count-zeros f N*) / *2^N*)^2

⟨*proof*⟩

end
theory *Day16-Block-Form*
imports *Day25-Arveson-Correspondence Day02-11-Supplement*
begin

26 Day 16: $H_A \otimes H_B \cong H_A^{(p)}$ and $L(H_A \otimes H_B) \cong M_p(L(H_A))$ (D16-2)

The notes identify $H_A \otimes H_B$ with p copies of H_A through $\sum_i h_i \otimes f_i \leftrightarrow (h_1, \ldots, h_p)$, with $\|v\|^2 = \sum_i \|h_i\|^2$, and an operator X with its block matrix (X_{ij}) over $L(H_A)$, acting by $(h_j) \mapsto (\sum_j X_{ij} h_j)$, so that $X = \sum_{i,j} X_{ij} \otimes E_{ij}$, $\mathrm{tr}_B(X) = X_{11} + \cdots + X_{pp}$ and $\mathrm{tr}_A(X) = (\mathrm{Tr}\, X_{ij})$. Here $f_i = e_i$, and indices are A-major as everywhere in the development.

With $H_A = \mathbb{C}^m$, $H_B = \mathbb{C}^p$ and A-major indices $ap + i$, the coordinates of v along e_i form $h_i = (v_{ap+i})_a \in H_A$, the i-th column of *vec-to-mat m p v*, and an operator X has the blocks $X_{ij} = (X_{ap+i,\, bp+j})_{a,b} \in L(H_A)$.

definition *bblock* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
bblock m p X i j = *Matrix.mat m m* (λ(*a*, *b*). *X* $$ (*a* ∗ *p* + *i*, *b* ∗ *p* + *j*))

lemma *bblock-carrier* [*simp*]: *bblock m p X i j* ∈ *carrier-mat m m*
⟨*proof*⟩

lemma *bblock-dims* [*simp*]: *dim-row* (*bblock m p X i j*) = *m dim-col* (*bblock m p X i j*) = *m*
⟨*proof*⟩

lemma *sum-mp-reindex*:
fixes *f* :: *nat* ⇒ *complex*
shows (∑ *r*<*m* ∗ *p*. *f r*) = (∑ *b*<*m*. ∑ *j*<*p*. *f* (*b* ∗ *p* + *j*))
⟨*proof*⟩

$H_A \otimes H_B \cong H_A^{(p)}$ isometrically: $\langle v|w\rangle = \sum_i \langle h_i|k_i\rangle$.

theorem *vec-to-mat-cols-inner-prod*:
assumes *v*: *dim-vec v* = *m* ∗ *p* **and** *w*: *dim-vec w* = *m* ∗ *p*
shows *Quantum.inner-prod v w* = (∑ *i*<*p*. *Quantum.inner-prod* (*Matrix.col* (*vec-to-mat m p v*) *i*) (*Matrix.col* (*vec-to-mat m p w*) *i*))
⟨*proof*⟩

$v = \sum_i h_i \otimes e_i$.

theorem *vec-to-mat-cols-expansion*:
assumes *v*: *dim-vec v* = *m* ∗ *p*
shows *v* = *tensor-combination* (*m* ∗ *p*) (*map* (*Matrix.col* (*vec-to-mat m p v*)) [*0*..<*p*]) (*map* (*unit-vec p*) [*0*..<*p*])
⟨*proof*⟩

The block action: the i-th coordinate of Xv is $\sum_j X_{ij} h_j$.

theorem *bblock-action*:
assumes *X*: *X* ∈ *carrier-mat* (*m* ∗ *p*) (*m* ∗ *p*) **and** *v*: *dim-vec v* = *m* ∗ *p* **and** *i*: *i* < *p*
shows *Matrix.col* (*vec-to-mat m p* (*X* ∗*v v*)) *i* = *Matrix.vec m* (λ*a*. ∑ *j*<*p*. (*bblock m p X i j* ∗*v Matrix.col* (*vec-to-mat m p v*) *j*) $ *a*)
⟨*proof*⟩

$L(H_A \otimes H_B) \cong M_p(L(H_A))$: $X = \sum_{i,j} X_{ij} \otimes E_{ij}$, and the blocks are unique.

lemma *double-delta*:
fixes *c* :: *complex* **and** *a b p q* :: *nat*

assumes $a < p$ $b < q$
shows $(\sum i{<}p.\ \sum j{<}q.\ \mathit{if}\ i = a \wedge j = b\ \mathit{then}\ c\ \mathit{else}\ 0) = c$
⟨*proof*⟩

lemma *mat-unit-tensor-index*:
assumes *A*: $A \in \mathit{carrier\text{-}mat}\ m\ m$ **and** *r*: $r < m * p$ **and** *s*: $s < m * p$ **and** *i*: $i < p$ **and** *j*: $j < p$
shows $(A \bigotimes E[\![i,j\ ;\ p,p]\!])\ \$\$\ (r,\ s) = (\mathit{if}\ r\ \mathit{mod}\ p = i \wedge s\ \mathit{mod}\ p = j\ \mathit{then}\ A\ \$\$\ (r\ \mathit{div}\ p,\ s\ \mathit{div}\ p)\ \mathit{else}\ 0)$
⟨*proof*⟩

theorem *tensor-block-decomposition*:
assumes *X*: $X \in \mathit{carrier\text{-}mat}\ (m * p)\ (m * p)$
shows $X = \mathit{msum}\ (m * p)\ (m * p)\ (\lambda(i,\ j).\ \mathit{bblock}\ m\ p\ X\ i\ j \bigotimes E[\![i,j\ ;\ p,p]\!])\ (\mathit{List.product}\ [0..{<}p]\ [0..{<}p])$
⟨*proof*⟩

theorem *tensor-block-decomposition-unique*:
assumes *Y*: $\bigwedge i\ j.\ Y\ i\ j \in \mathit{carrier\text{-}mat}\ m\ m$
and *X*: $X = \mathit{msum}\ (m * p)\ (m * p)\ (\lambda(i,\ j).\ Y\ i\ j \bigotimes E[\![i,j\ ;\ p,p]\!])\ (\mathit{List.product}\ [0..{<}p]\ [0..{<}p])$
and *i*: $i < p$ **and** *j*: $j < p$
shows $Y\ i\ j = \mathit{bblock}\ m\ p\ X\ i\ j$
⟨*proof*⟩

The partial traces in block form (the definitions, restated): $\mathrm{tr}_B(X) = \sum_i X_{ii}$, $\mathrm{tr}_A(X) = (\mathrm{Tr}\, X_{ij})_{ij}$.

lemma *partial-trace-B-blocks*:
partial-trace-B m p X = *msum m m* (λ*i*. *bblock m p X i i*) [*0..<p*]
⟨*proof*⟩

lemma *partial-trace-A-blocks*:
partial-trace-A m p X = *Matrix.mat p p* (λ(*i*, *j*). *Tr* (*bblock m p X i j*))
⟨*proof*⟩

end
theory *Day17-Environment-Model*
imports *Day22-Minimal-Kraus*
begin

Day 17, the environment model of noise: a state ρ of lab A meets an environment in the state $|\varphi\rangle$, the pair evolves by a unitary U on $\mathbb{C}^m \otimes \mathbb{C}^p$, and lab A sees $\mathrm{tr}_E(U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*) = \sum_i W_i \rho W_i^*$ with $\sum_i W_i^* W_i = I$. The index of $\mathbb{C}^m \otimes \mathbb{C}^p$ is $a\,p + i$ (lab index a, environment index i), and W_i is the matrix with entries $\sum_k U_{(a,i),(c,k)} \varphi_k$, i.e. the notes' $W_i = \sum_k U_{ik} \alpha_k$ for the blocks U_{ik} of U.

definition *env-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
env-kraus m p U φ i = *Matrix.mat m m* (λ(*a*,*c*). ∑*k*<*p*. *U* $$ (*a* ∗ *p* + *i*, *c* ∗ *p* + *k*) ∗ *φ* $ *k*)

lemma *env-kraus-carrier* [*simp*]: *env-kraus m p U φ i* ∈ *carrier-mat m m*
⟨*proof*⟩

lemma *env-kraus-dims* [*simp*]: *dim-row* (*env-kraus m p U φ i*) = *m dim-col* (*env-kraus m p U φ i*) = *m*
⟨*proof*⟩

lemma *env-kraus-index*: *a* < *m* ⟹ *c* < *m* ⟹ *env-kraus m p U φ i* $$ (*a*,*c*) = (∑*k*<*p*. *U* $$ (*a* ∗ *p* + *i*, *c* ∗ *p* + *k*) ∗ *φ* $ *k*)
⟨*proof*⟩

lemma *sum-split-mp*:
fixes $f :: nat \Rightarrow 'a$::*comm-monoid-add*
shows $(\sum x < m * p.\ f\ x) = (\sum c{<}m.\ \sum k{<}p.\ f\ (c * p + k))$
⟨*proof*⟩

lemma *tensor-outer-index*:
fixes ϱ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes ϱ: $\varrho \in$ *carrier-mat m m* **and** φ: *dim-vec* $\varphi = p$
and c: $c < m$ **and** d: $d < m$ **and** k: $k < p$ **and** l: $l < p$
shows *Tensor.tensor-mat* ϱ $(\varphi \oslash \varphi)$ \$\$ $(c * p + k,\ d * p + l) = \varrho$ \$\$ $(c,d) * (\varphi$ \$ $k * cnj\ (\varphi$ \$ $l))$
⟨*proof*⟩

The entries of $U(\rho \otimes |\varphi\rangle\langle\varphi|)U^*$.

lemma *env-entry*:
fixes U ϱ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes U: $U \in$ *carrier-mat* $(m * p)$ $(m * p)$ **and** ϱ: $\varrho \in$ *carrier-mat m m* **and** φ: *dim-vec* $\varphi = p$
and x: $x < m * p$ **and** y: $y < m * p$
shows $(U *$ *Tensor.tensor-mat* ϱ $(\varphi \oslash \varphi) * (U^\dagger))$ \$\$ (x,y)
$= (\sum d{<}m.\ \sum c{<}m.\ (\sum k{<}p.\ U$ \$\$ $(x,\ c * p + k) * \varphi$ \$ $k) * \varrho$ \$\$ (c,d)
$* cnj\ (\sum l{<}p.\ U$ \$\$ $(y,\ d * p + l) * \varphi$ \$ $l))$
⟨*proof*⟩

theorem *environment-model*:
fixes U ϱ :: *complex Matrix.mat* **and** φ :: *complex Matrix.vec*
assumes U: $U \in$ *carrier-mat* $(m * p)$ $(m * p)$ **and** *Uu*: *unitary U*
and φ: *dim-vec* $\varphi = p$ **and** *φ1*: $\langle\varphi|\varphi\rangle = 1$ **and** ϱ: $\varrho \in$ *carrier-mat m m*
defines *Ws* $\equiv$ *map* (*env-kraus m p U* φ) $[0..{<}p]$
shows *partial-trace-B m p* $(U *$ *Tensor.tensor-mat* ϱ $(\varphi \oslash \varphi) * (U^\dagger)) =$ *kraus-map-dim m Ws* ϱ
and *kraus-operators m m Ws*
⟨*proof*⟩

end
theory *Day18-Positive-Maps*
imports *QC-Prelims*
begin

27 Days 17–18: densities to densities iff positive and trace-preserving (D17-4, D18-2)

theorem *density-preserving-iff-positive-tp*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow B \in$ *carrier-mat n n* $\Longrightarrow$
$\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow \Phi\ A \in$ *carrier-mat d d*
and *npos*: $0 < n$
shows $(\forall \varrho.$ *density-matrix n* $\varrho \longrightarrow$ *density-matrix d* $(\Phi\ \varrho)) \longleftrightarrow$
(*positive-map n d* $\Phi \wedge$ *trace-preserving n* Φ)
⟨*proof*⟩

28 Day 18: the transpose is positive but not 2-positive (D18-4)

lemma *transpose-positive-map*: *positive-map n n* (*transpose-mat* :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*)
⟨*proof*⟩

The notes' witness: $X = (E_{ij}) \in M_2(M_2)$ is positive (it is $|w\rangle\langle w|$ for $w = e_0 \otimes e_0 + e_1 \otimes e_1$) while

$\Phi^{(2)}(X)$ has the eigenvector $(0,1,-1,0)$ with eigenvalue -1 (the notes compute $\det \Phi^{(2)}(X) = -1$ instead).

lemma *choi-seed-2-outer*: *choi-seed 2 = outer-sum 4 [Matrix.vec 4 (λi. if i = 0 ∨ i = 3 then 1 else 0)]*
⟨*proof*⟩

lemma *choi-seed-2-psd*: *psd (choi-seed 2)*
⟨*proof*⟩

lemma *transpose-not-2-positive*:
¬ *p-positive 2 2 2 (transpose-mat :: complex Matrix.mat ⇒ complex Matrix.mat)*
⟨*proof*⟩

corollary *transpose-not-completely-positive*:
¬ *completely-positive 2 2 (transpose-mat :: complex Matrix.mat ⇒ complex Matrix.mat)*
⟨*proof*⟩

end
theory *Day19-Choi-TP-Unital*
imports *Day25-Arveson-Correspondence*
begin

Day 19, notes (4) and (5) after Choi's theorem: write $P_\Phi = (R_{ij})$ with $R_{ij} = \Phi(E_{ij}) \in M_d$. Then Φ is trace preserving iff $\operatorname{Tr} R_{ij} = \delta_{ij}$, and unital iff $\sum_i R_{ii} = I_d$; with *choi-theorem* this characterises CPTP and UCP maps by their Choi matrix. Linearity on M_n is *lin-on n d (carrier-mat n n) Φ*.

lemma *choi-block*:
assumes *range*: ∀ *A*. *A ∈ carrier-mat n n ⟶ Φ A ∈ carrier-mat d d* **and** *i*: *i < n* **and** *j*: *j < n*
shows *block-of-mat d (choi-matrix n d Φ) i j = Φ (E⟦i,j ; n,n⟧)*
⟨*proof*⟩

lemma *msum-cong*: *(⋀x. x ∈ set xs ⟹ g x = h x) ⟹ msum r c g xs = msum r c h xs*
⟨*proof*⟩

lemma *Tr-msum*:
assumes ∀ *x ∈ set xs. g x ∈ carrier-mat d d*
shows *Tr (msum d d g xs) = (∑ x←xs. Tr (g x))*
⟨*proof*⟩

lemma *unit-expansion*:
assumes *A*: *A ∈ carrier-mat n n*
shows *A = msum n n (λ(i,j). A $$ (i,j) ·m E⟦i,j ; n,n⟧) (List.product [0..<n] [0..<n])*
⟨*proof*⟩

lemma *id-expansion*: *1m n = msum n n (λi. E⟦i,i ; n,n⟧) [0..<n]*
⟨*proof*⟩

lemma *lin-expansion*:
assumes *L*: *lin-on n d (carrier-mat n n) Φ* **and** *A*: *A ∈ carrier-mat n n*
shows *Φ A = msum d d (λ(i,j). A $$ (i,j) ·m Φ (E⟦i,j ; n,n⟧)) (List.product [0..<n] [0..<n])*
⟨*proof*⟩

D19-3, note (4): trace preservation in Choi-matrix form.

theorem *choi-trace-preserving-iff*:
assumes *L*: *lin-on n d (carrier-mat n n) Φ*
shows *trace-preserving n Φ ⟷*
(∀ i < n. ∀ j < n. Tr (block-of-mat d (choi-matrix n d Φ) i j) = (if i = j then 1 else 0))

⟨*proof*⟩

D19-3, note (5): unitality in Choi-matrix form, $\sum_i R_{ii} = I_d$.

theorem *choi-unital-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ
shows *unital-map n d* Φ ⟷ *msum d d* (λ*i*. *block-of-mat d* (*choi-matrix n d* Φ) *i i*) [*0*..<*n*] = 1_m *d*
⟨*proof*⟩

lemma *lin-on-choi-hyps*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ
shows ⋀*A B*. *A* ∈ *carrier-mat n n* ⟹ *B* ∈ *carrier-mat n n* ⟹ Φ (*A* + *B*) = Φ *A* + Φ *B*
and ⋀*c A*. *A* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*
and ∀ *A*. *A* ∈ *carrier-mat n n* ⟶ Φ *A* ∈ *carrier-mat d d*
⟨*proof*⟩

Notes (4) and (5) with Choi's theorem: CPTP iff $P_\Phi \geq 0$ and $\operatorname{Tr} R_{ij} = \delta_{ij}$; UCP iff $P_\Phi \geq 0$ and $\sum_i R_{ii} = I_d$.

corollary *choi-cptp-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *n*: *n* > *0* **and** *d*: *d* > *0*
shows *completely-positive n d* Φ ∧ *trace-preserving n* Φ ⟷
psd (*choi-matrix n d* Φ) ∧
(∀ *i* < *n*. ∀ *j* < *n*. *Tr* (*block-of-mat d* (*choi-matrix n d* Φ) *i j*) = (*if i* = *j then 1 else 0*))
⟨*proof*⟩

corollary *choi-ucp-iff*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *n*: *n* > *0* **and** *d*: *d* > *0*
shows *completely-positive n d* Φ ∧ *unital-map n d* Φ ⟷
psd (*choi-matrix n d* Φ) ∧ *msum d d* (λ*i*. *block-of-mat d* (*choi-matrix n d* Φ) *i i*) [*0*..<*n*] = 1_m *d*
⟨*proof*⟩

end
theory *Day20-Choi-Rank*
imports *Day34-Knill-Laflamme Day22-Kraus-Independence*
begin

29 Day 20: Choi's rank theorem $\operatorname{cr}(\Phi) = \operatorname{rank} P_\Phi$ (D20-4)

Rank is JNF's *vec-space.rank*, the dimension of the column space.

abbreviation *mat-rank* :: *nat* ⇒ *complex Matrix.mat* ⇒ *nat* **where**
mat-rank m A ≡ *vec-space.rank m A*

29.1 $\operatorname{rank}(\sum_{l<L} w_l w_l^*) \leq L$

lemma *rank-outer-prod-le-1*:
assumes *vd*: *dim-vec v* = *m*
shows *mat-rank m* (*v* ⊘ *v*) ≤ *1*
⟨*proof*⟩

lemma *rank-foldr-outer-le*:
assumes *vsd*: ∀ *v* ∈ *set vs*. *dim-vec v* = *m*
shows *mat-rank m* (*foldr* (λ*v acc*. *v* ⊘ *v* + *acc*) *vs* (0_m *m m*)) ≤ *length vs*
⟨*proof*⟩

lemma *rank-outer-sum-le*:
assumes *vsd*: ∀ *v* ∈ *set vs*. *dim-vec v* = *m*
shows *mat-rank m* (*outer-sum m vs*) ≤ *length vs*
⟨*proof*⟩

29.2 Independent vectors inside the column space bound the rank from below

lemma *mult-mat-vec-in-span*:
assumes *Ac*: $A \in$ *carrier-mat m k* **and** *xc*: $x \in$ *carrier-vec k*
shows $A *_v x \in$ *module.span class-ring* (*module-vec TYPE*(*complex*) *m*) (*set* (*cols A*))
⟨*proof*⟩

lemma *rank-ge-indep-in-colspace*:
assumes *Ac*: $A \in$ *carrier-mat m k* **and** *li*: *lin-indep-vecs m ws*
and *insp*: $\bigwedge w.\ w \in$ *set ws* $\Longrightarrow$ $w \in$ *module.span class-ring* (*module-vec TYPE*(*complex*) *m*) (*set* (*cols A*))
shows *length ws* $\leq$ *mat-rank m A*
⟨*proof*⟩

29.3 The theorem

lemma *choi-rank-attained*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ A \in$ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\exists$ *As* :: *complex Matrix.mat list*. *length As* = *choi-rank n d* Φ $\wedge$
$(\forall A \in$ *set As*. $A \in$ *carrier-mat d n*$) \wedge$
$(\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ X =$ *kraus-map-dim d As X*$)$
⟨*proof*⟩

theorem *choi-rank-eq-rank*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ A \in$ *carrier-mat d d*
and *cp*: *completely-positive n d* Φ
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows *choi-rank n d* Φ = *mat-rank* $(n*d)$ (*choi-matrix n d* Φ)
⟨*proof*⟩

end
theory *Day21-Unitary-Freedom*
imports *QC-Prelims*
begin

30 Day 21: unitary freedom of rank-one decompositions and Kraus representations (D21-3, D21-5, D21-6)

30.1 The Gram matrix of an independent list is invertible; dual vectors

lemma *gram-det-nonzero*:
assumes *li*: *lin-indep-vecs n vs*
shows *det* $((\textit{mat-of-cols}\ n\ \textit{vs})^\dagger *$ *mat-of-cols n vs*$) \neq 0$
⟨*proof*⟩

lemma *gram-inverse*:
assumes *li*: *lin-indep-vecs n vs*
obtains *B* **where** $B \in$ *carrier-mat* (*length vs*) (*length vs*)

and $((\textit{mat-of-cols } n \textit{ vs})^\dagger * \textit{mat-of-cols } n \textit{ vs}) * B = 1_m\ (\textit{length vs})$
and $B * ((\textit{mat-of-cols } n \textit{ vs})^\dagger * \textit{mat-of-cols } n \textit{ vs}) = 1_m\ (\textit{length vs})$
and $B^\dagger = B$
⟨*proof*⟩

Dual vectors (the notes' "T^* is onto"): for independent $v_1, \ldots, v_k$ there are $\varphi_1, \ldots, \varphi_k$ with $\langle \varphi_i | v_j \rangle = \delta_{ij}$.

lemma *dual-vectors-exist*:
assumes *li*: *lin-indep-vecs n vs*
shows $\exists\, \varphi s :: \textit{complex Matrix.vec list}.\ \textit{length } \varphi s = \textit{length vs} \wedge$
$(\forall\, \varphi \in \textit{set } \varphi s.\ \textit{dim-vec } \varphi = n) \wedge$
$(\forall\, i < \textit{length vs}.\ \forall\, j < \textit{length vs}.\ \langle \varphi s\ !\ i \mid \textit{vs}\ !\ j \rangle = (\textit{if } i = j \textit{ then 1 else 0}))$
⟨*proof*⟩

30.2 D21-3: independent vectors give independent rank-one matrices

definition *outer-prods* :: *complex Matrix.vec list* ⇒ *complex Matrix.mat list* **where**
outer-prods vs = *map* (λ*k*. *vs* ! (*k div length vs*) ⊘ *vs* ! (*k mod length vs*)) [*0*..<*length vs* ∗ *length vs*]

lemma *length-outer-prods* [*simp*]: *length* (*outer-prods vs*) = *length vs* ∗ *length vs*
⟨*proof*⟩

lemma *outer-prods-nth*:
assumes *k* < *length vs* ∗ *length vs*
shows *outer-prods vs* ! *k* = *vs* ! (*k div length vs*) ⊘ *vs* ! (*k mod length vs*)
⟨*proof*⟩

lemma *inner-prod-cnj-delta*:
assumes *dim-vec* $\varphi = n$ **and** *dim-vec* $v = n$ **and** $\langle \varphi \mid v \rangle = (\textit{if } P \textit{ then 1 else 0})$
shows $\langle v \mid \varphi \rangle = (\textit{if } P \textit{ then 1 else 0})$
⟨*proof*⟩

proposition *outer-prods-lin-indep*:
assumes *li*: *lin-indep-vecs n vs*
shows *mat-lin-indep-complex n* (*outer-prods vs*)
⟨*proof*⟩

30.3 Isometric mixing of columns preserves VV^*

lemma *dagger-transpose-mult*:
assumes *Uc*: $U \in \textit{carrier-mat } m\ r$ **and** *UU*: $(U^\dagger) * U = 1_m\ r$
shows $\textit{transpose-mat } U * ((\textit{transpose-mat } U)^\dagger) = 1_m\ r$
⟨*proof*⟩

lemma *isometry-mix-outer*:
assumes *Vc*: $V \in \textit{carrier-mat } n\ r$ **and** *Uc*: $U \in \textit{carrier-mat } m\ r$ **and** *UU*: $(U^\dagger) * U = 1_m\ r$
shows $(V * \textit{transpose-mat } U) * ((V * \textit{transpose-mat } U)^\dagger) = V * (V^\dagger)$
⟨*proof*⟩

lemma *dagger-mult-zero-imp-zero*:
fixes *Y* :: *complex Matrix.mat*
assumes *Yc*: $Y \in \textit{carrier-mat } p\ q$ **and** *YY*: $(Y^\dagger) * Y = 0_m\ q\ q$
shows $Y = 0_m\ p\ q$
⟨*proof*⟩

30.4 D21-5: two rank-one decompositions are related by an isometry

If $P = \sum_{j<r} v_j v_j^* = \sum_{l<m} w_l w_l^*$ with $v_1, \ldots, v_r$ linearly independent (the notes assume rank $P = r$, which is equivalent), then $w_l = \sum_j \alpha_{lj} v_j$ with $U = (\alpha_{lj})$ an $m \times r$ isometry, $U^* U = I_r$. Matrix form: $W = V U^t$.

theorem *rank-one-decomp-isometry*:
  assumes *li*: *lin-indep-vecs n vs*
    and *wsd*: $\forall\, w \in$ *set ws. dim-vec w* $= n$
    and *same*: *outer-sum n vs* $=$ *outer-sum n ws*
  shows $\exists\, U ::$ *complex Matrix.mat. U* $\in$ *carrier-mat* (*length ws*) (*length vs*) $\wedge$
      ($U^\dagger$) $*$ *U* $= 1_m$ (*length vs*) $\wedge$
      mat-of-cols n ws $=$ *mat-of-cols n vs* $*$ *transpose-mat U*
⟨*proof*⟩

corollary *rank-one-decomp-isometry-cols*:
  assumes *li*: *lin-indep-vecs n vs*
    and *wsd*: $\forall\, w \in$ *set ws. dim-vec w* $= n$
    and *same*: *outer-sum n vs* $=$ *outer-sum n ws*
  shows $\exists\, U ::$ *complex Matrix.mat. U* $\in$ *carrier-mat* (*length ws*) (*length vs*) $\wedge$
      ($U^\dagger$) $*$ *U* $= 1_m$ (*length vs*) $\wedge$
      ($\forall\, l <$ *length ws. ws* ! *l* $=$ *mat-of-cols n vs* $*_v$ *Matrix.row U l*)
⟨*proof*⟩

30.5 D21-6 (Choi 2): unitary freedom of Kraus representations

lemma *vec-of-kraus-mat-of-cols-index*:
  assumes *Vs-c*: $\forall$ *V* $\in$ *set Vs. V* $\in$ *carrier-mat d n* **and** *dpos*: *0* $<$ *d*
    and *i*: *i* $<$ *d* **and** *a*: *a* $<$ *n* **and** *k*: *k* $<$ *length Vs*
  shows *mat-of-cols* (*n*$*$*d*) (*map* (*vec-of-kraus d n*) *Vs*) $$ (*a*$*$*d* $+$ *i*, *k*) $=$ *Vs* ! *k* $$ (*i*, *a*)
⟨*proof*⟩

lemma *lin-indep-complex-mat-vec-of-kraus*:
  assumes *li*: *lin-indep-complex-mat d n Vs* **and** *dpos*: *0* $<$ *d*
  shows *lin-indep-vecs* (*n*$*$*d*) (*map* (*vec-of-kraus d n*) *Vs*)
⟨*proof*⟩

Choi's Theorem 2: if $\Phi(X) = \sum_{j<r} V_j X V_j^* = \sum_{l<m} W_l X W_l^*$ with $V_1, \ldots, V_r$ linearly independent (the notes: $r = \mathrm{cr}(\Phi)$), then $W_l = \sum_j \alpha_{lj} V_j$ with $U = (\alpha_{lj})$ an isometry, $U^* U = I_r$.

theorem *choi-2-unitary-freedom*:
  assumes *npos*: *0* $<$ *n* **and** *dpos*: *0* $<$ *d*
    and *Ws-c*: $\forall$ *W* $\in$ *set Ws. W* $\in$ *carrier-mat d n*
    and *li*: *lin-indep-complex-mat d n Vs*
    and *same*: $\forall$ *X. X* $\in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim d Vs X* $=$ *kraus-map-dim d Ws X*
  shows $\exists\, U ::$ *complex Matrix.mat. U* $\in$ *carrier-mat* (*length Ws*) (*length Vs*) $\wedge$
      ($U^\dagger$) $*$ *U* $= 1_m$ (*length Vs*) $\wedge$
      ($\forall\, l <$ *length Ws. Ws* ! *l* $=$
        foldr (λj *acc. U* $$ (*l*,*j*) $\cdot_m$ (*Vs* ! *j*) $+$ *acc*) [*0*..<*length Vs*] (*0*$_m$ *d n*))
⟨*proof*⟩

end
theory *Day21-Ranges*
  imports *Day04-29-Supplement Day02-11-Supplement*
begin

31 Day 21: ranges of positive matrices (D21-4)

The notes write $R(P)$ for the range of P (here *mat-range k P*) and use two facts: if $0 \le ww^* \le P$ then $w \in R(P)$, and $R(\sum_l w_l w_l^*) = \mathrm{span}\{w_l\}$ (*col-span*). Both rest on one lemma: for Hermitian P, every vector orthogonal to $\ker P$ lies in $R(P)$; it is proved from the spectral decomposition $P = UDU^*$ by inverting the nonzero eigenvalues. The audit's earlier formal counterpart was the kernel form *loewner-ker-subset*.

definition *mat-range* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec set* **where**
  mat-range k P = {*P* ∗*v h* | *h*. *dim-vec h* = *k*}

lemma *diag-mat-of-real-mult-vec*:
  assumes *length D* = *k dim-vec v* = *k*
  shows *diag-mat-of-real D* ∗*v v* = *Matrix.vec k* (λ*i*. *complex-of-real* (*D* ! *i*) ∗ *v* $ *i*)
⟨*proof*⟩

For Hermitian P, a vector orthogonal to $\ker P$ lies in the range of P.

lemma *hermitian-perp-ker-in-range*:
  assumes *P*: *P* ∈ *carrier-mat k k* **and** *h*: *hermitian P* **and** *w*: *dim-vec w* = *k*
    and *perp*: ⋀*x*. *dim-vec x* = *k* ⟹ *P* ∗*v x* = 0*v k* ⟹ *Quantum.inner-prod x w* = 0
  shows *w* ∈ *mat-range k P*
⟨*proof*⟩

The notes: if $0 \le ww^* \le P$ then $w \in R(P)$.

theorem *dominated-rank-one-in-range*:
  assumes *P*: *P* ∈ *carrier-mat k k* **and** *Pp*: *psd P* **and** *w*: *dim-vec w* = *k*
    and *dom*: *psd* (*P* − *outer-sum k* [*w*])
  shows *w* ∈ *mat-range k P*
⟨*proof*⟩

The notes: if $P = \sum_l w_l w_l^*$ then $R(P) = \mathrm{span}\{w_1, \ldots, w_m\}$.

theorem *outer-sum-range*:
  assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *k*
  shows *mat-range k* (*outer-sum k vs*) = *col-span k vs*
⟨*proof*⟩

31.1 Day 4 revisited: positive definiteness and spanning (D4-8)

The notes: $P = \sum_i |v_i\rangle\langle v_i| > 0$ iff $\mathrm{span}\{v_i\} = \mathbb{C}^n$.

theorem *pd-iff-span-all*:
  assumes *vs*: ∀ *v* ∈ *set vs*. *dim-vec v* = *n*
  shows *pd* (*outer-sum n vs*) ⟷ *col-span n vs* = *carrier-vec n*
⟨*proof*⟩

end
theory *Day21-Average-Conjecture*
  imports *Quantum-Computing11*
begin

32 Day 21: the conjecture on averages of unital maps (D21-2)

The notes ask whether every unital completely positive $\Phi : M_n \to M_d$ is the average of n unital completely positive maps of Choi rank at most d (*D21-2*). This is the strong Ruskai–Audenaert conjecture for the dual channel, open in general. Proved here: Fourier mixing settles every map whose Kraus operators split into n groups with vanishing cross sums (*fourier-split*); this covers

measurement maps (*ra-measurement*), and Horn's lemma (*horn-lemma*, the case $d = 1$) covers maps with diagonal values (*ra-cq*). The notes' form (i) is false as written (*notes-form-i-false*).

lemma *conj-entry*:
assumes *W*: $W \in$ *carrier-mat d n* **and** *X*: $X \in$ *carrier-mat n n* **and** *a*: $a < d$ **and** *b*: $b < d$
shows $(W * X * (W^\dagger))$ $\$\$ (a, b) = (\sum p{<}n.\ \sum q{<}n.\ W$ $\$\$ (a, p) * X$ $\$\$ (p, q) * cnj (W$ $\$\$ (b, q)))$
⟨*proof*⟩

lemma *sum-list-concat-upt*:
sum-list (*map f* (*concat* (*map* (λ*m*. *map* (*g m*) [*0*..<*d*]) [*0*..<*n*])))) = $(\sum m{<}n.\ \sum j{<}d.\ f\ (g\ m\ j))$
⟨*proof*⟩

Characters of $\mathbb{Z}_n$.

definition *om* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex* **where**
om n l m = *cis* (*2* ∗ *pi* ∗ *real l* ∗ *real m* / *real n*)

lemma *om-norm* [*simp*]: *om n l m* ∗ *cnj* (*om n l m*) = *1*
⟨*proof*⟩

lemma *om-orth*:
assumes *m*: $m < n$ **and** *m′*: $m' < n$
shows $(\sum l{<}n.$ *om n l m* ∗ *cnj* (*om n l m′*)) = (*if m* = *m′ then of-nat n else 0*)
⟨*proof*⟩

lemma *om-mix*:
$(\sum l{<}n.\ (\sum m{<}n.$ *om n l m* ∗ *x m*) ∗ *cnj* $(\sum m{<}n.$ *om n l m* ∗ *y m*)) = *of-nat n* ∗ $(\sum m{<}n.$ *x m* ∗ *cnj* (*y m*))
⟨*proof*⟩

lemma *kraus-concat-entry*:
assumes *g*: $\bigwedge m\ j.\ m < k \Longrightarrow j < d \Longrightarrow$ *g m j* ∈ *carrier-mat d n* **and** *X*: $X \in$ *carrier-mat n n* **and** *a*: $a < d$ **and** *b*: $b < d$
shows *kraus-map-dim d* (*concat* (*map* (λ*m*. *map* (*g m*) [*0*..<*d*]) [*0*..<*k*])) *X* $\$\$ (a, b) = (\sum m{<}k.$ $\sum j{<}d.$ (*g m j* ∗ *X* ∗ ((*g m j*)†)) $\$\$ (a, b))$
⟨*proof*⟩

lemma *kraus-concat-carrier*:
assumes *g*: $\bigwedge m\ j.\ m < k \Longrightarrow j < d \Longrightarrow$ *g m j* ∈ *carrier-mat d n* **and** *X*: $X \in$ *carrier-mat n n*
shows *kraus-map-dim d* (*concat* (*map* (λ*m*. *map* (*g m*) [*0*..<*d*]) [*0*..<*k*])) *X* ∈ *carrier-mat d d*
⟨*proof*⟩

lemma *prod-dagger-entry*:
assumes *A*: $A \in$ *carrier-mat d n* **and** *B*: $B \in$ *carrier-mat d n* **and** *a*: $a < d$ **and** *b*: $b < d$
shows $(A * (B^\dagger))$ $\$\$ (a, b) = (\sum p{<}n.\ A$ $\$\$ (a, p) * cnj (B$ $\$\$ (b, p)))$
⟨*proof*⟩

D21-2 in the notes' main form: a unital $\Phi : M_n \to M_d$ is the average of n unital completely positive maps of Choi rank at most d.

definition *ra-split* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
ra-split n d Φ ⟷ (∃ *Bs*. *length Bs* = *n* ∧ (∀ *l*<*n*. *length* (*Bs* ! *l*) ≤ *d* ∧ (∀ *B* ∈ *set* (*Bs* ! *l*). *B* ∈ *carrier-mat d n*)
∧ *kraus-map-dim d* (*Bs* ! *l*) (*1*$_m$ *n*) = *1*$_m$ *d*) ∧ (∀ *X* ∈ *carrier-mat n n*. Φ *X* = (*1* / *of-nat n*) ·$_m$ *kraus-map-dim d* (*concat Bs*) *X*))

The conjecture itself, for maps given by Kraus operators (every completely positive map is one). It is the strong Ruskai–Audenaert conjecture for the dual channel $M_d \to M_n$, open in general; the cases below are the ones settled in the literature by direct constructions.

definition *D21-2* :: *bool* **where**

D21-2 $\longleftrightarrow$ $(\forall\, n\ d\ As.\ 0 < n \longrightarrow (\forall\, A \in set\ As.\ A \in carrier\text{-}mat\ d\ n) \longrightarrow kraus\text{-}map\text{-}dim\ d\ As\ (1_m\ n) = 1_m\ d$
$\longrightarrow$ *ra-split n d* (*kraus-map-dim d As*))

definition *fmix* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ (*nat* $\Rightarrow$ *nat* $\Rightarrow$ *complex Matrix.mat*) $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *complex Matrix.mat* **where**
fmix n d A l j = *Matrix.mat d n* ($\lambda(a,\ p).\ \sum m{<}n.\ om\ n\ l\ m * A\ m\ j\ \$\$\ (a,\ p)$)

lemma *fmix-carrier* [*simp*]: *fmix n d A l j* $\in$ *carrier-mat d n* ⟨*proof*⟩

lemma *fmix-index*: $a < d \Longrightarrow p < n \Longrightarrow$ *fmix n d A l j* \$\$ (*a*, *p*) = ($\sum m{<}n.\ om\ n\ l\ m * A\ m\ j\ \$\$\ (a,\ p)$)
⟨*proof*⟩

If the Kraus operators fall into n groups of d with $\sum_j A_{mj} A^*_{m'j} = 0$ for $m \neq m'$, Fourier mixing $B_{lj} = \sum_m \omega^{lm} A_{mj}$ gives the decomposition.

theorem *fourier-split*:
assumes *n*: $0 < n$ **and** *A*: $\bigwedge m\ j.\ m < n \Longrightarrow j < d \Longrightarrow A\ m\ j \in carrier\text{-}mat\ d\ n$
and *u*: *kraus-map-dim d* (*concat* (*map* ($\lambda m.\ map\ (A\ m)\ [0..{<}d]$) $[0..{<}n]$)) $(1_m\ n) = 1_m\ d$
and *orth*: $\bigwedge m\ m'\ a\ b.\ m < n \Longrightarrow m' < n \Longrightarrow m \neq m' \Longrightarrow a < d \Longrightarrow b < d \Longrightarrow (\sum j{<}d.\ (A\ m\ j * ((A\ m'\ j)^\dagger))\ \$\$\ (a,\ b)) = 0$
shows *ra-split n d* (*kraus-map-dim d* (*concat* (*map* ($\lambda m.\ map\ (A\ m)\ [0..{<}d]$) $[0..{<}n]$)))
⟨*proof*⟩

32.1 Positive matrices as sums of d rank-one terms

lemma *udu-entry*:
assumes *U*: $U \in carrier\text{-}mat\ d\ d$ **and** *D*: *length D* = *d* **and** *a*: $a < d$ **and** *b*: $b < d$
shows ($U * diag\text{-}mat\text{-}of\text{-}real\ D * (U^\dagger)$) \$\$ (*a*, *b*) = ($\sum k{<}d.\ U\ \$\$\ (a,\ k) * complex\text{-}of\text{-}real\ (D\ !\ k) * cnj\ (U\ \$\$\ (b,\ k))$)
⟨*proof*⟩

lemma *psd-eig*:
assumes *Q*: $Q \in carrier\text{-}mat\ d\ d\ psd\ Q$
shows $\exists\, U\ D.\ U \in carrier\text{-}mat\ d\ d \wedge length\ D = d \wedge (\forall\, k{<}d.\ 0 \leq D\ !\ k) \wedge (U^\dagger) * U = 1_m\ d$
$\wedge\ (\forall\, a{<}d.\ \forall\, b{<}d.\ Q\ \$\$\ (a,\ b) = (\sum k{<}d.\ U\ \$\$\ (a,\ k) * complex\text{-}of\text{-}real\ (D\ !\ k) * cnj\ (U\ \$\$\ (b,\ k))))$
⟨*proof*⟩

lemma *psd-vecs*:
assumes *Q*: $Q \in carrier\text{-}mat\ d\ d\ psd\ Q$
shows $\exists\, q.\ \forall\, a{<}d.\ \forall\, b{<}d.\ Q\ \$\$\ (a,\ b) = (\sum k{<}d.\ q\ k\ a * cnj\ (q\ k\ b))$
⟨*proof*⟩

lemma *ra-split-cong*: $(\bigwedge X.\ X \in carrier\text{-}mat\ n\ n \Longrightarrow \Phi\ X = \Psi\ X) \Longrightarrow ra\text{-}split\ n\ d\ \Psi \Longrightarrow ra\text{-}split\ n\ d\ \Phi$
⟨*proof*⟩

lemma *delta-sum2*:
fixes *n i* :: *nat*
assumes *i*: $i < n$
shows ($\sum p{<}n.\ \sum p'{<}n.\ (if\ p = i\ then\ x\ else\ 0) * f\ p\ p' * cnj\ (if\ p' = i\ then\ y\ else\ 0)) = x * f\ i\ i * cnj\ (y :: complex)$
⟨*proof*⟩

32.2 Measurement maps

$\Phi(X) = \sum_i X_{ii} Q_i$ with $Q_i \geq 0$, $\sum_i Q_i = I$ (dual to a channel with classical output): D21-2 holds.

definition *povm-map* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
povm-map n d Q X = *Matrix.mat d d* (λ(*a*, *b*). ∑ *i*<*n*. *X* $$ (*i*, *i*) ∗ *Q i* $$ (*a*, *b*))

theorem *ra-measurement*:
assumes *n*: *0* < *n* **and** *Q*: ⋀*i*. *i* < *n* ⟹ *Q i* ∈ *carrier-mat d d* ∧ *psd* (*Q i*)
and *QI*: ⋀*a b*. *a* < *d* ⟹ *b* < *d* ⟹ (∑ *i*<*n*. *Q i* $$ (*a*, *b*)) = *1*$_m$ *d* $$ (*a*, *b*)
shows *ra-split n d* (*povm-map n d Q*)
⟨*proof*⟩

32.3 The case $d = 1$: Horn's lemma

Every density matrix on $\mathbb{C}^n$ is the average of n pure states (Fourier mixing of an eigenbasis).

theorem *horn-lemma*:
assumes ϱ: *density-matrix n* ϱ **and** *n*: *0* < *n*
shows ∃ φ. (∀ *l*<*n*. *dim-vec* (φ *l*) = *n* ∧ ‖φ *l*‖ = *1*)
∧ (∀ *a*<*n*. ∀ *b*<*n*. ϱ $$ (*a*, *b*) = (*1* / *of-nat n*) ∗ (∑ *l*<*n*. φ *l* $ *a* ∗ *cnj* (φ *l* $ *b*)))
⟨*proof*⟩

lemma *delta-pair*:
fixes *c c*′ *d* :: *nat*
assumes *c* < *d*
shows (∑ *a*<*d*. *if c* = *a* ∧ *c*′ = *a then f a else 0*) = (*if c* = *c*′ *then f c else* (*0*::*complex*))
⟨*proof*⟩

32.4 Maps with diagonal values

$\Phi(X) = \mathrm{diag}(\mathrm{tr}(R_1 X), \ldots, \mathrm{tr}(R_d X))$ with density matrices R_a (dual to a channel with classical input): D21-2 holds, by Horn's lemma for each R_a.

definition *cq-map* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
cq-map n d Rh X = *Matrix.mat d d* (λ(*a*, *b*). *if a* = *b then* (∑ *p*<*n*. ∑ *q*<*n*. *Rh a* $$ (*q*, *p*) ∗ *X* $$ (*p*, *q*)) *else 0*)

theorem *ra-cq*:
assumes *n*: *0* < *n* **and** *Rh*: ⋀*a*. *a* < *d* ⟹ *density-matrix n* (*Rh a*)
shows *ra-split n d* (*cq-map n d Rh*)
⟨*proof*⟩

32.5 The notes' form (i) is false as written

Form (i) asks for n trace-preserving maps $M_n \to M_d$ of Choi rank at most d. For $n = 2$, $d = 1$ there is no trace-preserving map $M_2 \to \mathbb{C}$ of Choi rank at most one, while the trace is trace preserving. (The equivalent dual form has the dimensions exchanged: maps $M_d \to M_n$.)

lemma *no-tp-rank-one*: *length Ws* ≤ *1* ⟹ ¬ *kraus-operators 2 1 Ws*
⟨*proof*⟩

theorem *notes-form-i-false*:
¬ (∀ *Ws*. *kraus-operators 2 1 Ws* ⟶ (∃ *Bs*. *length Bs* = *2* ∧ (∀ *l*<*2*. *length* (*Bs* ! *l*) ≤ *1* ∧ *kraus-operators 2 1* (*Bs* ! *l*))
∧ (∀ *X* ∈ *carrier-mat 2 2*. *kraus-map-dim 1 Ws X* = (*1* / *2*) ·$_m$ *kraus-map-dim 1* (*concat Bs*) *X*)))
⟨*proof*⟩

end
theory *Day21-Average-Two*
imports *Day21-Average-Conjecture*

begin

33 D21-2 for $n = 2$

Every unital completely positive $\Phi : M_2 \to M_d$ is the average of two unital maps of Choi rank at most d (*ra-two*; Bacon, Smolin and Wolf). The off-diagonal block of the Choi matrix is $R_1 K R_2^*$ with $R_i R_i^*$ the diagonal blocks and K a contraction (*wide-factor*, *bessel-rows*), and a contraction is the average of two unitaries (*contraction-avg*, through a singular value decomposition, *svd-square*).

33.1 Completing an orthonormal family to an orthonormal basis

lemma *onb-complete*:
fixes d :: *nat* **and** J :: *nat set* **and** w :: *nat* ⇒ *nat* ⇒ *complex*
assumes *J*: $J \subseteq \{..<d\}$
and *on*: $\bigwedge k\ k'.\ k \in J \Longrightarrow k' \in J \Longrightarrow (\sum a{<}d.\ cnj\ (w\ k\ a) * w\ k'\ a) = (if\ k = k'\ then\ 1\ else\ 0)$
shows $\exists W.\ W \in carrier\text{-}mat\ d\ d \wedge (W^\dagger) * W = 1_m\ d \wedge (\forall k{\in}J.\ \forall a{<}d.\ W\ \$\$\ (a,\ k) = w\ k\ a)$
⟨*proof*⟩

33.2 Singular value decomposition

lemma *unitary-sq-right*: $V \in carrier\text{-}mat\ d\ d \Longrightarrow (V^\dagger) * V = 1_m\ d \Longrightarrow V * (V^\dagger) = 1_m\ d$
⟨*proof*⟩

lemma *svd-square*:
assumes *Km*: $Km \in carrier\text{-}mat\ d\ d$
shows $\exists W\ V.\ W \in carrier\text{-}mat\ d\ d \wedge (W^\dagger) * W = 1_m\ d \wedge V \in carrier\text{-}mat\ d\ d \wedge (V^\dagger) * V = 1_m\ d$
$\wedge\ (\forall a{<}d.\ \forall b{<}d.\ Km\ \$\$\ (a,\ b) = (\sum k{<}d.\ W\ \$\$\ (a,\ k) * complex\text{-}of\text{-}real\ (sqrt\ (\sum c{<}d.\ (cmod\ ((Km * V)\ \$\$\ (c,\ k)))^2)) * cnj\ (V\ \$\$\ (b,\ k))))$
⟨*proof*⟩

33.3 A contraction is the average of two unitaries

definition *phase-diag* :: *nat* ⇒ (*nat* ⇒ *real*) ⇒ *complex Matrix.mat* **where**
$phase\text{-}diag\ d\ \vartheta = Matrix.mat\ d\ d\ (\lambda(i,\ j).\ if\ i = j\ then\ cis\ (\vartheta\ i)\ else\ 0)$

lemma *phase-diag-carrier* [*simp*]: $phase\text{-}diag\ d\ \vartheta \in carrier\text{-}mat\ d\ d$ ⟨*proof*⟩

lemma *phase-diag-unitary*: $(phase\text{-}diag\ d\ \vartheta)^\dagger * phase\text{-}diag\ d\ \vartheta = 1_m\ d$
⟨*proof*⟩

lemma *wdv-entry*:
assumes *W*: $W \in carrier\text{-}mat\ d\ d$ **and** *V*: $V \in carrier\text{-}mat\ d\ d$ **and** *a*: $a < d$ **and** *b*: $b < d$
shows $(W * phase\text{-}diag\ d\ \vartheta * (V^\dagger))\ \$\$\ (a,\ b) = (\sum k{<}d.\ W\ \$\$\ (a,\ k) * cis\ (\vartheta\ k) * cnj\ (V\ \$\$\ (b,\ k)))$
⟨*proof*⟩

lemma *unitary-triple*:
assumes *W*: $W \in carrier\text{-}mat\ d\ d\ (W^\dagger) * W = 1_m\ d$ **and** *V*: $V \in carrier\text{-}mat\ d\ d\ (V^\dagger) * V = 1_m\ d$
and *D*: $D \in carrier\text{-}mat\ d\ d\ (D^\dagger) * D = 1_m\ d$
shows $(W * D * (V^\dagger))^\dagger * (W * D * (V^\dagger)) = 1_m\ d$
⟨*proof*⟩

theorem *contraction-avg*:
assumes *Km*: $Km \in carrier\text{-}mat\ d\ d$ **and** *ct*: $\bigwedge x.\ dim\text{-}vec\ x = d \Longrightarrow \|Km *_v x\| \leq \|x\|$

shows $\exists$ *U1 U2*. *U1* $\in$ *carrier-mat d d* $\wedge$ *U2* $\in$ *carrier-mat d d* $\wedge$ (*U1*†) $*$ *U1* = 1_m *d* $\wedge$ (*U2*†) $*$ *U2* = 1_m *d*
$\wedge$ ($\forall$ *a*<*d*. $\forall$ *b*<*d*. *Km* $$ (*a*, *b*) = (*U1* $$ (*a*, *b*) + *U2* $$ (*a*, *b*)) / 2)
⟨*proof*⟩

33.4 Bessel's inequality for rows that are orthonormal or zero

lemma *cmod-sq-sum*: *complex-of-real* ($\sum$ *k*∈*A*. (*cmod* (*f k*))^2) = ($\sum$ *k*∈*A*. *f k* $*$ *cnj* (*f k*))
⟨*proof*⟩

lemma *bessel-rows*:
fixes *s* :: *nat* ⇒ *nat* ⇒ *complex* **and** *y* :: *nat* ⇒ *complex* **and** *d K* :: *nat*
assumes *on*: $\bigwedge$*j j'*. *j* < *d* ⟹ *j'* < *d* ⟹ ($\sum$ *k*<*K*. *s j k* $*$ *cnj* (*s j' k*)) = (*if j* = *j'* $\wedge$ *j* ∈ *J then 1 else 0*)
shows ($\sum$ *j*<*d*. (*cmod* ($\sum$ *k*<*K*. *s j k* $*$ *y k*))^2) ≤ ($\sum$ *k*<*K*. (*cmod* (*y k*))^2)
⟨*proof*⟩

33.5 Factoring a wide matrix

$M = RS$ with $RR^* = MM^*$ and the rows of S orthonormal or zero.

lemma *wide-factor*:
fixes *m* :: *nat* ⇒ *nat* ⇒ *complex* **and** *d Kn* :: *nat*
shows $\exists$ *R S J*. *R* ∈ *carrier-mat d d*
$\wedge$ ($\forall$ *k*<*Kn*. $\forall$ *a*<*d*. *m k a* = ($\sum$ *j*<*d*. *R* $$ (*a*, *j*) $*$ *S j k*))
$\wedge$ ($\forall$ *a*<*d*. $\forall$ *b*<*d*. ($\sum$ *j*<*d*. *R* $$ (*a*, *j*) $*$ *cnj* (*R* $$ (*b*, *j*))) = ($\sum$ *k*<*Kn*. *m k a* $*$ *cnj* (*m k b*)))
$\wedge$ ($\forall$ *j*<*d*. $\forall$ *j'*<*d*. ($\sum$ *k*<*Kn*. *S j k* $*$ *cnj* (*S j' k*)) = (*if j* = *j'* $\wedge$ *j* ∈ *J then 1 else 0*))
⟨*proof*⟩

33.6 The case $n = 2$

definition *gk* :: *complex Matrix.mat list* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex* **where**
gk Fs p q i i' = ($\sum$ *k*<*length Fs*. *Fs* ! *k* $$ (*i*, *p*) $*$ *cnj* (*Fs* ! *k* $$ (*i'*, *q*)))

lemma *kraus2-entry*:
assumes *F*: $\forall$ *A* ∈ *set Fs*. *A* ∈ *carrier-mat d 2* **and** *X*: *X* ∈ *carrier-mat 2 2* **and** *i*: *i* < *d* **and** *i'*: *i'* < *d*
shows *kraus-map-dim d Fs X* $$ (*i*, *i'*) = ($\sum$ *p*<*2*. $\sum$ *q*<*2*. *X* $$ (*p*, *q*) $*$ *gk Fs p q i i'*)
⟨*proof*⟩

lemma *gk-map*:
gk (*map f* [*0*..<*d*]) *p q i i'* = ($\sum$ *j*<*d*. *f j* $$ (*i*, *p*) $*$ *cnj* (*f j* $$ (*i'*, *q*)))
⟨*proof*⟩

lemma *mult-entry3*:
assumes *A*: *A* ∈ *carrier-mat d d* **and** *B*: *B* ∈ *carrier-mat d d* **and** *C*: *C* ∈ *carrier-mat d d* **and** *i*: *i* < *d* **and** *i'*: *i'* < *d*
shows (*A* $*$ *B* $*$ (*C*†)) $$ (*i*, *i'*) = ($\sum$ *j*<*d*. $\sum$ *m*<*d*. *A* $$ (*i*, *j*) $*$ *B* $$ (*j*, *m*) $*$ *cnj* (*C* $$ (*i'*, *m*)))
⟨*proof*⟩

Every unital $\Phi : M_2 \to M_d$ is the average of two unital maps of Choi rank at most d (Bacon–Smolin–Wolf): the off-diagonal Choi block is $R_1 K R_2^*$ with K a contraction, and $K = \frac{1}{2}(U_1 + U_2)$.

theorem *ra-two*:
assumes *As*: $\forall$ *A* ∈ *set As*. *A* ∈ *carrier-mat d 2* **and** *u*: *kraus-map-dim d As* (1_m *2*) = 1_m *d*
shows *ra-split 2 d* (*kraus-map-dim d As*)
⟨*proof*⟩

end

theory *Day22-Kraus-Independence*
  imports *QC-Prelims Jordan-Normal-Form.DL-Rank*
begin

34 Day 22: linear independence bounds the number of Kraus operators (D22-6)

A linearly independent list of vectors in $\mathbb{C}^n$ has at most n members. The bridge is to JNF's *vec-space*: kernel-triviality of *mat-of-cols* is JNF's *lin-indpt* (via *vec-space.lin-depE*), and *li-le-dim* bounds the cardinality.

lemma *lin-indep-vecs-distinct*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *distinct vs*
⟨*proof*⟩

lemma *lin-indep-vecs-length-le*:
  assumes *li*: *lin-indep-vecs n vs*
  shows *length vs* ≤ *n*
⟨*proof*⟩

Flattening a matrix into a vector carries *mat-lin-indep-complex* to *lin-indep-vecs*, so a linearly independent family in M_n has at most n^2 members.

definition *flatten-mat* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* **where**
  flatten-mat n B = *Matrix.vec* (*n*∗*n*) (λ*k*. *B* $$ (*k div n*, *k mod n*))

lemma *flatten-mat-dim* [*simp*]: *dim-vec* (*flatten-mat n B*) = *n*∗*n*
  ⟨*proof*⟩

lemma *flatten-mat-index*:
  assumes *i* < *n* **and** *j* < *n*
  shows *flatten-mat n B* $ (*i*∗*n* + *j*) = *B* $$ (*i*,*j*)
⟨*proof*⟩

lemma *mat-lin-indep-complex-flatten*:
  assumes *li*: *mat-lin-indep-complex n Bs*
  shows *lin-indep-vecs* (*n*∗*n*) (*map* (*flatten-mat n*) *Bs*)
⟨*proof*⟩

lemma *mat-lin-indep-complex-length-le*:
  assumes *mat-lin-indep-complex n Bs*
  shows *length Bs* ≤ *n*∗*n*
  ⟨*proof*⟩

Corollary to Choi's Theorem 3 (D22-6): an extreme CPTP map $M_n \to M_d$ with a minimal Kraus representation $(V_i)_{i<r}$ has $r^2 \le n^2$, i.e. $\mathrm{cr}(\Phi) \le n$.

corollary *extreme-CPTP-kraus-length-le*:
  assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
    and Φ-*in*: Φ ∈ *CPTP n d*
    and *rep*: *cptp-kraus-rep n d* Φ *Vs*
    and *minimal*: *lin-indep-complex-mat d n Vs*
    and *ext*: *extreme-CPTP-carrier n d* Φ
  shows *length Vs* ≤ *n*
⟨*proof*⟩

corollary *extreme-CPTP-choi-rank-le*:

assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
and Φ-*in*: $\Phi \in$ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP-carrier n d* Φ
shows *choi-rank n d* $\Phi \leq n$
⟨*proof*⟩

end
theory *Day22-Minimal-Kraus*
imports *Day32-Recovery-Proposition*
begin

35 Day 22: a minimal Kraus representation is linearly independent (D22-1)

35.1 Mixing a Kraus list by an isometry does not change the map

definition *kraus-mix* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* **where**
kraus-mix d n U Vs =
map (λ*l*. *foldr* (λ*j acc*. *U* $$ (*l*,*j*) ·$_m$ (*Vs* ! *j*) + *acc*) [*0*..<*length Vs*] (*0*$_m$ *d n*)) [*0*..<*dim-row U*]

lemma *kraus-mix-length* [*simp*]: *length* (*kraus-mix d n U Vs*) = *dim-row U*
⟨*proof*⟩

lemma *kraus-mix-nth*:
assumes *l* < *dim-row U*
shows *kraus-mix d n U Vs* ! *l* = *foldr* (λ*j acc*. *U* $$ (*l*,*j*) ·$_m$ (*Vs* ! *j*) + *acc*) [*0*..<*length Vs*] (*0*$_m$ *d n*)
⟨*proof*⟩

lemma *kraus-mix-carrier*:
assumes ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
shows ∀ *W* ∈ *set* (*kraus-mix d n U Vs*). *W* ∈ *carrier-mat d n*
⟨*proof*⟩

lemma *kraus-mix-entry*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *l*: *l* < *dim-row U* **and** *i*: *i* < *d* **and** *a*: *a* < *n*
shows *kraus-mix d n U Vs* ! *l* $$ (*i*,*a*) = (∑ *j* = *0*..<*length Vs*. *U* $$ (*l*,*j*) ∗ *Vs* ! *j* $$ (*i*,*a*))
⟨*proof*⟩

lemma *kraus-mix-mat-of-cols*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n* **and** *Uc*: *U* ∈ *carrier-mat m* (*length Vs*) **and** *dpos*: *0* < *d*
shows *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) (*kraus-mix d n U Vs*))
= *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) *Vs*) ∗ *transpose-mat U*
⟨*proof*⟩

lemma *kraus-mix-same-map*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *Uc*: *U* ∈ *carrier-mat m* (*length Vs*) **and** *UU*: (*U*†) ∗ *U* = *1*$_m$ (*length Vs*)
and *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*kraus-mix d n U Vs*) *M* = *kraus-map-dim d Vs M*
⟨*proof*⟩

35.2 A unitary with a prescribed first row

lemma *mat-mult-entry*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat r s B* ∈ *carrier-mat s t k* < *r j* < *t*
shows (*A* ∗ *B*) $$ (*k*,*j*) = (∑ *l*<*s*. *A* $$ (*k*,*l*) ∗ *B* $$ (*l*,*j*))
⟨*proof*⟩

lemma *transpose-unitary-isometry*:
assumes *Mc*: *M* ∈ *carrier-mat r r* **and** *MM*: *M* ∗ (*M*†) = *1*$_m$ *r*
shows ((*transpose-mat M*)†) ∗ *transpose-mat M* = *1*$_m$ *r*
⟨*proof*⟩

lemma *isometry-with-first-row*:
assumes *ud*: *dim-vec u* = *r* **and** *un*: ‖*u*‖ = *1*
shows ∃ *U* :: *complex Matrix.mat*. *U* ∈ *carrier-mat r r* ∧ (*U*†) ∗ *U* = *1*$_m$ *r* ∧ (∀ *j* < *r*. *U* $$ (*0*,*j*) = *u* $ *j*)
⟨*proof*⟩

35.3 D22-1

lemma *kraus-map-dim-zero-head*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n* **and** *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*0*$_m$ *d n* # *Ws*) *M* = *kraus-map-dim d Ws M*
⟨*proof*⟩

If $\Phi(X) = \sum_{j<r} V_j X V_j^*$ with $r = \mathrm{cr}(\Phi)$, then $V_0, \ldots, V_{r-1}$ are linearly independent: a dependency $\sum_j c_j V_j = 0$ would allow a unitary change of representation whose first Kraus operator is 0, giving a representation with $r - 1$ terms.

theorem *minimal-kraus-rep-lin-indep*:
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *minimal*: *choi-rank n d* (*kraus-map-dim d Vs*) = *length Vs*
shows *lin-indep-complex-mat d n Vs*
⟨*proof*⟩

end
theory *Day22-Adjoint-Maps*
imports *Day19-Choi-TP-Unital Day04-29-Supplement Day26-CP-Factorization*
begin

36 Day 22: the adjoint map, an affine bijection from UCP onto CPTP (D22-5)

The notes: UCP(M_n, M_d) and CPTP(M_d, M_n) are convex, and $\Gamma(\Phi) = \Phi^*$ is an affine bijection between them with affine inverse, so the two convex sets are affinely isomorphic. The adjoint was available only for Kraus maps (*kraus-adjoint-dim*); here it is defined for every linear Φ. Maps are total functions on matrices, so Γ is a bijection up to agreement on M_n, resp. M_d: Γ sends linear UCP maps to linear CPTP maps and back, and $\Gamma(\Gamma(\Phi)) = \Phi$ on M_n. The set *UCP* does not record linearity, which the notes' maps have, so it is assumed.

For linear $\Phi : M_n \to M_d$, $\Phi^* : M_d \to M_n$ is determined by $\langle Y|\Phi(X)\rangle = \langle \Phi^*(Y)|X\rangle$; taking $X = E_{ij}$ gives $\Phi^*(Y)_{ij} = \langle \Phi(E_{ij})|Y\rangle$.

definition *map-adjoint* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**

map-adjoint n d Φ Y = Matrix.mat n n (λ(i, j). ⟨⟨ Φ (E⟦i,j ; n,n⟧) | Y ⟩⟩)

lemma *map-adjoint-carrier* [*simp*]: *map-adjoint n d Φ Y ∈ carrier-mat n n*
⟨*proof*⟩

lemma *map-adjoint-dims* [*simp*]: *dim-row* (*map-adjoint n d Φ Y*) = *n dim-col* (*map-adjoint n d Φ Y*) = *n*
⟨*proof*⟩

lemma *lin-entry*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X ∈ carrier-mat n n* **and** *a*: *a < d* **and** *b*: *b < d*
shows Φ *X* $$ (*a*, *b*) = (∑ *i*<*n*. ∑ *j*<*n*. *X* $$ (*i*, *j*) ∗ Φ (*E⟦i,j ; n,n⟧*) $$ (*a*, *b*))
⟨*proof*⟩

theorem *map-adjoint-correct*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X ∈ carrier-mat n n* **and** *Y*: *Y ∈ carrier-mat d d*
shows ⟨⟨ *Y* | Φ *X* ⟩⟩ = ⟨⟨ *map-adjoint n d Φ Y* | *X* ⟩⟩
⟨*proof*⟩

lemma *hs-inner-prod-mat-unit*:
assumes *A*: *A ∈ carrier-mat n n* **and** *i*: *i < n* **and** *j*: *j < n*
shows ⟨⟨ *A* | *E⟦i,j ; n,n⟧* ⟩⟩ = *cnj* (*A* $$ (*i*, *j*))
⟨*proof*⟩

lemma *hs-inner-prod-ext*:
assumes *A*: *A ∈ carrier-mat n n* **and** *B*: *B ∈ carrier-mat n n*
and *eq*: ⋀*X*. *X ∈ carrier-mat n n* ⟹ ⟨⟨ *A* | *X* ⟩⟩ = ⟨⟨ *B* | *X* ⟩⟩
shows *A* = *B*
⟨*proof*⟩

lemma *map-adjoint-lin*: *lin-on d n* (*carrier-mat d d*) (*map-adjoint n d* Φ)
⟨*proof*⟩

lemma *map-adjoint-cong*:
assumes ⋀*X*. *X ∈ carrier-mat n n* ⟹ Φ *X* = Ψ *X*
shows *map-adjoint n d* Φ = *map-adjoint n d* Ψ
⟨*proof*⟩

For a Kraus map the adjoint is *kraus-adjoint-dim*.

lemma *map-adjoint-kraus*:
assumes *As*: ∀ *A ∈ set As*. *A ∈ carrier-mat d n* **and** *Y*: *Y ∈ carrier-mat d d*
shows *map-adjoint n d* (*kraus-map-dim d As*) *Y* = *kraus-adjoint-dim n As Y*
⟨*proof*⟩

$\Phi^{**} = \Phi$.

theorem *map-adjoint-adjoint*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *X*: *X ∈ carrier-mat n n*
shows *map-adjoint d n* (*map-adjoint n d* Φ) *X* = Φ *X*
⟨*proof*⟩

lemma *cp-kraus-form*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *cp*: *completely-positive n d* Φ **and** *n*: *0 < n* **and** *d*: *0 < d*
shows ∃ *Ws*. (∀ *W ∈ set Ws*. *W ∈ carrier-mat d n*) ∧ (∀ *X*. *X ∈ carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d Ws X*)
⟨*proof*⟩

lemma *trace-preserving-cong*:

assumes $\bigwedge X.\ X \in \mathit{carrier\text{-}mat}\ n\ n \Longrightarrow \Phi\ X = \Psi\ X$
shows *trace-preserving* $n\ \Phi \longleftrightarrow$ *trace-preserving* $n\ \Psi$
⟨*proof*⟩

$\Gamma(\Phi) = \Phi^*$ maps the unital CP maps $M_n \to M_d$ to the CPTP maps $M_d \to M_n$ and back.

theorem *adjoint-ucp-imp-cptp*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *U*: $\Phi \in \mathit{UCP}\ n\ d$ **and** *n*: $0 < n$ **and** *d*: $0 < d$
shows *map-adjoint n d* $\Phi \in \mathit{CPTP}\ d\ n$
⟨*proof*⟩

theorem *adjoint-cptp-imp-ucp*:
assumes *L*: *lin-on d n* (*carrier-mat d d*) Ψ **and** *C*: $\Psi \in \mathit{CPTP}\ d\ n$ **and** *n*: $0 < n$ **and** *d*: $0 < d$
shows *map-adjoint d n* $\Psi \in \mathit{UCP}\ n\ d$
⟨*proof*⟩

Γ is affine (for real weights), and $\Gamma^{-1} = \Gamma$ by *map-adjoint-adjoint*.

lemma *hs-inner-prod-left-real-comb*:
fixes *t s* :: *real*
assumes *A*: $A \in \mathit{carrier\text{-}mat}\ d\ d$ **and** *B*: $B \in \mathit{carrier\text{-}mat}\ d\ d$ **and** *Y*: $Y \in \mathit{carrier\text{-}mat}\ d\ d$
shows ⟨⟨ *complex-of-real t* $\cdot_m$ *A* + *complex-of-real s* $\cdot_m$ *B* | *Y* ⟩⟩
= *complex-of-real t* $*$ ⟨⟨ *A* | *Y* ⟩⟩ + *complex-of-real s* $*$ ⟨⟨ *B* | *Y* ⟩⟩
⟨*proof*⟩

theorem *map-adjoint-affine*:
fixes *t* :: *real*
assumes *L1*: *lin-on n d* (*carrier-mat n n*) Φ*1* **and** *L2*: *lin-on n d* (*carrier-mat n n*) Φ*2* **and** *Y*: $Y \in$ *carrier-mat d d*
shows *map-adjoint n d* (λ*X*. *complex-of-real t* $\cdot_m$ Φ*1 X* + *complex-of-real* (1 − *t*) $\cdot_m$ Φ*2 X*) *Y*
= *complex-of-real t* $\cdot_m$ *map-adjoint n d* Φ*1 Y* + *complex-of-real* (1 − *t*) $\cdot_m$ *map-adjoint n d* Φ*2 Y*
⟨*proof*⟩

end
theory *Day22-UCP-Compact*
imports *Day16-Block-Form Day19-Choi-TP-Unital Day27-Positive-Map-Criterion*
begin

37 Day 22: $\mathrm{UCP}(M_n, M_d)$ is compact (D22-4)

The notes: $\mathrm{UCP}(M_n, M_d)$ is convex and compact, closed because the conditions are closed and bounded because $\Phi(I) = I$ forces $0 \le \Phi(E_{ii}) \le I$. The space $L(M_n, M_d)$ carries no topology in this development; the Choi matrix is a linear bijection onto M_{nd}, so the statement is made in Choi coordinates: the Choi matrices of linear UCP maps are exactly the positive P whose diagonal blocks sum to I_d (*ucp-choi-image*, with the inverse Choi map *choi-inv*), and that set is compact (*compact-ucp-choi-set*): it is closed, and it lies in d times the compact set of unit-trace positive matrices.

In Choi coordinates a linear map $\Phi : M_n \to M_d$ is UCP iff $P_\Phi \ge 0$ and $\sum_i R_{ii} = I_d$ (*choi-ucp-iff*); the set of such P is compact. Dimensions are cardinalities of finite types, $n = |'n|$, $d = |'d|$, so that the typed matrices of *Separable-Cone* (a Euclidean space) can be used.

definition *ucp-choi-set* :: (*complex*, *'n* :: *finite* × *'d* :: *finite*, *'n* × *'d*) *lmat set* **where**
ucp-choi-set = {*M*. *psd-lmat M* ∧ (∀ *a* < *CARD*(*'d*). ∀ *b* < *CARD*(*'d*).
(∑ *i*<*CARD*(*'n*). *unwrap-lm M* $$ (*i* $*$ *CARD*(*'d*) + *a*, *i* $*$ *CARD*(*'d*) + *b*)) = (*if a* = *b then 1 else 0*))}

lemma *closed-block-sum-eq*:
assumes *a*: $a < CARD('d :: finite)$ **and** *b*: $b < CARD('d)$
shows *closed* {$M :: (complex, 'n :: finite \times 'd, 'n \times 'd)$ *lmat*.
$(\sum i{<}CARD('n).$ *unwrap-lm M* $$ $(i * CARD('d) + a, i * CARD('d) + b)) = c\}$
⟨*proof*⟩

lemma *closed-ucp-choi-set*: *closed* (*ucp-choi-set* :: (*complex*, $'n :: finite \times 'd :: finite, 'n \times 'd$) *lmat set*)
⟨*proof*⟩

lemma *ucp-choi-set-trace*:
assumes $M \in$ (*ucp-choi-set* :: (*complex*, $'n :: finite \times 'd :: finite, 'n \times 'd$) *lmat set*)
shows *Tr-lmat M* = *of-nat CARD*('*d*)
⟨*proof*⟩

theorem *compact-ucp-choi-set*: *compact* (*ucp-choi-set* :: (*complex*, $'n :: finite \times 'd :: finite, 'n \times 'd$) *lmat set*)
⟨*proof*⟩

The inverse of the Choi map: $\Phi_P(X) = \sum_{i,j} x_{ij} P_{ij}$ with P_{ij} the (i, j) block.

definition *choi-inv* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat*
where
choi-inv n d P X = *Matrix.mat d d* (λ(*a*, *b*). $\sum i{<}n. \sum j{<}n.$ *X* $$ (*i*, *j*) * *P* $$ (*i* * *d* + *a*, *j* * *d* + *b*))

lemma *choi-inv-lin*: *lin-on n d* (*carrier-mat n n*) (*choi-inv n d P*)
⟨*proof*⟩

lemma *choi-matrix-choi-inv*:
assumes *P*: $P \in$ *carrier-mat* $(n * d)$ $(n * d)$
shows *choi-matrix n d* (*choi-inv n d P*) = *P*
⟨*proof*⟩

lemma *block-sum-entry*:
assumes *P*: $P \in$ *carrier-mat* $(n * d)$ $(n * d)$ **and** *a*: $a < d$ **and** *b*: $b < d$
shows *msum d d* (λ*i*. *block-of-mat d P i i*) [*0..<n*] $$ (*a*, *b*) = $(\sum i{<}n.$ *P* $$ $(i * d + a, i * d + b))$
⟨*proof*⟩

UCP in Choi coordinates is exactly *ucp-choi-set*, so it is compact.

theorem *ucp-choi-image*:
(λΦ. *wrap-lm* (*choi-matrix CARD*('*n*) *CARD*('*d*) Φ) :: (*complex*, $'n :: finite \times 'd :: finite, 'n \times 'd$) *lmat*)
‘ {Φ. *lin-on CARD*('*n*) *CARD*('*d*) (*carrier-mat CARD*('*n*) *CARD*('*n*)) Φ ∧ Φ ∈ *UCP CARD*('*n*) *CARD*('*d*)}
= *ucp-choi-set*
⟨*proof*⟩

corollary *compact-ucp*:
compact ((λΦ. *wrap-lm* (*choi-matrix CARD*('*n*) *CARD*('*d*) Φ) :: (*complex*, $'n :: finite \times 'd :: finite, 'n \times 'd$) *lmat*)
‘ {Φ. *lin-on CARD*('*n*) *CARD*('*d*) (*carrier-mat CARD*('*n*) *CARD*('*n*)) Φ ∧ Φ ∈ *UCP CARD*('*n*) *CARD*('*d*)})
⟨*proof*⟩

end
theory *Day06-15-Universal*
imports *Day02-11-Supplement*
begin

38 Day 6 and Day 15: the universal properties in the Kronecker model

D6-1: for $\mathbb{C}^n \otimes \mathbb{C}^m = \mathbb{C}^{nm}$ with $(v \otimes w)_{im+j} = v_i w_j$, every bilinear map $\beta : \mathbb{C}^n \times \mathbb{C}^m \to \mathbb{C}^p$ factors as $\beta(v, w) = L(v \otimes w)$ for a unique linear L. First for bilinear forms ($p = 1$), then coordinatewise.

definition *bilinear-on* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex*) ⇒ *bool* **where**
bilinear-on n m β ⟷
(∀ *v1 v2 w*. *dim-vec v1* = *n* ⟶ *dim-vec v2* = *n* ⟶ *dim-vec w* = *m* ⟶ *β* (*v1* + *v2*) *w* = *β v1 w* + *β v2 w*)
∧ (∀ *v w1 w2*. *dim-vec v* = *n* ⟶ *dim-vec w1* = *m* ⟶ *dim-vec w2* = *m* ⟶ *β v* (*w1* + *w2*) = *β v w1* + *β v w2*)
∧ (∀ *a v w*. *dim-vec v* = *n* ⟶ *dim-vec w* = *m* ⟶ *β* (*a* ·$_v$ *v*) *w* = *a* ∗ *β v w* ∧ *β v* (*a* ·$_v$ *w*) = *a* ∗ *β v w*)

definition *linear-on* :: *nat* ⇒ (*complex Matrix.vec* ⇒ *complex*) ⇒ *bool* **where**
linear-on K L ⟷ (∀ *x y*. *dim-vec x* = *K* ⟶ *dim-vec y* = *K* ⟶ *L* (*x* + *y*) = *L x* + *L y*)
∧ (∀ *a x*. *dim-vec x* = *K* ⟶ *L* (*a* ·$_v$ *x*) = *a* ∗ *L x*)

definition *trunc-vec* :: *complex Matrix.vec* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
trunc-vec v k = *Matrix.vec* (*dim-vec v*) (λ*i*. *if i* < *k then v* $ *i else 0*)

lemma *dim-trunc-vec* [*simp*]: *dim-vec* (*trunc-vec v k*) = *dim-vec v* ⟨*proof*⟩

lemma *trunc-vec-full*: *trunc-vec v* (*dim-vec v*) = *v* ⟨*proof*⟩

lemma *trunc-vec-Suc*: *k* < *dim-vec v* ⟹ *trunc-vec v* (*Suc k*) = *trunc-vec v k* + *v* $ *k* ·$_v$ *unit-vec* (*dim-vec v*) *k*
⟨*proof*⟩

lemma *trunc-vec-zero*: *trunc-vec v 0* = 0_v (*dim-vec v*) ⟨*proof*⟩

A linear map is determined by its values on the unit vectors.

lemma *linear-on-zero*: *linear-on K L* ⟹ *L* (0_v *K*) = *0*
⟨*proof*⟩

lemma *linear-on-expand*:
assumes *l*: *linear-on K L* **and** *x*: *dim-vec x* = *K*
shows *L x* = (∑ *k*<*K*. *x* $ *k* ∗ *L* (*unit-vec K k*))
⟨*proof*⟩

A bilinear map is determined by its values on pairs of unit vectors.

lemma *bilinear-on-expand*:
assumes *b*: *bilinear-on n m β* **and** *v*: *dim-vec v* = *n* **and** *w*: *dim-vec w* = *m*
shows *β v w* = (∑ *i*<*n*. ∑ *j*<*m*. (*v* $ *i* ∗ *w* $ *j*) ∗ *β* (*unit-vec n i*) (*unit-vec m j*))
⟨*proof*⟩

lemma *sum-div-mod*:
fixes *n m* :: *nat* **and** *g* :: *nat* ⇒ *nat* ⇒ *'a*::*comm-monoid-add*
shows (∑ *k*<*n* ∗ *m*. *g* (*k div m*) (*k mod m*)) = (∑ *i*<*n*. ∑ *j*<*m*. *g i j*)
⟨*proof*⟩

theorem *tensor-universal-property*:
assumes *b*: *bilinear-on n m β*
shows ∃ *L*. *linear-on* (*n* ∗ *m*) *L* ∧ (∀ *v w*. *dim-vec v* = *n* ⟶ *dim-vec w* = *m* ⟶ *L* (*tensor-vec v w*) = *β v w*)

$\wedge$ ($\forall$ L'. *linear-on* (n * m) L' $\wedge$ ($\forall$ v w. *dim-vec* v = n $\longrightarrow$ *dim-vec* w = m $\longrightarrow$ L' (*tensor-vec* v w) = β v w)
$\longrightarrow$ ($\forall$ x. *dim-vec* x = n * m $\longrightarrow$ L' x = L x))
⟨*proof*⟩

definition *bilinear-vec-on* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ (*complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec*) ⇒ *bool* **where**
bilinear-vec-on n m p β ⟷
($\forall$ v w. *dim-vec* v = n $\longrightarrow$ *dim-vec* w = m $\longrightarrow$ *dim-vec* (β v w) = p)
$\wedge$ ($\forall$ *v1 v2 w*. *dim-vec v1* = n $\longrightarrow$ *dim-vec v2* = n $\longrightarrow$ *dim-vec* w = m $\longrightarrow$ β (*v1* + *v2*) w = β *v1* w + β *v2* w)
$\wedge$ ($\forall$ v *w1 w2*. *dim-vec* v = n $\longrightarrow$ *dim-vec w1* = m $\longrightarrow$ *dim-vec w2* = m $\longrightarrow$ β v (*w1* + *w2*) = β v *w1* + β v *w2*)
$\wedge$ ($\forall$ a v w. *dim-vec* v = n $\longrightarrow$ *dim-vec* w = m $\longrightarrow$ β (a $\cdot_v$ v) w = a $\cdot_v$ β v w $\wedge$ β v (a $\cdot_v$ w) = a $\cdot_v$ β v w)

definition *linear-vec-on* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.vec* ⇒ *complex Matrix.vec*) ⇒ *bool* **where**
linear-vec-on K p L ⟷ ($\forall$ x. *dim-vec* x = K $\longrightarrow$ *dim-vec* (L x) = p)
$\wedge$ ($\forall$ x y. *dim-vec* x = K $\longrightarrow$ *dim-vec* y = K $\longrightarrow$ L (x + y) = L x + L y)
$\wedge$ ($\forall$ a x. *dim-vec* x = K $\longrightarrow$ L (a $\cdot_v$ x) = a $\cdot_v$ L x)

lemma *bilinear-vec-coord*: *bilinear-vec-on n m p β* ⟹ r < p ⟹ *bilinear-on n m* (λv w. β v w $ r)
⟨*proof*⟩

lemma *linear-vec-coord*: *linear-vec-on K p L* ⟹ r < p ⟹ *linear-on K* (λx. L x $ r)
⟨*proof*⟩

theorem *tensor-universal-property-vec*:
assumes *b*: *bilinear-vec-on n m p β*
shows $\exists$ L. *linear-vec-on* (n * m) p L $\wedge$ ($\forall$ v w. *dim-vec* v = n $\longrightarrow$ *dim-vec* w = m $\longrightarrow$ L (*tensor-vec* v w) = β v w)
$\wedge$ ($\forall$ L'. *linear-vec-on* (n * m) p L' $\wedge$ ($\forall$ v w. *dim-vec* v = n $\longrightarrow$ *dim-vec* w = m $\longrightarrow$ L' (*tensor-vec* v w) = β v w)
$\longrightarrow$ ($\forall$ x. *dim-vec* x = n * m $\longrightarrow$ L' x = L x))
⟨*proof*⟩

D15-3: in the coordinate model $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) = M_{m^2 \times p^2}$ the elementary tensor $R \otimes T$ is the matrix $(R_{ij}T_{kl})_{(im+j,\, kp+l)}$. The isomorphism Γ of *operator-tensor-space-isomorphism* sends it to the Kronecker product $R \otimes T$, and Γ is the only linear map that does.

definition *op-elem* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
op-elem m p Ra Tb = *Matrix.mat* (*m* * *m*) (*p* * *p*) (λ(*a*, *b*). *Ra* $$ (*a div m*, *a mod m*) * *Tb* $$ (*b div p*, *b mod p*))

lemma *op-elem-carrier* [*simp*]: *op-elem m p Ra Tb* ∈ *carrier-mat* (*m* * *m*) (*p* * *p*) ⟨*proof*⟩

theorem *gamma-elementary*:
assumes *Ra*: *Ra* ∈ *carrier-mat m m* **and** *Tb*: *Tb* ∈ *carrier-mat p p*
shows *operator-tensor-iso-map m p* (*op-elem m p Ra Tb*) = *Ra* ⨂ *Tb*
⟨*proof*⟩

definition *unit-mat* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**
unit-mat K1 K2 a b = *Matrix.mat K1 K2* (λ(*i*, *j*). *if i* = *a* ∧ *j* = *b then 1 else 0*)

lemma *unit-mat-carrier* [*simp*]: *unit-mat K1 K2 a b* ∈ *carrier-mat K1 K2* ⟨*proof*⟩

The unit matrices of $M_{m^2 \times p^2}$ are elementary tensors of unit matrices.

lemma *unit-mat-op-elem*:
assumes *a*: $a < m * m$ **and** *b*: $b < p * p$
shows *unit-mat* $(m * m)$ $(p * p)$ a b = *op-elem* m p (*unit-mat* m m $(a \ div \ m)$ $(a \ mod \ m)$) (*unit-mat* p p $(b \ div \ p)$ $(b \ mod \ p)$)
⟨*proof*⟩

A linear map on $M_{K_1 \times K_2}$ is determined by its values on the unit matrices.

definition *linear-mat-on* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
linear-mat-on K1 K2 Φ ⟷
(∀ *x y*. *x* ∈ *carrier-mat K1 K2* ⟶ *y* ∈ *carrier-mat K1 K2* ⟶ Φ (*x* + *y*) = Φ *x* + Φ *y*)
∧ (∀ *c x*. *x* ∈ *carrier-mat K1 K2* ⟶ Φ (*c* ·$_m$ *x*) = *c* ·$_m$ Φ *x*)

definition *flat-mat* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec* **where**
flat-mat K2 c = *Matrix.vec* (*dim-row c* ∗ *K2*) (λ*k*. *c* $$ (*k div K2*, *k mod K2*))

lemma *vec-to-mat-flat*: *c* ∈ *carrier-mat K1 K2* ⟹ *vec-to-mat K1 K2* (*flat-mat K2 c*) = *c*
⟨*proof*⟩

lemma *dim-flat*: *c* ∈ *carrier-mat K1 K2* ⟹ *dim-vec* (*flat-mat K2 c*) = *K1* ∗ *K2* ⟨*proof*⟩

lemma *vec-to-mat-unit*:
assumes *k*: *k* < *K1* ∗ *K2*
shows *vec-to-mat K1 K2* (*unit-vec* (*K1* ∗ *K2*) *k*) = *unit-mat K1 K2* (*k div K2*) (*k mod K2*)
⟨*proof*⟩

lemma *linear-mat-unique*:
assumes *l1*: *linear-mat-on K1 K2* Φ **and** *l2*: *linear-mat-on K1 K2* Ψ
and *dims*: ⋀*x*. *x* ∈ *carrier-mat K1 K2* ⟹ *dim-row* (Φ *x*) = *Q1* ∧ *dim-col* (Φ *x*) = *Q2* ∧ *dim-row* (Ψ *x*) = *Q1* ∧ *dim-col* (Ψ *x*) = *Q2*
and *e*: ⋀*a b*. *a* < *K1* ⟹ *b* < *K2* ⟹ Φ (*unit-mat K1 K2 a b*) = Ψ (*unit-mat K1 K2 a b*)
and *c*: *c* ∈ *carrier-mat K1 K2*
shows Φ *c* = Ψ *c*
⟨*proof*⟩

theorem *gamma-unique*:
assumes *p0*: *0* < *p* **and** *l*: *linear-mat-on* $(m * m)$ $(p * p)$ Φ
and *dims*: ⋀*x*. *x* ∈ *carrier-mat* $(m * m)$ $(p * p)$ ⟹ Φ *x* ∈ *carrier-mat* $(m * m)$ $(m * p)$
and *e*: ⋀*Ra Tb*. *Ra* ∈ *carrier-mat m m* ⟹ *Tb* ∈ *carrier-mat p p* ⟹ Φ (*op-elem m p Ra Tb*) = *Ra* ⊗ *Tb*
and *c*: *c* ∈ *carrier-mat* $(m * m)$ $(p * p)$
shows Φ *c* = *operator-tensor-iso-map m p c*
⟨*proof*⟩

end
theory *Day29-Classical-Codes*
imports *Quantum-Computing11*
begin

Days 29–30: classical codes over $\mathbb{Z}_2$. Words are lists over $\{0,1\}$; all finite checks are decided by *code-simp* (proof-producing evaluation, no oracle).

definition *bin-words* :: *nat* ⇒ *nat list set* **where**
bin-words n = *set* (*List.n-lists n* [*0*, *1*])

definition *xor-word* :: *nat list* ⇒ *nat list* ⇒ *nat list* **where**
xor-word v w = *map2* (λ*a b*. (*a* + *b*) *mod 2*) *v w*

38.1 The notes' "Hamming [7,1,3]" code does not correct one error

The notes encode $(a_1,\ldots,a_7) \mapsto (a_1,\ldots,a_7;\ a_1{+}a_2{+}a_3{+}a_4;\ a_1{+}a_2{+}a_5{+}a_6;\ a_1{+}a_3{+}a_5{+}a_7)$ and claim it detects and corrects one error. It detects one error, but two codewords are at distance 2: the word e_4 is at distance 1 from the codewords of 0 and of e_4, so no decoder corrects every single error.

definition *notes-hamming-encode* :: *nat list* ⇒ *nat list* **where**
notes-hamming-encode a = *a* @ [(*a*!*0* + *a*!*1* + *a*!*2* + *a*!*3*) *mod 2*, (*a*!*0* + *a*!*1* + *a*!*4* + *a*!*5*) *mod 2*, (*a*!*0* + *a*!*2* + *a*!*4* + *a*!*6*) *mod 2*]

definition *notes-hamming-code* :: *nat list set* **where**
notes-hamming-code = *notes-hamming-encode* ‘ *bin-words 7*

definition *flip-at* :: *nat* ⇒ *nat list* ⇒ *nat list* **where**
flip-at i w = *w*[*i* := (*w*!*i* + *1*) *mod 2*]

lemma *notes-hamming-code-iff*:
w ∈ *notes-hamming-code* ⟷ *take 7 w* ∈ *bin-words 7* ∧ *w* = *notes-hamming-encode* (*take 7 w*)
⟨*proof*⟩

lemma *bin-words-iff*: *w* ∈ *bin-words n* ⟷ *length w* = *n* ∧ *set w* ⊆ {*0*, *1*}
⟨*proof*⟩

lemma *bin-nth-lt2*: *set v* ⊆ {*0*,*1*} ⟹ *k* < *length v* ⟹ *v* ! *k* < (*2*::*nat*)
⟨*proof*⟩

lemma *xor-word-length* [*simp*]: *length* (*xor-word v w*) = *min* (*length v*) (*length w*)
⟨*proof*⟩

lemma *xor-word-nth*: *k* < *length v* ⟹ *k* < *length w* ⟹ *xor-word v w* ! *k* = (*v* ! *k* + *w* ! *k*) *mod 2*
⟨*proof*⟩

lemma *xor-word-cancel*:
assumes *length c* = *length v length c* = *length w set v* ⊆ {*0*,*1*} *set w* ⊆ {*0*,*1*}
shows *xor-word c v* = *xor-word c w* ⟷ *v* = *w*
⟨*proof*⟩

lemma *mod2-eq-iff*: (*p*::*nat*) *mod 2* = *q mod 2* ⟷ (*even p* ⟷ *even q*)
⟨*proof*⟩

The encoding is linear over $\mathbb{Z}_2$.

lemma *notes-hamming-encode-xor*:
assumes *length a* = *7 length b* = *7*
shows *notes-hamming-encode* (*xor-word a b*) = *xor-word* (*notes-hamming-encode a*) (*notes-hamming-encode b*)
⟨*proof*⟩

lemma *flip-at-xor*:
assumes *set c* ⊆ {*0*,*1*} *i* < *length c*
shows *flip-at i c* = *xor-word c* ((*replicate* (*length c*) *0*)[*i* := *1*])
⟨*proof*⟩

theorem *notes-hamming-detects-one-error*:
∀ *c* ∈ *notes-hamming-code*. ∀ *i* < *10*. *flip-at i c* ∉ *notes-hamming-code*
⟨*proof*⟩

theorem *notes-hamming-ambiguous-word*:

defines $w \equiv [0,0,0,1,0,0,0,0,0,0]$
shows *notes-hamming-encode* (*replicate 7 0*) ∈ *notes-hamming-code*
and *notes-hamming-encode* $[0,0,0,1,0,0,0]$ ∈ *notes-hamming-code*
and *notes-hamming-encode* (*replicate 7 0*) ≠ *notes-hamming-encode* $[0,0,0,1,0,0,0]$
and *hamming-dist* (*notes-hamming-encode* (*replicate 7 0*)) *w* = *1*
and *hamming-dist* (*notes-hamming-encode* $[0,0,0,1,0,0,0]$) *w* = *1*
⟨*proof*⟩

theorem *notes-hamming-not-correcting*: ¬ *corrects-errors notes-hamming-code 1*
⟨*proof*⟩

38.2 The Hamming [7,4,3] code corrects one error

definition *hamming74-encode* :: *nat list* ⇒ *nat list* **where**
hamming74-encode a = *a* @ [(*a*!*0* + *a*!*1* + *a*!*3*) *mod 2*, (*a*!*0* + *a*!*2* + *a*!*3*) *mod 2*, (*a*!*1* + *a*!*2* + *a*!*3*) *mod 2*]

definition *hamming74-code* :: *nat list set* **where**
hamming74-code = *hamming74-encode* ‘ *bin-words 4*

lemma *ex1-in-set*: (∃!*c*. *c* ∈ *set xs* ∧ *P c*) ⟷ *card* (*set* (*filter P xs*)) = *1*
⟨*proof*⟩

theorem *hamming74-unique-decoding*:
∀ *w* ∈ *bin-words 7*. ∃!*c*. *c* ∈ *hamming74-code* ∧ *hamming-dist c w* ≤ *1*
⟨*proof*⟩

theorem *hamming74-corrects-one-error*: *corrects-errors hamming74-code 1*
⟨*proof*⟩

38.3 Probabilities of undetected and uncorrected errors (Day 30)

Each bit flips independently with probability p; an error pattern e of weight k on n bits has probability $p^k(1-p)^{n-k}$. An error is undetected when it turns the sent codeword into another codeword.

definition *word-prob* :: *real* ⇒ *nat* ⇒ *nat list* ⇒ *real* **where**
word-prob p n e = *p* ^ *hamming-weight e* ∗ (*1* − *p*) ^ (*n* − *hamming-weight e*)

definition *err-prob* :: *real* ⇒ *nat* ⇒ *nat list set* ⇒ *real* **where**
err-prob p n ES = (∑ *e* ∈ *ES*. *word-prob p n e*)

lemma *err-prob-list*: *distinct es* ⟹ *err-prob p n* (*set es*) = (∑ *e*←*es*. *word-prob p n e*)
⟨*proof*⟩

definition *rep-code* :: *nat list set* **where** *rep-code* = *repetition-encode* ‘ *bin-words 3*
definition *par-code* :: *nat list set* **where** *par-code* = *parity-check-encode* ‘ *bin-words 3*

lemma *rep-code-list*: *rep-code* = *set* (*map repetition-encode* (*List.n-lists 3* [*0*, *1*]))
⟨*proof*⟩

lemma *par-code-list*: *par-code* = *set* (*map parity-check-encode* (*List.n-lists 3* [*0*, *1*]))
⟨*proof*⟩

definition *undetected* :: *nat list set* ⇒ *nat* ⇒ *nat list* ⇒ *nat list set* **where**
undetected C n c = {*e* ∈ *bin-words n*. *e* ≠ *replicate n 0* ∧ *xor-word c e* ∈ *C*}

lemma *set-filter-bin*: {*e* ∈ *bin-words n*. *P e*} = *set* (*filter P* (*List.n-lists n* [*0*, *1*]))

⟨*proof*⟩

Repetition on three bits: $3p^2(1-p)^4 + 3p^4(1-p)^2 + p^6$.

theorem *repetition-undetected-prob*:
assumes *c*: *c* ∈ *rep-code*
shows *err-prob p 6* (*undetected rep-code 6 c*) = *3 * p^2 * (1 − p)^4 + 3 * p^4 * (1 − p)^2 + p^6*
⟨*proof*⟩

Parity on three bits: $\binom{4}{2}p^2(1-p)^2 + p^4$.

theorem *parity-undetected-prob*:
assumes *c*: *c* ∈ *par-code*
shows *err-prob p 4* (*undetected par-code 4 c*) = *6 * p^2 * (1 − p)^2 + p^4*
⟨*proof*⟩

The notes compare the two for $p < \frac{1}{2}$; the inequality in fact holds for all $0 < p < 1$.

theorem *repetition-better-than-parity*:
assumes *p0*: *0 < p* **and** *p1*: *p < 1*
shows *3 * p^2 * (1 − p)^4 + 3 * p^4 * (1 − p)^2 + p^6 < 6 * p^2 * (1 − p)^2 + (p::real)^4*
⟨*proof*⟩

corollary *repetition-better-than-parity-notes*:
*0 < p ⟹ p < 1/2 ⟹ 3 * p^2 * (1 − p)^4 + 3 * p^4 * (1 − p)^2 + p^6 < 6 * p^2 * (1 − p)^2 + (p::real)^4*
⟨*proof*⟩

Majority rule $0 \mapsto 000$, $1 \mapsto 111$: decoding fails with probability $3p^2(1-p) + p^3$.

definition *maj-decode* :: *nat list* ⇒ *nat* **where**
maj-decode w = (*if 2 ≤ sum-list w then 1 else 0*)

theorem *majority-uncorrected-prob*:
assumes *b*: *b* ∈ {*0*, *1*}
shows *err-prob p 3* {*e* ∈ *bin-words 3*. *maj-decode* (*xor-word* (*replicate 3 b*) *e*) ≠ *b*}
= *3 * p^2 * (1 − p) + p^3*
⟨*proof*⟩

end
theory *Day30-BitFlip-Code*
imports *QC-Prelims*
begin

39 Day 30: the three-qubit bit-flip code and its recovery map (D30-3, D30-4)

The syndrome projections $P_0 = |000\rangle\langle 000| + |111\rangle\langle 111|$, …, as diagonal 0/1 matrices ($|abc\rangle \leftrightarrow e_{4a+2b+c}$), and the single flips $X \otimes I \otimes I$, $I \otimes X \otimes I$, $I \otimes I \otimes X$ as permutation matrices.

definition *syndrome-proj* :: *nat set* ⇒ *complex Matrix.mat* **where**
syndrome-proj K = *Matrix.mat 8 8* (λ(*i*,*j*). *if i = j ∧ i ∈ K then 1 else 0*)

abbreviation P_0 ≡ *syndrome-proj* {*0*, *7*}
abbreviation P_1 ≡ *syndrome-proj* {*4*, *3*}
abbreviation P_2 ≡ *syndrome-proj* {*2*, *5*}
abbreviation P_3 ≡ *syndrome-proj* {*1*, *6*}

definition *flip-bit-mat* :: *nat* ⇒ *complex Matrix.mat* **where**

flip-bit-mat b = Matrix.mat 8 8 (λ(i,j). if j = (if i div 2^b mod 2 = 0 then i + 2^b else i − 2^b) then 1 else 0)

abbreviation $X_1 \equiv$ *flip-bit-mat 2*
abbreviation $X_2 \equiv$ *flip-bit-mat 1*
abbreviation $X_3 \equiv$ *flip-bit-mat 0*

lemma *syndrome-proj-carrier* [*simp*]: *syndrome-proj K ∈ carrier-mat 8 8*
⟨*proof*⟩

lemma *flip-bit-mat-carrier* [*simp*]: *flip-bit-mat b ∈ carrier-mat 8 8*
⟨*proof*⟩

lemma *syndrome-proj-dims* [*simp*]: *dim-row (syndrome-proj K) = 8 dim-col (syndrome-proj K) = 8*
⟨*proof*⟩

lemma *flip-bit-mat-dims* [*simp*]: *dim-row (flip-bit-mat b) = 8 dim-col (flip-bit-mat b) = 8*
⟨*proof*⟩

lemma *encode-mat-dims* [*simp*]: *dim-row encode-mat = 8 dim-col encode-mat = 2*
⟨*proof*⟩

lemma *flip-proj-carrier* [*simp*]: *flip-bit-mat b * syndrome-proj K ∈ carrier-mat 8 8*
⟨*proof*⟩

lemma *b3-cases*: $b < (3::nat) \Longrightarrow b = 0 \vee b = 1 \vee b = 2$ ⟨*proof*⟩

Entry of an 8×8 product, kept as an abstract sum: *simp* unrolls $\sum k{<}8$ into eight terms and then splits every *if*, which does not terminate in practice.

lemma *mat-mult-entry-8*:
fixes *A B* :: *complex Matrix.mat*
assumes *A ∈ carrier-mat 8 8 B ∈ carrier-mat 8 8 i < 8 j < 8*
shows *(A * B) $$ (i,j) = (∑k<8. A $$ (i,k) * B $$ (k,j))*
⟨*proof*⟩

lemma *syndrome-proj-index*:
assumes *i < 8 j < 8*
shows *syndrome-proj K $$ (i,j) = (if i = j ∧ i ∈ K then 1 else 0)*
⟨*proof*⟩

lemma *syndrome-proj-two*:
assumes *a*: *a < 8* **and** *b*: *b < 8* **and** *ab*: *a ≠ b*
shows *syndrome-proj {a, b} = unit-vec 8 a ⊘ unit-vec 8 a + unit-vec 8 b ⊘ unit-vec 8 b*
⟨*proof*⟩

lemma *syndrome-proj-as-outer*:
P_0 = *unit-vec 8 0 ⊘ unit-vec 8 0 + unit-vec 8 7 ⊘ unit-vec 8 7*
P_1 = *unit-vec 8 4 ⊘ unit-vec 8 4 + unit-vec 8 3 ⊘ unit-vec 8 3*
P_2 = *unit-vec 8 2 ⊘ unit-vec 8 2 + unit-vec 8 5 ⊘ unit-vec 8 5*
P_3 = *unit-vec 8 1 ⊘ unit-vec 8 1 + unit-vec 8 6 ⊘ unit-vec 8 6*
⟨*proof*⟩

The flips are the notes' tensors.

lemma *tensor3-index*:
fixes *A B C* :: *complex Matrix.mat*
assumes *A*: *A ∈ carrier-mat 2 2* **and** *B*: *B ∈ carrier-mat 2 2* **and** *C*: *C ∈ carrier-mat 2 2*
and *i*: *i < 8* **and** *j*: *j < 8*
shows *Tensor.tensor-mat (Tensor.tensor-mat A B) C $$ (i,j)*

$= A \$\$ (i \mathbin{div} 4, j \mathbin{div} 4) * B \$\$ (i \mathbin{div} 2 \bmod 2, j \mathbin{div} 2 \bmod 2) * C \$\$ (i \bmod 2, j \bmod 2)$
⟨*proof*⟩

lemma *flips-as-tensors*:
$X_1 = Tensor.tensor\text{-}mat\ (Tensor.tensor\text{-}mat\ Pauli\text{-}X\ (1_m\ 2))\ (1_m\ 2)$
$X_2 = Tensor.tensor\text{-}mat\ (Tensor.tensor\text{-}mat\ (1_m\ 2)\ Pauli\text{-}X)\ (1_m\ 2)$
$X_3 = Tensor.tensor\text{-}mat\ (Tensor.tensor\text{-}mat\ (1_m\ 2)\ (1_m\ 2))\ Pauli\text{-}X$
⟨*proof*⟩

lemma *flip-bit-mat-mult-vec*:
assumes *b*: $b < 3$ **and** *vd*: $dim\text{-}vec\ v = 8$ **and** *i*: $i < 8$
shows $(flip\text{-}bit\text{-}mat\ b *_v v) \$ i = v \$ (if\ i \mathbin{div} 2\hat{}b \bmod 2 = 0\ then\ i + 2\hat{}b\ else\ i - 2\hat{}b)$
⟨*proof*⟩

lemma *syndrome-proj-mult-vec*:
assumes *vd*: $dim\text{-}vec\ v = 8$ **and** *i*: $i < 8$
shows $(syndrome\text{-}proj\ K *_v v) \$ i = (if\ i \in K\ then\ v \$ i\ else\ 0)$
⟨*proof*⟩

lemma *syndrome-proj-hermitian*: $(syndrome\text{-}proj\ K)^\dagger = syndrome\text{-}proj\ K$
⟨*proof*⟩

lemma *syndrome-proj-idem*: $syndrome\text{-}proj\ K * syndrome\text{-}proj\ K = syndrome\text{-}proj\ K$
⟨*proof*⟩

lemma *syndrome-proj-sum*:
$P_0 + P_1 + P_2 + P_3 = 1_m\ 8$
⟨*proof*⟩

The flip *flip-bit-mat b* permutes the basis by the involution *flip-tgt b*.

definition *flip-tgt* :: *nat* ⇒ *nat* ⇒ *nat* **where**
$flip\text{-}tgt\ b\ i = (if\ i \mathbin{div} 2\hat{}b \bmod 2 = 0\ then\ i + 2\hat{}b\ else\ i - 2\hat{}b)$

lemma *flip-tgt-lt*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **shows** $flip\text{-}tgt\ b\ i < 8$
⟨*proof*⟩

lemma *flip-tgt-invol*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **shows** $flip\text{-}tgt\ b\ (flip\text{-}tgt\ b\ i) = i$
⟨*proof*⟩

lemma *flip-bit-mat-index*:
assumes *i*: $i < 8$ **and** *j*: $j < 8$
shows $flip\text{-}bit\text{-}mat\ b \$\$ (i,j) = (if\ j = flip\text{-}tgt\ b\ i\ then\ 1\ else\ 0)$
⟨*proof*⟩

lemma *flip-tgt-iff*:
assumes *b*: $b < 3$ **and** *i*: $i < 8$ **and** *j*: $j < 8$
shows $i = flip\text{-}tgt\ b\ j \longleftrightarrow j = flip\text{-}tgt\ b\ i$
⟨*proof*⟩

lemma *flip-bit-mat-hermitian*:
assumes *b*: $b < 3$
shows $(flip\text{-}bit\text{-}mat\ b)^\dagger = flip\text{-}bit\text{-}mat\ b$
⟨*proof*⟩

lemma *flip-bit-mat-squared*:
assumes *b*: $b < 3$

shows *flip-bit-mat b ∗ flip-bit-mat b = 1*$_m$ *8*
⟨*proof*⟩

Kraus operators of the recovery map $R(Y) = P_0 Y P_0 + \sum_k X_k P_k Y P_k X_k$.

definition *recovery-ops* :: *complex Matrix.mat list* **where**
recovery-ops = [P_0, $X_1 * P_1$, $X_2 * P_2$, $X_3 * P_3$]

definition *recovery-map* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
recovery-map = *kraus-map-dim 8 recovery-ops*

lemma *flip-proj-dag-mult*:
assumes *b*: *b* < *3*
shows ((*flip-bit-mat b ∗ syndrome-proj K*)†) ∗ (*flip-bit-mat b ∗ syndrome-proj K*) = *syndrome-proj K*
⟨*proof*⟩

theorem *recovery-ops-kraus*: *kraus-operators 8 8 recovery-ops*
⟨*proof*⟩

corollary *recovery-map-CPTP*: *recovery-map* ∈ *CPTP 8 8*
⟨*proof*⟩

The encoded state $\psi = a|000\rangle + b|111\rangle$ and its single-flip corruptions.

lemma *encode-mat-mult-vec*:
assumes *xd*: *dim-vec x = 2* **and** *i*: *i* < *8*
shows (*encode-mat* ∗$_v$ *x*) $ *i* = (*if i = 0 then x $ 0 else if i = 7 then x $ 1 else 0*)
⟨*proof*⟩

lemma *encode-mat-dim* [*simp*]: *dim-vec* (*encode-mat* ∗$_v$ *x*) = *8* ⟨*proof*⟩

lemma *recovery-kraus-term*:
fixes *φ* :: *complex Matrix.vec*
assumes *φd*: *dim-vec φ = 8*
shows *recovery-map* (*φ* ⊘ *φ*) =
(*P*$_0$ ∗$_v$ *φ*) ⊘ (*P*$_0$ ∗$_v$ *φ*) + ((*X*$_1$ ∗$_v$ (*P*$_1$ ∗$_v$ *φ*)) ⊘ (*X*$_1$ ∗$_v$ (*P*$_1$ ∗$_v$ *φ*))
+ ((*X*$_2$ ∗$_v$ (*P*$_2$ ∗$_v$ *φ*)) ⊘ (*X*$_2$ ∗$_v$ (*P*$_2$ ∗$_v$ *φ*))
+ ((*X*$_3$ ∗$_v$ (*P*$_3$ ∗$_v$ *φ*)) ⊘ (*X*$_3$ ∗$_v$ (*P*$_3$ ∗$_v$ *φ*)) + *0*$_m$ *8 8*)))
⟨*proof*⟩

theorem *bit-flip-code-recovery*:
fixes *x* :: *complex Matrix.vec*
assumes *xd*: *dim-vec x = 2*
defines *ψ* ≡ *encode-mat* ∗$_v$ *x*
shows *recovery-map* (*ψ* ⊘ *ψ*) = *ψ* ⊘ *ψ*
and *recovery-map* ((*X*$_1$ ∗$_v$ *ψ*) ⊘ (*X*$_1$ ∗$_v$ *ψ*)) = *ψ* ⊘ *ψ*
and *recovery-map* ((*X*$_2$ ∗$_v$ *ψ*) ⊘ (*X*$_2$ ∗$_v$ *ψ*)) = *ψ* ⊘ *ψ*
and *recovery-map* ((*X*$_3$ ∗$_v$ *ψ*) ⊘ (*X*$_3$ ∗$_v$ *ψ*)) = *ψ* ⊘ *ψ*
⟨*proof*⟩

end
theory *Day31-Code-Fidelity*
imports *Day30-BitFlip-Code*
begin

Day 31: the three-qubit code against independent bit flips. Each qubit flips with probability p, so $E(Y) = (1-p)^3 Y + p(1-p)^2 \sum_i X_i Y X_i + p^2(1-p) \sum_{i<j} X_i X_j Y X_j X_i + p^3\, X_1X_2X_3 Y X_3X_2X_1$. On an encoded state the recovery R leaves $((1-p)^3 + 3p(1-p)^2)|\tilde\psi\rangle\langle\tilde\psi| + (3p^2(1-p) + p^3)|X_1X_2X_3\tilde\psi\rangle\langle X_1X_2X_3\tilde\psi|$. The notes state this formula for a general Y; it holds for Y supported on the code space (for $Y = |100\rangle\langle 100|$ and $p = 0$ one gets $R(Y) = |000\rangle\langle 000| \neq Y$).

definition *three-bit-channel* :: *real* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
three-bit-channel p M =
complex-of-real ((1 − *p*)^3) ·$_m$ *M*
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_1 ∗ *M* ∗ (X_1†))
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_2 ∗ *M* ∗ (X_2†))
+ *complex-of-real* (*p* ∗ (1 − *p*)^2) ·$_m$ (X_3 ∗ *M* ∗ (X_3†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_2 ∗ X_3) ∗ *M* ∗ ((X_2 ∗ X_3)†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_1 ∗ X_3) ∗ *M* ∗ ((X_1 ∗ X_3)†))
+ *complex-of-real* (*p*^2 ∗ (1 − *p*)) ·$_m$ ((X_1 ∗ X_2) ∗ *M* ∗ ((X_1 ∗ X_2)†))
+ *complex-of-real* (*p*^3) ·$_m$ ((X_1 ∗ X_2 ∗ X_3) ∗ *M* ∗ ((X_1 ∗ X_2 ∗ X_3)†))

definition *swap2* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
swap2 x = *Matrix.vec 2* (λ*i*. *x* $ (1 − *i*))

lemma *swap2-dim* [*simp*]: *dim-vec* (*swap2 x*) = 2 ⟨*proof*⟩

lemma *flips-on-codeword*:
assumes *x*: *dim-vec x* = 2
defines ψ ≡ *encode-mat* ∗$_v$ *x* **and** φ ≡ *encode-mat* ∗$_v$ *swap2 x*
shows (X_1 ∗ X_2 ∗ X_3) ∗$_v$ ψ = φ
and (X_2 ∗ X_3) ∗$_v$ ψ = X_1 ∗$_v$ φ
and (X_1 ∗ X_3) ∗$_v$ ψ = X_2 ∗$_v$ φ
and (X_1 ∗ X_2) ∗$_v$ ψ = X_3 ∗$_v$ φ
⟨*proof*⟩

39.1 Errors followed by recovery

lemma *recovery-ops-carrier'*: ∀ *W* ∈ *set recovery-ops*. *W* ∈ *carrier-mat 8 8*
⟨*proof*⟩

lemma *recovery-map-add*: *A* ∈ *carrier-mat 8 8* ⟹ *B* ∈ *carrier-mat 8 8* ⟹ *recovery-map* (*A* + *B*) = *recovery-map A* + *recovery-map B*
⟨*proof*⟩

lemma *recovery-map-smult*: *A* ∈ *carrier-mat 8 8* ⟹ *recovery-map* (*c* ·$_m$ *A*) = *c* ·$_m$ *recovery-map A*
⟨*proof*⟩

lemma *comb8*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat n n* **and** *B*: *B* ∈ *carrier-mat n n*
shows *complex-of-real a* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A* + *complex-of-real b* ·$_m$ *A*
+ *complex-of-real c* ·$_m$ *B* + *complex-of-real c* ·$_m$ *B* + *complex-of-real c* ·$_m$ *B* + *complex-of-real d* ·$_m$ *B*
= *complex-of-real* (*a* + 3 ∗ *b*) ·$_m$ *A* + *complex-of-real* (3 ∗ *c* + *d*) ·$_m$ *B*
⟨*proof*⟩

lemma *outer8*: *dim-vec v* = 8 ⟹ *v* ⊘ *v* ∈ *carrier-mat 8 8*
⟨*proof*⟩

D31-2: errors followed by recovery on an encoded state.

theorem *three-bit-code-recovery*:
assumes *x*: *dim-vec x* = 2
defines ψ ≡ *encode-mat* ∗$_v$ *x* **and** φ ≡ *encode-mat* ∗$_v$ *swap2 x*
shows *recovery-map* (*three-bit-channel p* (ψ ⊘ ψ))
= *complex-of-real* ((1 − *p*)^3 + 3 ∗ *p* ∗ (1 − *p*)^2) ·$_m$ (ψ ⊘ ψ)
+ *complex-of-real* (3 ∗ *p*^2 ∗ (1 − *p*) + *p*^3) ·$_m$ (φ ⊘ φ)
⟨*proof*⟩

The code is better than doing nothing iff $(1-p)^3 + 3p(1-p)^2 \geq 1-p$ iff $p \leq \frac{1}{2}$.

theorem *three-bit-code-better-iff*:
assumes *p0*: $0 \leq p$ **and** *p1*: $p < 1$
shows $(1 - p) \leq (1 - p)\hat{}3 + 3 * p * (1 - p)\hat{}2 \longleftrightarrow p \leq (1/2 :: real)$
⟨*proof*⟩

39.2 Minimum fidelities (D31-4)

lemma *quad-two-outer*:
fixes $\psi\ \varphi$ :: *complex Matrix.vec*
assumes ψd: *dim-vec* $\psi = n$ **and** φd: *dim-vec* $\varphi = n$
shows $\langle \psi \mid (a \cdot_m (\psi \oslash \psi) + b \cdot_m (\varphi \oslash \varphi)) *_v \psi \rangle$
$= a * (\langle \psi|\psi \rangle * \langle \psi|\psi \rangle) + b * (\langle \varphi|\psi \rangle * \langle \psi|\varphi \rangle)$
⟨*proof*⟩

lemma *encode-inner*:
assumes *u*: *dim-vec u = 2* **and** *v*: *dim-vec v = 2*
shows ⟨*encode-mat* $*_v$ *u* | *encode-mat* $*_v$ *v*⟩ = ⟨*u*|*v*⟩
⟨*proof*⟩

theorem *code-min-fidelity*:
assumes *p0*: $0 \leq p$ **and** *p1*: $p \leq 1$
defines *c1* ≡ $(1 - p)\hat{}3 + 3 * p * (1 - p)\hat{}2$
shows ∀ *x*. *dim-vec x = 2* ⟶ ⟨*x*|*x*⟩ = *1* ⟶
sqrt c1 ≤ *fidelity* (*encode-mat* $*_v$ *x*) (*recovery-map* (*three-bit-channel p* ((*encode-mat* $*_v$ *x*) ⊘ (*encode-mat* $*_v$ *x*))))
and *fidelity* (*encode-mat* $*_v$ *unit-vec 2 0*)
(*recovery-map* (*three-bit-channel p* ((*encode-mat* $*_v$ *unit-vec 2 0*) ⊘ (*encode-mat* $*_v$ *unit-vec 2 0*)))) = *sqrt c1*
⟨*proof*⟩

theorem *nothing-min-fidelity*:
assumes *p0*: $0 \leq p$ **and** *p1*: $p \leq 1$
shows ∀ ψ. *dim-vec* ψ = *2* ⟶ ⟨$\psi|\psi$⟩ = *1* ⟶ *sqrt* (*1* − *p*) ≤ *fidelity* ψ (*bit-flip-channel p* ($\psi \oslash \psi$))
and *fidelity* (*unit-vec 2 0*) (*bit-flip-channel p* (*unit-vec 2 0* ⊘ *unit-vec 2 0*)) = *sqrt* (*1* − *p*)
⟨*proof*⟩

end
theory *Day32-Pauli-Expansion*
imports *QC-Prelims*
begin

40 Day 32: the Pauli expansion identity for unitaries (D32-7)

lemma *cmod-parallelogram*:
fixes *a b* :: *complex*
shows $(cmod\ (a + b))^2 + (cmod\ (a - b))^2 = 2 * (cmod\ a)^2 + 2 * (cmod\ b)^2$
⟨*proof*⟩

proposition *pauli-expansion-unitary*:
fixes *U* :: *complex Matrix.mat* **and** *a0 a1 a2 a3* :: *complex*
assumes *Uu*: *unitary U*
and *U-eq*: *U* = *a0* $\cdot_m$ 1_m *2* + *a1* $\cdot_m$ *Pauli-X* + *a2* $\cdot_m$ *Pauli-Y* + *a3* $\cdot_m$ *Pauli-Z*
shows $(cmod\ a0)^2 + (cmod\ a1)^2 + (cmod\ a2)^2 + (cmod\ a3)^2 = 1$
⟨*proof*⟩

end

theory *Day32-Recovery-Proposition*
imports *Day21-Unitary-Freedom*
begin

41 Days 32–33: composition of Kraus maps and the general recovery proposition (D32-4, D33-2)

41.1 Algebra of *foldr*-sums of matrices

lemma *foldr-add-mat-mult-left*:
fixes $f :: 'a \Rightarrow$ *complex Matrix.mat*
assumes *Ac*: $A \in$ *carrier-mat m n* **and** *fc*: $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat n p*
shows $A * \mathit{foldr}\ (\lambda x\ \mathit{acc}.\ f\ x + \mathit{acc})\ \mathit{xs}\ (0_m\ n\ p) = \mathit{foldr}\ (\lambda x\ \mathit{acc}.\ A * f\ x + \mathit{acc})\ \mathit{xs}\ (0_m\ m\ p)$
⟨*proof*⟩

lemma *foldr-add-mat-mult-right*:
fixes $f :: 'a \Rightarrow$ *complex Matrix.mat*
assumes *Bc*: $B \in$ *carrier-mat n p* **and** *fc*: $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat m n*
shows $\mathit{foldr}\ (\lambda x\ \mathit{acc}.\ f\ x + \mathit{acc})\ \mathit{xs}\ (0_m\ m\ n) * B = \mathit{foldr}\ (\lambda x\ \mathit{acc}.\ f\ x * B + \mathit{acc})\ \mathit{xs}\ (0_m\ m\ p)$
⟨*proof*⟩

lemma *foldr-add-mat-dagger*:
fixes $f :: 'a \Rightarrow$ *complex Matrix.mat*
assumes $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat m n*
shows $(\mathit{foldr}\ (\lambda x\ \mathit{acc}.\ f\ x + \mathit{acc})\ \mathit{xs}\ (0_m\ m\ n))^\dagger = \mathit{foldr}\ (\lambda x\ \mathit{acc}.\ (f\ x)^\dagger + \mathit{acc})\ \mathit{xs}\ (0_m\ n\ m)$
⟨*proof*⟩

lemma *foldr-add-mat-cong*:
fixes $f\ g :: 'a \Rightarrow$ *complex Matrix.mat*
assumes $\bigwedge x.\ x \in$ *set xs* $\Longrightarrow f\ x = g\ x$
shows $\mathit{foldr}\ (\lambda x\ \mathit{acc}.\ f\ x + \mathit{acc})\ \mathit{xs}\ z = \mathit{foldr}\ (\lambda x\ \mathit{acc}.\ g\ x + \mathit{acc})\ \mathit{xs}\ z$
⟨*proof*⟩

lemma *foldr-add-mat-zero*:
$\mathit{foldr}\ (\lambda x\ \mathit{acc}.\ (0_m\ m\ n :: \mathit{complex\ Matrix.mat}) + \mathit{acc})\ \mathit{xs}\ (0_m\ m\ n) = 0_m\ m\ n$
⟨*proof*⟩

lemma *foldr-add-mat-delta*:
fixes $f :: \mathit{nat} \Rightarrow$ *complex Matrix.mat*
assumes *fc*: $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat m n* **and** *d*: *distinct xs* **and** *i*: $i \in$ *set xs*
shows $\mathit{foldr}\ (\lambda x\ \mathit{acc}.\ (\mathit{if}\ x = i\ \mathit{then}\ f\ x\ \mathit{else}\ 0_m\ m\ n) + \mathit{acc})\ \mathit{xs}\ (0_m\ m\ n) = f\ i$
⟨*proof*⟩

lemma *foldr-add-mat-smult-pull*:
fixes $f :: 'a \Rightarrow$ *complex Matrix.mat* **and** $c :: 'a \Rightarrow$ *complex*
assumes *fc*: $\forall x \in$ *set xs*. $f\ x = c\ x \cdot_m M$ **and** *Mc*: $M \in$ *carrier-mat m n*
shows $\mathit{foldr}\ (\lambda x\ \mathit{acc}.\ f\ x + \mathit{acc})\ \mathit{xs}\ (0_m\ m\ n) = \mathit{sum\text{-}list}\ (\mathit{map}\ c\ \mathit{xs}) \cdot_m M$
⟨*proof*⟩

lemma *foldr-list-as-index*:
fixes $g ::$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
shows $\mathit{foldr}\ (\lambda A\ \mathit{acc}.\ g\ A + \mathit{acc})\ \mathit{As}\ z = \mathit{foldr}\ (\lambda l\ \mathit{acc}.\ g\ (\mathit{As}\ !\ l) + \mathit{acc})\ [0..<\mathit{length}\ \mathit{As}]\ z$
⟨*proof*⟩

41.2 Composition of Kraus maps

definition *kraus-compose* :: *complex Matrix.mat list* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat list* **where**
kraus-compose As Es = *concat* (*map* (λ*E*. *map* (λ*A*. *A* ∗ *E*) *As*) *Es*)

lemma *kraus-compose-Nil* [*simp*]: *kraus-compose As* [] = []
⟨*proof*⟩

lemma *kraus-compose-Cons*: *kraus-compose As* (*E* # *Es*) = *map* (λ*A*. *A* ∗ *E*) *As* @ *kraus-compose As Es*
⟨*proof*⟩

lemma *length-kraus-compose* [*simp*]: *length* (*kraus-compose As Es*) = *length Es* ∗ *length As*
⟨*proof*⟩

lemma *kraus-compose-nth*:
assumes *i*: *i* < *length Es* **and** *l*: *l* < *length As*
shows *kraus-compose As Es* ! (*i* ∗ *length As* + *l*) = *As* ! *l* ∗ *Es* ! *i*
⟨*proof*⟩

lemma *kraus-compose-carrier*:
assumes ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat d n*
shows ∀ *W* ∈ *set* (*kraus-compose As Es*). *W* ∈ *carrier-mat m n*
⟨*proof*⟩

lemma *kraus-map-dim-map-mult-right*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** *Ec*: *E* ∈ *carrier-mat d n* **and** *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim m* (*map* (λ*A*. *A* ∗ *E*) *As*) *M* = *kraus-map-dim m As* (*E* ∗ *M* ∗ (*E*†))
⟨*proof*⟩

lemma *kraus-map-dim-compose*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat d n*
and *Xc*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim m* (*kraus-compose As Es*) *M* = *kraus-map-dim m As* (*kraus-map-dim d Es M*)
⟨*proof*⟩

lemma *sum-wdag-w-map-mult-right*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** *Ec*: *E* ∈ *carrier-mat d n*
shows *sum-wdag-w n* (*map* (λ*A*. *A* ∗ *E*) *As*) = (*E*†) ∗ *sum-wdag-w d As* ∗ *E*
⟨*proof*⟩

lemma *sum-wdag-w-compose*:
assumes *As-c*: ∀ *A* ∈ *set As*. *A* ∈ *carrier-mat m d* **and** *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat d n*
and *As-tp*: *sum-wdag-w d As* = 1_m *d*
shows *sum-wdag-w n* (*kraus-compose As Es*) = *sum-wdag-w n Es*
⟨*proof*⟩

lemma *kraus-operators-compose*:
assumes *As*: *kraus-operators d m As* **and** *Es*: *kraus-operators n d Es*
shows *kraus-operators n m* (*kraus-compose As Es*)
⟨*proof*⟩

41.3 The general recovery proposition (D32-4) and its linear-combination form (D33-2)

41.4 Matrix sums acting on vectors

lemma *smult-zero-vec* [*simp*]: $(c :: complex) \cdot_v 0_v\ n = 0_v\ n$
⟨*proof*⟩

lemma *zero-mat-mult-vec*:
assumes *dim-vec* $w = n$ **shows** $(0_m\ m\ n :: complex\ Matrix.mat) *_v w = 0_v\ m$
⟨*proof*⟩

lemma *mult-mat-vec-zero-vec*:
assumes *Ac*: $A \in carrier\text{-}mat\ m\ n$ **shows** $A *_v (0_v\ n :: complex\ Matrix.vec) = 0_v\ m$
⟨*proof*⟩

lemma *foldr-add-mat-mult-vec-zero*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ n\ n$ **and** *wd*: $dim\text{-}vec\ w = n$
and *z*: $\bigwedge x.\ x \in set\ xs \Longrightarrow f\ x *_v w = 0_v\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ n\ n) *_v w = 0_v\ n$
⟨*proof*⟩

lemma *foldr-add-mat-mult-vec-delta*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ n\ n$ **and** *wd*: $dim\text{-}vec\ w = n$
and *d*: *distinct xs* **and** *j*: $j \in set\ xs$
and *del*: $\bigwedge x.\ x \in set\ xs \Longrightarrow f\ x *_v w = (if\ x = j\ then\ w\ else\ 0_v\ n)$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ n\ n) *_v w = w$
⟨*proof*⟩

lemma *vec-sum-delta*:
assumes *fin*: *finite Js* **and** *j*: $j \in Js$ **and** *wd*: $\bigwedge i.\ i \in Js \Longrightarrow dim\text{-}vec\ (w\ i) = n$
shows $vec\text{-}sum\ n\ Js\ (\lambda i.\ if\ i = j\ then\ w\ i\ else\ 0_v\ n) = w\ j$
⟨*proof*⟩

Let $V = \operatorname{ran} P$ for a projection P and let $U_0, \ldots, U_{t-1}$ be unitaries with $U_i V \perp U_j V$ for $i \neq j$, i.e. $P U_i^* U_j P = 0$. With $P_i = U_i P U_i^*$, $Q = I - \sum_i P_i$ and the Kraus operators $P U_i^*$ (the notes write $U_i^* P_i = P U_i^*$) the map $R(X) = QXQ + \sum_i P U_i^* X U_i P$ is CPTP and recovers every $\psi \in V$ from any $\varphi = \sum_i a_i U_i \psi$: $R(|\varphi\rangle\langle\varphi|) = (\sum_i |a_i|^2)|\psi\rangle\langle\psi|$.

definition *image-proj-sum* :: $nat \Rightarrow complex\ Matrix.mat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat$ **where**
$image\text{-}proj\text{-}sum\ n\ P\ Us = foldr\ (\lambda i\ acc.\ Us\ !\ i * P * ((Us\ !\ i)^\dagger) + acc)\ [0..<length\ Us]\ (0_m\ n\ n)$

definition *recovery-ops* :: $nat \Rightarrow complex\ Matrix.mat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.mat\ list$ **where**
$recovery\text{-}ops\ n\ P\ Us = (1_m\ n - image\text{-}proj\text{-}sum\ n\ P\ Us)\ \#\ map\ (\lambda i.\ P * ((Us\ !\ i)^\dagger))\ [0..<length\ Us]$

The hypotheses of the proposition, bundled: P a projection and $P U_i^* U_j P = \delta_{ij} P$. For unitaries U_i the case $i = j$ is automatic (*recovery-setting-unitaries*); the weaker form also covers the rescaled errors of the Knill–Laflamme theorem.

definition *recovery-setting* :: $nat \Rightarrow complex\ Matrix.mat \Rightarrow complex\ Matrix.mat\ list \Rightarrow bool$ **where**
$recovery\text{-}setting\ n\ P\ Us \longleftrightarrow projection\ n\ P \wedge (\forall U \in set\ Us.\ U \in carrier\text{-}mat\ n\ n)$
$\wedge\ (\forall i < length\ Us.\ \forall j < length\ Us.\ P * (((Us\ !\ i)^\dagger) * Us\ !\ j) * P = (if\ i = j\ then\ P\ else\ 0_m\ n\ n))$

lemma *recovery-settingD*:
assumes *recovery-setting n P Us*
shows $projection\ n\ P\ \forall U \in set\ Us.\ U \in carrier\text{-}mat\ n\ n$

$\forall\, i < \mathit{length}\ \mathit{Us}.\ \forall\, j < \mathit{length}\ \mathit{Us}.\ i \neq j \longrightarrow P * (((\mathit{Us}\ !\ i)^\dagger) * \mathit{Us}\ !\ j) * P = 0_m\ n\ n$
$\forall\, i < \mathit{length}\ \mathit{Us}.\ P * (((\mathit{Us}\ !\ i)^\dagger) * \mathit{Us}\ !\ i) * P = P$
⟨*proof*⟩

lemma *recovery-setting-unitaries*:
assumes *P*: *projection n P* **and** *Us*: $\forall\, U \in \mathit{set}\ \mathit{Us}.\ U \in \mathit{carrier\text{-}mat}\ n\ n \wedge \mathit{unitary}\ U$
and *orth*: $\forall\, i < \mathit{length}\ \mathit{Us}.\ \forall\, j < \mathit{length}\ \mathit{Us}.\ i \neq j \longrightarrow P * (((\mathit{Us}\ !\ i)^\dagger) * \mathit{Us}\ !\ j) * P = 0_m\ n\ n$
shows *recovery-setting n P Us*
⟨*proof*⟩

lemma
assumes *S*: *recovery-setting n P Us*
shows *rec-Pc*: $P \in \mathit{carrier\text{-}mat}\ n\ n$ **and** *rec-Ph*: $P^\dagger = P$ **and** *rec-PP*: $P * P = P$
⟨*proof*⟩

lemma
assumes *S*: *recovery-setting n P Us*
shows *rec-Uc*: $i < \mathit{length}\ \mathit{Us} \Longrightarrow \mathit{Us}\ !\ i \in \mathit{carrier\text{-}mat}\ n\ n$
and *rec-PUUP*: $i < \mathit{length}\ \mathit{Us} \Longrightarrow P * (((\mathit{Us}\ !\ i)^\dagger) * \mathit{Us}\ !\ i) * P = P$
⟨*proof*⟩

lemma *rec-Pi-c*:
assumes *S*: *recovery-setting n P Us* **and** *i*: $i < \mathit{length}\ \mathit{Us}$
shows $\mathit{Us}\ !\ i * P * ((\mathit{Us}\ !\ i)^\dagger) \in \mathit{carrier\text{-}mat}\ n\ n$
⟨*proof*⟩

lemma *rec-Pi-mult*:
assumes *S*: *recovery-setting n P Us* **and** *i*: $i < \mathit{length}\ \mathit{Us}$ **and** *j*: $j < \mathit{length}\ \mathit{Us}$
shows $(\mathit{Us}\ !\ i * P * ((\mathit{Us}\ !\ i)^\dagger)) * (\mathit{Us}\ !\ j * P * ((\mathit{Us}\ !\ j)^\dagger))$
$= (\mathit{if}\ i = j\ \mathit{then}\ \mathit{Us}\ !\ i * P * ((\mathit{Us}\ !\ i)^\dagger)\ \mathit{else}\ 0_m\ n\ n)$
⟨*proof*⟩

lemma *image-proj-sum-carrier*:
assumes *S*: *recovery-setting n P Us*
shows *image-proj-sum n P Us* $\in$ *carrier-mat n n*
⟨*proof*⟩

lemma *image-proj-sum-hermitian*:
assumes *S*: *recovery-setting n P Us*
shows $(\mathit{image\text{-}proj\text{-}sum}\ n\ P\ \mathit{Us})^\dagger = \mathit{image\text{-}proj\text{-}sum}\ n\ P\ \mathit{Us}$
⟨*proof*⟩

lemma *image-proj-sum-idem*:
assumes *S*: *recovery-setting n P Us*
shows *image-proj-sum n P Us* $*$ *image-proj-sum n P Us* $=$ *image-proj-sum n P Us*
⟨*proof*⟩

lemma *image-proj-sum-projection*:
assumes *S*: *recovery-setting n P Us*
shows *projection n* (*image-proj-sum n P Us*)
⟨*proof*⟩

lemma *dagger-one-minus*:
assumes *Sc*: $M \in \mathit{carrier\text{-}mat}\ n\ n$ **and** *Sh*: $M^\dagger = M$
shows $(1_m\ n - M)^\dagger = 1_m\ n - M$
⟨*proof*⟩

lemma *recovery-ops-carrier*:

assumes *S*: *recovery-setting n P Us*
shows ∀ *W* ∈ *set* (*recovery-ops n P Us*). *W* ∈ *carrier-mat n n*
⟨*proof*⟩

theorem *recovery-ops-kraus-operators*:
assumes *S*: *recovery-setting n P Us*
shows *kraus-operators n n* (*recovery-ops n P Us*)
⟨*proof*⟩

corollary *recovery-map-CPTP'*:
assumes *S*: *recovery-setting n P Us* **and** *npos*: *0* < *n*
shows *kraus-map-dim n* (*recovery-ops n P Us*) ∈ *CPTP n n*
⟨*proof*⟩

Action of the Kraus operators on $\varphi = \sum_i a_i U_i \psi$, $\psi \in V$.

lemma *rec-kraus-on-image*:
assumes *S*: *recovery-setting n P Us*
and *ψd*: *dim-vec ψ* = *n* **and** *ψV*: *P* ∗*v ψ* = *ψ* **and** *i*: *i* < *length Us* **and** *j*: *j* < *length Us*
shows (*P* ∗ ((*Us* ! *i*)†)) ∗*v* (*Us* ! *j* ∗*v ψ*) = (*if i* = *j then ψ else* 0*v n*)
⟨*proof*⟩

lemma *rec-Q-on-image*:
assumes *S*: *recovery-setting n P Us*
and *ψd*: *dim-vec ψ* = *n* **and** *ψV*: *P* ∗*v ψ* = *ψ* **and** *j*: *j* < *length Us*
shows (1*m n* − *image-proj-sum n P Us*) ∗*v* (*Us* ! *j* ∗*v ψ*) = 0*v n*
⟨*proof*⟩

theorem *recovery-linear-combination*:
fixes *a* :: *nat* ⇒ *complex*
assumes *S*: *recovery-setting n P Us* **and** *ψd*: *dim-vec ψ* = *n* **and** *ψV*: *P* ∗*v ψ* = *ψ*
defines *φ* ≡ *vec-sum n* {*i*. *i* < *length Us*} (λ*i*. *a i* ·*v* (*Us* ! *i* ∗*v ψ*))
shows *kraus-map-dim n* (*recovery-ops n P Us*) (*φ* ⊘ *φ*)
= *complex-of-real* (∑ *i* < *length Us*. (*cmod* (*a i*))2) ·*m* (*ψ* ⊘ *ψ*)
⟨*proof*⟩

corollary *recovery-single-error*:
assumes *S*: *recovery-setting n P Us*
and *ψd*: *dim-vec ψ* = *n* **and** *ψV*: *P* ∗*v ψ* = *ψ* **and** *j*: *j* < *length Us*
shows *kraus-map-dim n* (*recovery-ops n P Us*) ((*Us* ! *j* ∗*v ψ*) ⊘ (*Us* ! *j* ∗*v ψ*)) = *ψ* ⊘ *ψ*
⟨*proof*⟩

end
theory *Day32-Y-Code*
imports *Day33-Shor-Code*
begin

Day 32: the code for Y flips. With $\beta = Y|+\rangle$ one has $|+\rangle \perp \beta$, $Y|+\rangle = \beta$, $Y\beta = |+\rangle$, and the code $|0\rangle \mapsto |++\!+\rangle$, $|1\rangle \mapsto |\beta\beta\beta\rangle$ corrects one Y flip exactly as the bit-flip code corrects one X flip.

definition *ket-beta* :: *complex Matrix.vec* **where** *ket-beta* = i ·*v ket-minus*

definition *ub* :: *bool* ⇒ *complex Matrix.vec* **where** *ub s* = (*if s then ket-plus else ket-beta*)

lemma *ket-beta-dim* [*simp*]: *dim-vec ket-beta* = *2* ⟨*proof*⟩
lemma *ub-dim* [*simp*]: *dim-vec* (*ub s*) = *2* ⟨*proof*⟩

lemma *ket-beta-index*: *ket-beta* $ *0* = i / *sqrt 2 ket-beta* $ *1* = − i / *sqrt 2*

⟨*proof*⟩

lemma *Y-plus*: *Pauli-Y* $*_v$ *ket-plus* = *ket-beta*
⟨*proof*⟩

lemma *Y-beta*: *Pauli-Y* $*_v$ *ket-beta* = *ket-plus*
⟨*proof*⟩

lemma *Y-ub*: *Pauli-Y* $*_v$ *ub s* = *ub* (¬ *s*)
⟨*proof*⟩

lemma *inner2*: *dim-vec u* = *2* ⟹ *dim-vec v* = *2* ⟹ ⟨*u*|*v*⟩ = *cnj* (*u* $ *0*) * *v* $ *0* + *cnj* (*u* $ *1*) * *v* $ *1*
⟨*proof*⟩

lemma *ub-inner*: ⟨*ub s* | *ub t*⟩ = (*if s* = *t then 1 else 0*)
⟨*proof*⟩

theorem *ket-plus-beta-orth*: ⟨*ket-plus* | *ket-beta*⟩ = *0*
⟨*proof*⟩

41.5 Codes with orthogonal error images

For a code with orthonormal basis $v_1, \ldots, v_k$ (the columns of V, projection VV^*) and unitary errors whose images of the basis are mutually orthogonal, the recovery proposition gives one CPTP map correcting each error on every code state Vc.

lemma *proj-of-orthonormal*:
assumes *orth*: *orthonormal-list n vs*
defines *V* ≡ *mat-of-cols n vs*
shows *projection n* (*V* * (*V*†))
⟨*proof*⟩

lemma *proj-fixes-code*:
assumes *orth*: *orthonormal-list n vs* **and** *c*: *dim-vec c* = *length vs*
defines *V* ≡ *mat-of-cols n vs*
shows (*V* * (*V*†)) $*_v$ (*V* $*_v$ *c*) = *V* $*_v$ *c*
⟨*proof*⟩

theorem *orthogonal-images-recovery*:
assumes *orth*: *orthonormal-list n vs*
and *Us*: ∀ *U* ∈ *set Us*. *U* ∈ *carrier-mat n n* ∧ *unitary U*
and *img*: ∀ *i* < *length Us*. ∀ *j* < *length Us*. *i* ≠ *j* ⟶
(∀ *a* < *length vs*. ∀ *b* < *length vs*. ⟨*Us* ! *i* $*_v$ *vs* ! *a* | *Us* ! *j* $*_v$ *vs* ! *b*⟩ = *0*)
defines *V* ≡ *mat-of-cols n vs*
shows *recovery-setting n* (*V* * (*V*†)) *Us*
⟨*proof*⟩

corollary *orthogonal-images-correct*:
assumes *orth*: *orthonormal-list n vs*
and *Us*: ∀ *U* ∈ *set Us*. *U* ∈ *carrier-mat n n* ∧ *unitary U*
and *img*: ∀ *i* < *length Us*. ∀ *j* < *length Us*. *i* ≠ *j* ⟶
(∀ *a* < *length vs*. ∀ *b* < *length vs*. ⟨*Us* ! *i* $*_v$ *vs* ! *a* | *Us* ! *j* $*_v$ *vs* ! *b*⟩ = *0*)
shows ∃ *Rs*. *kraus-operators n n Rs* ∧
(∀ (*c* :: *complex Matrix.vec*) *j*. *dim-vec c* = *length vs* ⟶ *j* < *length Us* ⟶
kraus-map-dim n Rs (*Us* ! *j* * ((*mat-of-cols n vs* $*_v$ *c*) ⊘ (*mat-of-cols n vs* $*_v$ *c*)) * ((*Us* ! *j*)†))
= (*mat-of-cols n vs* $*_v$ *c*) ⊘ (*mat-of-cols n vs* $*_v$ *c*))

⟨*proof*⟩

41.6 The Y-flip code

definition *tv2* :: *bool* ⇒ *bool* ⇒ *bool* ⇒ *complex Matrix.vec* **where**
tv2 a b c = *tensor-vec* (*tensor-vec* (*ub a*) (*ub b*)) (*ub c*)

definition *y-errors* :: *complex Matrix.mat list* **where**
y-errors = [1_m *8*, *bop 0 Pauli-Y*, *bop 1 Pauli-Y*, *bop 2 Pauli-Y*]

definition *y-codewords* :: *complex Matrix.vec list* **where**
y-codewords = [*tv2 True True True*, *tv2 False False False*]

lemma *tv2-dim* [*simp*]: *dim-vec* (*tv2 a b c*) = *8*
⟨*proof*⟩

lemma *tv2-inner*: ⟨*tv2 a b c* | *tv2 a′ b′ c′*⟩ = (*if a* = *a′* ∧ *b* = *b′* ∧ *c* = *c′ then 1 else 0*)
⟨*proof*⟩

lemma *y-actions*:
bop 0 Pauli-Y ∗$_v$ *tv2 a b c* = *tv2* (¬ *a*) *b c*
bop 1 Pauli-Y ∗$_v$ *tv2 a b c* = *tv2 a* (¬ *b*) *c*
bop 2 Pauli-Y ∗$_v$ *tv2 a b c* = *tv2 a b* (¬ *c*)
⟨*proof*⟩

lemma *y-codewords-orthonormal*: *orthonormal-list 8 y-codewords*
⟨*proof*⟩

lemma *y-errors-unitary*: ∀ *U* ∈ *set y-errors*. *U* ∈ *carrier-mat 8 8* ∧ *unitary U*
⟨*proof*⟩

lemma *y-images-orthogonal*:
∀ *i* < *length y-errors*. ∀ *j* < *length y-errors*. *i* ≠ *j* ⟶
(∀ *a* < *length y-codewords*. ∀ *b* < *length y-codewords*.
⟨*y-errors* ! *i* ∗$_v$ *y-codewords* ! *a* | *y-errors* ! *j* ∗$_v$ *y-codewords* ! *b*⟩ = *0*)
⟨*proof*⟩

D32-2: one CPTP map corrects every single Y flip on every state $a|++\rangle + b|\beta\beta\beta\rangle$.

theorem *y-code-corrects*:
∃ *Rs*. *kraus-operators 8 8 Rs* ∧
(∀ *c j*. *dim-vec c* = *2* ⟶ *j* < *4* ⟶
kraus-map-dim 8 Rs (*y-errors* ! *j* ∗ ((*mat-of-cols 8 y-codewords* ∗$_v$ *c*) ⊘ (*mat-of-cols 8 y-codewords* ∗$_v$ *c*)) ∗ ((*y-errors* ! *j*)†))
= (*mat-of-cols 8 y-codewords* ∗$_v$ *c*) ⊘ (*mat-of-cols 8 y-codewords* ∗$_v$ *c*))
⟨*proof*⟩

end
theory *Day33-Shor-Code*
imports *Day32-Recovery-Proposition Day32-Pauli-Expansion Day30-BitFlip-Code*
begin

Days 32–33: the nine-qubit Shor code. The 2^9-dimensional space is never handled entrywise: nine qubits are three blocks of three, every single-qubit operator acts inside one block, and all inner products factor over the blocks (*tensor-inner-prod*).

41.7 Tensor products of matrices

lemma *tensor-index-c*:

fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat r c B* ∈ *carrier-mat p q i* < *r* ∗ *p j* < *c* ∗ *q* 0 < *c* 0 < *q*
shows *Tensor.tensor-mat A B* $$ (*i*,*j*) = *A* $$ (*i div p*, *j div q*) ∗ *B* $$ (*i mod p*, *j mod q*)
⟨*proof*⟩

lemma *tensor-one-left*:
fixes *A* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m m* **and** *m*: 0 < *m*
shows *Tensor.tensor-mat* (1$_m$ 1) *A* = *A*
⟨*proof*⟩

lemma *tensor-one-right*:
fixes *A* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m m* **and** *m*: 0 < *m*
shows *Tensor.tensor-mat A* (1$_m$ 1) = *A*
⟨*proof*⟩

lemma *tensor-mat-add-right*:
fixes *A B C* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *Cc*: *C* ∈ *carrier-mat p q*
and *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat A* (*B* + *C*) = *Tensor.tensor-mat A B* + *Tensor.tensor-mat A C*
⟨*proof*⟩

lemma *tensor-mat-add-left*:
fixes *A B C* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat r c* **and** *Cc*: *C* ∈ *carrier-mat p q*
and *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat* (*A* + *B*) *C* = *Tensor.tensor-mat A C* + *Tensor.tensor-mat B C*
⟨*proof*⟩

lemma *tensor-mat-smult-right*:
fixes *A B* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat A* (*k* ·$_m$ *B*) = *k* ·$_m$ *Tensor.tensor-mat A B*
⟨*proof*⟩

lemma *tensor-mat-smult-left*:
fixes *A B* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *c*: 0 < *c* **and** *q*: 0 < *q*
shows *Tensor.tensor-mat* (*k* ·$_m$ *A*) *B* = *k* ·$_m$ *Tensor.tensor-mat A B*
⟨*proof*⟩

41.8 Single-qubit operators on nine qubits, and on a block of three

$\tilde{U} = I \otimes \cdots \otimes I \otimes U \otimes I \otimes \cdots \otimes I$ with U in position i ($i = 0, \ldots, 8$).

definition *qubit-op* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
qubit-op i U = *Tensor.tensor-mat* (*Tensor.tensor-mat* (1$_m$ (2^*i*)) *U*) (1$_m$ (2^(8−*i*)))

definition *bop* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
bop s U = *Tensor.tensor-mat* (*Tensor.tensor-mat* (1$_m$ (2^*s*)) *U*) (1$_m$ (2^(2−*s*)))

definition *blk* :: *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
blk b M = *Tensor.tensor-mat* (*Tensor.tensor-mat* (1$_m$ (8^*b*)) *M*) (1$_m$ (8^(2−*b*)))

lemma *qubit-op-carrier*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *i*: *i* < *9*

shows *qubit-op i U* ∈ *carrier-mat 512 512*
⟨*proof*⟩

lemma *qubit-op-blk*:
assumes *b*: *b* < *3* **and** *s*: *s* < *3*
shows *qubit-op* (*3* ∗ *b* + *s*) *U* = *blk b* (*bop s U*)
⟨*proof*⟩

lemma *qubit-op-blk-div*:
assumes *i*: *i* < *9*
shows *qubit-op i U* = *blk* (*i div 3*) (*bop* (*i mod 3*) *U*)
⟨*proof*⟩

lemma *bop-cases*:
fixes *U* :: *complex Matrix.mat*
assumes *U*: *U* ∈ *carrier-mat 2 2*
shows *bop 0 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat U* (*1*$_m$ *2*)) (*1*$_m$ *2*)
and *bop 1 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ *2*) *U*) (*1*$_m$ *2*)
and *bop 2 U* = *Tensor.tensor-mat* (*Tensor.tensor-mat* (*1*$_m$ *2*) (*1*$_m$ *2*)) *U*
⟨*proof*⟩

lemma *bop-carrier*:
assumes *U*: *U* ∈ *carrier-mat 2 2* **and** *s*: *s* < *3*
shows *bop s U* ∈ *carrier-mat 8 8*
⟨*proof*⟩

definition *tv3* :: *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* ⇒ *complex Matrix.vec* **where**
tv3 x y z = *tensor-vec* (*tensor-vec x y*) *z*

lemma *tv3-dim*: *dim-vec x* = *8* ⟹ *dim-vec y* = *8* ⟹ *dim-vec z* = *8* ⟹ *dim-vec* (*tv3 x y z*) = *512*
⟨*proof*⟩

lemma *blk-tv3*:
assumes *M*: *M* ∈ *carrier-mat 8 8* **and** *x*: *dim-vec x* = *8* **and** *y*: *dim-vec y* = *8* **and** *z*: *dim-vec z* = *8*
shows *blk 0 M* ∗$_v$ *tv3 x y z* = *tv3* (*M* ∗$_v$ *x*) *y z*
and *blk 1 M* ∗$_v$ *tv3 x y z* = *tv3 x* (*M* ∗$_v$ *y*) *z*
and *blk 2 M* ∗$_v$ *tv3 x y z* = *tv3 x y* (*M* ∗$_v$ *z*)
⟨*proof*⟩

41.9 The block vectors $|m\rangle \pm |7-m\rangle$

In a block of three qubits, *gvec p 0* is $|000\rangle \pm |111\rangle$; a bit flip moves it to *gvec p m* with $m \in \{4, 2, 1\}$. These eight vectors are mutually orthogonal.

definition *pm* :: *bool* ⇒ *complex* **where** *pm p* = (*if p then 1 else* −*1*)

definition *gvec* :: *bool* ⇒ *nat* ⇒ *complex Matrix.vec* **where**
gvec p m = *unit-vec 8 m* + *pm p* ·$_v$ *unit-vec 8* (*7* − *m*)

lemma *gvec-dim* [*simp*]: *dim-vec* (*gvec p m*) = *8*
⟨*proof*⟩

lemma *gvec-index*:
assumes *r*: *r* < *8* **and** *m*: *m* < *8*
shows *gvec p m* $ *r* = (*if r* = *m then 1 else 0*) + *pm p* ∗ (*if r* = *7* − *m then 1 else 0*)
⟨*proof*⟩

lemma *mult-gvec0*:
assumes *M*: $M \in$ *carrier-mat 8 8* **and** *r*: $r < 8$
shows $(M *_v$ *gvec p 0*$)$ \$ $r = M$ \$\$ $(r,0) +$ *pm p* $*$ M \$\$ $(r,7)$
⟨*proof*⟩

lemma *bop-entry-cases*:
assumes *U*: $U \in$ *carrier-mat 2 2* **and** *r*: $r < 8$ **and** *k*: $k < 8$
shows *bop 0 U* \$\$ $(r,k) = U$ \$\$ $(r$ *div 4*, *k div 4*$) * 1_m$ *2* \$\$ $(r$ *div 2 mod 2*, *k div 2 mod 2*$) * 1_m$ *2* \$\$ $(r$ *mod 2*, *k mod 2*$)$
and *bop 1 U* \$\$ $(r,k) = 1_m$ *2* \$\$ $(r$ *div 4*, *k div 4*$) * U$ \$\$ $(r$ *div 2 mod 2*, *k div 2 mod 2*$) * 1_m$ *2* \$\$ $(r$ *mod 2*, *k mod 2*$)$
and *bop 2 U* \$\$ $(r,k) = 1_m$ *2* \$\$ $(r$ *div 4*, *k div 4*$) * 1_m$ *2* \$\$ $(r$ *div 2 mod 2*, *k div 2 mod 2*$) * U$ \$\$ $(r$ *mod 2*, *k mod 2*$)$
⟨*proof*⟩

lemma *bop-X-gvec*:
assumes *s*: $s < 3$
shows *bop s Pauli-X* $*_v$ *gvec p 0* $=$ *gvec p* $(2\hat{}(2-s))$
⟨*proof*⟩

lemma *bop-Y-gvec*:
assumes *s*: $s < 3$
shows *bop s Pauli-Y* $*_v$ *gvec p 0* $= (-\mathrm{i}) \cdot_v$ *gvec* $(\neg p)$ $(2\hat{}(2-s))$
⟨*proof*⟩

lemma *bop-Z-gvec*:
assumes *s*: $s < 3$
shows *bop s Pauli-Z* $*_v$ *gvec p 0* $=$ *gvec* $(\neg p)$ *0*
⟨*proof*⟩

lemma *inner-two-units*:
assumes *a*: $a < n$ **and** *b*: $b < n$ **and** *c*: $c < n$ **and** *d*: $d < n$
shows ⟨*unit-vec n a* $+ \alpha \cdot_v$ *unit-vec n b* | *unit-vec n c* $+ \beta \cdot_v$ *unit-vec n d*⟩
$=$ (*if a* $=$ *c then 1 else 0*) $+ \beta *$ (*if a* $=$ *d then 1 else 0*)
$+$ *cnj* $\alpha *$ (*if b* $=$ *c then 1 else 0*) $+$ *cnj* $\alpha * \beta *$ (*if b* $=$ *d then 1 else 0*)
⟨*proof*⟩

lemma *gvec-inner*:
assumes *m*: $m \in \{0,1,2,4\}$ **and** *m'*: $m' \in \{0,1,2,4\}$
shows ⟨*gvec p m* | *gvec p' m'*⟩ $=$ (*if* $m = m' \wedge p = p'$ *then 2 else 0*)
⟨*proof*⟩

41.10 Code words and the images of the errors

definition *gv* :: *bool* $\times$ *nat* $\Rightarrow$ *complex Matrix.vec* **where** *gv x* $=$ *gvec* (*fst x*) (*snd x*)

A label lists the block vectors of a product state, one per block.

definition *lab-vec* :: (*bool* $\times$ *nat*) *list* $\Rightarrow$ *complex Matrix.vec* **where**
lab-vec l $=$ *tv3* (*gv* (*l* ! *0*)) (*gv* (*l* ! *1*)) (*gv* (*l* ! *2*))

definition *valid-label* :: (*bool* $\times$ *nat*) *list* $\Rightarrow$ *bool* **where**
valid-label l $\longleftrightarrow$ *length l* $= 3 \wedge (\forall x \in$ *set l*. *snd x* $\in \{0,1,2,4\})$

definition *sk* :: *complex* **where** *sk* $=$ *complex-of-real* (*1 / sqrt 8*)

definition *shor-cw* :: *bool* $\Rightarrow$ *complex Matrix.vec* **where**
shor-cw p $=$ *sk* $\cdot_v$ *lab-vec* (*replicate 3* (*p*, *0*))

The notes' code words $|0_L\rangle, |1_L\rangle$.

lemma *shor-cw-notes*:
shor-cw True = complex-of-real (1 / sqrt (2^3)) ·$_v$
tensor-vec (tensor-vec (unit-vec 8 0 + unit-vec 8 7) (unit-vec 8 0 + unit-vec 8 7)) (unit-vec 8 0 + unit-vec 8 7)
shor-cw False = complex-of-real (1 / sqrt (2^3)) ·$_v$
tensor-vec (tensor-vec (unit-vec 8 0 − unit-vec 8 7) (unit-vec 8 0 − unit-vec 8 7)) (unit-vec 8 0 − unit-vec 8 7)
⟨*proof*⟩

lemma *gv-dim* [*simp*]: *dim-vec (gv x) = 8*
⟨*proof*⟩

lemma *lab-vec-dim* [*simp*]: *dim-vec (lab-vec l) = 512*
⟨*proof*⟩

lemma *shor-cw-dim* [*simp*]: *dim-vec (shor-cw p) = 512*
⟨*proof*⟩

lemma *tv3-smult*:
assumes *dim-vec x = 8 dim-vec y = 8 dim-vec z = 8*
shows *tv3 (k ·$_v$ x) y z = k ·$_v$ tv3 x y z tv3 x (k ·$_v$ y) z = k ·$_v$ tv3 x y z*
tv3 x y (k ·$_v$ z) = k ·$_v$ tv3 x y z
⟨*proof*⟩

lemma *qubit-op-tl*:
assumes *i*: *i < 9* **and** *U*: *U ∈ carrier-mat 2 2*
and *act*: *bop (i mod 3) U *$_v$ gvec p 0 = κ ·$_v$ gvec q m*
shows *qubit-op i U *$_v$ lab-vec (replicate 3 (p, 0)) = κ ·$_v$ lab-vec ((replicate 3 (p, 0))[i div 3 := (q, m)])*
⟨*proof*⟩

lemma *qubit-op-X-tl*:
*i < 9 ⟹ qubit-op i Pauli-X *$_v$ lab-vec (replicate 3 (p, 0))*
= lab-vec ((replicate 3 (p, 0))[i div 3 := (p, 2^(2 − i mod 3))])
⟨*proof*⟩

lemma *qubit-op-Y-tl*:
*i < 9 ⟹ qubit-op i Pauli-Y *$_v$ lab-vec (replicate 3 (p, 0))*
= (−i) ·$_v$ lab-vec ((replicate 3 (p, 0))[i div 3 := (¬ p, 2^(2 − i mod 3))])
⟨*proof*⟩

lemma *qubit-op-Z-tl*:
*i < 9 ⟹ qubit-op i Pauli-Z *$_v$ lab-vec (replicate 3 (p, 0))*
= lab-vec ((replicate 3 (p, 0))[i div 3 := (¬ p, 0)])
⟨*proof*⟩

lemma *lab-vec-inner*:
assumes *l*: *valid-label l* **and** *l′*: *valid-label l′*
shows ⟨*lab-vec l* | *lab-vec l′*⟩ = (*if l = l′ then 8 else 0*)
⟨*proof*⟩

41.11 The 22 correctable errors

I, $X_1, \ldots, X_9$, $Y_1, \ldots, Y_9$, Z_1, Z_4, Z_7 (positions counted from 0 here).

definition *shor-errors* :: *complex Matrix.mat list* **where**
shor-errors = 1$_m$ 512 # map (λi. qubit-op i Pauli-X) [0..<9] @ map (λi. qubit-op i Pauli-Y) [0..<9]
*@ map (λb. qubit-op (3 * b) Pauli-Z) [0..<3]*

definition *shor-labels* :: *bool* ⇒ (*bool* × *nat*) *list list* **where**
shor-labels p = *replicate 3* (*p*, *0*)
map (λ*i*. (*replicate 3* (*p*, *0*))[*i div 3* := (*p*, *2*^(*2* − *i mod 3*))]) [*0*..<*9*]
@ *map* (λ*i*. (*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *2*^(*2* − *i mod 3*))]) [*0*..<*9*]
@ *map* (λ*b*. (*replicate 3* (*p*, *0*))[*b* := (¬ *p*, *0*)]) [*0*..<*3*]

definition *shor-kappa* :: *complex list* **where**
shor-kappa = *1* # *replicate 9 1* @ *replicate 9* (−i) @ *replicate 3 1*

lemma *shor-lengths* [*simp*]: *length shor-errors* = *22 length* (*shor-labels p*) = *22 length shor-kappa* = *22*
⟨*proof*⟩

lemma *shor-errors-cases*:
assumes *j*: *j* < *22*
obtains *j* = *0 shor-errors* ! *j* = *1*$_m$ *512 shor-labels p* ! *j* = *replicate 3* (*p*, *0*) *shor-kappa* ! *j* = *1*
| *i* **where** *i* < *9 j* = *Suc i shor-errors* ! *j* = *qubit-op i Pauli-X*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*i div 3* := (*p*, *2*^(*2* − *i mod 3*))] *shor-kappa* ! *j* = *1*
| *i* **where** *i* < *9 j* = *10* + *i shor-errors* ! *j* = *qubit-op i Pauli-Y*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*i div 3* := (¬ *p*, *2*^(*2* − *i mod 3*))] *shor-kappa* ! *j* = −i
| *b* **where** *b* < *3 j* = *19* + *b shor-errors* ! *j* = *qubit-op* (*3* ∗ *b*) *Pauli-Z*
shor-labels p ! *j* = (*replicate 3* (*p*, *0*))[*b* := (¬ *p*, *0*)] *shor-kappa* ! *j* = *1*
⟨*proof*⟩

lemma *shor-errors-carrier*: *j* < *22* ⟹ *shor-errors* ! *j* ∈ *carrier-mat 512 512*
⟨*proof*⟩

lemma *shor-errors-action*:
assumes *j*: *j* < *22*
shows *shor-errors* ! *j* ∗$_v$ *shor-cw p* = (*shor-kappa* ! *j* ∗ *sk*) ·$_v$ *lab-vec* (*shor-labels p* ! *j*)
⟨*proof*⟩

lemma *shor-labels-valid*: *j* < *22* ⟹ *valid-label* (*shor-labels p* ! *j*)
⟨*proof*⟩

lemma *shor-labels-distinct*: *distinct* (*shor-labels True* @ *shor-labels False*)
⟨*proof*⟩

lemma *shor-labels-differ*:
assumes *i*: *i* < *22* **and** *j*: *j* < *22* **and** *ij*: *i* ≠ *j*
shows *shor-labels p* ! *i* ≠ *shor-labels p′* ! *j*
⟨*proof*⟩

lemma *shor-errors-orth*:
assumes *i*: *i* < *22* **and** *j*: *j* < *22* **and** *ij*: *i* ≠ *j*
shows ⟨*shor-errors* ! *i* ∗$_v$ *shor-cw p* | *shor-errors* ! *j* ∗$_v$ *shor-cw p′*⟩ = *0*
⟨*proof*⟩

lemma *sk-sq*: *cnj sk* ∗ *sk* = *1* / *8*
⟨*proof*⟩

lemma *shor-cw-inner*: ⟨*shor-cw p* | *shor-cw p′*⟩ = (*if p* = *p′ then 1 else 0*)
⟨*proof*⟩

41.12 The code space and its projection

definition *shor-V* :: *complex Matrix.mat* **where** *shor-V* = *mat-of-cols 512* [*shor-cw True*, *shor-cw False*]

definition *shor-P* :: *complex Matrix.mat* **where** *shor-P* = *shor-V* ∗ (*shor-V*†)

definition *shor-state* :: *complex* ⇒ *complex* ⇒ *complex Matrix.vec* **where**
shor-state α β = α ·$_v$ *shor-cw True* + β ·$_v$ *shor-cw False*

lemma *shor-state-dim* [*simp*]: *dim-vec* (*shor-state* α β) = *512*
⟨*proof*⟩

lemma *shor-V-carrier*: *shor-V* ∈ *carrier-mat 512 2*
⟨*proof*⟩

lemma *shor-orthonormal*: *orthonormal-list 512* [*shor-cw True*, *shor-cw False*]
⟨*proof*⟩

lemma *shor-VdV*: (*shor-V*†) ∗ *shor-V* = 1_m *2*
⟨*proof*⟩

lemma *mat-vec-unit-col*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat n k* **and** *a*: *a* < *k*
shows *M* ∗$_v$ *unit-vec k a* = *Matrix.col M a*
⟨*proof*⟩

lemma *shor-V-col*: *a* < *2* ⟹ *Matrix.col shor-V a* = [*shor-cw True*, *shor-cw False*] ! *a*
⟨*proof*⟩

lemma *shor-V-unit*: *shor-V* ∗$_v$ *unit-vec 2 0* = *shor-cw True shor-V* ∗$_v$ *unit-vec 2 1* = *shor-cw False*
⟨*proof*⟩

lemma *shor-P-projection*: *projection 512 shor-P*
⟨*proof*⟩

lemma *shor-P-cw*: *shor-P* ∗$_v$ *shor-cw p* = *shor-cw p*
⟨*proof*⟩

lemma *shor-P-fixes*: *shor-P* ∗$_v$ *shor-state* α β = *shor-state* α β
⟨*proof*⟩

41.13 Proposition (Day 33): orthogonal images, and $Z_1v = Z_2v = Z_3v$

lemma *gram-entry*:
fixes *A B* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat n k* **and** *B*: *B* ∈ *carrier-mat n k* **and** *a*: *a* < *k* **and** *b*: *b* < *k*
shows ((*A*†) ∗ *B*) $$ (*a*,*b*) = ⟨*Matrix.col A a* | *Matrix.col B b*⟩
⟨*proof*⟩

lemma *sandwich-VVd*:
fixes *V W* :: *complex Matrix.mat*
assumes *V*: *V* ∈ *carrier-mat n k* **and** *W*: *W* ∈ *carrier-mat n n*
shows (*V* ∗ (*V*†)) ∗ *W* ∗ (*V* ∗ (*V*†)) = *V* ∗ ((*V*†) ∗ *W* ∗ *V*) ∗ (*V*†)
⟨*proof*⟩

lemma *qubit-op-unitary*: *unitary U* ⟹ *unitary* (*qubit-op i U*)
⟨*proof*⟩

lemma *shor-errors-unitary*: *U* ∈ *set shor-errors* ⟹ *U* ∈ *carrier-mat 512 512* ∧ *unitary U*
⟨*proof*⟩

theorem *shor-orthogonal-images*:
assumes *i*: $i < 22$ **and** *j*: $j < 22$ **and** *ij*: $i \neq j$
shows $shor\text{-}P * (((shor\text{-}errors\ !\ i)^\dagger) * shor\text{-}errors\ !\ j) * shor\text{-}P = 0_m\ 512\ 512$
⟨*proof*⟩

lemma *shor-recovery-setting*: *recovery-setting 512 shor-P shor-errors*
⟨*proof*⟩

lemma *qubit-op-Z-cw*:
assumes *i*: $i < 9$
shows $qubit\text{-}op\ i\ Pauli\text{-}Z *_v shor\text{-}cw\ p = qubit\text{-}op\ (3 * (i\ div\ 3))\ Pauli\text{-}Z *_v shor\text{-}cw\ p$
⟨*proof*⟩

theorem *shor-Z-coincide*:
assumes *i*: $i < 9$ **and** *j*: $j < 9$ **and** *ij*: $i\ div\ 3 = j\ div\ 3$
shows $qubit\text{-}op\ i\ Pauli\text{-}Z *_v shor\text{-}state\ \alpha\ \beta = qubit\text{-}op\ j\ Pauli\text{-}Z *_v shor\text{-}state\ \alpha\ \beta$
⟨*proof*⟩

The Day 32 proposition as first stated (distinct 1-Paulis map V to orthogonal subspaces) is false: $Z_1 \neq Z_2$, yet $Z_1|0_L\rangle = Z_2|0_L\rangle \neq 0$. The notes correct it on Day 33.

lemma *shor-notes-Z1-Z2*:
$qubit\text{-}op\ 0\ Pauli\text{-}Z \neq qubit\text{-}op\ 1\ Pauli\text{-}Z$
$\langle qubit\text{-}op\ 0\ Pauli\text{-}Z *_v shor\text{-}cw\ True \mid qubit\text{-}op\ 1\ Pauli\text{-}Z *_v shor\text{-}cw\ True\rangle = 1$
⟨*proof*⟩

41.14 Shor's theorem

lemma *pauli-decomp*:
fixes *U* :: *complex Matrix.mat*
assumes *U*: $U \in carrier\text{-}mat\ 2\ 2$
shows $U = ((U\ \$\$\ (0,0) + U\ \$\$\ (1,1)) / 2) \cdot_m 1_m\ 2 + ((U\ \$\$\ (0,1) + U\ \$\$\ (1,0)) / 2) \cdot_m Pauli\text{-}X$
$+ (-\mathrm{i} * (U\ \$\$\ (0,1) - U\ \$\$\ (1,0)) / 2) \cdot_m Pauli\text{-}Y + ((U\ \$\$\ (0,0) - U\ \$\$\ (1,1)) / 2) \cdot_m Pauli\text{-}Z$
(**is** $U = ?R$)
⟨*proof*⟩

lemma *qubit-op-add*:
assumes *A*: $A \in carrier\text{-}mat\ 2\ 2$ **and** *B*: $B \in carrier\text{-}mat\ 2\ 2$
shows $qubit\text{-}op\ i\ (A + B) = qubit\text{-}op\ i\ A + qubit\text{-}op\ i\ B$
⟨*proof*⟩

lemma *qubit-op-smult*:
assumes *A*: $A \in carrier\text{-}mat\ 2\ 2$
shows $qubit\text{-}op\ i\ (k \cdot_m A) = k \cdot_m qubit\text{-}op\ i\ A$
⟨*proof*⟩

lemma *qubit-op-one*:
assumes *i*: $i < 9$
shows $qubit\text{-}op\ i\ (1_m\ 2) = 1_m\ 512$
⟨*proof*⟩

lemma *shor-errors-nth*:
assumes *i*: $i < 9$ **and** *b*: $b < 3$
shows $shor\text{-}errors\ !\ 0 = 1_m\ 512$ $shor\text{-}errors\ !\ Suc\ i = qubit\text{-}op\ i\ Pauli\text{-}X$
$shor\text{-}errors\ !\ (10 + i) = qubit\text{-}op\ i\ Pauli\text{-}Y$ $shor\text{-}errors\ !\ (19 + b) = qubit\text{-}op\ (3 * b)\ Pauli\text{-}Z$
⟨*proof*⟩

Shor: one CPTP recovery map R corrects every single-qubit unitary error on every qubit, for every state $\psi = a|0_L\rangle + b|1_L\rangle$ of the code space.

theorem *shor-code-theorem*:
  $\exists$ *Rs. kraus-operators 512 512 Rs* $\wedge$
    ($\forall$ *U i* α β. *U* $\in$ *carrier-mat 2 2* $\longrightarrow$ *unitary U* $\longrightarrow$ *i* $<$ *9* $\longrightarrow$
      kraus-map-dim 512 Rs (*qubit-op i U* $*$ (*shor-state* α β $\oslash$ *shor-state* α β) $*$ ((*qubit-op i U*)†))
      = *shor-state* α β $\oslash$ *shor-state* α β)
⟨*proof*⟩

end
theory *Day34-Knill-Laflamme*
  imports *Day22-Minimal-Kraus*
begin

42 Day 34: the Knill–Laflamme conditions (D34-1, D34-2, D34-3)

42.1 Vectors in the range of a projection; rank-one decompositions inside V

lemma *projection-fixed-of-orth-kernel*:
  assumes *P*: *projection n P* **and** *vd*: *dim-vec v* = *n*
    and *orth*: $\bigwedge$*x. dim-vec x* = *n* $\Longrightarrow$ *P* $*_v$ *x* = 0_v *n* $\Longrightarrow$ $\langle$*v* | *x*$\rangle$ = *0*
  shows *P* $*_v$ *v* = *v*
⟨*proof*⟩

lemma *outer-sum-Nil*: *outer-sum n* [] = 0_m *n n*
  ⟨*proof*⟩

lemma *outer-sum-singleton-eq*:
  assumes *vd*: *dim-vec v* = *n*
  shows *outer-sum n* [*v*] = *v* $\oslash$ *v*
⟨*proof*⟩

lemma *outer-sum-as-foldr*:
  assumes $\forall$ *v* $\in$ *set vs. dim-vec v* = *n*
  shows *outer-sum n vs* = *foldr* (λ*v acc. v* $\oslash$ *v* + *acc*) *vs* (0_m *n n*)
  ⟨*proof*⟩

For $X \geq 0$ the matrix PXP is a sum of $|\psi_l\rangle\langle\psi_l|$ with $\psi_l \in V$ (the notes: "non-zero eigenvectors of PXP are all in V").

lemma *psd-PXP-decomp-in-range*:
  assumes *P*: *projection n P* **and** *Xc*: *M* $\in$ *carrier-mat n n* **and** *Xpsd*: *psd M*
  shows $\exists$ *vs* :: *complex Matrix.vec list*. ($\forall$ *v* $\in$ *set vs. dim-vec v* = *n* $\wedge$ *P* $*_v$ *v* = *v*) $\wedge$
      P $*$ *M* $*$ *P* = *foldr* (λ*v acc. v* $\oslash$ *v* + *acc*) *vs* (0_m *n n*)
⟨*proof*⟩

42.2 Linearity facts for the composite map $\mathcal{R} \circ \mathbb{E}$

definition *RE* :: *nat* $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat* **where**
  RE n As Es M = *kraus-map-dim n As* (*kraus-map-dim n Es M*)

lemma *RE-carrier*: **assumes** *As-c*: $\forall$ *A* $\in$ *set As. A* $\in$ *carrier-mat n n* **and** *Es-c*: $\forall$ *E* $\in$ *set Es. E* $\in$ *carrier-mat n n* **and** *M*: *M* $\in$ *carrier-mat n n* **shows** *RE n As Es M* $\in$ *carrier-mat n n*
  ⟨*proof*⟩

lemma *RE-add*: **assumes** *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ n\ n$ **and** *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$ **and** *M*: $M \in carrier\text{-}mat\ n\ n$ **and** *N*: $N \in carrier\text{-}mat\ n\ n$ **shows** $RE\ n\ As\ Es\ (M + N) = RE\ n\ As\ Es\ M + RE\ n\ As\ Es\ N$
⟨*proof*⟩

lemma *RE-smult*: **assumes** *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ n\ n$ **and** *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$ **and** *M*: $M \in carrier\text{-}mat\ n\ n$ **shows** $RE\ n\ As\ Es\ (c \cdot_m M) = c \cdot_m RE\ n\ As\ Es\ M$
⟨*proof*⟩

lemma *RE-minus*:
assumes *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ n\ n$ **and** *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$
and *Xc*: $M \in carrier\text{-}mat\ n\ n$ **and** *Yc*: $N \in carrier\text{-}mat\ n\ n$
shows $RE\ n\ As\ Es\ (M - N) = RE\ n\ As\ Es\ M - RE\ n\ As\ Es\ N$
⟨*proof*⟩

lemma *RE-foldr*:
assumes *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ n\ n$ **and** *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$ **and** $\forall v \in set\ vs.\ dim\text{-}vec\ v = n$
shows $RE\ n\ As\ Es\ (foldr\ (\lambda v\ acc.\ v \oslash v + acc)\ vs\ (0_m\ n\ n)) = foldr\ (\lambda v\ acc.\ RE\ n\ As\ Es\ (v \oslash v) + acc)\ vs\ (0_m\ n\ n)$
⟨*proof*⟩

D34-1: $\mathcal{R} \circ \mathbb{E}$ fixes $|\psi\rangle\langle\psi|$ for all $\psi \in V$ iff it fixes PXP for all X.

proposition *recovery-states-iff-PXP*:
assumes *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ n\ n$ **and** *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$ **and** *P*: *projection n P*
shows $(\forall \psi.\ dim\text{-}vec\ \psi = n \longrightarrow P *_v \psi = \psi \longrightarrow RE\ n\ As\ Es\ (\psi \oslash \psi) = \psi \oslash \psi)$
$\longleftrightarrow (\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow RE\ n\ As\ Es\ (P * X * P) = P * X * P)$
⟨*proof*⟩

42.3 Knill–Laflamme, Theorem 1, necessity: $PE_i^* E_j P = \alpha_{ij} P$

lemma *lin-indep-complex-mat-single*:
assumes *Pc*: $P \in carrier\text{-}mat\ d\ n$ **and** *Pnz*: $P \neq 0_m\ d\ n$
shows *lin-indep-complex-mat d n* $[P]$
⟨*proof*⟩

The core of the necessity proof: by Choi's Theorem 2 applied to the two Kraus representations $\{P\}$ and $\{A_l E_i P\}$ of $X \mapsto PXP$, every $A_l E_i P$ is a multiple of P.

lemma *kl-kraus-scalars*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As-c*: $\forall A \in set\ As.\ A \in carrier\text{-}mat\ n\ n$ **and** *Es-c*: $\forall E \in set\ Es.\ E \in carrier\text{-}mat\ n\ n$
and *corr*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
$kraus\text{-}map\text{-}dim\ n\ As\ (kraus\text{-}map\text{-}dim\ n\ Es\ (P * X * P)) = P * X * P$
shows $\exists \beta :: nat \Rightarrow nat \Rightarrow complex.\ \forall l < length\ As.\ \forall i < length\ Es.$
$As\ !\ l * Es\ !\ i * P = \beta\ l\ i \cdot_m P$
⟨*proof*⟩

theorem *knill-laflamme-necessity*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As*: *kraus-operators n n As* **and** *Es*: *kraus-operators n n Es*
and *corr*: $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow$
$kraus\text{-}map\text{-}dim\ n\ As\ (kraus\text{-}map\text{-}dim\ n\ Es\ (P * X * P)) = P * X * P$
shows $\exists \alpha :: nat \Rightarrow nat \Rightarrow complex.\ \forall i < length\ Es.\ \forall j < length\ Es.$
$P * (((Es\ !\ i)^\dagger) * Es\ !\ j) * P = \alpha\ i\ j \cdot_m P$
⟨*proof*⟩

42.4 Kraus lists consisting of multiples of one operator

lemma *kraus-map-dim-scalar-list*:
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Xc*: $M \in$ *carrier-mat n n*
shows *kraus-map-dim n* (*map* ($\lambda\gamma.\ \gamma \cdot_m P$) *gs*) *M*
$=$ *sum-list* (*map* ($\lambda\gamma.$ *cnj* $\gamma * \gamma$) *gs*) $\cdot_m$ ($P * M * (P^\dagger)$)
⟨*proof*⟩

lemma *sum-wdag-w-scalar-list*:
assumes *Pc*: $P \in$ *carrier-mat n n*
shows *sum-wdag-w n* (*map* ($\lambda\gamma.\ \gamma \cdot_m P$) *gs*) $=$ *sum-list* (*map* ($\lambda\gamma.$ *cnj* $\gamma * \gamma$) *gs*) $\cdot_m$ ($(P^\dagger) * P$)
⟨*proof*⟩

lemma *smult-eq-self-imp-one*:
fixes P :: *complex Matrix.mat*
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *eq*: $s \cdot_m P = P$
shows $s = 1$
⟨*proof*⟩

42.5 Theorem 2

If $\mathcal{R} \circ \mathbb{E}$ fixes PXP and $G_i = \sum_j g_{ij} E_j$ with $\sum_i G_i^* G_i = I$, then $\mathcal{R} \circ \mathbb{G}$ fixes PXP as well (same $\mathcal{R}$).

theorem *knill-laflamme-closure*:
assumes *P*: *projection n P* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *npos*: $0 < n$
and *As*: *kraus-operators n n As*
and *Es-c*: $\forall E \in$ *set Es*. $E \in$ *carrier-mat n n*
and *corr*: $\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow$
kraus-map-dim n As (*kraus-map-dim n Es* ($P * X * P$)) $= P * X * P$
and *Gc*: $G \in$ *carrier-mat t* (*length Es*)
and *Gs-tp*: *sum-wdag-w n* (*kraus-mix n n G Es*) $= 1_m\ n$
shows $\forall X.\ X \in$ *carrier-mat n n* $\longrightarrow$
kraus-map-dim n As (*kraus-map-dim n* (*kraus-mix n n G Es*) ($P * X * P$)) $= P * X * P$
⟨*proof*⟩

end
theory *Day34-KL-Sufficiency*
imports *Day34-Knill-Laflamme*
begin

D34-2, sufficiency: if $PE_i^* E_j P = \alpha_{ij} P$ then a recovery channel exists. Diagonalise $(\alpha_{ij}) = u\,\mathrm{diag}(d)\,u^*$; the errors $F_k = \sum_i u_{ik} E_i$ give the same channel and satisfy $PF_k^* F_l P = \delta_{kl} d_k P$, so the $F_k/\sqrt{d_k}$ with $d_k > 0$ satisfy the hypotheses of the recovery proposition (*recovery-setting*).

lemma *smult-cancel-nonzero*:
fixes P :: *complex Matrix.mat*
assumes *Pc*: $P \in$ *carrier-mat n n* **and** *Pnz*: $P \neq 0_m\ n\ n$ **and** *eq*: $c \cdot_m P = d \cdot_m P$
shows $c = d$
⟨*proof*⟩

lemma *outer-prod-smult-vec*:
fixes v :: *complex Matrix.vec*
shows ($s \cdot_v v$) $\oslash$ ($s \cdot_v v$) $=$ ($s * cnj\ s$) $\cdot_m$ ($v \oslash v$)
⟨*proof*⟩

lemma *kraus-map-dim-foldr-add*:
assumes *Ws-c*: $\forall W \in$ *set Ws*. $W \in$ *carrier-mat n n* **and** *fc*: $\forall k \in$ *set xs*. $f\,k \in$ *carrier-mat n n*

shows *kraus-map-dim n Ws* (*foldr* (λ*k acc*. *f k* + *acc*) *xs* (0_m *n n*))
= *foldr* (λ*k acc*. *kraus-map-dim n Ws* (*f k*) + *acc*) *xs* (0_m *n n*)
⟨*proof*⟩

lemma *kraus-map-dim-outer-as-foldr*:
assumes *Ws-c*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat n n* **and** ψ*d*: *dim-vec* ψ = *n*
shows *kraus-map-dim n Ws* (ψ ⊘ ψ)
= *foldr* (λ*k acc*. (*Ws* ! *k* ∗$_v$ ψ) ⊘ (*Ws* ! *k* ∗$_v$ ψ) + *acc*) [*0*..<*length Ws*] (0_m *n n*)
⟨*proof*⟩

lemma *foldr-smult-sandwich*:
fixes *V* :: *nat* ⇒ *complex Matrix.mat*
assumes *Mc*: *M* ∈ *carrier-mat n n* **and** *Nc*: *N* ∈ *carrier-mat n n* **and** *Pc*: *P* ∈ *carrier-mat n n*
and *Vc*: ∀ *j* < *r*. *V j* ∈ *carrier-mat n n* **and** *h*: ∀ *j* < *r*. *M* ∗ *V j* ∗ *N* = β *j* ·$_m$ *P*
shows *M* ∗ *foldr* (λ*j acc*. *b j* ·$_m$ *V j* + *acc*) [*0*..<*r*] (0_m *n n*) ∗ *N* = (∑*j*<*r*. *b j* ∗ β *j*) ·$_m$ *P*
⟨*proof*⟩

lemma *foldr-smult-dagger*:
fixes *V* :: *nat* ⇒ *complex Matrix.mat*
assumes *Vc*: ∀ *j* < *r*. *V j* ∈ *carrier-mat n n*
shows (*foldr* (λ*j acc*. *b j* ·$_m$ *V j* + *acc*) [*0*..<*r*] (0_m *n n*))†
= *foldr* (λ*j acc*. *cnj* (*b j*) ·$_m$ ((*V j*)†) + *acc*) [*0*..<*r*] (0_m *n n*)
⟨*proof*⟩

The bracket $P\,(\sum_i a_i E_i)^*(\sum_j b_j E_j)\,P$ under the Knill–Laflamme conditions.

lemma *kl-mix-bracket*:
assumes *Es-c*: ∀ *E* ∈ *set Es*. *E* ∈ *carrier-mat n n* **and** *Pc*: *P* ∈ *carrier-mat n n*
and *kl*: ∀ *i* < *length Es*. ∀ *j* < *length Es*. *P* ∗ (((*Es* ! *i*)†) ∗ *Es* ! *j*) ∗ *P* = α *i j* ·$_m$ *P*
and *r*: *r* = *length Es*
shows *P* ∗ (((*foldr* (λ*i acc*. *a i* ·$_m$ (*Es* ! *i*) + *acc*) [*0*..<*r*] (0_m *n n*))†)
∗ *foldr* (λ*j acc*. *b j* ·$_m$ (*Es* ! *j*) + *acc*) [*0*..<*r*] (0_m *n n*)) ∗ *P*
= (∑*j*<*r*. *b j* ∗ (∑ *i*<*r*. *cnj* (*a i*) ∗ α *i j*)) ·$_m$ *P*
⟨*proof*⟩

lemma *dagger-sandwich*:
assumes *Pc*: *P* ∈ *carrier-mat n n* **and** *Ph*: *P*† = *P* **and** *Ac*: *A* ∈ *carrier-mat n n* **and** *Bc*: *B* ∈ *carrier-mat n n*
shows (*P* ∗ ((*A*†) ∗ *B*) ∗ *P*)† = *P* ∗ ((*B*†) ∗ *A*) ∗ *P*
⟨*proof*⟩

lemma *smult-bracket*:
assumes *Pc*: *P* ∈ *carrier-mat n n* **and** *Ac*: *A* ∈ *carrier-mat n n* **and** *Bc*: *B* ∈ *carrier-mat n n*
shows *P* ∗ (((*x* ·$_m$ *A*)†) ∗ (*y* ·$_m$ *B*)) ∗ *P* = (*cnj x* ∗ *y*) ·$_m$ (*P* ∗ ((*A*†) ∗ *B*) ∗ *P*)
⟨*proof*⟩

theorem *knill-laflamme-sufficiency*:
assumes *P*: *projection n P* **and** *Pnz*: *P* ≠ 0_m *n n* **and** *npos*: *0* < *n*
and *Es*: *kraus-operators n n Es*
and *kl*: ∀ *i* < *length Es*. ∀ *j* < *length Es*. *P* ∗ (((*Es* ! *i*)†) ∗ *Es* ! *j*) ∗ *P* = α *i j* ·$_m$ *P*
shows ∃ *As*. *kraus-operators n n As* ∧
(∀ *X*. *X* ∈ *carrier-mat n n* ⟶ *kraus-map-dim n As* (*kraus-map-dim n Es* (*P* ∗ *X* ∗ *P*)) = *P* ∗ *X* ∗ *P*)
⟨*proof*⟩

D34-2 (Theorem 1): the code is correctable iff $PE_i^* E_j P = \alpha_{ij} P$.

theorem *knill-laflamme*:
assumes *P*: *projection n P* **and** *Pnz*: *P* ≠ 0_m *n n* **and** *npos*: *0* < *n*
and *Es*: *kraus-operators n n Es*

shows ($\exists$ *As. kraus-operators n n As* $\wedge$
($\forall$ *X. X* $\in$ *carrier-mat n n* $\longrightarrow$ *kraus-map-dim n As* (*kraus-map-dim n Es* (*P* $*$ *X* $*$ *P*)) = *P* $*$ *X* $*$ *P*))
$\longleftrightarrow$ ($\exists$ α :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *complex*. $\forall$ *i* < *length Es*. $\forall$ *j* < *length Es*.
P $*$ (((*Es* ! *i*)†) $*$ *Es* ! *j*) $*$ *P* = α *i j* $\cdot_m$ *P*)
⟨*proof*⟩

end

Part III

Lectures 23–26: extreme points, operator systems and Arveson's theorem

theory *Day23-ChoiExtremeUCP*
imports
Quantum-Computing11
begin

43 Day 23 (19 October 2011): Choi's Theorem 3, extreme points of UCP and CPTP

Day 22 stated and Day 23 proves:

Theorem (notes) (Choi 3)**.** If $\Phi : M_n \to M_d$ is unital and completely positive, $\Phi(X) = \sum_{i=1}^{r} V_i^* X V_i$ with $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $UCP(M_n, M_d)$ if and only if $\{V_i^* V_j : 1 \le i, j \le r\} \subseteq M_d$ is linearly independent.

Corollary (notes). If $\Phi : M_d \to M_n$ is completely positive and trace preserving, $\Phi(X) = \sum_{i=1}^{r} V_i X V_i^*$ with $r = \mathrm{cr}(\Phi)$, then Φ is extreme in $CPTP(M_d, M_n)$ iff $\{V_i^* V_j\}$ is linearly independent (apply the theorem to Φ^*).

The notes' proof. ($\Rightarrow$) If $\sum_{ij} \lambda_{ij} V_i^* V_j = 0$ it suffices to treat Hermitian $\mu = (\mu_{ij})$ with $-I \le \mu \le I$: writing $I \pm \mu = (\alpha_{ij})^*(\alpha_{ij})$ and $W_i = \sum_j \alpha_{ij} V_j$ shows that $\psi_\pm(X) = \sum_i V_i^* X V_i \pm \sum_{ij} \mu_{ij} V_i^* X V_j$ are UCP with $\Phi = \frac{1}{2}(\psi_+ + \psi_-)$, so extremality gives $\psi_+ = \Phi$, and Choi 2 (unitary freedom) forces $I + \mu = U^*U = I$, i.e. $\mu = 0$. ($\Leftarrow$) If $\Phi = \frac{1}{2}(\psi_1 + \psi_2)$ with $\psi_k(X) = \sum_p W_p^* X W_p$, Choi 2 gives $W_p/\sqrt{2} = \sum_j \alpha_{pj} V_j$ with $U^*U = I_r$; unitality of ψ_1 and linear independence of the $V_i^* V_j$ force $U_1^* U_1 = \frac{1}{2} I_r$, whence $\psi_1 = \Phi = \psi_2$. Examples: $X \mapsto U^* X U$ is extreme; $\frac{1}{2}(U_1^* X U_1 + U_2^* X U_2)$, $X \mapsto \mathrm{tr}(X) I_d$ and $X \mapsto \mathrm{diag}(X)$ are not.
What is formalised here is the corollary, i.e. the CPTP form, for a fixed Kraus list *Vs* of linearly independent operators (the consequence of $r = \mathrm{cr}(\Phi)$ proved on Day 22): extremality (in the carrier sense *extreme-CPTP-carrier*) holds if and only if the products $V_j^\dagger V_i$ are linearly independent over $\mathbb{C}$ (*choi-theorem-3-cptp-iff*). The direction ($\Rightarrow$) follows the notes' perturbation argument (*kraus-quad-map*, *small-eps-psd*); the direction ($\Leftarrow$) replaces Choi 2 by a CP Radon–Nikodym theorem proved below from the Moore–Penrose pseudoinverse of the Choi matrix. Unitary freedom itself is *choi-2-unitary-freedom* in *Day21-Unitary-Freedom*, proved afterwards and independently of this theory; the examples are in theory Day23_Extremality_Examples (the UCP form is not formalised).

43.1 Extreme UCP maps

An *extreme UCP map* is one that cannot be written as a proper convex combination of two distinct UCP maps. Following Day-22's *extreme-point-mat*, we use the half-half mid-point formulation; this is equivalent to the standard definition for convex sets.

definition *extreme-UCP* ::
  $nat \Rightarrow nat \Rightarrow (complex\ Matrix.mat \Rightarrow complex\ Matrix.mat) \Rightarrow bool$
where
  $extreme\text{-}UCP\ n\ d\ \Phi \equiv$
    $\Phi \in UCP\ n\ d \wedge$
    $\neg\ (\exists\, \Phi_1 \in UCP\ n\ d.\ \exists\, \Phi_2 \in UCP\ n\ d.$
        $\Phi_1 \neq \Phi \wedge \Phi \neq \Phi_2 \wedge$
        $(\forall\, X.\ X \in carrier\text{-}mat\ n\ n$
            $\longrightarrow \Phi\ X = (1/2 :: complex) \cdot_m \Phi_1\ X$
                $+ (1/2 :: complex) \cdot_m \Phi_2\ X))$

lemma *extreme-UCP-member*:
  assumes *extreme-UCP n d* Φ
  shows $\Phi \in UCP\ n\ d$
  ⟨*proof*⟩

Trivial unfolding: an extreme UCP map satisfies no non-trivial mid-point decomposition.

lemma *extreme-UCP-no-split*:
  assumes *ext*: *extreme-UCP n d* Φ
    and Φ*1-in*: $\Phi_1 \in UCP\ n\ d$
    and Φ*2-in*: $\Phi_2 \in UCP\ n\ d$
    and Φ*1-neq*: $\Phi_1 \neq \Phi$
    and Φ*2-neq*: $\Phi \neq \Phi_2$
  shows $\exists\, X \in carrier\text{-}mat\ n\ n.$
        $\Phi\ X \neq (1/2 :: complex) \cdot_m \Phi_1\ X$
            $+ (1/2 :: complex) \cdot_m \Phi_2\ X$
  ⟨*proof*⟩

43.2 UCP-Kraus representation

A *UCP Kraus representation* for a map $\Phi : M_n \to M_d$ is a list *Vs* of $d \times n$ matrices such that

- each $V \in Vs$ is $d \times n$;
- the unitality identity $\sum_i\ V_i\ {V_i}^\dagger = I_d$ holds;
- $\Phi\ X = \sum_i\ V_i\ X\ {V_i}^\dagger$ for every $X \in M_n$.

Note that the standard *kraus-operators* predicate of Day 17 uses the *trace-preserving* identity $\sum\ {V_i}^\dagger\ V_i = I_n$; the *unital* form $\sum\ V_i\ {V_i}^\dagger = I_d$ is the Heisenberg-dual condition appropriate for *UCP*.

definition *ucp-kraus-rep* ::
  $nat \Rightarrow nat \Rightarrow (complex\ Matrix.mat \Rightarrow complex\ Matrix.mat) \Rightarrow$
    $complex\ Matrix.mat\ list \Rightarrow bool$
where
  $ucp\text{-}kraus\text{-}rep\ n\ d\ \Phi\ Vs \equiv$
    $(\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n) \wedge$
    $foldr\ (\lambda V\ acc.\ V * (V^\dagger) + acc)\ Vs\ (0_m\ d\ d) = 1_m\ d \wedge$
    $(\forall\, X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ X = kraus\text{-}map\text{-}dim\ d\ Vs\ X)$

The *products set* of a Kraus list *Vs* is the list of $n \times n$ matrices ${V_j}^\dagger\ V_i$ for all pairs of indices (i,j).

definition *kraus-products* ::
  complex Matrix.mat list ⇒ *complex Matrix.mat list*
where
  kraus-products Vs ≡
    concat (*map* (λ*i*. *map* (λ*j*. (*Vs*!*j*)† ∗ *Vs*!*i*) [*0*..<*length Vs*])
          [*0*..<*length Vs*])

Real linear independence for a list of complex matrices. A list *Bs* is independent over $\mathbb{R}$ iff the only $\mathbb{R}$-coefficients making $\sum_k c_k\ B_k = 0$ are all zero. We use the real-coefficient form because Choi's perturbation argument constructs real combinations of hermitian generators.

definition *mat-lin-indep-real* ::
  nat ⇒ *complex Matrix.mat list* ⇒ *bool*
where
  mat-lin-indep-real n Bs ≡
    (∀ *B* ∈ *set Bs*. *B* ∈ *carrier-mat n n*) ∧
    (∀ *c* :: *nat* ⇒ *real*.
      foldr (λ*k acc*. *complex-of-real* (*c k*) ·$_m$ (*Bs*!*k*) + *acc*)
          [*0*..<*length Bs*] (*0*$_m$ *n n*)
        = *0*$_m$ *n n*
      ⟶ (∀ *k* < *length Bs*. *c k* = *0*))

Complex linear independence: the classical Choi statement uses **complex**-coefficient linear independence. This is strictly stronger than *mat-lin-indep-real*.

definition *mat-lin-indep-complex* ::
  nat ⇒ *complex Matrix.mat list* ⇒ *bool*
where
  mat-lin-indep-complex n Bs ≡
    (∀ *B* ∈ *set Bs*. *B* ∈ *carrier-mat n n*) ∧
    (∀ *c* :: *nat* ⇒ *complex*.
      foldr (λ*k acc*. *c k* ·$_m$ (*Bs*!*k*) + *acc*)
          [*0*..<*length Bs*] (*0*$_m$ *n n*)
        = *0*$_m$ *n n*
      ⟶ (∀ *k* < *length Bs*. *c k* = *0*))

Complex linear independence implies real.

lemma *mat-lin-indep-complex-imp-real*:
  assumes *mat-lin-indep-complex n Bs*
  shows *mat-lin-indep-real n Bs*
⟨*proof*⟩

43.3 Unital condition from the Kraus list

If *Ws* is a list of $d \times n$ matrices, then evaluating the Kraus map at the identity $1_m\ n$ gives exactly the sum $\sum\ W\ W^\dagger$. Combined with unitality, this gives the $\sum\ W\ W^\dagger = 1_m\ d$ identity that distinguishes UCP from plain CP.

lemma *kraus-map-dim-one*:
  assumes ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
  shows *kraus-map-dim d Ws* (*1*$_m$ *n*)
        = *foldr* (λ*W acc*. *W* ∗ (*W*†) + *acc*) *Ws* (*0*$_m$ *d d*)
  ⟨*proof*⟩

43.4 UCP-Kraus existence

Every linear UCP map admits a UCP Kraus representation of length *n d*. Linearity, scalar-multiplication and range hypotheses match those of *kraus-rep-of-psd-choi*; the unital condition $\sum\ V\ V^\dagger = 1_m\ d$ is supplied by Φ (*1*$_m$ *n*) = *1*$_m$ *d* + *kraus-map-dim-one*.

lemma *ucp-has-kraus-rep*:
fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
assumes *linear*: $\bigwedge A\ B.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $B \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (A + B) = \Phi\ A + \Phi\ B$
and *smult*: $\bigwedge c\ A.\ A \in$ *carrier-mat n n* $\Longrightarrow$ $\Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A$
and *range*: $\forall A.\ A \in$ *carrier-mat n n* $\longrightarrow$ $\Phi\ A \in$ *carrier-mat d d*
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
and *ucp*: $\Phi \in$ *UCP n d*
shows $\exists$ *Vs* :: *complex Matrix.mat list*.
ucp-kraus-rep n d Φ *Vs* $\wedge$ *length Vs* $= n * d$
⟨*proof*⟩

43.5 Sanity facts about *kraus-products*

lemma *kraus-products-carrier*:
assumes $\forall V \in$ *set Vs*. $V \in$ *carrier-mat d n*
shows $\forall B \in$ *set* (*kraus-products Vs*). $B \in$ *carrier-mat n n*
⟨*proof*⟩

43.6 Quadratic Kraus map (perturbation primitive)

For a Kraus list *Vs* of $d \times n$ matrices and a coefficient matrix $T \in M_k$ (where $k =$ *length Vs*), the *quadratic Kraus map* is (*kraus-quad-map d Vs T*) $X = \sum_{\{i,j\}} T_{\{ij\}}\ V_i\ X\ V_j{}^\dagger$. When $T = I_k$ this reduces to the ordinary Kraus map *kraus-map-dim d Vs* $X = \sum\ V_i\ X\ V_i{}^\dagger$; for PSD T the result is again a Kraus map (Lemma A below). The perturbations $T_\pm = I \pm \varepsilon\ A$ for small ε and hermitian A will provide the *splittings* witnessing non-extremity.

definition *kraus-quad-row* ::
nat $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$
complex Matrix.mat $\Rightarrow$ *nat* $\Rightarrow$ *complex Matrix.mat*
where
kraus-quad-row d Vs Tm rho i $\equiv$
foldr (λj *acc*. *Tm* $\$\$$ (*i*, *j*) $\cdot_m$ (*Vs*!*i* $*$ *rho* $*$ ((*Vs*!*j*)†)) $+$ *acc*)
[*0*..<*length Vs*] (0_m *d d*)

definition *kraus-quad-map* ::
nat $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *complex Matrix.mat* $\Rightarrow$
complex Matrix.mat $\Rightarrow$ *complex Matrix.mat*
where
kraus-quad-map d Vs Tm rho $\equiv$
foldr (λi *acc*. *kraus-quad-row d Vs Tm rho i* $+$ *acc*)
[*0*..<*length Vs*] (0_m *d d*)

Generic carrier-preservation lemma for accumulating folds over [*0*..<*N*]: if each per-index term lies in M_d, so does the fold.

lemma *foldr-upt-add-carrier*:
fixes *f* :: *nat* $\Rightarrow$ *complex Matrix.mat*
assumes $\forall k < N.\ f\ k \in$ *carrier-mat d d*
shows *foldr* (λk *acc*. *f k* $+$ *acc*) [*0*..<*N*] (0_m *d d*) $\in$ *carrier-mat d d*
⟨*proof*⟩

lemma *kraus-quad-row-carrier*:
assumes *Vs-c*: $\forall V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *i-lt*: *i* $<$ *length Vs*
shows *kraus-quad-row d Vs Tm rho i* $\in$ *carrier-mat d d*
⟨*proof*⟩

lemma *kraus-quad-map-carrier*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ Tm\ rho \in carrier\text{-}mat\ d\ d$
⟨*proof*⟩

Per-entry index formula for the accumulating fold (with arbitrary base).

lemma *foldr-upt-add-acc-index*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *carr*: $\forall\, k < N.\ f\ k \in carrier\text{-}mat\ d\ d$
and *z-c*: $z \in carrier\text{-}mat\ d\ d$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $foldr\ (\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ z\ \$\$\ (r,\ c)$
$= (\sum k = 0..<N.\ f\ k\ \$\$\ (r,\ c)) + z\ \$\$\ (r,\ c)$
⟨*proof*⟩

lemma *foldr-upt-add-index*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *carr*: $\forall\, k < N.\ f\ k \in carrier\text{-}mat\ d\ d$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $foldr\ (\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ (0_m\ d\ d)\ \$\$\ (r,\ c)$
$= (\sum k = 0..<N.\ f\ k\ \$\$\ (r,\ c))$
⟨*proof*⟩

Per-entry formula for *kraus-quad-row*.

lemma *kraus-quad-row-index*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *i-lt*: $i < length\ Vs$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $kraus\text{-}quad\text{-}row\ d\ Vs\ Tm\ rho\ i\ \$\$\ (r,\ c)$
$= (\sum j = 0..<length\ Vs.$
$Tm\ \$\$\ (i,\ j) * (Vs!i * rho * ((Vs!j)^\dagger))\ \$\$\ (r,\ c))$
⟨*proof*⟩

Per-entry formula for *kraus-quad-map*: double sum over indices.

lemma *kraus-quad-map-index*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $kraus\text{-}quad\text{-}map\ d\ Vs\ Tm\ rho\ \$\$\ (r,\ c)$
$= (\sum i = 0..<length\ Vs.$
$\sum j = 0..<length\ Vs.$
$Tm\ \$\$\ (i,\ j) * (Vs!i * rho * ((Vs!j)^\dagger))\ \$\$\ (r,\ c))$
⟨*proof*⟩

Carrier-preservation in a different dimension pair.

lemma *foldr-add-carrier-dn*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes $\forall\, x \in set\ xs.\ f\ x \in carrier\text{-}mat\ d\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ d\ n) \in carrier\text{-}mat\ d\ n$
⟨*proof*⟩

Linear combination of Kraus operators with complex coefficients. *kraus-combo d n Vs u* is $\sum_i u_i\ V_i$, a $d \times n$ matrix.

definition *kraus-combo* ::
$nat \Rightarrow nat \Rightarrow complex\ Matrix.mat\ list \Rightarrow complex\ Matrix.vec \Rightarrow$

complex Matrix.mat
where
kraus-combo d n Vs u ≡
foldr (λ*i acc. u* $ *i* ·$_m$ (*Vs*!*i*) + *acc*) [*0*..<*length Vs*] (*0*$_m$ *d n*)

lemma *kraus-combo-carrier*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs. V* ∈ *carrier-mat d n*
shows *kraus-combo d n Vs u* ∈ *carrier-mat d n*
⟨*proof*⟩

Per-entry formula for *kraus-combo*: foldr index lemma in d×n.

lemma *foldr-upt-add-acc-index-dn*:
fixes *f* :: *nat* ⇒ *complex Matrix.mat*
assumes *carr*: ∀ *k* < *N. f k* ∈ *carrier-mat d n*
and *z-c*: *z* ∈ *carrier-mat d n*
and *r-lt*: *r* < *d* **and** *c-lt*: *c* < *n*
shows *foldr* (λ*k acc. f k* + *acc*) [*0*..<*N*] *z* $$ (*r*, *c*)
= (∑ *k* = *0*..<*N. f k* $$ (*r*, *c*)) + *z* $$ (*r*, *c*)
⟨*proof*⟩

lemma *kraus-combo-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs. V* ∈ *carrier-mat d n*
and *r-lt*: *r* < *d* **and** *c-lt*: *c* < *n*
shows *kraus-combo d n Vs u* $$ (*r*, *c*)
= (∑ *i* = *0*..<*length Vs. u* $ *i* ∗ (*Vs*!*i*) $$ (*r*, *c*))
⟨*proof*⟩

A direct per-entry formula for matrix products.

lemma *mat-mult-index-sum*:
assumes *A-c*: *A* ∈ *carrier-mat p q*
and *B-c*: *B* ∈ *carrier-mat q s*
and *r-lt*: *r* < *p* **and** *c-lt*: *c* < *s*
shows (*A* ∗ *B*) $$ (*r*, *c*) = (∑ *k* = *0*..<*q. A* $$ (*r*, *k*) ∗ *B* $$ (*k*, *c*))
⟨*proof*⟩

Per-entry formula for *W* ∗ *rho*, where *W* = *kraus-combo d n Vs u*.

lemma *kraus-combo-times-rho-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs. V* ∈ *carrier-mat d n*
and *rho-c*: *rho* ∈ *carrier-mat n n*
and *r-lt*: *r* < *d* **and** *k-lt*: *k* < *n*
shows (*kraus-combo d n Vs u* ∗ *rho*) $$ (*r*, *k*)
= (∑ *i* = *0*..<*length Vs. u* $ *i* ∗ (*Vs*!*i* ∗ *rho*) $$ (*r*, *k*))
⟨*proof*⟩

The full quadratic per-entry formula: (*W rho* $W^\dagger$) $$ (*r*,*c*) = ∑$_{\{i,j\}}$ u_i |u_j| (V_i *rho* $V_j{}^\dagger$) $$ (*r*,*c*).

lemma *kraus-combo-quad-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs. V* ∈ *carrier-mat d n*
and *rho-c*: *rho* ∈ *carrier-mat n n*
and *r-lt*: *r* < *d* **and** *c-lt*: *c* < *d*
shows (*kraus-combo d n Vs u* ∗ *rho* ∗ ((*kraus-combo d n Vs u*)†)) $$ (*r*, *c*)
= (∑ *i* = *0*..<*length Vs.* ∑ *j* = *0*..<*length Vs.*
u $ *i* ∗ *cnj* (*u* $ *j*) ∗ (*Vs*!*i* ∗ *rho* ∗ ((*Vs*!*j*)†)) $$ (*r*, *c*))
⟨*proof*⟩

Linearity of *kraus-quad-map* in the coefficient matrix (needed for the rank-1 decomposition below).

lemma *kraus-quad-map-add-Tm*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *T1-c*: *T1* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
and *T2-c*: *T2* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
shows *kraus-quad-map d Vs* (*T1* + *T2*) *rho*
= *kraus-quad-map d Vs T1 rho* + *kraus-quad-map d Vs T2 rho*
⟨*proof*⟩

Carrier of *outer-sum k vs*.

lemma *outer-sum-carrier*:
outer-sum k vs $\in$ *carrier-mat k k*
⟨*proof*⟩

Cons-recursion for *outer-sum*: adds a rank-1 piece.

lemma *outer-sum-Cons-split*:
assumes $\forall\, v \in$ *set* (*u* # *us*). *dim-vec v* = *k*
shows *outer-sum k* (*u* # *us*)
= *outer-sum k* [*u*] + *outer-sum k us*
⟨*proof*⟩

Rank-1 identity: *kraus-quad-map d Vs* (*outer-sum k* [*u*]) *rho* = *W rho* $W^\dagger$ where W = *kraus-combo d n Vs u*.

lemma *kraus-quad-map-outer-singleton*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *u-dim*: *dim-vec u* = *length Vs*
shows *kraus-quad-map d Vs* (*outer-sum* (*length Vs*) [*u*]) *rho*
= *kraus-combo d n Vs u* ∗ *rho* ∗ ((*kraus-combo d n Vs u*)†)
⟨*proof*⟩

Vanishing of the quadratic map when the coefficient matrix is zero.

lemma *kraus-quad-map-zero-Tm*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
shows *kraus-quad-map d Vs* (0_m (*length Vs*) (*length Vs*)) *rho* = 0_m *d d*
⟨*proof*⟩

Sum-decomposition: a PSD-style coefficient matrix *outer-sum k us* gives *kraus-quad-map* as a kraus_map_dim of the combos.

lemma *kraus-quad-map-outer-sum-eq-kraus-map-dim*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *us-dim*: $\forall\, v \in$ *set us*. *dim-vec v* = *length Vs*
shows *kraus-quad-map d Vs* (*outer-sum* (*length Vs*) *us*) *rho*
= *kraus-map-dim d* (*map* (*kraus-combo d n Vs*) *us*) *rho*
⟨*proof*⟩

43.7 Lemma A: PSD coefficient matrix gives a CP perturbation

Since *amplification* only evaluates Φ on $n \times n$ blocks, two maps that agree on *carrier-mat n n* yield the same amplification, hence the same notion of complete positivity.

lemma *amplification-cong*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Xm* :: *complex Matrix.mat*
assumes *eq*: $\bigwedge r.\ r \in$ *carrier-mat n n* ⟹ *Phi r* = *Psi r*

shows *amplification n d p Phi Xm = amplification n d p Psi Xm*
⟨*proof*⟩

lemma *completely-positive-cong*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes ⋀*r*. *r* ∈ *carrier-mat n n* ⟹ *Phi r* = *Psi r*
shows *completely-positive n d Phi* ⟷ *completely-positive n d Psi*
⟨*proof*⟩

Lemma A: if T is PSD then *kraus-quad-map d Vs T* is completely positive.

Proof sketch. Use ⟦*?P* ∈ *carrier-mat ?n ?n*; *?P* ≥$_M$ *0*⟧ ⟹ ∃ *vs*. *length vs* = *?n* ∧ (∀ *v*∈*set vs*. *dim-vec v* = *?n*) ∧ *outer-sum ?n vs* = *?P* to write T = *outer-sum k us*. Then by ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *?rho* ∈ *carrier-mat ?n ?n*; ∀ *v*∈*set ?us*. *dim-vec v* = *length ?Vs*⟧ ⟹ *kraus-quad-map ?d ?Vs* (*outer-sum* (*length ?Vs*) *?us*) *?rho* = *kraus-map-dim ?d* (*map* (*kraus-combo ?d ?n ?Vs*) *?us*) *?rho*, *kraus-quad-map d Vs T* agrees on *carrier-mat n n* with *kraus-map-dim d Ws*, where *Ws* := *map* (*kraus-combo d n Vs*) *us*. Apply ⟦∀ *W*∈*set ?Ws*. *W* ∈ *carrier-mat ?d ?n*; *0* < *?n*; *0* < *?d*⟧ ⟹ *completely-positive ?n ?d* (*kraus-map-dim ?d ?Ws*).

lemma *kraus-perturbation-cp*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *Tm-c*: *Tm* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *Tm-psd*: *psd Tm*
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *completely-positive n d* (*kraus-quad-map d Vs Tm*)
⟨*proof*⟩

43.8 Small-ε PSD perturbation of the identity

For any hermitian $A : k \times k$, the matrix $1_m\ k + \delta \cdot_m A$ is PSD whenever $|\delta|$ is small enough. The bound uses the entry-wise sum $S := \sum_{\{i,j\}} |A_{\{ij\}}|$, taking $\varepsilon := 1/(1 + S)$; this avoids any operator-norm computation and just uses Cauchy-Schwarz on entries.

Bilinearity of the inner product in the right slot.

lemma *inner-prod-add-right*:
fixes *u v w* :: *complex Matrix.vec*
assumes *ud*: *dim-vec u* = *k* **and** *vd*: *dim-vec v* = *k* **and** *wd*: *dim-vec w* = *k*
shows ⟨*u* | *v* + *w*⟩ = ⟨*u* | *v*⟩ + ⟨*u* | *w*⟩
⟨*proof*⟩

lemma *inner-prod-smult-right*:
fixes *u v* :: *complex Matrix.vec*
assumes *ud*: *dim-vec u* = *k* **and** *vd*: *dim-vec v* = *k*
shows ⟨*u* | *c* ·$_v$ *v*⟩ = *c* ∗ ⟨*u* | *v*⟩
⟨*proof*⟩

The hermitian sum.

Entry-wise bound on the quadratic form of a hermitian matrix.

lemma *quad-form-bound-by-entry-sum*:
fixes *A* :: *complex Matrix.mat* **and** *x* :: *complex Matrix.vec*
assumes *A-c*: *A* ∈ *carrier-mat k k* **and** *x-d*: *dim-vec x* = *k*
shows *cmod* (⟨*x* | *A* ∗$_v$ *x*⟩)
≤ (∑ *i* = *0*..<*k*. ∑ *j* = *0*..<*k*. *cmod* (*A* $$ (*i*,*j*))) ∗ *Re* ⟨*x* | *x*⟩
⟨*proof*⟩

The main lemma: small ε keeps the perturbation PSD.

lemma *small-eps-psd*:

fixes A :: *complex Matrix.mat*
assumes *A-c*: $A \in$ *carrier-mat k k* **and** *herm*: *hermitian A*
shows $\exists \varepsilon > 0.\ \forall d ::$ *real*. $|d| \le \varepsilon \longrightarrow$
psd (1_m *k* + *complex-of-real d* $\cdot_m$ *A*)
⟨*proof*⟩

43.9 Symmetric / anti-hermitian split of the coefficient matrix

The *dual* linear combination *kraus-pair-sum n Vs A* = $\sum_{\{i,j\}}$ $A_{\{ij\}}$ $V_j{}^\dagger$ V_i is an $n \times n$ matrix. The dependence assumption ¬ *mat-lin-indep-real n* (*kraus-products Vs*) says that some real-coefficient A makes this sum vanish.

definition *kraus-pair-row* ::
nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒
nat ⇒ *complex Matrix.mat*
where
kraus-pair-row n Vs Amat i ≡
foldr (λ*j acc*. *Amat* $$ (*i*, *j*) $\cdot_m$ ((*Vs*!*j*)† ∗ *Vs*!*i*) + *acc*)
[*0*..<*length Vs*] (*0*$_m$ *n n*)

definition *kraus-pair-sum* ::
nat ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat* ⇒
complex Matrix.mat
where
kraus-pair-sum n Vs Amat ≡
foldr (λ*i acc*. *kraus-pair-row n Vs Amat i* + *acc*)
[*0*..<*length Vs*] (*0*$_m$ *n n*)

lemma *kraus-pair-row-carrier*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *i-lt*: *i* < *length Vs*
shows *kraus-pair-row n Vs Amat i* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *kraus-pair-sum-carrier*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
shows *kraus-pair-sum n Vs Amat* ∈ *carrier-mat n n*
⟨*proof*⟩

Per-entry index for the dual sum.

lemma *kraus-pair-row-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *i-lt*: *i* < *length Vs*
and *r-lt*: *r* < *n* **and** *c-lt*: *c* < *n*
shows *kraus-pair-row n Vs Amat i* $$ (*r*, *c*)
= ($\sum$ *j* = *0*..<*length Vs*.
Amat $$ (*i*, *j*) ∗ ((*Vs*!*j*)† ∗ *Vs*!*i*) $$ (*r*, *c*))
⟨*proof*⟩

lemma *kraus-pair-sum-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *r-lt*: *r* < *n* **and** *c-lt*: *c* < *n*
shows *kraus-pair-sum n Vs Amat* $$ (*r*, *c*)
= ($\sum$ *i* = *0*..<*length Vs*. $\sum$ *j* = *0*..<*length Vs*.
Amat $$ (*i*, *j*) ∗ ((*Vs*!*j*)† ∗ *Vs*!*i*) $$ (*r*, *c*))
⟨*proof*⟩

Linearity in the coefficient matrix.

lemma *kraus-pair-sum-add-A*:
  assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
    and *A1-c*: $A1 \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
    and *A2-c*: $A2 \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
  shows *kraus-pair-sum n Vs (A1 + A2)*
      $= kraus\text{-}pair\text{-}sum\ n\ Vs\ A1 + kraus\text{-}pair\text{-}sum\ n\ Vs\ A2$
⟨*proof*⟩

lemma *kraus-pair-sum-smult-A*:
  assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
    and *Amat-c*: $Amat \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
  shows *kraus-pair-sum n Vs (c ·m Amat)*
      $= c \cdot_m kraus\text{-}pair\text{-}sum\ n\ Vs\ Amat$
⟨*proof*⟩

Dagger identity: taking the dagger of *kraus-pair-sum n Vs A* swaps and conjugates the coefficient indices.

lemma *kraus-pair-sum-dagger*:
  assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
    and *Amat-c*: $Amat \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
  shows $(kraus\text{-}pair\text{-}sum\ n\ Vs\ Amat)^\dagger$
      $= kraus\text{-}pair\text{-}sum\ n\ Vs$
        $(Matrix.mat\ (length\ Vs)\ (length\ Vs)$
          $(\lambda(i,j).\ cnj\ (Amat\ \$\$\ (j,\ i))))$
⟨*proof*⟩

Existence of a hermitian witness: if *A-orig* (with real entries) gives *kraus-pair-sum n Vs A-orig = 0* and *A-orig ≠ 0*, then either the symmetric or the anti-hermitian part yields a non-zero hermitian matrix with the same property.

lemma *sym-or-antisym-witness*:
  fixes *A-orig* :: *complex Matrix.mat*
  assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
    and *A-c*: $A\text{-}orig \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
    and *A-real*: $\bigwedge i\ j.\ i < length\ Vs \Longrightarrow j < length\ Vs \Longrightarrow$
        $cnj\ (A\text{-}orig\ \$\$\ (i,\ j)) = A\text{-}orig\ \$\$\ (i,\ j)$
    and *A-zero*: $kraus\text{-}pair\text{-}sum\ n\ Vs\ A\text{-}orig = 0_m\ n\ n$
    and *A-nontriv*: $A\text{-}orig \neq 0_m\ (length\ Vs)\ (length\ Vs)$
  shows $\exists\, H.\ H \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
      $\wedge\ hermitian\ H$
      $\wedge\ H \neq 0_m\ (length\ Vs)\ (length\ Vs)$
      $\wedge\ kraus\text{-}pair\text{-}sum\ n\ Vs\ H = 0_m\ n\ n$
⟨*proof*⟩

43.10 Bridge: ¬ *mat-lin-indep-real* gives a real *A-orig* witness

Generic concat-nth for uniform-length rows.

lemma *concat-uniform-length*:
  assumes $\forall\, xs \in set\ L.\ length\ xs = m$
  shows $length\ (concat\ L) = length\ L * m$
  ⟨*proof*⟩

lemma *concat-uniform-nth*:
  assumes *uniform*: $\forall\, xs \in set\ L.\ length\ xs = m$
    and *i-lt*: $i < length\ L$
    and *j-lt*: $j < m$
  shows $concat\ L\ !\ (i * m + j) = L\ !\ i\ !\ j$
  ⟨*proof*⟩

kraus-products Vs has length k^2 with *kraus-products Vs* ! $(i{*}k + j) = (V_j)^\dagger\ V_i$.

lemma *length-kraus-products*:
  length (*kraus-products Vs*) = *length Vs* ∗ *length Vs*
⟨*proof*⟩

lemma *kraus-products-nth*:
  assumes *i-lt*: *i* < *length Vs* **and** *j-lt*: *j* < *length Vs*
  shows *kraus-products Vs* ! (*i* ∗ *length Vs* + *j*) = (*Vs*!*j*)† ∗ *Vs*!*i*
⟨*proof*⟩

Per-entry of the foldr that mat_lin_indep_real builds.

lemma *foldr-smult-nth-index*:
  fixes *c* :: *nat* ⇒ *real* **and** *Bs* :: *complex Matrix.mat list*
  assumes *Bs-c*: ∀ *B* ∈ *set Bs*. *B* ∈ *carrier-mat n n*
    and *r-lt*: *r* < *n* **and** *c-lt*: *c1* < *n*
  shows *foldr* (λ*k acc*. *complex-of-real* (*c k*) ·$_m$ (*Bs* ! *k*) + *acc*)
        [*0*..<*length Bs*] (*0*$_m$ *n n*) $$ (*r*, *c1*)
      = (∑ *k* = *0*..<*length Bs*.
        complex-of-real (*c k*) ∗ (*Bs* ! *k*) $$ (*r*, *c1*))
⟨*proof*⟩

Reindex $\sum_{\{\kappa = 0..<n{\cdot}k\}}$ via $\kappa = i{*}k + j$.

lemma *sum-split-prod-atLeastLessThan*:
  fixes *f* :: *nat* ⇒ 'a::*comm-monoid-add*
  shows (∑ κ = *0*..<*N* ∗ *k*. *f* κ) = (∑ *i* = *0*..<*N*. ∑ *j* = *0*..<*k*. *f* (*i* ∗ *k* + *j*))
⟨*proof*⟩

The main bridge: a non-trivial real dependence yields a real, non-zero matrix *A-orig* for which *kraus-pair-sum* vanishes.

lemma *mat-lin-indep-real-neg-witness*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *npos*: *0* < *n*
    and *dep*: ¬ *mat-lin-indep-real n* (*kraus-products Vs*)
  shows ∃ *A-orig* :: *complex Matrix.mat*.
      A-orig ∈ *carrier-mat* (*length Vs*) (*length Vs*)
    ∧ (∀ *i j*. *i* < *length Vs* ⟶ *j* < *length Vs* ⟶
        cnj (*A-orig* $$ (*i*, *j*)) = *A-orig* $$ (*i*, *j*))
    ∧ *A-orig* ≠ *0*$_m$ (*length Vs*) (*length Vs*)
    ∧ *kraus-pair-sum n Vs A-orig* = *0*$_m$ *n n*
⟨*proof*⟩

The same with complex coefficients: a non-trivial complex relation among the products $V_j^\dagger V_i$ gives a non-zero complex coefficient matrix with vanishing pair sum, and then (closure of the family under adjoints) a non-zero *Hermitian* one.

lemma *mat-lin-indep-complex-neg-witness*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *npos*: *0* < *n*
    and *dep*: ¬ *mat-lin-indep-complex n* (*kraus-products Vs*)
  shows ∃ *A-orig* :: *complex Matrix.mat*.
      A-orig ∈ *carrier-mat* (*length Vs*) (*length Vs*)
    ∧ *A-orig* ≠ *0*$_m$ (*length Vs*) (*length Vs*)
    ∧ *kraus-pair-sum n Vs A-orig* = *0*$_m$ *n n*
⟨*proof*⟩

lemma *herm-witness-of-complex*:
  fixes *A-orig* :: *complex Matrix.mat*

assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *A-c*: *A-orig* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
and *A-zero*: *kraus-pair-sum n Vs A-orig* $= 0_m$ *n n*
and *A-nontriv*: *A-orig* $\neq 0_m$ (*length Vs*) (*length Vs*)
shows $\exists\, H$. $H \in$ *carrier-mat* (*length Vs*) (*length Vs*)
$\wedge$ *hermitian H*
$\wedge$ $H \neq 0_m$ (*length Vs*) (*length Vs*)
$\wedge$ *kraus-pair-sum n Vs H* $= 0_m$ *n n*
⟨*proof*⟩

43.11 Auxiliary lemmas for the assembly

Scalar-multiplication linearity of *kraus-quad-map* in T.

lemma *kraus-quad-map-smult-Tm*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
and *Tm-c*: *Tm* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
shows *kraus-quad-map d Vs* (*c* $\cdot_m$ *Tm*) *rho*
$=$ *c* $\cdot_m$ *kraus-quad-map d Vs Tm rho*
⟨*proof*⟩

The identity-coefficient case: *kraus-quad-map* with 1_m *k* reduces to the ordinary *kraus-map-dim*.

lemma *kraus-quad-map-one-eq-kraus-map-dim*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
shows *kraus-quad-map d Vs* (1_m (*length Vs*)) *rho* $=$ *kraus-map-dim d Vs rho*
⟨*proof*⟩

The pair-sum at the identity coefficient is exactly *sum-wdag-w*.

lemma *kraus-pair-sum-one-eq-sum-wdag-w*:
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
shows *kraus-pair-sum n Vs* (1_m (*length Vs*)) $=$ *sum-wdag-w n Vs*
⟨*proof*⟩

43.12 Minimality and the distinctness lemma (Lemma B)

Linear independence of a complex-matrix list (over $\mathbb{C}$). This is the *minimality* assumption needed to make the reverse direction of Choi's Theorem 3 go through: without it, perturbations of a non-minimal Kraus list can produce the same map.

definition *lin-indep-complex-mat* ::
nat $\Rightarrow$ *nat* $\Rightarrow$ *complex Matrix.mat list* $\Rightarrow$ *bool*
where
lin-indep-complex-mat d n Ws $\equiv$
$(\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n*$) \wedge$
$(\forall\, c ::$ *nat* $\Rightarrow$ *complex*.
foldr $(\lambda k$ *acc*. *c k* $\cdot_m$ (*Ws*!*k*) $+$ *acc*$)$ [*0..<length Ws*] (0_m *d n*)
$=$ 0_m *d n*
$\longrightarrow (\forall\, k <$ *length Ws*. *c k* $= 0))$

Lemma B: under minimality of *Vs*, a non-zero coefficient matrix gives a non-vanishing quadratic Kraus map for some ϱ.

The witness $\varrho :=$ *mat-unit p q n n* (a single 1 at (p,q)) makes (V_i ϱ $V_j{}^\dagger$) \$\$ $(r,c) = V_i$ \$\$ (r,p) $|V_j$ \$\$ $(c,q)|$, so universal vanishing of *kraus-quad-map* on all (p,q,r,c) combined with linear independence of *Vs* (applied twice, separated by conjugation) forces $A = 0$.

A convenient form of minimality: if the per-entry sum vanishes for all (r,p), then all coefficients are zero.

lemma *lin-indep-zero-combo*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *minimal*: *lin-indep-complex-mat d n Vs*
and *coeff-kills*: $\bigwedge r\ p.\ r < d \Longrightarrow p < n \Longrightarrow$
$(\sum i = 0..<length\ Vs.\ c\ i * (Vs!i)\ \$\$\ (r,\ p)) = 0$
shows $\forall\, i < length\ Vs.\ c\ i = 0$
⟨*proof*⟩

The per-entry computation for the rank-1 $\varrho = mat\text{-}unit\ p\ q\ n\ n$.

lemma *kraus-quad-map-mat-unit-index*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *i-lt*: $i < length\ Vs$ **and** *j-lt*: $j < length\ Vs$
and *p-lt*: $p < n$ **and** *q-lt*: $q < n$
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows $(Vs!i * mat\text{-}unit\ p\ q\ n\ n * ((Vs!j)^\dagger))\ \$\$\ (r,\ c)$
$= (Vs!i)\ \$\$\ (r,\ p) * cnj\ ((Vs!j)\ \$\$\ (c,\ q))$
⟨*proof*⟩

Lemma B. Under minimality, a non-zero coefficient matrix gives a non-vanishing quadratic Kraus map for some rank-1 ϱ.

lemma *kraus-quad-map-nonzero-witness*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *minimal*: *lin-indep-complex-mat d n Vs*
and *A-c*: $A \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
and *A-nz*: $A \neq 0_m\ (length\ Vs)\ (length\ Vs)$
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows $\exists\, rho \in carrier\text{-}mat\ n\ n.\ kraus\text{-}quad\text{-}map\ d\ Vs\ A\ rho \neq 0_m\ d\ d$
⟨*proof*⟩

Trace-preservation lemma for *kraus-quad-map*.

The proof shows $Tr\ (kraus\text{-}quad\text{-}map\ d\ Vs\ T\ \varrho) = Tr\ (kraus\text{-}pair\text{-}sum\ n\ Vs\ T * \varrho)$ by per-entry sum manipulation and one application of $[\![dim\text{-}col\ ?A = dim\text{-}row\ ?B;\ dim\text{-}row\ ?A = dim\text{-}col\ ?B]\!] \Longrightarrow Tr\ (?A * ?B) = Tr\ (?B * ?A)$ to flip $V_i\ \varrho\ {V_j}^\dagger$ into ${V_j}^\dagger\ V_i\ \varrho$; the TP conclusion follows when the pair-sum equals $1_m\ n$.

lemma *Tr-kraus-quad-map-eq*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *Tmat-c*: $Tmat \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
and *rho-c*: $rho \in carrier\text{-}mat\ n\ n$
shows $Tr\ (kraus\text{-}quad\text{-}map\ d\ Vs\ Tmat\ rho)$
$= Tr\ (kraus\text{-}pair\text{-}sum\ n\ Vs\ Tmat * rho)$
⟨*proof*⟩

lemma *kraus-quad-map-is-TP*:
assumes *Vs-c*: $\forall\, V \in set\ Vs.\ V \in carrier\text{-}mat\ d\ n$
and *Tmat-c*: $Tmat \in carrier\text{-}mat\ (length\ Vs)\ (length\ Vs)$
and *Tmat-pair-sum*: $kraus\text{-}pair\text{-}sum\ n\ Vs\ Tmat = 1_m\ n$
and *npos*: $0 < n$
shows *trace-preserving n* (*kraus-quad-map d Vs Tmat*)
⟨*proof*⟩

43.13 CPTP-Kraus representation and extreme CPTP maps

A *cptp-kraus-rep n d* Φ *Vs* says *Vs* is a Kraus operator list (trace-preserving) and Φ equals the corresponding *kraus-map-dim* on every $X \in carrier\text{-}mat\ n\ n$.

definition *cptp-kraus-rep* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒
      complex Matrix.mat list ⇒ *bool*
where
  cptp-kraus-rep n d Φ *Vs* ≡
    kraus-operators n d Vs ∧
    (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ *X* = *kraus-map-dim d Vs X*)

Extreme points of $CPTP(n,d)$: analog of *extreme-UCP*.

definition *extreme-CPTP* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
  extreme-CPTP n d Φ ≡
    Φ ∈ *CPTP n d* ∧
    ¬ (∃ Φ₁ ∈ *CPTP n d*. ∃ Φ₂ ∈ *CPTP n d*.
        Φ₁ ≠ Φ ∧ Φ ≠ Φ₂ ∧
        (∀ *X*. *X* ∈ *carrier-mat n n*
            ⟶ Φ *X* = (*1/2* :: *complex*) ·ₘ Φ₁ *X*
                + (*1/2* :: *complex*) ·ₘ Φ₂ *X*))

43.14 Vectorisation of Kraus operators

The *vectorisation* of a $d \times n$ matrix V is the length-$n{*}d$ vector with r-th entry $V(r \bmod d,\ r \operatorname{div} d)$. This is the column-stacking isomorphism $M_{\{d\times n\}} \cong \mathbb{C}^{\{nd\}}$ used in the Choi/Stinespring picture.

definition *vec-of-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.vec*
  where
  vec-of-kraus d n V = *Matrix.vec* (*n*d*) (λ*r*. *V* $$ (*r mod d*, *r div d*))

lemma *vec-of-kraus-dim* [*simp*]: *dim-vec* (*vec-of-kraus d n V*) = *n*d*
  ⟨*proof*⟩

lemma *vec-of-kraus-index*:
  assumes *r-lt*: *r* < *n*d*
  shows *vec-of-kraus d n V* $ *r* = *V* $$ (*r mod d*, *r div d*)
  ⟨*proof*⟩

For $i < d$ and $a < n$, indexing the vectorisation at the *combined* index $a{*}d + i$ recovers the matrix entry $V(i,\ a)$.

lemma *vec-of-kraus-combined-index*:
  assumes *V-c*: *V* ∈ *carrier-mat d n*
      and *a-lt*: *a* < *n* **and** *i-lt*: *i* < *d* **and** *dpos*: *0* < *d*
  shows *vec-of-kraus d n V* $ (*a*d* + *i*) = *V* $$ (*i*, *a*)
⟨*proof*⟩

43.15 Choi matrix of a Kraus map

Key structural identity: the Choi matrix of a Kraus map $\Phi(X) = \sum_i V_i\ X\ {V_i}^\dagger$ equals the outer sum $\sum_i v_i\ {v_i}^\dagger$ where $v_i = \textit{vec-of-kraus}(V_i)$. In particular, its range is the linear span of the vectorisations of *Vs*.

lemma *kraus-map-dim-mat-unit-entry*:
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
      and *a-lt*: *a* < *n* **and** *b-lt*: *b* < *n*
      and *i-lt*: *i* < *d* **and** *j-lt*: *j* < *d*
  shows (*kraus-map-dim d Vs* (*mat-unit a b n n*)) $$ (*i*, *j*)
        = (∑ *k*<*length Vs*. *Vs*!*k* $$ (*i*, *a*) * *cnj* (*Vs*!*k* $$ (*j*, *b*)))

⟨*proof*⟩

lemma *choi-matrix-kraus-eq*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-matrix n d* (*kraus-map-dim d Vs*)
= *outer-sum* (*n*∗*d*) (*map* (*vec-of-kraus d n*) *Vs*)
⟨*proof*⟩

43.16 Choi matrix is linear in the map

The Choi matrix is a linear function of the underlying map (evaluated on the basis *mat-unit*'s). In particular, $P_{\{}\Phi - \Psi\} = P_\Phi - P_\Psi$.

lemma *choi-matrix-carrier* [*simp*]:
choi-matrix n d Φ ∈ *carrier-mat* (*n*∗*d*) (*n*∗*d*)
⟨*proof*⟩

lemma *choi-matrix-diff*:
fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *Phi-range*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* ∈ *carrier-mat d d*
and *Psi-range*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Psi X* ∈ *carrier-mat d d*
and *npos*: *0* < *n* **and** *dpos*: *0* < *d*
shows *choi-matrix n d* (λ*X*. *Phi X* − *Psi X*)
= *choi-matrix n d Phi* − *choi-matrix n d Psi*
⟨*proof*⟩

The Choi matrix agrees on maps that agree on the carrier.

lemma *choi-matrix-cong*:
assumes *agree*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* = *Psi X*
shows *choi-matrix n d Phi* = *choi-matrix n d Psi*
⟨*proof*⟩

43.17 The Choi matrix of a *kraus-quad-map*

The Choi matrix of $X \mapsto \sum_{\{} ij\} \ T_{\{} ij\} \ V_i \ X \ {V_j}^\dagger$ equals *V T* $V^\dagger$ where *V* = *mat-of-cols* (*n*∗*d*) (*map vec-of-kraus Vs*).

This is the bridge from the classical dominated-PSD parametrisation back to a Kraus quadratic form.

definition *kraus-vec-mat* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* ⇒ *complex Matrix.mat*
where *kraus-vec-mat d n Vs* = *mat-of-cols* (*n*∗*d*) (*map* (*vec-of-kraus d n*) *Vs*)

lemma *kraus-vec-mat-carrier* [*simp*]:
kraus-vec-mat d n Vs ∈ *carrier-mat* (*n*∗*d*) (*length Vs*)
⟨*proof*⟩

lemma *kraus-vec-mat-index*:
assumes *r-lt*: *r* < *n*∗*d* **and** *k-lt*: *k* < *length Vs*
shows *kraus-vec-mat d n Vs* $$ (*r*, *k*) = *vec-of-kraus d n* (*Vs*!*k*) $ *r*
⟨*proof*⟩

lemma *kraus-vec-mat-dagger-index*:
assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
and *k-lt*: *k* < *length Vs* **and** *c-lt*: *c* < *n*∗*d*
shows (*kraus-vec-mat d n Vs*)† $$ (*k*, *c*) = *cnj* (*vec-of-kraus d n* (*Vs*!*k*) $ *c*)
⟨*proof*⟩

The Choi matrix of *kraus-quad-map d Vs T* is the sandwich $V\ T\ V^\dagger$, where V = *kraus-vec-mat d n Vs*.

lemma *choi-matrix-kraus-quad-eq*:
fixes *Tmat* :: *complex Matrix.mat*
assumes *Vs-c*: $\forall\, V \in$ *set Vs*. $V \in$ *carrier-mat d n*
and *T-c*: *Tmat* $\in$ *carrier-mat* (*length Vs*) (*length Vs*)
and *npos*: $0 < n$ **and** *dpos*: $0 < d$
shows *choi-matrix n d* (*kraus-quad-map d Vs Tmat*)
= (*kraus-vec-mat d n Vs*) $*$ *Tmat* $*$ ((*kraus-vec-mat d n Vs*)†)
⟨*proof*⟩

43.18 CP Radon–Nikodym — the dominated-PSD kernel

Dominated PSD parametrisation. This is the genuinely classical fact behind CP Radon–Nikodym: any PSD matrix B dominated by the outer sum $\sum\ v_i\ {v_i}^\dagger$ can be sandwiched as $V\ T\ V^\dagger$ for some PSD T, where V = *mat-of-cols k vs*.

Geometric content: *range*(B) $\subseteq$ *range*(*outer-sum k vs*) (equivalent to B being Loewner-below outer_sum k vs after PSD), and that range is spanned by the columns of V. Pulling B back through that span gives T.

Proved below as *dominated-outer-sum-param* through the Moore–Penrose pseudoinverse of the outer sum (no axioms).

Loewner–kernel inclusion. If $A \geq B \geq 0$ in Loewner order, then every vector annihilated by A is also annihilated by B. Used to derive the range inclusion *range*(B) $\subseteq$ *range*(A).

Proof structure:

1. $\langle x \mid A\ x\rangle = 0$ from $A\ x = 0$.
2. Linearity of the inner product: $\langle x \mid A\ x\rangle = \langle x \mid B\ x\rangle + \langle x \mid (A-B)\ x\rangle$.
3. Both summands have non-negative real part (PSD); the sum is 0, so each is 0.
4. Hermiticity of B makes $\langle x \mid B\ x\rangle$ real, so it equals 0.
5. Decompose $B = N\ N^\dagger$ via ⟦*?P* $\in$ *carrier-mat ?n ?n*; *?P* $\geq_M 0$⟧ $\Longrightarrow$ $\exists$ *vs*. *length vs* = *?n* $\wedge$ ($\forall\, v \in$ *set vs*. *dim-vec v* = *?n*) $\wedge$ *outer-sum ?n vs* = *?P*, then $\langle x \mid B\ x\rangle = \langle N^\dagger\ x \mid N^\dagger\ x\rangle = 0$ forces $N^\dagger\ x = 0$, hence $B\ x = N\ (N^\dagger\ x) = 0$.

lemma *loewner-ker-subset*:
fixes $A\ B$ :: *complex Matrix.mat* **and** x :: *complex Matrix.vec*
assumes *A-c*: $A \in$ *carrier-mat k k*
and *B-c*: $B \in$ *carrier-mat k k*
and *B-psd*: *psd B*
and *diff-psd*: *psd* $(A - B)$
and *x-dim*: *dim-vec x* = *k*
and *Ax-zero*: $A *_v x = 0_v\ k$
shows $B *_v x = 0_v\ k$
⟨*proof*⟩

PSD congruence. If P is PSD and M is any matrix of compatible shape, then $M^\dagger\ P\ M$ is PSD. This is the standard "sandwich" fact, generalising *?M* $\in$ *carrier-mat ?n ?m* $\Longrightarrow$ *?M*† $*$ *?M* $\geq_M 0$ (the case $P = 1_m$).

lemma *psd-conjugate*:
fixes P :: *complex Matrix.mat* **and** M :: *complex Matrix.mat*
assumes *P-c*: $P \in$ *carrier-mat k k*

and *P-psd*: *psd P*
and *M-c*: $M \in$ *carrier-mat k n*
shows *psd* $(M^\dagger * P * M)$
⟨*proof*⟩

43.18.1 Diagonal pseudo-inverse list and its identity

Pseudo-inverse of a real list (interpreted as a diagonal). Zero entries are kept at zero; non-zero entries are reciprocated. This is the diagonal Moore–Penrose pseudoinverse.

definition *pinv-list* :: *real list* ⇒ *real list* **where**
pinv-list D = *map* (λ*d*. *if d* = *0 then 0 else 1*/*d*) *D*

lemma *pinv-list-length* [*simp*]: *length* (*pinv-list D*) = *length D*
⟨*proof*⟩

lemma *pinv-list-index*:
assumes *i* < *length D*
shows *pinv-list D* ! *i* = (*if D* ! *i* = *0 then 0 else 1* / (*D* ! *i*))
⟨*proof*⟩

lemma *pinv-list-nonneg*:
assumes ⋀*i*. *i* < *length D* ⟹ *0* ≤ *D* ! *i*
shows ⋀*i*. *i* < *length D* ⟹ *0* ≤ *pinv-list D* ! *i*
⟨*proof*⟩

The central identity: $D \cdot D^+ \cdot D = D$ entrywise on the diagonal. For $D_i = 0$ the left-hand side is $0 \cdot 0 \cdot 0 = 0 = D_i$; for $D_i \neq 0$ it is $D_i \cdot (1/D_i) \cdot D_i = D_i$. This is the core diagonal pseudoinverse identity that lifts to operator level through unitary conjugation.

lemma *diag-pinv-diag-eq*:
fixes *D* :: *real list*
shows *diag-mat-of-real D*
∗ *diag-mat-of-real* (*pinv-list D*)
∗ *diag-mat-of-real D*
= *diag-mat-of-real D*
⟨*proof*⟩

Pseudoinverse identity at operator level. Given a hermitian spectral decomposition $A = U\ D\ U^\dagger$, define the Moore–Penrose pseudoinverse via $A^+ = U\ D^+\ U^\dagger$. Then $A\ A^+\ A = A$. This is the unitary-conjugation lift of *diag-mat-of-real ?D* ∗ *diag-mat-of-real* (*pinv-list ?D*) ∗ *diag-mat-of-real ?D* = *diag-mat-of-real ?D*.

lemma *A-Aplus-A-eq*:
fixes *U* :: *complex Matrix.mat* **and** *D* :: *real list*
assumes *Uc*: *U* ∈ *carrier-mat k k*
and *UU*: *unitary U*
and *Dlen*: *length D* = *k*
shows (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real* (*pinv-list D*) ∗ (*U*†))
∗ (*U* ∗ *diag-mat-of-real D* ∗ (*U*†))
= *U* ∗ *diag-mat-of-real D* ∗ (*U*†)
⟨*proof*⟩

Two square matrices agreeing on every vector are equal.

lemma *mat-eq-of-mult-vec-eq*:
fixes *M N* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat k k*
and *N-c*: *N* ∈ *carrier-mat k k*

and *eq*: $\bigwedge x.\ x \in$ *carrier-vec k* $\Longrightarrow M *_v x = N *_v x$
shows $M = N$
⟨*proof*⟩

Loewner-implied pseudoinverse identity, proved at operator level: with $Q = A^+ A$ one has $A\ Q = A$, so $x - Q\ x \in ker\ A \subseteq ker\ B$, whence $B = B\ Q$; taking adjoints gives $B = A\ A^+\ B$.

lemma *A-Aplus-B-eq-B*:
fixes U :: *complex Matrix.mat* **and** D :: *real list* **and** B :: *complex Matrix.mat*
assumes *Uc*: $U \in$ *carrier-mat k k*
and *UU*: *unitary U*
and *Dlen*: *length D* $= k$
and *Dnn*: $\bigwedge i.\ i < k \Longrightarrow 0 \le D\ !\ i$
and *B-c*: $B \in$ *carrier-mat k k*
and *B-psd*: *psd B*
and *dom-psd*: *psd* $((U * diag\text{-}mat\text{-}of\text{-}real\ D * (U^\dagger)) - B)$
shows $(U * diag\text{-}mat\text{-}of\text{-}real\ D * (U^\dagger))$
$* (U * diag\text{-}mat\text{-}of\text{-}real\ (pinv\text{-}list\ D) * (U^\dagger)) * B = B$
⟨*proof*⟩

Spectral decomposition of a psd matrix with non-negative diagonal.

Dominated psd matrices are parametrised through the dominating frame: if $0 \le B \le V\ V^\dagger$ then $B = V\ T\ V^\dagger$ with $T = V^\dagger\ A^+\ B\ A^+\ V \ge 0$, $A = V\ V^\dagger$, A^+ the Moore–Penrose pseudoinverse.

lemma *dominated-outer-sum-param*:
fixes *vs* :: *complex Matrix.vec list*
and B :: *complex Matrix.mat*
assumes *vs-dim*: $\forall v \in$ *set vs. dim-vec v* $= k$
and *k-pos*: $0 < k$
and *B-c*: $B \in$ *carrier-mat k k*
and *B-psd*: *psd B*
and *B-dom*: *psd* $(outer\text{-}sum\ k\ vs - B)$
shows $\exists T.\ T \in carrier\text{-}mat\ (length\ vs)\ (length\ vs) \wedge psd\ T \wedge$
$B = (mat\text{-}of\text{-}cols\ k\ vs) * T * ((mat\text{-}of\text{-}cols\ k\ vs)^\dagger)$
⟨*proof*⟩

43.19 Linearity of *kraus-quad-map* in its matrix argument

Per-entry computation shows *kraus-quad-map d Vs Tm* is linear in its third (matrix) argument: $\varrho \mapsto \sum_{\{ij\}}\ T_{\{ij\}}\ V_i\ \varrho\ {V_j}^\dagger$.

lemma *kraus-quad-map-add-rho*:
assumes *Vs-c*: $\forall V \in$ *set Vs. V* $\in$ *carrier-mat d n*
and *rho1-c*: *rho1* $\in$ *carrier-mat n n*
and *rho2-c*: *rho2* $\in$ *carrier-mat n n*
shows *kraus-quad-map d Vs Tm* $(rho1 + rho2)$
$=$ *kraus-quad-map d Vs Tm rho1 + kraus-quad-map d Vs Tm rho2*
⟨*proof*⟩

lemma *kraus-quad-map-smult-rho*:
assumes *Vs-c*: $\forall V \in$ *set Vs. V* $\in$ *carrier-mat d n*
and *rho-c*: *rho* $\in$ *carrier-mat n n*
shows *kraus-quad-map d Vs Tm* $(c \cdot_m rho)$
$=$ $c \cdot_m$ *kraus-quad-map d Vs Tm rho*
⟨*proof*⟩

43.20 Choi matrix determines a linear CP map on carrier

If two maps that are linear on carrier have the same Choi matrix and the same target carrier, they agree pointwise on carrier.

lemma *map-eq-from-choi-when-linear*:
  fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
  assumes *Phi-add*: ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
    ⟹ *Phi* (*X* + *Y*) = *Phi X* + *Phi Y*
    and *Phi-smult*: ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ *Phi* (*c* ·$_m$ *X*) = *c* ·$_m$ *Phi X*
    and *Phi-car*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* ∈ *carrier-mat d d*
    and *Psi-add*: ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
    ⟹ *Psi* (*X* + *Y*) = *Psi X* + *Psi Y*
    and *Psi-smult*: ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ *Psi* (*c* ·$_m$ *X*) = *c* ·$_m$ *Psi X*
    and *Psi-car*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Psi X* ∈ *carrier-mat d d*
    and *choi-eq*: *choi-matrix n d Phi* = *choi-matrix n d Psi*
    and *npos*: *n* > *0* **and** *dpos*: *d* > *0*
    and *A-c*: *A* ∈ *carrier-mat n n*
  shows *Phi A* = *Psi A*
⟨*proof*⟩

43.21 CP Radon–Nikodym

CP Radon–Nikodym (Choi level). At the Choi-matrix level, the parametrisation falls out from ⟦∀ *v*∈*set ?vs*. *dim-vec v* = *?k*; *0* < *?k*; *?B* ∈ *carrier-mat ?k ?k*; *?B* ≥$_M$ *0*; *outer-sum ?k ?vs* − *?B* ≥$_M$ *0*⟧ ⟹ ∃ *T*. *T* ∈ *carrier-mat* (*length ?vs*) (*length ?vs*) ∧ *T* ≥$_M$ *0* ∧ *?B* = *mat-of-cols ?k ?vs* ∗ *T* ∗ *mat-of-cols ?k ?vs*† composed with the structural identities ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *0* < *?n*; *0* < *?d*⟧ ⟹ P_Φ *?n ?d* (*kraus-map-dim ?d ?Vs*) = *outer-sum* (*?n* ∗ *?d*) (*map* (*vec-of-kraus ?d ?n*) *?Vs*) and ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *?Tmat* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*); *0* < *?n*; *0* < *?d*⟧ ⟹ P_Φ *?n ?d* (*kraus-quad-map ?d ?Vs ?Tmat*) = *kraus-vec-mat ?d ?n ?Vs* ∗ *?Tmat* ∗ *kraus-vec-mat ?d ?n ?Vs*†. This proves what is needed at the matrix level; passing to the ∀ *X*-quantified form requires linearity of Ψ, which is the project's *completely-positive* does *not* include (see the lemma after).

lemma *cp-radon-nikodym-choi*:
  fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
    and *Phi-kraus*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* = *kraus-map-dim d Vs X*
    and *Psi-cp*: *completely-positive n d Psi*
    and *Psi-range*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Psi X* ∈ *carrier-mat d d*
    and *diff-cp*: *completely-positive n d* (λ*X*. *Phi X* − *Psi X*)
    and *npos*: *0* < *n*
    and *dpos*: *0* < *d*
  shows ∃ *T*. *T* ∈ *carrier-mat* (*length Vs*) (*length Vs*) ∧ *psd T* ∧
      choi-matrix n d Psi = *choi-matrix n d* (*kraus-quad-map d Vs T*)
⟨*proof*⟩

CP Radon–Nikodym (operator level). With Ψ linear on *carrier-mat n n*, the Choi-level parametrisation gives the pointwise equality $\Psi(X) = \sum\ T_{\{ij\}}\ V_i\ X\ V_j{}^\dagger$.
Linearity of Ψ is supplied as an explicit hypothesis: the project's *completely-positive* predicate does *not* imply linearity (it constrains only positivity under amplification). This corresponds to the classical fact that "CP map" standardly includes linearity; we make it explicit so the consumer can supply it.

Operator-level CP Radon–Nikodym, with linearity assumption. This version is fully discharged via ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; ⋀*X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Phi X*

= *kraus-map-dim ?d ?Vs X*; *completely-positive ?n ?d ?Psi*; ⋀*X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Psi X* ∈ *carrier-mat ?d ?d*; *completely-positive ?n ?d* (λ*X*. *?Phi X* − *?Psi X*); *0* < *?n*; *0* < *?d*⟧ ⟹ ∃ *T*. *T* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*) ∧ *T* ≥*M* *0* ∧ *P*Φ *?n ?d ?Psi* = *P*Φ *?n ?d* (*kraus-quad-map ?d ?Vs T*) + ⟦⋀*X Y*. ⟦*X* ∈ *carrier-mat ?n ?n*; *Y* ∈ *carrier-mat ?n ?n*⟧ ⟹ *?Phi* (*X* + *Y*) = *?Phi X* + *?Phi Y*; ⋀*c X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Phi* (*c* ·*m* *X*) = *c* ·*m* *?Phi X*; ⋀*X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Phi X* ∈ *carrier-mat ?d ?d*; ⋀*X Y*. ⟦*X* ∈ *carrier-mat ?n ?n*; *Y* ∈ *carrier-mat ?n ?n*⟧ ⟹ *?Psi* (*X* + *Y*) = *?Psi X* + *?Psi Y*; ⋀*c X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Psi* (*c* ·*m* *X*) = *c* ·*m* *?Psi X*; ⋀*X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Psi X* ∈ *carrier-mat ?d ?d*; *P*Φ *?n ?d ?Phi* = *P*Φ *?n ?d ?Psi*; *0* < *?n*; *0* < *?d*; *?A* ∈ *carrier-mat ?n ?n*⟧ ⟹ *?Phi ?A* = *?Psi ?A*, which rest on ⟦∀ *v*∈*set ?vs*. *dim-vec v* = *?k*; *0* < *?k*; *?B* ∈ *carrier-mat ?k ?k*; *?B* ≥*M* *0*; *outer-sum ?k ?vs* − *?B* ≥*M* *0*⟧ ⟹ ∃ *T*. *T* ∈ *carrier-mat* (*length ?vs*) (*length ?vs*) ∧ *T* ≥*M* *0* ∧ *?B* = *mat-of-cols ?k ?vs* ∗ *T* ∗ *mat-of-cols ?k ?vs*†. It requires *Psi* to be linear on *carrier-mat n n*, which is not implied by the project's *completely-positive* predicate.

lemma *cp-radon-nikodym-linear*:
  fixes *Phi Psi* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *Vs-c*: ∀ *V* ∈ *set Vs*. *V* ∈ *carrier-mat d n*
      and *Phi-kraus*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Phi X* = *kraus-map-dim d Vs X*
      and *Psi-cp*: *completely-positive n d Psi*
      and *Psi-range*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ *Psi X* ∈ *carrier-mat d d*
      and *Psi-add*: ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
                  ⟹ *Psi* (*X* + *Y*) = *Psi X* + *Psi Y*
      and *Psi-smult*: ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ *Psi* (*c* ·*m* *X*) = *c* ·*m* *Psi X*
      and *diff-cp*: *completely-positive n d* (λ*X*. *Phi X* − *Psi X*)
      and *npos*: *0* < *n*
      and *dpos*: *0* < *d*
  shows ∃ *T*. *T* ∈ *carrier-mat* (*length Vs*) (*length Vs*) ∧ *psd T* ∧
          (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ *Psi X* = *kraus-quad-map d Vs T X*)
⟨*proof*⟩

The use site *choi-theorem-3-cptp-forward-complex* obtains linearity of its Ψ through the strengthened *cptp-pair-mid* predicate (which carries *lin-on-carrier* for both midpoint witnesses) and dispatches to ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; ⋀*X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Phi X* = *kraus-map-dim ?d ?Vs X*; *completely-positive ?n ?d ?Psi*; ⋀*X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Psi X* ∈ *carrier-mat ?d ?d*; ⋀*X Y*. ⟦*X* ∈ *carrier-mat ?n ?n*; *Y* ∈ *carrier-mat ?n ?n*⟧ ⟹ *?Psi* (*X* + *Y*) = *?Psi X* + *?Psi Y*; ⋀*c X*. *X* ∈ *carrier-mat ?n ?n* ⟹ *?Psi* (*c* ·*m* *X*) = *c* ·*m* *?Psi X*; *completely-positive ?n ?d* (λ*X*. *?Phi X* − *?Psi X*); *0* < *?n*; *0* < *?d*⟧ ⟹ ∃ *T*. *T* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*) ∧ *T* ≥*M* *0* ∧ (∀ *X*. *X* ∈ *carrier-mat ?n ?n* ⟶ *?Psi X* = *kraus-quad-map ?d ?Vs T X*). No operator-level *cp-radon-nikodym* without linearity is needed any more.

43.22 Choi's Theorem 3 — reverse direction

Reverse direction of Choi's Theorem 3 (CPTP form). Under minimality of the Kraus list *Vs*, the non-linear-independence of the products {*V*$_j$† *V*$_i$} forces Φ to fail to be an extreme point of *CPTP n d*.

The witness uses the explicit perturbation Φ±(ϱ) := ∑(*I*{*ij*} ± ε *H*{*ij*}) *V*$_i$ ϱ *V*$_j$† for a hermitian *H* (from ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *?A-orig* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*); ⋀*i j*. ⟦*i* < *length ?Vs*; *j* < *length ?Vs*⟧ ⟹ *cnj* (*?A-orig* $$ (*i*, *j*)) = *?A-orig* $$ (*i*, *j*); *kraus-pair-sum ?n ?Vs ?A-orig* = *0*$_m$ *?n ?n*; *?A-orig* ≠ *0*$_m$ (*length ?Vs*) (*length ?Vs*)⟧ ⟹ ∃ *H*. *H* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*) ∧ *hermitian H* ∧ *H* ≠ *0*$_m$ (*length ?Vs*) (*length ?Vs*) ∧ *kraus-pair-sum ?n ?Vs H* = *0*$_m$ *?n ?n*) and a small ε (from ⟦*?A* ∈ *carrier-mat*

?k ?k; *hermitian ?A*⟧ ⟹ ∃ε>0. ∀ *d*. |*d*| ≤ ε ⟶ *1*$_m$ *?k* + *complex-of-real d* ·$_m$ *?A* ≥$_M$ *0*). Distinctness uses ⟦∀ *V*∈*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *lin-indep-complex-mat ?d ?n ?Vs*; *?A* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*); *?A* ≠ *0*$_m$ (*length ?Vs*) (*length ?Vs*); *0* < *?n*; *0* < *?d*⟧ ⟹ ∃ *rho*∈*carrier-mat ?n ?n*. *kraus-quad-map ?d ?Vs ?A rho* ≠ *0*$_m$ *?d ?d* (Lemma B).

theorem *choi-theorem-3-cptp-reverse*:
  fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
    and *Vs* :: *complex Matrix.mat list*
  assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
      and Φ*-in*: Φ ∈ *CPTP n d*
      and *rep*: *cptp-kraus-rep n d* Φ *Vs*
      and *minimal*: *lin-indep-complex-mat d n Vs*
      and *dep*: ¬ *mat-lin-indep-real n* (*kraus-products Vs*)
  shows ¬ *extreme-CPTP n d* Φ
⟨*proof*⟩

Caveat on *extreme-CPTP* **(forward shadow).** Because both *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*, the raw function inequality Φ$_1$ ≠ Φ in *extreme-CPTP* is trivially satisfiable. We therefore introduce a *carrier*-point non-extremity notion below.

To keep *extreme-CPTP-carrier*'s definition manageable for automated proofs, we factor it through a helper predicate *cptp-pair-mid*: the data of a CPTP midpoint pair witnessing non-extremity (range, linearity on carrier, off-equal at some carrier-point, and the midpoint identity). Unfolding only the outer definition keeps the goal small.

definition *lin-on-carrier* ::
  nat ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
  lin-on-carrier n Φ ≡
    (∀ *X Y*. *X* ∈ *carrier-mat n n* ⟶ *Y* ∈ *carrier-mat n n*
        ⟶ Φ (*X* + *Y*) = Φ *X* + Φ *Y*) ∧
    (∀ *c X*. *X* ∈ *carrier-mat n n* ⟶ Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*)

lemma *lin-on-carrier-add*:
  assumes *lin-on-carrier n* Φ *X* ∈ *carrier-mat n n Y* ∈ *carrier-mat n n*
  shows Φ (*X* + *Y*) = Φ *X* + Φ *Y*
  ⟨*proof*⟩

lemma *lin-on-carrier-smult*:
  assumes *lin-on-carrier n* Φ *X* ∈ *carrier-mat n n*
  shows Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*
  ⟨*proof*⟩

lemma *lin-on-carrierI*:
  assumes ⋀*X Y*. *X* ∈ *carrier-mat n n* ⟹ *Y* ∈ *carrier-mat n n*
        ⟹ Φ (*X* + *Y*) = Φ *X* + Φ *Y*
    and ⋀*c X*. *X* ∈ *carrier-mat n n* ⟹ Φ (*c* ·$_m$ *X*) = *c* ·$_m$ Φ *X*
  shows *lin-on-carrier n* Φ
  ⟨*proof*⟩

definition *cptp-pair-mid* ::
  nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
    ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
    ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
  cptp-pair-mid n d Φ Φ$_1$ Φ$_2$ ≡
    Φ$_1$ ∈ *CPTP n d* ∧ Φ$_2$ ∈ *CPTP n d* ∧
    (∀ *X*. *X* ∈ *carrier-mat n n* ⟶ Φ$_1$ *X* ∈ *carrier-mat d d*) ∧

$(\forall X.\ X \in \mathit{carrier\text{-}mat}\ n\ n \longrightarrow \Phi_2\ X \in \mathit{carrier\text{-}mat}\ d\ d) \wedge$
$\mathit{lin\text{-}on\text{-}carrier}\ n\ \Phi_1 \wedge \mathit{lin\text{-}on\text{-}carrier}\ n\ \Phi_2 \wedge$
$(\exists X_1 \in \mathit{carrier\text{-}mat}\ n\ n.\ \Phi_1\ X_1 \neq \Phi\ X_1) \wedge$
$(\exists X_2 \in \mathit{carrier\text{-}mat}\ n\ n.\ \Phi\ X_2 \neq \Phi_2\ X_2) \wedge$
$(\forall X.\ X \in \mathit{carrier\text{-}mat}\ n\ n \longrightarrow$
$\Phi\ X = (1/2 :: \mathit{complex}) \cdot_m \Phi_1\ X + (1/2 :: \mathit{complex}) \cdot_m \Phi_2\ X)$

definition *extreme-CPTP-carrier* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool*
where
extreme-CPTP-carrier n d Φ ≡
Φ ∈ *CPTP n d* ∧ ¬ (∃ Φ₁ Φ₂. *cptp-pair-mid n d* Φ Φ₁ Φ₂)

The same construction as in ⟦*0 < ?n*; *0 < ?d*; *?*Φ ∈ *CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?*Φ *?Vs*; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ ⟹ ¬ *extreme-CPTP ?n ?d ?*Φ yields the stronger fact that Φ fails the *extreme-CPTP-carrier* test: the perturbed maps *Phip*, *Phim* are CPTP, have range *carrier-mat d d*, and disagree with Φ at the carrier-point *rho*.

lemma *not-extreme-CPTP-carrier-of-herm-witness*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list* **and** *H* :: *complex Matrix.mat*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *H-c*: *H* ∈ *carrier-mat* (*length Vs*) (*length Vs*)
and *H-herm*: *hermitian H*
and *H-nz*: *H* ≠ 0_m (*length Vs*) (*length Vs*)
and *H-pair*: *kraus-pair-sum n Vs H* = 0_m *n n*
shows ¬ *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

Immediate contrapositive: extreme-in-carrier-sense forces linear independence of the products.

theorem *choi-theorem-3-cptp-reverse-carrier*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *dep*: ¬ *mat-lin-indep-real n* (*kraus-products Vs*)
shows ¬ *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

theorem *choi-theorem-3-cptp-reverse-carrier-complex*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *dep*: ¬ *mat-lin-indep-complex n* (*kraus-products Vs*)
shows ¬ *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

corollary *choi-theorem-3-cptp-carrier-forward*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*

and *Vs* :: *complex Matrix.mat list*
assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
and Φ-*in*: $\Phi \in CPTP\ n\ d$
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
and *ext*: *extreme-CPTP-carrier n d* Φ
shows *mat-lin-indep-real n* (*kraus-products Vs*)
⟨*proof*⟩

43.23 Trace pairing duality and TP-converse

If *Tr* $(M * \varrho) =$ *Tr* $(N * \varrho)$ for every ϱ in carrier, then $M = N$. Used to invert $[\![\forall V{\in}$*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; *?Tmat* ∈ *carrier-mat* (*length ?Vs*) (*length ?Vs*); *kraus-pair-sum ?n ?Vs ?Tmat* = 1_m *?n*; $0 <$ *?n*$]\!] \Longrightarrow$ *trace-preserving ?n* (*kraus-quad-map ?d ?Vs ?Tmat*).

lemma *Tr-mat-unit-mult*:
fixes *M* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat n n*
and *r-lt*: $r < n$ **and** *s-lt*: $s < n$
shows *Tr* (*M* ∗ *mat-unit s r n n*) = *M* $$ (*r*, *s*)
⟨*proof*⟩

lemma *mat-trace-pairing-unique*:
fixes *M N* :: *complex Matrix.mat*
assumes *M-c*: *M* ∈ *carrier-mat n n*
and *N-c*: *N* ∈ *carrier-mat n n*
and *pair-eq*: $\bigwedge$*rho*. *rho* ∈ *carrier-mat n n* $\Longrightarrow$ *Tr* (*M* ∗ *rho*) = *Tr* (*N* ∗ *rho*)
shows *M* = *N*
⟨*proof*⟩

43.24 Choi's Theorem 3 — forward direction

Important caveat on *extreme-CPTP***.** The current definition uses raw function inequality $\Phi_1 \neq \Phi$, but both *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*. Hence for *any* CPTP Φ one can construct Φ_1 that agrees with Φ on the carrier but disagrees on (say) a *1*×*1* matrix; this Φ_1 is itself in CPTP, so the negation of *extreme-CPTP* is vacuously satisfied. Consequently *extreme-CPTP* **as currently defined is always false**, and Choi's Theorem 3 as stated (an iff with *mat-lin-indep-real*) does not hold.

The fix is to use a *carrier*-point notion of inequality (defined above as *extreme-CPTP-carrier*).

The forward direction targets the *carrier*-flavoured version: *mat-lin-indep-complex* $\Longrightarrow$ *extreme-CPTP-carrier*. Proof uses $[\![\forall V{\in}$*set ?Vs*. *V* ∈ *carrier-mat ?d ?n*; $\bigwedge$*X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Phi X* = *kraus-map-dim ?d ?Vs X*; *completely-positive ?n ?d ?Psi*; $\bigwedge$*X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi X* ∈ *carrier-mat ?d ?d*; $\bigwedge$*X Y*. $[\![$*X* ∈ *carrier-mat ?n ?n*; *Y* ∈ *carrier-mat ?n ?n*$]\!] \Longrightarrow$ *?Psi* (*X* + *Y*) = *?Psi X* + *?Psi Y*; $\bigwedge$*c X*. *X* ∈ *carrier-mat ?n ?n* $\Longrightarrow$ *?Psi* (*c* ·$_m$ *X*) = *c* ·$_m$ *?Psi X*; *completely-positive ?n ?d* (λ*X*. *?Phi X* − *?Psi X*); $0 <$ *?n*; $0 <$ *?d*$]\!] \Longrightarrow \exists T.\ T \in$ *carrier-mat* (*length ?Vs*) (*length ?Vs*) ∧ *T* $\geq_M$ *0* ∧ (∀*X*. *X* ∈ *carrier-mat ?n ?n* ⟶ *?Psi X* = *kraus-quad-map ?d ?Vs T X*), obtaining linearity of Ψ from the *lin-on-carrier* conjuncts carried by *cptp-pair-mid*.

theorem *choi-theorem-3-cptp-forward-complex*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: $0 < n$ **and** *dpos*: $0 < d$
and Φ-*in*: $\Phi \in CPTP\ n\ d$
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *lin-indep*: *mat-lin-indep-complex n* (*kraus-products Vs*)

shows *extreme-CPTP-carrier n d* Φ
⟨*proof*⟩

43.25 Choi's Theorem 3 (statement)

NOTE on conventions. Choi (1975, Theorem 3) gives an extreme-point characterisation in terms of *linear independence of certain products of Kraus operators*. Which products appear depends on which of the two natural normalisations the underlying convex set uses:

- **CPTP** ($\sum\ V_i^\dagger\ V_i = I_n$): products are $\{V_j^\dagger\ V_i\}$ in M_n. The perturbation $\Phi_\varepsilon(X) = \sum(\delta_{\{ij\}} + \varepsilon\ a_{\{ij\}})\ V_i\ X\ V_j^\dagger$ stays trace-preserving iff $\sum\ a_{\{ij\}}\ V_j^\dagger\ V_i = 0$.
- **UCP** ($\sum\ V_i\ V_i^\dagger = I_d$): products are $\{V_i\ V_j^\dagger\}$ in M_d. Same perturbation stays unital iff $\sum\ a_{\{ij\}}\ V_i\ V_j^\dagger = 0$.

The two are interchanged by the Hilbert–Schmidt adjoint $\Phi \mapsto \Phi^*$; this provides a UCP $\leftrightarrow$ CPTP bijection.

We state both versions, parametrised by a fixed Kraus representation; the "any/every" character follows from the unitary-freedom of Kraus reps (Day 22 / "Kraus uniqueness"), which is not yet formalised in this development.

Reverse direction (extreme ⇒ lin. indep.): contrapositive. For the CPTP version: if $\sum\ a_{\{ij\}}\ V_j^\dagger\ V_i = 0$ non-trivially, define Φ_ε as above; for small hermitian a, both $\Phi_{\pm\varepsilon}$ are CPTP and distinct, giving a non-trivial midpoint of Φ.

Forward direction (lin. indep. ⇒ extreme): requires Radon–Nikodym for CP maps (every CP $\Psi \leq_{\{cp\}} \lambda\Phi$ has $\Psi(X) = \sum\ T_{\{ij\}}\ V_i\ X\ V_j^\dagger$ for a PSD matrix T) which is itself a substantial development.

The CPTP form of Choi's Theorem 3 (carrier-flavoured). Φ is extreme in CPTP iff the products $\{V_j^\dagger\ V_i\}$ are linearly independent in M_n.

The project's *extreme-CPTP* uses a raw function inequality $\Phi_1 \neq \Phi$; since *completely-positive* and *trace-preserving* only constrain behaviour on *carrier-mat n n*, this is technically vacuous (every CPTP map admits a CPTP Φ_1 that agrees with it on carrier but disagrees off carrier). The theorem is therefore stated in terms of *extreme-CPTP-carrier*, the carrier-point variant.

The first biconditional below uses different lin-indep notions on each direction (the historical form); *choi-theorem-3-cptp-iff* states the genuine iff:

- *extreme-CPTP-carrier* ⟹ *mat-lin-indep-real* (contrapositive of ⟦*0 < ?n*; *0 < ?d*; *?*Φ ∈ *CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?*Φ *?Vs*; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ ⟹ ¬ *extreme-CPTP-carrier ?n ?d ?*Φ);
- *mat-lin-indep-complex* ⟹ *extreme-CPTP-carrier* (the strengthened Choi forward direction ⟦*0 < ?n*; *0 < ?d*; *?*Φ ∈ *CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?*Φ *?Vs*; *mat-lin-indep-complex ?n* (*kraus-products ?Vs*)⟧ ⟹ *extreme-CPTP-carrier ?n ?d ?*Φ, modulo CP Radon–Nikodym).

  The complex-coefficient witness *herm-witness-of-complex* bridges the two notions, so the direction "extreme ⇒ independent" also holds over ℂ.

theorem *choi-theorem-3-cptp*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0 < n* **and** *dpos*: *0 < d*
and Φ-*in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*

and *minimal*: *lin-indep-complex-mat d n Vs*
shows (*extreme-CPTP-carrier n d* Φ
⟶ *mat-lin-indep-real n* (*kraus-products Vs*))
∧ (*mat-lin-indep-complex n* (*kraus-products Vs*)
⟶ *extreme-CPTP-carrier n d* Φ)
⟨*proof*⟩

With the complex-coefficient witness, the two directions use the same notion of linear independence: Choi's Theorem 3 in the form of the notes' corollary.

theorem *choi-theorem-3-cptp-iff*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
and *Vs* :: *complex Matrix.mat list*
assumes *npos*: *0* < *n* **and** *dpos*: *0* < *d*
and Φ*-in*: Φ ∈ *CPTP n d*
and *rep*: *cptp-kraus-rep n d* Φ *Vs*
and *minimal*: *lin-indep-complex-mat d n Vs*
shows *extreme-CPTP-carrier n d* Φ ⟷ *mat-lin-indep-complex n* (*kraus-products Vs*)
⟨*proof*⟩

The UCP form of Choi's Theorem 3. In the UCP / Heisenberg convention, the relevant products live in M_d with the *opposite* order $\{V_i\ {V_j}^\dagger\}$.
The project's *kraus-products* uses the CPTP-side convention $\{{V_j}^\dagger\ V_i\}$, and there is no UCP-side carrier-extreme notion or analogue of ⟦*0* < *?n*; *0* < *?d*; *?*Φ ∈ *CPTP ?n ?d*; *cptp-kraus-rep ?n ?d ?*Φ *?Vs*; *lin-indep-complex-mat ?d ?n ?Vs*; ¬ *mat-lin-indep-real ?n* (*kraus-products ?Vs*)⟧ ⟹ ¬ *extreme-CPTP-carrier ?n ?d ?*Φ in this development. A proper UCP form of Choi's theorem requires either:

- a parallel *kraus-products-ucp* $(V_i\ {V_j}^\dagger)$, an *extreme-UCP-carrier* predicate, and full UCP-side analogues of the carrier-flavoured lemmas; or
- a Hilbert–Schmidt duality between *extreme-UCP* and *extreme-CPTP* that transports the CPTP theorem above.

  The second route is taken in theory Day23_Choi3_UCP, which states the UCP form.

43.26 Sanity / easy facts about extreme UCP

If a UCP map has a non-trivial mid-point representation by two distinct UCP maps, it is not extreme *by definition*.

lemma *not-extreme-UCP-if-midpoint*:
assumes Φ₁ ∈ *UCP n d* Φ₂ ∈ *UCP n d*
and Φ₁ ≠ Φ **and** Φ ≠ Φ₂
and Φ ∈ *UCP n d*
and ⋀*X*. *X* ∈ *carrier-mat n n*
⟹ Φ *X* = (*1/2* :: *complex*) ·$_m$ Φ₁ *X*
+ (*1/2* :: *complex*) ·$_m$ Φ₂ *X*
shows ¬ *extreme-UCP n d* Φ
⟨*proof*⟩

The contrapositive: every extreme UCP map, when written as a mid-point of two UCP maps, must have those maps coincide with itself on every *X* ∈ *carrier-mat n n*.

lemma *extreme-UCP-midpoint-collapses*:
assumes *ext*: *extreme-UCP n d* Φ
and Φ*1-in*: Φ₁ ∈ *UCP n d*

and $\Phi 2\text{-}in$: $\Phi_2 \in UCP\ n\ d$
and *mid*: $\bigwedge X.\ X \in carrier\text{-}mat\ n\ n$
$\Longrightarrow \Phi\ X = (1/2 :: complex) \cdot_m \Phi_1\ X$
$+ (1/2 :: complex) \cdot_m \Phi_2\ X$
shows $\Phi_1 = \Phi \vee \Phi = \Phi_2$
⟨*proof*⟩

This closes the Day-23 stub. The full content of Choi's Theorem 3 awaits a Kraus-list formalisation; the definitions collected above will plug into that proof once available.

end
theory *Day23-Choi3-UCP*
imports *Day22-Adjoint-Maps*
begin

44 Day 23: Choi's Theorem 3 for unital maps (D23-1)

The notes state Choi's Theorem 3 for UCP maps; *choi-theorem-3-cptp-iff* proves the CPTP form. The UCP form follows by the duality of Day 22: $\Gamma(\Phi) = \Phi^*$ maps midpoint decompositions of a UCP map to midpoint decompositions of its adjoint and back (*extreme-UCP-iff-adjoint*), and the Kraus operators of Φ^* are the adjoints W_i^*, which are linearly independent with the W_i. Extremality is the carrier-point notion used for CPTP maps.

The UCP-side midpoint relation and extremality, mirroring *cptp-pair-mid* and *extreme-CPTP-carrier*.

definition *ucp-pair-mid* ::
nat ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*)
⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
ucp-pair-mid n d Φ Φ_1 Φ_2 ≡
$\Phi_1 \in UCP\ n\ d \wedge \Phi_2 \in UCP\ n\ d \wedge$
$(\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi_1\ X \in carrier\text{-}mat\ d\ d) \wedge$
$(\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi_2\ X \in carrier\text{-}mat\ d\ d) \wedge$
lin-on-carrier n $\Phi_1 \wedge$ *lin-on-carrier n* $\Phi_2 \wedge$
$(\exists X_1 \in carrier\text{-}mat\ n\ n.\ \Phi_1\ X_1 \neq \Phi\ X_1) \wedge$
$(\exists X_2 \in carrier\text{-}mat\ n\ n.\ \Phi\ X_2 \neq \Phi_2\ X_2) \wedge$
$(\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ X = (1/2 :: complex) \cdot_m \Phi_1\ X + (1/2 :: complex) \cdot_m \Phi_2\ X)$

definition *extreme-UCP-carrier* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
extreme-UCP-carrier n d Φ ≡ $\Phi \in UCP\ n\ d \wedge \neg\ (\exists \Phi_1\ \Phi_2.$ *ucp-pair-mid n d* Φ Φ_1 $\Phi_2)$

lemma *lin-on-from-carrier*:
assumes $\forall X.\ X \in carrier\text{-}mat\ n\ n \longrightarrow \Phi\ X \in carrier\text{-}mat\ d\ d$ *lin-on-carrier n* Φ
shows *lin-on n d* (*carrier-mat n n*) Φ
⟨*proof*⟩

lemma *lin-on-carrier-from-lin-on*: *lin-on n d* (*carrier-mat n n*) Φ ⟹ *lin-on-carrier n* Φ
⟨*proof*⟩

lemma *map-adjoint-half*:
assumes *L1*: *lin-on n d* (*carrier-mat n n*) *Φ1* **and** *L2*: *lin-on n d* (*carrier-mat n n*) *Φ2* **and** *Y*: $Y \in carrier\text{-}mat\ d\ d$
shows *map-adjoint n d* ($\lambda X.$ $(1/2 :: complex) \cdot_m \Phi 1\ X + (1/2 :: complex) \cdot_m \Phi 2\ X$) *Y*
$= (1/2 :: complex) \cdot_m$ *map-adjoint n d Φ1 Y* $+ (1/2 :: complex) \cdot_m$ *map-adjoint n d Φ2 Y*
⟨*proof*⟩

Γ carries midpoint decompositions to midpoint decompositions, in both directions.

lemma *ucp-mid-imp-cptp-mid*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *mid*: *ucp-pair-mid n d* Φ Φ*1* Φ*2* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *cptp-pair-mid d n* (*map-adjoint n d* Φ) (*map-adjoint n d* Φ*1*) (*map-adjoint n d* Φ*2*)
⟨*proof*⟩

lemma *cptp-mid-imp-ucp-mid*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *mid*: *cptp-pair-mid d n* (*map-adjoint n d* Φ) Ψ*1* Ψ*2* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *ucp-pair-mid n d* Φ (*map-adjoint d n* Ψ*1*) (*map-adjoint d n* Ψ*2*)
⟨*proof*⟩

theorem *extreme-UCP-iff-adjoint*:
assumes *L*: *lin-on n d* (*carrier-mat n n*) Φ **and** *U*: Φ ∈ *UCP n d* **and** *n*: *0* < *n* **and** *d*: *0* < *d*
shows *extreme-UCP-carrier n d* Φ ⟷ *extreme-CPTP-carrier d n* (*map-adjoint n d* Φ)
⟨*proof*⟩

Linear independence passes to the adjoints: $\sum_k c_k W_k^* = 0$ gives $\sum_k \bar{c}_k W_k = 0$.

lemma *lin-indep-complex-mat-dagger*:
assumes *li*: *lin-indep-complex-mat d n Ws*
shows *lin-indep-complex-mat n d* (*map* (λ*W*. *W*†) *Ws*)
⟨*proof*⟩

lemma *kraus-operators-dagger-of-unital*:
assumes *Wc*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n* **and** *un*: *kraus-map-dim d Ws* (*1*$_m$ *n*) = *1*$_m$ *d*
shows *kraus-operators d n* (*map* (λ*W*. *W*†) *Ws*)
⟨*proof*⟩

Choi's Theorem 3: a unital CP map $\Phi(X) = \sum_i W_i X W_i^*$ with linearly independent W_i is extreme iff the products $W_j W_i^*$ are linearly independent (*kraus-products* of the adjoints, whose entries are $(W_j^*)^* W_i^* = W_j W_i^*$). The notes write $\Phi(X) = \sum_i V_i^* X V_i$, i.e. $V_i = W_i^*$, and the products $V_i^* V_j$.

theorem *choi-theorem-3-ucp*:
assumes *n*: *0* < *n* **and** *d*: *0* < *d* **and** *U*: Φ ∈ *UCP n d*
and *Wc*: ∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat d n*
and *rep*: ⋀*X*. *X* ∈ *carrier-mat n n* ⟹ Φ *X* = *kraus-map-dim d Ws X*
and *li*: *lin-indep-complex-mat d n Ws*
shows *extreme-UCP-carrier n d* Φ ⟷ *mat-lin-indep-complex d* (*kraus-products* (*map* (λ*W*. *W*†) *Ws*))
⟨*proof*⟩

end
theory *Day23-Extremality-Examples*
imports *Day22-Kraus-Independence Day25-Arveson-Correspondence*
begin

Day 23, examples after Choi's Theorem 3, in the CPTP form and with the Kraus convention $\Phi(X) = \sum_i W_i X W_i^*$ of the formalisation (the notes write $U^* X U$; replace U by U^*).

44.1 Linear dependence tests

lemma *foldr-smult-index*:
assumes *Bs*: ∀ *B* ∈ *set Bs*. *B* ∈ *carrier-mat n m* **and** *r*: *r* < *n* **and** *s*: *s* < *m*
shows *foldr* (λ*k acc*. *c k* ·$_m$ (*Bs* ! *k*) + *acc*) [*0*..<*length Bs*] (*0*$_m$ *n m*) $$ (*r*,*s*)
= (∑ *k*<*length Bs*. *c k* ∗ *Bs* ! *k* $$ (*r*,*s*))
⟨*proof*⟩

lemma *not-indep-of-dup*:
assumes *a*: $a < length\ Bs$ **and** *b*: $b < length\ Bs$ **and** *ab*: $a \neq b$ **and** *eq*: $Bs\ !\ a = Bs\ !\ b$
shows $\neg$ *mat-lin-indep-complex n Bs*
⟨*proof*⟩

lemma *not-indep-of-zero*:
assumes *a*: $a < length\ Bs$ **and** *z*: $Bs\ !\ a = 0_m\ n\ n$
shows $\neg$ *mat-lin-indep-complex n Bs*
⟨*proof*⟩

lemma *single-lin-indep*:
assumes *W*: $W \in carrier\text{-}mat\ d\ n$ **and** *r*: $r < d$ **and** *s*: $s < n$ **and** *nz*: $W\ \$\$\ (r,s) \neq 0$
shows *lin-indep-complex-mat d n* [*W*]
⟨*proof*⟩

lemma *single-mat-lin-indep*:
assumes *B*: $B \in carrier\text{-}mat\ n\ n$ **and** *r*: $r < n$ **and** *s*: $s < n$ **and** *nz*: $B\ \$\$\ (r,s) \neq 0$
shows *mat-lin-indep-complex n* [*B*]
⟨*proof*⟩

lemma *nonzero-entry*:
fixes *A* :: *complex Matrix.mat*
assumes *A*: $A \in carrier\text{-}mat\ m\ k$ **and** *nz*: $A \neq 0_m\ m\ k$
obtains *r s* **where** $r < m\ s < k\ A\ \$\$\ (r,s) \neq 0$
⟨*proof*⟩

44.2 Example 1: unitary conjugation is extreme

theorem *unitary-conjugation-extreme*:
assumes *U*: $U \in carrier\text{-}mat\ n\ n$ **and** *Uu*: *unitary U* **and** *n*: $0 < n$
shows *extreme-CPTP-carrier n n* (*kraus-map-dim n* [*U*])
⟨*proof*⟩

44.3 Example 2: an average of two unitary conjugations

definition *avg2* :: *complex Matrix.mat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat list* **where**
avg2 U1 U2 = [(*1 / csqrt 2*) $\cdot_m$ *U1*, (*1 / csqrt 2*) $\cdot_m$ *U2*]

lemma *half-sq*: *cnj* (*1 / csqrt 2*) $*$ (*1 / csqrt 2*) = *1/2*
⟨*proof*⟩

lemma *smult-prod*:
fixes *A B* :: *complex Matrix.mat*
assumes $A \in carrier\text{-}mat\ m\ k\ B \in carrier\text{-}mat\ k\ l$
shows $(a \cdot_m A) * (b \cdot_m B) = (a * b) \cdot_m (A * B)$
⟨*proof*⟩

lemma *smult-triple*:
fixes *A X B* :: *complex Matrix.mat*
assumes $A \in carrier\text{-}mat\ m\ k\ X \in carrier\text{-}mat\ k\ k\ B \in carrier\text{-}mat\ k\ l$
shows $(a \cdot_m A) * X * (b \cdot_m B) = (a * b) \cdot_m (A * X * B)$
⟨*proof*⟩

lemma *avg2-products*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
shows $(avg2\ U1\ U2\ !\ 0)^\dagger * avg2\ U1\ U2\ !\ 0 = (1/2) \cdot_m 1_m\ n$
and $(avg2\ U1\ U2\ !\ 1)^\dagger * avg2\ U1\ U2\ !\ 1 = (1/2) \cdot_m 1_m\ n$
⟨*proof*⟩

lemma *avg2-kraus*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
shows *kraus-operators n n* (*avg2 U1 U2*)
⟨*proof*⟩

lemma *avg2-map*:
assumes *U1*: *U1* ∈ *carrier-mat n n* **and** *U2*: *U2* ∈ *carrier-mat n n* **and** *M*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim n* (*avg2 U1 U2*) *M* = (*1/2*) ·$_m$ (*U1* ∗ *M* ∗ (*U1*†) + *U2* ∗ *M* ∗ (*U2*†))
⟨*proof*⟩

The notes' claim needs U_1, U_2 linearly independent: then the products $V_i^* V_j$ contain $\frac{1}{2}I$ twice and Φ is not extreme.

theorem *avg2-not-extreme*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *U2*: *U2* ∈ *carrier-mat n n unitary U2*
and *n*: *0* < *n* **and** *ind*: *lin-indep-complex-mat n n* (*avg2 U1 U2*)
shows ¬ *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*avg2 U1 U2*))
⟨*proof*⟩

Without that hypothesis the claim fails: for $U_2 = \lambda U_1$ with $|\lambda| = 1$ the map is $X \mapsto U_1 X U_1^*$, which is extreme.

theorem *avg2-proportional-extreme*:
assumes *U1*: *U1* ∈ *carrier-mat n n unitary U1* **and** *lam*: *cmod z* = *1* **and** *n*: *0* < *n*
shows *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*avg2 U1* (*z* ·$_m$ *U1*)))
⟨*proof*⟩

44.4 Example 4: the diagonal map

lemma *unit-mult-left-entry*:
assumes *Bm*: *Bm* ∈ *carrier-mat n m* **and** *r*: *r* < *p* **and** *k*: *k* < *m* **and** *j*: *j* < *n*
shows (*E*⟦*i,j* ; *p,n*⟧ ∗ *Bm*) $$ (*r,k*) = (*if r* = *i then Bm* $$ (*j,k*) *else 0*)
⟨*proof*⟩

lemma *unit-mult-right-entry*:
assumes *Cm*: *Cm* ∈ *carrier-mat p n* **and** *r*: *r* < *p* **and** *s*: *s* < *m* **and** *i*: *i* < *n*
shows (*Cm* ∗ *E*⟦*i,j* ; *n,m*⟧) $$ (*r,s*) = (*if s* = *j then Cm* $$ (*r,i*) *else 0*)
⟨*proof*⟩

definition *diag-kraus* :: *nat* ⇒ *complex Matrix.mat list* **where**
diag-kraus n = *map* (λ*i*. *E*⟦*i,i* ; *n,n*⟧) [*0*..<*n*]

lemma *diag-kraus-carrier*: ∀ *W* ∈ *set* (*diag-kraus n*). *W* ∈ *carrier-mat n n*
⟨*proof*⟩

lemma *diag-kraus-map*:
assumes *M*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim n* (*diag-kraus n*) *M* = *Matrix.mat n n* (λ(*r,s*). *if r* = *s then M* $$ (*r,r*) *else 0*)
⟨*proof*⟩

lemma *diag-kraus-operators*: *kraus-operators n n* (*diag-kraus n*)
⟨*proof*⟩

lemma *diag-kraus-lin-indep*: *lin-indep-complex-mat n n* (*diag-kraus n*)
⟨*proof*⟩

theorem *diag-not-extreme*:
assumes *n*: *2* ≤ *n*
shows ¬ *extreme-CPTP-carrier n n* (*kraus-map-dim n* (*diag-kraus n*))

⟨*proof*⟩

44.5 Example 3: $X \mapsto \mathrm{Tr}(X)\, I_d$

As written, $\Phi(X) = \mathrm{Tr}(X) I_d$ is neither unital ($\Phi(I_n) = n I_d$) nor trace preserving ($\mathrm{Tr}\,\Phi(X) = d\,\mathrm{Tr}\,X$) unless $n = 1$ resp. $d = 1$. The CPTP normalisation is $\Phi(X) = \mathrm{Tr}(X) I_d/d$, with the nd Kraus operators $E_{ab}/\sqrt{d}$; these are linearly independent, so $\mathrm{cr}(\Phi) = nd > n$ for $d \geq 2$ and Φ is not extreme (D22-6).

definition *tr-kraus* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat list* **where**
tr-kraus n d = *map* (λ(*a*,*b*). *complex-of-real* (*1* / *sqrt* (*real d*)) ·$_m$ *E*⟦*a*,*b* ; *d*,*n*⟧) (*List.product* [*0*..<*d*] [*0*..<*n*])

lemma *tr-kraus-carrier*: ∀ *W* ∈ *set* (*tr-kraus n d*). *W* ∈ *carrier-mat d n*
⟨*proof*⟩

lemma *tr-kraus-length*: *length* (*tr-kraus n d*) = *d* ∗ *n*
⟨*proof*⟩

lemma *tr-scalar*:
assumes *d*: *0* < *d*
shows *cnj* (*complex-of-real* (*1* / *sqrt* (*real d*))) ∗ *complex-of-real* (*1* / *sqrt* (*real d*)) = *1* / *of-nat d*
and *complex-of-real* (*1* / *sqrt* (*real d*)) ∗ *cnj* (*complex-of-real* (*1* / *sqrt* (*real d*))) = *1* / *of-nat d*
⟨*proof*⟩

lemma *tr-kraus-operators*:
assumes *d*: *0* < *d*
shows *kraus-operators n d* (*tr-kraus n d*)
⟨*proof*⟩

lemma *tr-kraus-map*:
assumes *d*: *0* < *d* **and** *M*: *M* ∈ *carrier-mat n n*
shows *kraus-map-dim d* (*tr-kraus n d*) *M* = (*Tr M* / *of-nat d*) ·$_m$ *1*$_m$ *d*
⟨*proof*⟩

lemma *tr-kraus-lin-indep*:
assumes *d*: *0* < *d* **and** *n*: *0* < *n*
shows *lin-indep-complex-mat d n* (*tr-kraus n d*)
⟨*proof*⟩

theorem *tr-map-not-extreme*:
assumes *n*: *0* < *n* **and** *d*: *2* ≤ *d*
shows ¬ *extreme-CPTP-carrier n d* (*kraus-map-dim d* (*tr-kraus n d*))
⟨*proof*⟩

end
theory *Day24-Operator-Systems*
imports *Day17-Environment-Model*
begin

Day 24: operator systems $S \subseteq M_n$ (subspaces containing I and closed under adjoints), the matrices $M_p(S)$ over them, and completely positive maps on S. An element of $M_p(M_n)$ is a $pn \times pn$ matrix whose (a, b) block is *block-of-mat n X a b*; the ampliation $\Phi^{(p)}$ is *amplification n d p* Φ (Day 19).

definition *operator-system* :: *nat* ⇒ *complex Matrix.mat set* ⇒ *bool* **where**
operator-system n OS ⟷ *OS* ⊆ *carrier-mat n n* ∧ *1*$_m$ *n* ∈ *OS* ∧ (∀ *A* ∈ *OS*. *A*† ∈ *OS*)
∧ (∀ *A* ∈ *OS*. ∀ *B* ∈ *OS*. *A* + *B* ∈ *OS*) ∧ (∀ *A* ∈ *OS*. ∀ *c*. *c* ·$_m$ *A* ∈ *OS*)

definition *Mp* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ *complex Matrix.mat set* **where**
Mp p n OS = {*A* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*). ∀ *a* < *p*. ∀ *b* < *p*. *block-of-mat n A a b* ∈ *OS*}

definition *lin-on* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
lin-on n d OS Φ ⟷ (∀ *A* ∈ *OS*. Φ *A* ∈ *carrier-mat d d*) ∧ (∀ *A* ∈ *OS*. ∀ *B* ∈ *OS*. Φ (*A* + *B*) = Φ *A* + Φ *B*)
∧ (∀ *A* ∈ *OS*. ∀ *c*. Φ (*c* ·$_m$ *A*) = *c* ·$_m$ Φ *A*)

definition *p-positive-on* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
p-positive-on n d p OS Φ ⟷ (∀ *A* ∈ *Mp p n OS*. *psd A* ⟶ *psd* (*amplification n d p* Φ *A*))

definition *cp-on* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
cp-on n d OS Φ ⟷ (∀ *p* > *0*. *p-positive-on n d p OS* Φ)

lemma *block-of-mat-dims* [*simp*]: *dim-row* (*block-of-mat n A a b*) = *n dim-col* (*block-of-mat n A a b*) = *n*
⟨*proof*⟩

lemma *operator-systemD*:
assumes *operator-system n OS*
shows *OS* ⊆ *carrier-mat n n* 1$_m$ *n* ∈ *OS* ⋀*A*. *A* ∈ *OS* ⟹ *A*† ∈ *OS*
⋀*A B*. *A* ∈ *OS* ⟹ *B* ∈ *OS* ⟹ *A* + *B* ∈ *OS* ⋀*A c*. *A* ∈ *OS* ⟹ *c* ·$_m$ *A* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-zero*:
assumes *OS*: *operator-system n OS*
shows *0*$_m$ *n n* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-diff*:
assumes *OS*: *operator-system n OS* **and** *A*: *A* ∈ *OS* **and** *B*: *B* ∈ *OS*
shows *A* − *B* ∈ *OS*
⟨*proof*⟩

lemma *operator-system-full*: *operator-system n* (*carrier-mat n n*)
⟨*proof*⟩

lemma *Mp-full*: *Mp p n* (*carrier-mat n n*) = *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*)
⟨*proof*⟩

lemma *cp-on-full*: *cp-on n d* (*carrier-mat n n*) Φ ⟷ *completely-positive n d* Φ
⟨*proof*⟩

lemma *cp-on-mono*:
assumes *cp-on n d OS* Φ **and** *OS*′ ⊆ *OS*
shows *cp-on n d OS*′ Φ
⟨*proof*⟩

44.6 $M_p(OS)$ is an operator system

lemma *block-of-mat-add*:
assumes *A* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*) *B* ∈ *carrier-mat* (*p* ∗ *n*) (*p* ∗ *n*) *a* < *p b* < *p*
shows *block-of-mat n* (*A* + *B*) *a b* = *block-of-mat n A a b* + *block-of-mat n B a b*
⟨*proof*⟩

lemma *block-of-mat-smult*:
assumes $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ $a < p$ $b < p$
shows *block-of-mat n* $(k \cdot_m A)$ *a b* $= k \cdot_m$ *block-of-mat n A a b*
⟨*proof*⟩

lemma *block-of-mat-dagger*:
assumes $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ $a < p$ $b < p$
shows *block-of-mat n* $(A^\dagger)$ *a b* $=$ (*block-of-mat n A b a*)†
⟨*proof*⟩

lemma *block-of-mat-one*:
assumes $a < p$ $b < p$
shows *block-of-mat n* $(1_m\ (p * n))$ *a b* $=$ (*if a* $=$ *b then* 1_m *n else* 0_m *n n*)
⟨*proof*⟩

The note of Day 24: $M_p(S)$ is again an operator system.

lemma *Mp-operator-system*:
assumes *OS*: *operator-system n OS*
shows *operator-system* $(p * n)$ (*Mp p n OS*)
⟨*proof*⟩

44.7 Every element is a combination of four positives

lemma *herm-bound*:
assumes *Hmc*: *Hm* $\in$ *carrier-mat n n* **and** *Hmh*: *hermitian Hm*
shows $\exists$ *c*::*real*. $0 \le c \wedge$ *psd* (*complex-of-real c* $\cdot_m$ 1_m *n* $-$ *Hm*) $\wedge$ *psd* (*complex-of-real c* $\cdot_m$ 1_m *n* $+$ *Hm*)
⟨*proof*⟩

Proposition (Day 24): $X = (P_1 - P_2) + i(P_3 - P_4)$ with $P_k \in S^+$.

proposition *operator-system-four-positives*:
assumes *OS*: *operator-system n OS* **and** *A*: *A* $\in$ *OS*
shows $\exists$ *P1 P2 P3 P4*. *P1* $\in$ *OS* $\wedge$ *P2* $\in$ *OS* $\wedge$ *P3* $\in$ *OS* $\wedge$ *P4* $\in$ *OS* $\wedge$ *psd P1* $\wedge$ *psd P2* $\wedge$ *psd P3* $\wedge$ *psd P4*
$\wedge$ *A* $=$ (*P1* $-$ *P2*) $+$ i $\cdot_m$ (*P3* $-$ *P4*)
⟨*proof*⟩

end
theory *Day25-Arveson-Correspondence*
imports *Day24-Operator-Systems*
begin

Days 24–25: Arveson's correspondence between maps $\Phi : S \to M_d$ and functionals $S_\Phi : M_d(S) \to \mathbb{C}$, $S_\Phi((A_{ij})) = \frac{1}{d}\sum_{i,j} \Phi(A_{ij})_{ij}$, with inverse $f \mapsto \Phi_f$, $\Phi_f(M) = d\,(f(E_{ij} \otimes M))_{ij}$.

44.8 Finite sums of matrices

definition *msum* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ (*'a* $\Rightarrow$ *complex Matrix.mat*) $\Rightarrow$ *'a list* $\Rightarrow$ *complex Matrix.mat* **where**
msum r c g xs $=$ *foldr* $(\lambda x\ acc.\ g\ x + acc)$ *xs* $(0_m\ r\ c)$

lemma *msum-Nil* [*simp*]: *msum r c g* [] $= 0_m\ r\ c$
⟨*proof*⟩

lemma *msum-Cons* [*simp*]: *msum r c g* (*x* # *xs*) $=$ *g x* $+$ *msum r c g xs*
⟨*proof*⟩

lemma *msum-carrier*: $\forall$ *x* $\in$ *set xs*. *g x* $\in$ *carrier-mat r c* $\Longrightarrow$ *msum r c g xs* $\in$ *carrier-mat r c*

⟨*proof*⟩

lemma *msum-index*:
assumes $\forall x \in set\ xs.\ g\ x \in carrier\text{-}mat\ r\ c$ **and** $i < r$ **and** $j < c$
shows $msum\ r\ c\ g\ xs\ \$\$\ (i,j) = (\sum x{\leftarrow}xs.\ g\ x\ \$\$\ (i,j))$
⟨*proof*⟩

lemma *msum-in-os*:
assumes *OS*: *operator-system n OS* **and** *g*: $\forall x \in set\ xs.\ g\ x \in OS$
shows $msum\ n\ n\ g\ xs \in OS$
⟨*proof*⟩

lemma *lin-on-zero*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ
shows $\Phi\ (0_m\ n\ n) = 0_m\ d\ d$
⟨*proof*⟩

lemma *lin-on-msum*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *g*: $\forall x \in set\ xs.\ g\ x \in OS$
shows $\Phi\ (msum\ n\ n\ g\ xs) = msum\ d\ d\ (\lambda x.\ \Phi\ (g\ x))\ xs$
⟨*proof*⟩

lemma *sum-list-product-upt*:
$(\sum x{\leftarrow}List.product\ [0..<a]\ [0..<b].\ h\ x) = (\sum i{<}a.\ \sum j{<}b.\ h\ (i,j))$
⟨*proof*⟩

lemma *sum-delta2*:
fixes *a d b e* :: *nat*
assumes *a*: $a < d$ **and** *b*: $b < e$
shows $(\sum i{<}d.\ \sum j{<}e.\ if\ i = a \wedge j = b\ then\ v\ else\ 0) = v$
⟨*proof*⟩

44.9 $E_{ij} \otimes M$ and the block decomposition

definition *unit-block* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
unit-block d n i j M = *Matrix.mat* $(d * n)\ (d * n)$
$(\lambda(r,c).\ if\ r\ div\ n = i \wedge c\ div\ n = j\ then\ M\ \$\$\ (r\ mod\ n,\ c\ mod\ n)\ else\ 0)$

lemma *unit-block-carrier* [*simp*]: *unit-block d n i j M* ∈ *carrier-mat* $(d * n)\ (d * n)$
⟨*proof*⟩

lemma *unit-block-dims* [*simp*]: *dim-row* (*unit-block d n i j M*) = $d * n$ *dim-col* (*unit-block d n i j M*) = $d * n$
⟨*proof*⟩

unit-block d n i j M is $E_{ij} \otimes M$.

lemma *unit-block-tensor*:
assumes *M*: $M \in carrier\text{-}mat\ n\ n$ **and** *i*: $i < d$ **and** *j*: $j < d$
shows *unit-block d n i j M* = *Tensor.tensor-mat* (*Matrix.mat d d* $(\lambda(a,b).\ if\ a = i \wedge b = j\ then\ 1\ else\ 0))\ M$
⟨*proof*⟩

lemma *block-unit-block*:
assumes *M*: $M \in carrier\text{-}mat\ n\ n$ **and** *a*: $a < d$ **and** *b*: $b < d$
shows *block-of-mat n* (*unit-block d n i j M*) *a b* = (*if a* = *i* ∧ *b* = *j then M else* $0_m\ n\ n$)
⟨*proof*⟩

lemma *unit-block-Mp*:

assumes *OS*: *operator-system n OS* **and** *M*: $M \in OS$ **and** *i*: $i < d$ **and** *j*: $j < d$
shows *unit-block d n i j M* $\in$ *Mp d n OS*
⟨*proof*⟩

lemma *block-decomposition*:
assumes *A*: $A \in$ *carrier-mat* $(d * n)$ $(d * n)$
shows $A =$ *msum* $(d * n)$ $(d * n)$ $(\lambda(i,j).$ *unit-block d n i j* (*block-of-mat n A i j*)) (*List.product* [*0*..<*d*] [*0*..<*d*])
⟨*proof*⟩

44.10 The correspondence

definition *arveson-functional* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *complex Matrix.mat* ⇒ *complex* **where**
arveson-functional n d Φ *A* = ($\sum i$<*d*. $\sum j$<*d*. Φ (*block-of-mat n A i j*) $$ (*i*,*j*)) / *of-nat d*

definition *arveson-map* :: *nat* ⇒ *nat* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *complex Matrix.mat* ⇒ *complex Matrix.mat* **where**
arveson-map n d f M = *Matrix.mat d d* $(\lambda(i,j).$ *of-nat d* $*$ *f* (*unit-block d n i j M*))

definition *lin-fun-on* :: *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *bool* **where**
lin-fun-on TS f ⟷ ($\forall A \in TS.\ \forall B \in TS.$ *f* (*A* + *B*) = *f A* + *f B*) ∧ ($\forall A \in TS.\ \forall c.$ *f* (*c* ·$_m$ *A*) = *c* $*$ *f A*)

definition *positive-fun-on* :: *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *bool* **where**
positive-fun-on TS f ⟷ ($\forall A \in TS.$ *psd A* ⟶ *0* ≤ *f A*)

lemma *amplification-idx*:
assumes *r*: $r < q$ **and** *s*: $s < q$ **and** *i*: $i < d$ **and** *j*: $j < d$
shows *amplification n d q* Φ *A* $$ (*r* $*$ *d* + *i*, *s* $*$ *d* + *j*) = Φ (*block-of-mat n A r s*) $$ (*i*,*j*)
⟨*proof*⟩

lemma *arveson-map-index*: $i < d \Longrightarrow j < d \Longrightarrow$ *arveson-map n d f M* $$ (*i*,*j*) = *of-nat d* $*$ *f* (*unit-block d n i j M*)
⟨*proof*⟩

lemma *arveson-map-carrier* [*simp*]: *arveson-map n d f M* $\in$ *carrier-mat d d*
⟨*proof*⟩

lemma *arveson-map-dims* [*simp*]: *dim-row* (*arveson-map n d f M*) = *d dim-col* (*arveson-map n d f M*) = *d*
⟨*proof*⟩

lemma *lin-fun-zero*:
assumes *TS*: *operator-system N TS* **and** *f*: *lin-fun-on TS f*
shows *f* (*0*$_m$ *N N*) = *0*
⟨*proof*⟩

lemma *lin-fun-msum*:
assumes *TS*: *operator-system N TS* **and** *f*: *lin-fun-on TS f* **and** *g*: $\forall x \in$ *set xs. g x* $\in$ *TS*
shows *f* (*msum N N g xs*) = ($\sum x$←*xs*. *f* (*g x*))
⟨*proof*⟩

$S_{\Phi_f} = f$ on $M_d(S)$.

theorem *arveson-functional-of-map*:
assumes *OS*: *operator-system n OS* **and** *f*: *lin-fun-on* (*Mp d n OS*) *f* **and** *A*: *A* $\in$ *Mp d n OS* **and** *d*: *0* < *d*
shows *arveson-functional n d* (*arveson-map n d f*) *A* = *f A*

⟨*proof*⟩

$\Phi_{S_\Phi} = \Phi$ on S.

theorem *arveson-map-of-functional*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *M*: $M \in OS$ **and** *d*: $0 < d$
shows *arveson-map n d* (*arveson-functional n d* Φ) *M* = Φ *M*
⟨*proof*⟩

lemma *unit-block-add*:
assumes *M*: $M \in$ *carrier-mat n n* **and** *M'*: $M' \in$ *carrier-mat n n*
shows *unit-block d n i j* (*M* + *M'*) = *unit-block d n i j M* + *unit-block d n i j M'*
⟨*proof*⟩

lemma *unit-block-smult*:
assumes *M*: $M \in$ *carrier-mat n n*
shows *unit-block d n i j* (*c* ·$_m$ *M*) = *c* ·$_m$ *unit-block d n i j M*
⟨*proof*⟩

lemma *arveson-map-lin*:
assumes *OS*: *operator-system n OS* **and** *f*: *lin-fun-on* (*Mp d n OS*) *f*
shows *lin-on n d OS* (*arveson-map n d f*)
⟨*proof*⟩

lemma *arveson-functional-lin*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ
shows *lin-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

lemma *quad-split*:
fixes *M* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
assumes *M*: $M \in$ *carrier-mat* (*q* ∗ *d*) (*q* ∗ *d*) **and** *w*: *dim-vec w* = *q* ∗ *d*
shows ⟨*w* | *M* ∗$_v$ *w*⟩ = (∑ *r*<*q*. ∑ *i*<*d*. ∑ *s*<*q*. ∑ *j*<*d*.
cnj (*w* $ (*r* ∗ *d* + *i*)) ∗ *M* $$ (*r* ∗ *d* + *i*, *s* ∗ *d* + *j*) ∗ *w* $ (*s* ∗ *d* + *j*))
⟨*proof*⟩

44.11 Theorem (Arveson): completely positive iff d-positive iff S_Φ positive

lemma *if-zero-mult*: (*if P then x else 0*) ∗ (*y*::*complex*) = (*if P then x* ∗ *y else 0*)
⟨*proof*⟩

lemma *mult-if-zero*: (*y*::*complex*) ∗ (*if P then x else 0*) = (*if P then y* ∗ *x else 0*)
⟨*proof*⟩

lemma *cnj-if-zero*: *cnj* (*if P then x else 0*) = (*if P then cnj x else 0*)
⟨*proof*⟩

lemma *nonneg-of-real-quad*:
fixes *z* :: *complex*
assumes $z \in \mathbb{R}$ **and** $0 \le$ *Re z* **and** *d*: $0 < d$
shows $0 \le z$ / *of-nat d*
⟨*proof*⟩

lemma *diag-quad-amplification*:
fixes *d n* :: *nat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat* **and** *A* :: *complex Matrix.mat*
defines *v* ≡ *Matrix.vec* (*d* ∗ *d*) (λ*x*. *if x div d* = *x mod d then 1 else* (*0*::*complex*))
shows ⟨*v* | *amplification n d d* Φ *A* ∗$_v$ *v*⟩ = (∑ *i*<*d*. ∑ *j*<*d*. Φ (*block-of-mat n A i j*) $$ (*i*,*j*))
⟨*proof*⟩

lemma *dpos-imp-functional-pos*:
assumes *OS*: *operator-system n OS* **and** *d*: *0 < d* **and** *P*: *p-positive-on n d d OS* Φ
shows *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

lemma *conj-tensor-index*:
fixes *w* :: *complex Matrix.vec*
assumes *s*: *s < q* **and** *k*: *k < d* **and** *b'*: *b' < n* **and** *b*: *b < n*
defines *V* ≡ *Tensor.tensor-mat* (*Matrix.mat q d* (λ(*r*,*t*). *w* $ (*r* ∗ *d* + *t*))) (*1*$_m$ *n*)
shows *V* $$ (*s* ∗ *n* + *b'*, *k* ∗ *n* + *b*) = (*if b' = b then w* $ (*s* ∗ *d* + *k*) *else 0*)
⟨*proof*⟩

lemma *conj-block*:
fixes *A* :: *complex Matrix.mat* **and** *w* :: *complex Matrix.vec*
assumes *A*: *A* ∈ *carrier-mat* (*q* ∗ *n*) (*q* ∗ *n*) **and** *i*: *i < d* **and** *j*: *j < d* **and** *a*: *a < n* **and** *b*: *b < n*
defines *V* ≡ *Tensor.tensor-mat* (*Matrix.mat q d* (λ(*r*,*t*). *w* $ (*r* ∗ *d* + *t*))) (*1*$_m$ *n*)
shows ((*V*†) ∗ *A* ∗ *V*) $$ (*i* ∗ *n* + *a*, *j* ∗ *n* + *b*)
= (∑ *r*<*q*. ∑ *s*<*q*. *cnj* (*w* $ (*r* ∗ *d* + *i*)) ∗ *w* $ (*s* ∗ *d* + *j*) ∗ *A* $$ (*r* ∗ *n* + *a*, *s* ∗ *n* + *b*))
⟨*proof*⟩

lemma *functional-pos-imp-cp*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *d*: *0 < d*
and *Pos*: *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
shows *cp-on n d OS* Φ
⟨*proof*⟩

Theorem (Arveson, Day 24): for linear $\Phi : S \to M_d$, completely positive $\iff$ d-positive $\iff$ S_Φ positive.

theorem *arveson-cp-iff*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *d*: *0 < d*
shows *cp-on n d OS* Φ ⟷ *p-positive-on n d d OS* Φ
and *p-positive-on n d d OS* Φ ⟷ *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
⟨*proof*⟩

Theorem (Arveson correspondence, Day 25): $\Phi \mapsto S_\Phi$ maps $CP(S, M_d)$ onto the positive functionals on $M_d(S)$, with inverse $f \mapsto \Phi_f$; it is affine and sends unital maps to states.

theorem *arveson-correspondence*:
assumes *OS*: *operator-system n OS* **and** *d*: *0 < d*
shows *lin-on n d OS* Φ ⟹ *cp-on n d OS* Φ ⟷ *positive-fun-on* (*Mp d n OS*) (*arveson-functional n d* Φ)
and *lin-fun-on* (*Mp d n OS*) *f* ⟹ *cp-on n d OS* (*arveson-map n d f*) ⟷ *positive-fun-on* (*Mp d n OS*) *f*
and *lin-on n d OS* Φ ⟹ *M* ∈ *OS* ⟹ *arveson-map n d* (*arveson-functional n d* Φ) *M* = Φ *M*
and *lin-fun-on* (*Mp d n OS*) *f* ⟹ *A* ∈ *Mp d n OS* ⟹ *arveson-functional n d* (*arveson-map n d f*) *A* = *f A*
⟨*proof*⟩

lemma *arveson-functional-affine*:
assumes *OS*: *operator-system n OS* **and** *L1*: *lin-on n d OS* Φ **and** *L2*: *lin-on n d OS* Ψ **and** *A*: *A* ∈ *Mp d n OS*
shows *arveson-functional n d* (λ*M*. *t* ·$_m$ Φ *M* + (*1* − *t*) ·$_m$ Ψ *M*) *A*
= *t* ∗ *arveson-functional n d* Φ *A* + (*1* − *t*) ∗ *arveson-functional n d* Ψ *A*
⟨*proof*⟩

lemma *arveson-functional-unital*:
assumes *OS*: *operator-system n OS* **and** *L*: *lin-on n d OS* Φ **and** *u*: Φ (*1*$_m$ *n*) = *1*$_m$ *d* **and** *d*: *0 < d*
shows *arveson-functional n d* Φ (*1*$_m$ (*d* ∗ *n*)) = *1*
⟨*proof*⟩

end
theory *Day25-Unital-Functional-Norm*
imports *Day25-Arveson-Correspondence*
HOL−Analysis.L2-Norm
begin

Day 25 proposition (D25-4): for an operator system $S \subseteq M_n$ and a linear $f : S \to \mathbb{C}$ with $f(I) = 1$, f is positive iff $\|f\| = 1$. The operator norm is $\|A\| = \sup\{\|Av\| : \|v\| \le 1\}$ and, as in the notes, $\|f\|$ is defined for bounded f. The proof differs from the notes in one step: for $P \ge 0$ the bound $\|P - sI + itI\|^2 \le s^2 + t^2$ ($s = \|P\|/2$) comes from $P^2 \le \|P\|\,P$, which uses the square root of P, instead of from the spectrum of P. Cauchy–Schwarz is *cauchy-schwarz-ineq* of Isabelle_Marries_Dirac.

definition *op-norm* :: *nat* ⇒ *complex Matrix.mat* ⇒ *real* **where**
op-norm n A = *Sup* {‖*A* ∗*v v*‖ | *v*. *dim-vec v* = *n* ∧ ‖*v*‖ ≤ *1*}

definition *fun-norm* :: *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex*) ⇒ *real* **where**
fun-norm n TS f = *Sup* {*cmod* (*f A*) | *A*. *A* ∈ *TS* ∧ *op-norm n A* ≤ *1*}

44.12 Vectors

lemma *vnorm-nonneg*: ‖*v*‖ ≥ *0*
⟨*proof*⟩

lemma *cauchy-schwarz-norm*:
assumes *dim-vec v* = *dim-vec w*
shows *cmod* ⟨*v*|*w*⟩ ≤ ‖*v*‖ ∗ ‖*w*‖
⟨*proof*⟩

lemma *inner-add-smult-expand*:
assumes *dim-vec u* = *dim-vec x*
shows ⟨*u* + *z* ·*v x* | *u* + *z* ·*v x*⟩ = ⟨*u*|*u*⟩ + *z* ∗ ⟨*u*|*x*⟩ + *cnj z* ∗ ⟨*x*|*u*⟩ + *cnj z* ∗ *z* ∗ ⟨*x*|*x*⟩
⟨*proof*⟩

lemma *smult-mat-vec*: *A* ∈ *carrier-mat n m* ⟹ *dim-vec v* = *m* ⟹ (*c* ·*m A*) ∗*v v* = *c* ·*v* (*A* ∗*v v*)
⟨*proof*⟩

lemma *entry-le-vnorm*: *i* < *dim-vec v* ⟹ *cmod* (*v* $ *i*) ≤ ‖*v*‖
⟨*proof*⟩

lemma *vnorm-le-sum*: ‖*w*‖ ≤ (∑ *i*<*dim-vec w*. *cmod* (*w* $ *i*))
⟨*proof*⟩

lemma *vnorm-zero-iff*: *dim-vec v* = *n* ⟹ ‖*v*‖ = *0* ⟷ *v* = *0v n*
⟨*proof*⟩

44.13 The operator norm

lemma *mult-vec-norm-bound*:
assumes *A*: *A* ∈ *carrier-mat n n* **and** *v*: *dim-vec v* = *n*
shows ‖*A* ∗*v v*‖ ≤ (∑ *i*<*n*. ∑ *k*<*n*. *cmod* (*A* $$ (*i*,*k*))) ∗ ‖*v*‖
⟨*proof*⟩

lemma *op-norm-bdd*:
assumes *A*: *A* ∈ *carrier-mat n n*
shows *bdd-above* {‖*A* ∗*v v*‖ | *v*. *dim-vec v* = *n* ∧ ‖*v*‖ ≤ *1*}
⟨*proof*⟩

lemma *mult-zero-vec'*: $A \in$ *carrier-mat n n* $\Longrightarrow$ $A *_v 0_v\ n = 0_v\ n$
⟨*proof*⟩

lemma *op-norm-ge*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *v*: *dim-vec v* $= n$ $\|v\| \leq 1$
shows $\|A *_v v\| \leq$ *op-norm n A*
⟨*proof*⟩

lemma *op-norm-nonneg*:
assumes *A*: $A \in$ *carrier-mat n n*
shows $0 \leq$ *op-norm n A*
⟨*proof*⟩

lemma *op-norm-mult*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *v*: *dim-vec v* $= n$
shows $\|A *_v v\| \leq$ *op-norm n A* $* \|v\|$
⟨*proof*⟩

lemma *op-norm-le*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *c*: $0 \leq c$ **and** *b*: $\bigwedge v$. *dim-vec v* $= n \Longrightarrow \|A *_v v\| \leq c * \|v\|$
shows *op-norm n A* $\leq c$
⟨*proof*⟩

lemma *op-norm-one*:
assumes *n*: $0 < n$
shows *op-norm n* $(1_m\ n) = 1$
⟨*proof*⟩

lemma *op-norm-smult-le*:
assumes *A*: $A \in$ *carrier-mat n n*
shows *op-norm n* $(c \cdot_m A) \leq$ *cmod c* $*$ *op-norm n A*
⟨*proof*⟩

lemma *op-norm-zero-imp*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *z*: *op-norm n A* $= 0$
shows $A = 0_m\ n\ n$
⟨*proof*⟩

44.14 Unital functionals

context
fixes *n* :: *nat* **and** *TS* :: *complex Matrix.mat set* **and** *f* :: *complex Matrix.mat* $\Rightarrow$ *complex*
assumes *OS*: *operator-system n TS* **and** *L*: *lin-fun-on TS f* **and** *f1*: *f* $(1_m\ n) = 1$
begin

lemma *TSc*: $A \in$ *TS* $\Longrightarrow A \in$ *carrier-mat n n* ⟨*proof*⟩

lemma *lin2*: $A \in$ *TS* $\Longrightarrow B \in$ *TS* $\Longrightarrow$ *f* $(a \cdot_m A + b \cdot_m B) = a * f\ A + b * f\ B$
⟨*proof*⟩

lemma *comb-in*: $A \in$ *TS* $\Longrightarrow B \in$ *TS* $\Longrightarrow a \cdot_m A + b \cdot_m B \in$ *TS*
⟨*proof*⟩

lemma *f-zero*: *f* $(0_m\ n\ n) = 0$
⟨*proof*⟩

A Hermitian $A \in S$ with $|\Re\langle x|Ax\rangle| \leq c\|x\|^2$ satisfies $cI \pm A \geq 0$, so a positive f has $f(A)$ real

with $|f(A)| \le c$.

lemma *herm-bound*:
assumes *pos*: *positive-fun-on TS f* **and** *A*: $A \in$ *TS hermitian A* **and** *c*: $0 \le c$
and *b*: $\bigwedge x.$ *dim-vec* $x = n \Longrightarrow |Re\ \langle x|A *_v x\rangle| \le c * \|x\|^2$
shows $Im\ (f\ A) = 0$ **and** $|Re\ (f\ A)| \le c$
⟨*proof*⟩

lemma *adjoint-inner*:
assumes *A*: $A \in$ *carrier-mat n n* **and** *x*: *dim-vec* $x = n$
shows $\langle x|A^\dagger *_v x\rangle = cnj\ \langle x|A *_v x\rangle$
⟨*proof*⟩

($\Rightarrow$): a positive unital f has $|f(A)| \le \|A\|$.

lemma *positive-imp-bound*:
assumes *pos*: *positive-fun-on TS f* **and** *A*: $A \in$ *TS*
shows *cmod* $(f\ A) \le$ *op-norm n A*
⟨*proof*⟩

($\Leftarrow$): if $|f(A)| \le \|A\|$ on S then f is positive.

lemma *sq-le-norm-quad*:
assumes *P*: $P \in$ *carrier-mat n n psd P* **and** *x*: *dim-vec* $x = n$
shows $\|P *_v x\|^2 \le$ *op-norm n P* $*$ $Re\ \langle x|P *_v x\rangle$
⟨*proof*⟩

lemma *bound-imp-positive*:
assumes *n*: $0 < n$ **and** *b*: $\forall A \in$ *TS*. *cmod* $(f\ A) \le$ *op-norm n A*
shows *positive-fun-on TS f*
⟨*proof*⟩

lemma *fun-norm-one-iff*:
assumes *n*: $0 < n$ **and** *bdd*: $\exists C.\ \forall A \in$ *TS*. *cmod* $(f\ A) \le C *$ *op-norm n A*
shows *fun-norm n TS f* $= 1 \longleftrightarrow (\forall A \in$ *TS*. *cmod* $(f\ A) \le$ *op-norm n A*$)$
⟨*proof*⟩

D25-4.

theorem *unital-positive-iff-norm-one*:
assumes *n*: $0 < n$ **and** *bdd*: $\exists C.\ \forall A \in$ *TS*. *cmod* $(f\ A) \le C *$ *op-norm n A*
shows *positive-fun-on TS f* $\longleftrightarrow$ *fun-norm n TS f* $= 1$
⟨*proof*⟩

end

end
theory *Day26-Arveson-Extension*
imports *Day25-Arveson-Correspondence Quantum-Computing-by-VP-3-10-Lmat.Separable-Cone*
begin

Day 26: Arveson's extension theorem. The notes extend the state S_Φ by Hahn–Banach (norm one) and use that a unital functional of norm one is positive. Here the positive functional S_Φ on the operator system $M_d(S)$ is extended to a positive functional on $M_d(M_n)$ directly, by separating the positive definite cone from the kernel of S_Φ (*separating-hyperplane-set-0* in the Euclidean space of typed matrices); the extension is $X \mapsto \mathrm{Tr}(DX)$ with $D \ge 0$. Dimensions are cardinalities of finite types, as in *Separable-Cone*.

44.15 Auxiliary facts

lemma *operator-system-herm-parts*:

assumes *OS*: *operator-system n OS* **and** *A*: $A \in OS$
shows $\exists\, Hp\ Kp.\ Hp \in OS \wedge Kp \in OS \wedge$ *hermitian Hp* $\wedge$ *hermitian Kp* $\wedge\ A = Hp + \mathrm{i} \cdot_m Kp$
⟨*proof*⟩

lemma *Tr-herm-mult-real*:
assumes *Wc*: $W \in$ *carrier-mat n n* **and** *Mc*: $M \in$ *carrier-mat n n* **and** *Wh*: *hermitian W* **and** *Mh*: *hermitian M*
shows *Tr* $(W * M) \in \mathbb{R}$
⟨*proof*⟩

lemma *Tr-mult-outer*:
fixes *A* :: *complex Matrix.mat* **and** *v* :: *complex Matrix.vec*
assumes *A*: $A \in$ *carrier-mat n n* **and** *v*: *dim-vec v* $= n$
shows *Tr* $(A * (v \oslash v)) = \langle v \mid A *_v v \rangle$
⟨*proof*⟩

lemma *Tr-psd-mult-nonneg*:
assumes *Dc*: $D \in$ *carrier-mat n n* **and** *Pc*: $P \in$ *carrier-mat n n* **and** *Dp*: *psd D* **and** *Pp*: *psd P*
shows $0 \leq$ *Tr* $(D * P)$
⟨*proof*⟩

lemma *herm-entry*:
assumes $M \in$ *carrier-mat n n hermitian M* $i < n\ j < n$
shows *M* $$ $(i,j) =$ *cnj* $(M$ $$ $(j,i))$
⟨*proof*⟩

lemma *herm-real-comb*:
assumes *Ac*: $A \in$ *carrier-mat n n* **and** *Bc*: $B \in$ *carrier-mat n n* **and** *Ah*: *hermitian A* **and** *Bh*: *hermitian B*
shows *hermitian* (*complex-of-real u* $\cdot_m$ *A* + *complex-of-real v* $\cdot_m$ *B*)
⟨*proof*⟩

lemma *lin-t-zero*:
fixes *b c* :: *real*
assumes $\forall\, t.\ 0 \leq c + t * b$
shows $b = 0$
⟨*proof*⟩

lemma *eps-nonneg*:
fixes *x y* :: *real*
assumes $\forall \varepsilon{>}0.\ 0 \leq x + \varepsilon * y$
shows $0 \leq x$
⟨*proof*⟩

lemma *psd-one-mat*: *psd* $(1_m\ n)$
⟨*proof*⟩

44.16 Matrices as points of the Euclidean space *lmat*

lemma *unwrap-wrap-lm*:
fixes *M* :: *complex Matrix.mat*
assumes *M* ∈ *carrier-mat CARD('k::finite) CARD('k)*
shows *unwrap-lm* (*wrap-lm M* :: (*complex*, *'k*, *'k*) *lmat*) = *M*
⟨*proof*⟩

lemma *wrap-lm-add*:
fixes *M M'* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat CARD('k::finite) CARD('k)* **and** *M'*: *M'* ∈ *carrier-mat CARD('k)*

CARD('k)
shows (*wrap-lm* (*M* + *M'*) :: (*complex*, *'k*, *'k*) *lmat*) = *wrap-lm M* + *wrap-lm M'*
⟨*proof*⟩

lemma *wrap-lm-scale*:
fixes *M* :: *complex Matrix.mat*
assumes *M*: *M* ∈ *carrier-mat CARD('k::finite) CARD('k)*
shows (*wrap-lm* (*complex-of-real r* ·$_m$ *M*) :: (*complex*, *'k*, *'k*) *lmat*) = *r* ∗$_R$ *wrap-lm M*
⟨*proof*⟩

lemma *wrap-lm-zero*: (*wrap-lm* (0$_m$ *CARD('k::finite) CARD('k)*) :: (*complex*, *'k*, *'k*) *lmat*) = *0*
⟨*proof*⟩

44.17 Extension of positive functionals

A positive functional on an operator system $T \subseteq M_N$ is $X \mapsto \mathrm{Tr}(DX)$ on T for some $D \geq 0$; hence it extends to a positive functional on M_N.

theorem *positive-functional-extension*:
fixes *TS* :: *complex Matrix.mat set* **and** *f* :: *complex Matrix.mat* ⇒ *complex*
assumes *TS*: *operator-system CARD('k::finite) TS* **and** *f*: *lin-fun-on TS f* **and** *fpos*: *positive-fun-on TS f*
shows ∃ *D*. *D* ∈ *carrier-mat CARD('k) CARD('k)* ∧ *psd D* ∧ (∀ *A* ∈ *TS*. *f A* = *Tr* (*D* ∗ *A*))
⟨*proof*⟩

44.18 Arveson's extension theorem

theorem *arveson-extension*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS* Φ
and *CP*: *cp-on CARD('n) CARD('d) OS* Φ
shows ∃ Ψ. *lin-on CARD('n) CARD('d)* (*carrier-mat CARD('n) CARD('n)*) Ψ
∧ *completely-positive CARD('n) CARD('d)* Ψ ∧ (∀ *M* ∈ *OS*. Ψ *M* = Φ *M*)
⟨*proof*⟩

Corollary: a completely positive map on an operator system has a Kraus form.

corollary *arveson-kraus*:
fixes *OS* :: *complex Matrix.mat set* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS* Φ
and *CP*: *cp-on CARD('n) CARD('d) OS* Φ
shows ∃ *Ws*. (∀ *W* ∈ *set Ws*. *W* ∈ *carrier-mat CARD('d) CARD('n)*) ∧ (∀ *M* ∈ *OS*. Φ *M* = *kraus-map-dim CARD('d) Ws M*)
⟨*proof*⟩

end
theory *Day26-CP-Factorization*
imports *Day26-Arveson-Extension Day32-Recovery-Proposition*
begin

45 Day 26: factoring a CP map through a unital or a trace-preserving one (D26-1)

The i-th block embedding $B_i : \mathbb{C}^n \to \mathbb{C}^{kn}$. The B_i are isometries with orthogonal ranges, $\sum_i B_i B_i^* = I$, so a Kraus representation $\Phi(X) = \sum_i W_i X W_i^*$ gives $\Phi(X) = V^* \psi(X) V$ with $\psi(X) = \sum_i B_i X B_i^*$ unital and $V = \sum_i B_i W_i^*$.

definition *blk-emb* :: *nat* ⇒ *nat* ⇒ *nat* ⇒ *complex Matrix.mat* **where**

*blk-emb k n i = Matrix.mat (k * n) n (λ(x, y). if x div n = i ∧ x mod n = y then 1 else 0)*

lemma *blk-emb-carrier* [*simp*]: *blk-emb k n i ∈ carrier-mat (k * n) n*
⟨*proof*⟩

lemma *blk-emb-dims* [*simp*]: *dim-row (blk-emb k n i) = k * n dim-col (blk-emb k n i) = n*
⟨*proof*⟩

lemma *blk-cond*:
fixes *x y i n* :: *nat*
assumes *y*: *y < n*
shows *(x div n = i ∧ x mod n = y) ⟷ x = i * n + y*
⟨*proof*⟩

lemma *blk-emb-gram-entry*:
assumes *x*: *x < k * n* **and** *y*: *y < k * n*
shows *(blk-emb k n i * ((blk-emb k n i)†)) $$ (x, y) =*
(if x div n = i ∧ y div n = i ∧ x mod n = y mod n then 1 else 0)
⟨*proof*⟩

lemma *blk-emb-gram*:
*foldr (λB acc. B * (B†) + acc) (map (blk-emb k n) [0..<k]) (0_m (k * n) (k * n)) = 1_m (k * n)*
⟨*proof*⟩

lemma *kraus-lin-on*:
assumes *∀ W ∈ set Ws. W ∈ carrier-mat d n*
shows *lin-on n d (carrier-mat n n) (kraus-map-dim d Ws)*
⟨*proof*⟩

lemma *kraus-compose-single*: *kraus-compose [A] Es = map (λE. A * E) Es*
⟨*proof*⟩

V has the blocks W_i^* stacked vertically, so $V^*B_i = W_i$.

definition *kraus-stack-dag* :: *nat ⇒ nat ⇒ complex Matrix.mat list ⇒ complex Matrix.mat* **where**
*kraus-stack-dag n d Ws = Matrix.mat (length Ws * n) d (λ(x, c). cnj (Ws ! (x div n) $$ (c, x mod n)))*

lemma *kraus-stack-dag-blk*:
assumes *Wc*: *∀ W ∈ set Ws. W ∈ carrier-mat d n* **and** *i*: *i < length Ws*
shows *(kraus-stack-dag n d Ws)† * blk-emb (length Ws) n i = Ws ! i*
⟨*proof*⟩

Proposition (Day 26): a CP map $\Phi : S \to M_d$ on an operator system with $\Phi(I) = P$ is $\Phi(X) = V^*\psi(X)V$ with ψ unital and completely positive (here even on all of M_n) and $V^*V = P$. The notes compress Φ to the range of P; here the Kraus form of *arveson-kraus* is used.

theorem *cp-unital-factor*:
fixes *OS* :: *complex Matrix.mat set* **and** *Φ* :: *complex Matrix.mat ⇒ complex Matrix.mat*
assumes *OS*: *operator-system CARD('n::finite) OS* **and** *L*: *lin-on CARD('n) CARD('d::finite) OS Φ*
and *CP*: *cp-on CARD('n) CARD('d) OS Φ*
shows *∃ r V ψ. 0 < r ∧ V ∈ carrier-mat r CARD('d) ∧ ψ ∈ UCP CARD('n) r*
∧ lin-on CARD('n) r (carrier-mat CARD('n) CARD('n)) ψ
*∧ (∀ X ∈ OS. Φ X = (V†) * ψ X * V) ∧ (V†) * V = Φ (1_m CARD('n))*
⟨*proof*⟩

V with the blocks W_i stacked vertically, so $B_i^*V = W_i$.

definition *kraus-stack* :: *nat ⇒ nat ⇒ complex Matrix.mat list ⇒ complex Matrix.mat* **where**
*kraus-stack n d Ws = Matrix.mat (length Ws * d) n (λ(x, c). Ws ! (x div d) $$ (x mod d, c))*

lemma *kraus-stack-blk*:
  assumes *Wc*: $\forall\, W \in$ *set Ws*. $W \in$ *carrier-mat d n* **and** *i*: $i <$ *length Ws*
  shows (*blk-emb* (*length Ws*) *d i*)† $*$ *kraus-stack n d Ws* $=$ *Ws* ! *i*
⟨*proof*⟩

Proposition (Day 26): a CP map $\Phi : M_n \to M_d$ is $\Phi(X) = \psi(VXV^*)$ with $\psi : M_r \to M_d$ completely positive and trace preserving. The notes dualise the previous proposition; here $\psi(Y) = \sum_i B_i^* Y B_i$ and $V = \sum_i B_i W_i$ directly.

theorem *cp-tp-factor*:
  fixes Φ :: *complex Matrix.mat* $\Rightarrow$ *complex Matrix.mat*
  assumes *L*: *lin-on CARD('n::finite) CARD('d::finite)* (*carrier-mat CARD('n) CARD('n)*) Φ
    and *CP*: *completely-positive CARD('n) CARD('d)* Φ
  shows $\exists\, r\ V\ \psi.\ 0 < r \wedge V \in$ *carrier-mat r CARD('n)* $\wedge\ \psi \in$ *CPTP r CARD('d)*
      $\wedge$ *lin-on r CARD('d)* (*carrier-mat r r*) ψ
      $\wedge\ (\forall\, X \in$ *carrier-mat CARD('n) CARD('n)*. $\Phi\ X = \psi\ (V * X * (V^\dagger)))$
⟨*proof*⟩

end

Part IV
Functional-analytic infrastructure (Lecture 25: Hahn–Banach)

Part V
Lectures 26–28: separability and entanglement witnesses

theory *Day24-Separability*
  imports
    Quantum-Computing11
    Quantum-Computing-by-VP-3-10-Lmat.Separable-Cone
begin

46 Days 26–28 (28 October – 2 November 2011): separability and entanglement witnesses

Entanglement revisited. Ensembles $\{\psi_i, p_i\}$ on $\mathcal{H}_A$ and $\{\phi_j, q_j\}$ on $\mathcal{H}_B$ compose to the density matrix $\sum_{i,j} p_i q_j |\psi_i\rangle\langle\psi_i| \otimes |\phi_j\rangle\langle\phi_j|$.

Definition (notes) (Day 26)**.** A density matrix $P \in L(\mathcal{H}_A \otimes \mathcal{H}_B)$ is *separable* if $P = \sum_l p_l |\psi_l\rangle\langle\psi_l| \otimes |\phi_l\rangle\langle\phi_l|$ with unit vectors $\psi_l \in \mathcal{H}_A$, $\phi_l \in \mathcal{H}_B$, $p_l \geq 0$, $\sum p_l = 1$; otherwise P is *entangled*. More generally (Day 27) $Sep \subseteq M_r(M_n)$ is the set of matrices $\sum_l P_l \otimes Q_l$ with $P_l \geq 0$, $Q_l \geq 0$.

Are there entangled density matrices, and how can one tell? The notes answer with the theory of convex cones:

Theorem (notes) (Days 27–28)**.** Let V be a finite-dimensional real vector space, $K \subseteq V$ closed and convex, $y \notin K$. Then there is a linear $f : V \to \mathbb{R}$ and $\alpha \in \mathbb{R}$ with $f(y) < \alpha \leq f(K)$; if C is a closed cone and $w \notin C$ there is f with $f(w) < 0 \leq f(C)$.

The separable matrices form a closed cone C in the real vector space of Hermitian matrices; an entangled R is separated from C by a real functional, which complexifies to $\tilde{f}(H + iK) = f(H) + if(K)$, and $\tilde{f}$ corresponds to a Hermitian matrix W with $\mathrm{Tr}(WR) < 0 \leq \mathrm{Tr}(W\sigma)$ for all separable σ: an *entanglement witness*. The notes phrase the final criterion through positive maps Φ_f obtained from the Arveson correspondence; that form is *separable-iff-positive-maps* in *Day27-Positive-Map-Criterion*, derived from the witness form proved here.

The formal development uses the typed matrices *lmat* so that HOL-Analysis' separation theorem applies: *separable-cone-lmat* is the convex cone generated by tensor products of positive matrices and *Sprime* is the set of product density matrices.

46.1 The separable cone (Day 27)

The cone is convex, conic, contains 0, and consists of Hermitian matrices.

46.2 Separable density matrices (Day 26)

For a density matrix ρ (positive with $\mathrm{Tr}\,\rho = 1$), membership in the cone is the notes' definition: ρ is a convex combination of product density matrices $\sigma_i \otimes \tau_i$.

46.3 The separable cone is closed (Day 28)

The notes record that C is a closed cone (needed for the separation theorem); here the cone is closed in the Hilbert–Schmidt topology and the separable density matrices form a compact convex set.

46.4 Separation (Days 27–28)

The separation theorem applied to the cone: a non-separable ρ is separated from C by a real-linear functional (an inner product against some a), strictly for density matrices because the separable density matrices are compact.

46.5 Entanglement witnesses (Day 28)

An *entanglement witness* for ρ is a Hermitian W with $\mathrm{Tr}(W\sigma) \geq 0$ on every separable σ and $\mathrm{Tr}(W\rho) < 0$. (i) Every non-separable Hermitian ρ has a witness; (ii) density matrices have a witness with a strict margin; (iii) (Horodecki) ρ is separable iff every Hermitian W that is non-negative on the cone is non-negative on ρ.

This closes the entanglement chapter of the notes. The Horodecki–Peres criterion for two qubits is *Day28-Horodecki-Peres*, the positive-map formulation *Day27-Positive-Map-Criterion*; the universal-witness question (Day 28) is not formalised.

end
theory *Day27-Positive-Map-Criterion*
imports *Day26-Arveson-Extension Day34-Knill-Laflamme*
begin

Days 27–28: positive maps detect entanglement. For $R \in M_r(M_n)$ (block size n), $\Phi^{(r)}(R) \geq 0$ for every positive Φ when R is separable (D27-1); a functional f on $M_r(M_n)$ is nonnegative

on the separable matrices iff Φ_f is positive (D27-3); and with the entanglement witnesses of *Separable-Cone* this gives the positive-map criterion (D28-3).

46.6 Tensor products with a matrix in the first factor

lemma *tensor-add-first*:
fixes *A B P* :: *complex Matrix.mat*
assumes *A*: *A* ∈ *carrier-mat r r* **and** *B*: *B* ∈ *carrier-mat r r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (*A* + *B*) *P* = *Tensor.tensor-mat A P* + *Tensor.tensor-mat B P*
⟨*proof*⟩

lemma *tensor-zero-first*:
fixes *P* :: *complex Matrix.mat*
assumes *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (0_m *r r*) *P* = 0_m (*r* ∗ *n*) (*r* ∗ *n*)
⟨*proof*⟩

lemma *tensor-msum-first*:
fixes *P* :: *complex Matrix.mat*
assumes *g*: ∀ *x* ∈ *set xs*. *g x* ∈ *carrier-mat r r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows *Tensor.tensor-mat* (*msum r r g xs*) *P* = *msum* (*r* ∗ *n*) (*r* ∗ *n*) (λ*x*. *Tensor.tensor-mat* (*g x*) *P*) *xs*
⟨*proof*⟩

46.7 The quadratic form of Φ_f

definition *rank1* :: *complex Matrix.vec* ⇒ *complex Matrix.mat* **where**
rank1 w = *Matrix.mat* (*dim-vec w*) (*dim-vec w*) (λ(*i*,*j*). *cnj* (*w* $ *i*) ∗ *w* $ *j*)

lemma *rank1-carrier* [*simp*]: *rank1 w* ∈ *carrier-mat* (*dim-vec w*) (*dim-vec w*)
⟨*proof*⟩

lemma *rank1-dims* [*simp*]: *dim-row* (*rank1 w*) = *dim-vec w dim-col* (*rank1 w*) = *dim-vec w*
⟨*proof*⟩

lemma *rank1-outer*: *rank1* (*conjugate u*) = *u* ⊘ *u*
⟨*proof*⟩

lemma *rank1-psd*: *psd* (*rank1 w*)
⟨*proof*⟩

lemma *quad-arveson-map*:
fixes *f* :: *complex Matrix.mat* ⇒ *complex* **and** *w* :: *complex Matrix.vec*
assumes *f*: *lin-fun-on* (*carrier-mat* (*r* ∗ *n*) (*r* ∗ *n*)) *f* **and** *w*: *dim-vec w* = *r* **and** *P*: *P* ∈ *carrier-mat n n* **and** *n*: *0* < *n*
shows ⟨*w* | *arveson-map n r f P* ∗$_v$ *w*⟩ = *of-nat r* ∗ *f* (*Tensor.tensor-mat* (*rank1 w*) *P*)
⟨*proof*⟩

46.8 Proposition (Day 27): $f(\mathrm{Sep}) \geq 0$ iff Φ_f is positive

theorem *positive-map-iff-sep-nonneg*:
fixes *f* :: *complex Matrix.mat* ⇒ *complex*
assumes *f*: *lin-fun-on* (*carrier-mat* (*r* ∗ *n*) (*r* ∗ *n*)) *f* **and** *r*: *0* < *r* **and** *n*: *0* < *n*
shows *positive-map n r* (*arveson-map n r f*)
⟷ (∀ *Q P*. *Q* ∈ *carrier-mat r r* ⟶ *psd Q* ⟶ *P* ∈ *carrier-mat n n* ⟶ *psd P*
⟶ *0* ≤ *f* (*Tensor.tensor-mat Q P*))
⟨*proof*⟩

46.9 Proposition (Day 27): positive maps keep separable matrices positive

lemma *block-tensor*:
fixes $Q\ P$ :: *complex Matrix.mat*
assumes Q: $Q \in$ *carrier-mat r r* **and** P: $P \in$ *carrier-mat n n* **and** a: $a < r$ **and** b: $b < r$
shows *block-of-mat n* (*Tensor.tensor-mat Q P*) *a b* = Q $$ (a,b) $\cdot_m$ P
⟨*proof*⟩

lemma *amplification-tensor*:
fixes $Q\ P$ :: *complex Matrix.mat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes L: *lin-on n d* (*carrier-mat n n*) Φ **and** Q: $Q \in$ *carrier-mat r r* **and** P: $P \in$ *carrier-mat n n*
and d: $0 < d$ **and** n: $0 < n$
shows *amplification n d r* Φ (*Tensor.tensor-mat Q P*) = *Tensor.tensor-mat Q* (Φ P)
⟨*proof*⟩

lemma *amplification-lin*:
fixes Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes L: *lin-on n d* (*carrier-mat n n*) Φ **and** A: $A \in$ *carrier-mat* $(p * n)$ $(p * n)$ **and** B: $B \in$ *carrier-mat* $(p * n)$ $(p * n)$
shows *amplification n d p* Φ $(A + B)$ = *amplification n d p* Φ A + *amplification n d p* Φ B
and *amplification n d p* Φ $(c \cdot_m A)$ = $c \cdot_m$ *amplification n d p* Φ A
⟨*proof*⟩

lemma *unwrap-lm-scaleR-mat*:
fixes R :: (*complex*, $'k$ :: *finite*, $'k$) *lmat*
shows *unwrap-lm* (u $*_R$ R) = *complex-of-real u* $\cdot_m$ *unwrap-lm R*
⟨*proof*⟩

theorem *positive-map-separable*:
fixes R :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat* **and** Φ :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
assumes L: *lin-on CARD*($'p$) *d* (*carrier-mat CARD*($'p$) *CARD*($'p$)) Φ **and** *pos*: *positive-map CARD*($'p$) *d* Φ
and d: $0 < d$ **and** R: $R \in$ *separable-cone-lmat*
shows *psd* (*amplification CARD*($'p$) *d CARD*($'m$) Φ (*unwrap-lm R*))
⟨*proof*⟩

46.10 Theorem (Day 28): the positive-map criterion

A Hermitian $R \in M_r(M_n)$ is separable iff $\Phi^{(r)}(R) \geq 0$ for every positive $\Phi : M_n \to M_r$.

theorem *separable-iff-positive-maps*:
fixes R :: (*complex*, $'m$ :: *finite* × $'p$ :: *finite*, $'m$ × $'p$) *lmat*
assumes *Rh*: *hermitian* (*unwrap-lm R*)
shows $R \in$ *separable-cone-lmat* ⟷
($\forall \Phi$. *lin-on CARD*($'p$) *CARD*($'m$) (*carrier-mat CARD*($'p$) *CARD*($'p$)) Φ ⟶ *positive-map CARD*($'p$) *CARD*($'m$) Φ
⟶ *psd* (*amplification CARD*($'p$) *CARD*($'m$) *CARD*($'m$) Φ (*unwrap-lm R*)))
⟨*proof*⟩

end
theory *Day27-Entangled-Examples*
imports *Day18-Positive-Maps Day27-Positive-Map-Criterion*
begin

47 Day 27: entangled matrices detected by the transpose (D27-2)

For $f : \{0, \ldots, r-1\} \to \{0, \ldots, n-1\}$ the block matrix $(E_{f(a)f(b)})_{a,b} \in M_r(M_n)$. The notes' examples are $f = (i, j)$ with $r = 2$, giving $\begin{pmatrix} E_{ii} & E_{ij} \\ E_{ji} & E_{jj} \end{pmatrix}$, and $f = \mathrm{id}$ with $r = n$, giving (E_{ij}).

definition *unit-blocks* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.mat* **where**
unit-blocks r n f = *Matrix.mat* (*r* ∗ *n*) (*r* ∗ *n*)
(λ(*x*, *y*). *if x mod n* = *f* (*x div n*) ∧ *y mod n* = *f* (*y div n*) *then 1 else 0*)

lemma *unit-blocks-carrier* [*simp*]: *unit-blocks r n f* ∈ *carrier-mat* (*r* ∗ *n*) (*r* ∗ *n*)
⟨*proof*⟩

lemma *unit-blocks-block*:
assumes *a* < *r b* < *r f a* < *n f b* < *n*
shows *block-of-mat n* (*unit-blocks r n f*) *a b* = *mat-unit* (*f a*) (*f b*) *n n*
⟨*proof*⟩

definition *unit-blocks-vec* :: *nat* ⇒ *nat* ⇒ (*nat* ⇒ *nat*) ⇒ *complex Matrix.vec* **where**
unit-blocks-vec r n f = *Matrix.vec* (*r* ∗ *n*) (λ*x*. *if x mod n* = *f* (*x div n*) *then 1 else 0*)

lemma *unit-blocks-outer*: *unit-blocks r n f* = *unit-blocks-vec r n f* ⊘ *unit-blocks-vec r n f*
⟨*proof*⟩

lemma *unit-blocks-psd*: *psd* (*unit-blocks r n f*)
⟨*proof*⟩

lemma *sum-two-support*:
fixes *g* :: *nat* ⇒ *complex*
assumes *p* ≠ *q p* < *m q* < *m* ⋀*x*. *x* < *m* ⟹ *x* ≠ *p* ⟹ *x* ≠ *q* ⟹ *g x* = *0*
shows (∑ *x* = *0*..<*m*. *g x*) = *g p* + *g q*
⟨*proof*⟩

lemma *transpose-lin-on*: *lin-on n n* (*carrier-mat n n*) (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*)
⟨*proof*⟩

The partial transpose of $(E_{f(a)f(b)})$ is $(E_{f(b)f(a)})$; its quadratic form at $e_0 \otimes e_{f(1)} - e_1 \otimes e_{f(0)}$ is -2.

lemma *unit-blocks-partial-transpose-not-psd*:
assumes *r*: *2* ≤ *r* **and** *f*: *f 0* < *n f 1* < *n f 0* ≠ *f 1*
shows ¬ *psd* (*amplification n n r* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*unit-blocks r n f*))
⟨*proof*⟩

theorem *unit-blocks-entangled*:
fixes *R* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*
assumes *R*: *unwrap-lm R* = *unit-blocks CARD('m) CARD('p) f*
and *r*: *2* ≤ *CARD('m)* **and** *f*: *f 0* < *CARD('p) f 1* < *CARD('p) f 0* ≠ *f 1*
shows *psd* (*unwrap-lm R*) ∧ *R* ∉ *separable-cone-lmat*
⟨*proof*⟩

The notes' first example: $\begin{pmatrix} E_{ii} & E_{ij} \\ E_{ji} & E_{jj} \end{pmatrix} \in M_2(M_n)$, $i \neq j$.

corollary *entangled-example-2*:
fixes *R* :: (*complex*, *'m* :: *finite* × *'p* :: *finite*, *'m* × *'p*) *lmat*

assumes *m*: *CARD('m) = 2* **and** *ij*: *i < CARD('p) j < CARD('p) i ≠ j*
and *R*: *unwrap-lm R = unit-blocks 2 CARD('p) (λa. if a = 0 then i else j)*
shows *block-of-mat CARD('p) (unwrap-lm R) 0 0 = E⟦i,i ; CARD('p),CARD('p)⟧*
and *block-of-mat CARD('p) (unwrap-lm R) 0 1 = E⟦i,j ; CARD('p),CARD('p)⟧*
and *block-of-mat CARD('p) (unwrap-lm R) 1 0 = E⟦j,i ; CARD('p),CARD('p)⟧*
and *block-of-mat CARD('p) (unwrap-lm R) 1 1 = E⟦j,j ; CARD('p),CARD('p)⟧*
and *psd (unwrap-lm R) ∧ R ∉ separable-cone-lmat*
⟨*proof*⟩

The second: $(E_{ij}) \in M_n(M_n)$, $n \geq 2$, which is *choi-seed*.

lemma *choi-seed-unit-blocks*: *choi-seed n = unit-blocks n n id*
⟨*proof*⟩

corollary *entangled-example-choi-seed*:
fixes *R* :: (*complex*, *'p* :: *finite* × *'p*, *'p* × *'p*) *lmat*
assumes *n*: *2 ≤ CARD('p)* **and** *R*: *unwrap-lm R = choi-seed CARD('p)*
shows *psd (unwrap-lm R) ∧ R ∉ separable-cone-lmat*
⟨*proof*⟩

end
theory *Day28-Horodecki-Peres*
imports *QC-Prelims*
begin

48 Day 28: the Horodecki–Peres criterion for two qubits (D28-5)

Entries of the partial transpose $\Phi^{(2)}(X)$ of a 4×4 matrix.

lemma *partial-transpose-2-index*:
fixes *M* :: *complex Matrix.mat*
assumes *i*: *i < 4* **and** *j*: *j < 4*
shows *amplification 2 2 2 (transpose-mat* :: *complex Matrix.mat ⇒ complex Matrix.mat) M $$ (i,j)*
*= M $$ ((i div 2) * 2 + j mod 2, (j div 2) * 2 + i mod 2)*
⟨*proof*⟩

lemma *product-vec-iff-det-zero*:
fixes *v* :: *complex Matrix.vec*
assumes *vd*: *dim-vec v = 4*
shows *(∃α β. dim-vec α = 2 ∧ dim-vec β = 2 ∧ v = tensor-vec α β)*
*⟷ v $ 0 * v $ 3 − v $ 1 * v $ 2 = 0*
⟨*proof*⟩

The notes' witness $w = (-\overline{v_2}, \overline{v_1})$ gives $\langle w|\Phi^{(2)}(|v\rangle\langle v|)w\rangle = -2|v_0v_3 - v_1v_2|^2$ $(= 2|\langle v_2|v_1\rangle|^2 - 2\|v_1\|^2\|v_2\|^2$ by Lagrange's identity).

lemma *partial-transpose-witness-value*:
fixes *v* :: *complex Matrix.vec*
assumes *vd*: *dim-vec v = 4*
defines *w ≡ Matrix.vec 4 (λk. if k = 0 then − cnj (v $ 2) else if k = 1 then − cnj (v $ 3)*
else if k = 2 then cnj (v $ 0) else cnj (v $ 1))
shows *⟨w | amplification 2 2 2 (transpose-mat* :: *complex Matrix.mat ⇒ complex Matrix.mat) (v ⊘ v) *$_v$ w⟩*
*= − 2 * cnj (v $ 0 * v $ 3 − v $ 1 * v $ 2) * (v $ 0 * v $ 3 − v $ 1 * v $ 2)*
⟨*proof*⟩

theorem *horodecki-peres-two-qubits*:
fixes *v* :: *complex Matrix.vec*

assumes *vd*: *dim-vec v = 4*
shows *psd* (*amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*v* ⊘ *v*))
⟷ (∃ *α β*. *dim-vec α = 2* ∧ *dim-vec β = 2* ∧ *v = tensor-vec α β*)
⟨*proof*⟩

corollary *horodecki-peres-entangled*:
fixes *v* :: *complex Matrix.vec*
assumes *vd*: *dim-vec v = 4*
shows *entangled 2 2 v* ⟷
¬ *psd* (*amplification 2 2 2* (*transpose-mat* :: *complex Matrix.mat* ⇒ *complex Matrix.mat*) (*v* ⊘ *v*))
⟨*proof*⟩

end